

# **Jersey 4.0.0 User Guide**

---

## Jersey 4.0.0 User Guide

---

---

# Table of Contents

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| Preface .....                                                        | xvi |
| 1. Getting Started .....                                             | 1   |
| 1.1. Creating a New Project from Maven Archetype .....               | 1   |
| 1.2. Exploring the Newly Created Project .....                       | 1   |
| 1.3. Running the Project .....                                       | 3   |
| 1.4. Creating a JavaEE Web Application .....                         | 5   |
| 1.5. Creating a Web Application that can be deployed on Heroku ..... | 6   |
| 1.5.1. Deploy it on Heroku .....                                     | 8   |
| 1.6. Exploring Other Jersey Examples .....                           | 11  |
| 2. Modules and dependencies .....                                    | 12  |
| 2.1. Java SE Compatibility .....                                     | 12  |
| 2.1.1. Virtual Threads and Thread Factories .....                    | 12  |
| 2.2. Introduction to Jersey dependencies .....                       | 12  |
| 2.3. Common Jersey Use Cases .....                                   | 13  |
| 2.3.1. Servlet based application on Glassfish .....                  | 13  |
| 2.3.2. Servlet based server-side application .....                   | 13  |
| 2.3.3. Client application on JDK .....                               | 14  |
| 2.3.4. Server-side application on supported containers .....         | 14  |
| 2.4. List of modules .....                                           | 15  |
| 3. JAX-RS Application, Resources and Sub-Resources .....             | 38  |
| 3.1. Root Resource Classes .....                                     | 38  |
| 3.1.1. @Path .....                                                   | 39  |
| 3.1.2. @GET, @PUT, @POST, @DELETE, ... (HTTP Methods) .....          | 39  |
| 3.1.3. @Produces .....                                               | 40  |
| 3.1.4. @Consumes .....                                               | 42  |
| 3.2. Parameter Annotations (@*Param) .....                           | 42  |
| 3.3. Sub-resources .....                                             | 46  |
| 3.4. Life-cycle of Root Resource Classes .....                       | 49  |
| 3.5. Rules of Injection .....                                        | 50  |
| 3.6. Use of @Context .....                                           | 53  |
| 3.7. Programmatic resource model .....                               | 53  |
| 4. Application Deployment and Runtime Environments .....             | 54  |
| 4.1. Introduction .....                                              | 54  |
| 4.2. JAX-RS Application Model .....                                  | 54  |
| 4.3. Auto-Discoverable Features .....                                | 55  |
| 4.3.1. Configuring Feature Auto-discovery Mechanism .....            | 56  |
| 4.4. Feature and Dynamic Feature SPI automatic registration .....    | 56  |
| 4.5. Configuring the Classpath Scanning .....                        | 57  |
| 4.6. Java SE Deployment Environments .....                           | 58  |
| 4.6.1. HTTP servers .....                                            | 58  |
| 4.6.2. Jakarta REST Bootstrap API .....                              | 61  |
| 4.6.3. Jersey WebServer SPI .....                                    | 62  |
| 4.7. Creating programmatic JAX-RS endpoint .....                     | 63  |
| 4.8. Servlet-based Deployment .....                                  | 63  |
| 4.8.1. Servlet 2.x way .....                                         | 63  |
| 4.8.2. Servlet 5.x Container .....                                   | 65  |
| 4.8.3. Jersey Servlet container modules .....                        | 69  |
| 4.9. Jakarta EE Platform .....                                       | 69  |
| 4.9.1. Managed Beans .....                                           | 70  |
| 4.9.2. Context and Dependency Injection (CDI) .....                  | 70  |
| 4.9.3. Enterprise Java Beans (EJB) .....                             | 71  |

|                                                                                     |     |
|-------------------------------------------------------------------------------------|-----|
| 4.9.4. Jakarta EE Servers .....                                                     | 71  |
| 4.10. OSGi .....                                                                    | 72  |
| 4.10.1. Enabling the OSGi shell in Glassfish .....                                  | 73  |
| 4.10.2. WAB Example .....                                                           | 74  |
| 4.10.3. HTTP Service Example .....                                                  | 74  |
| 4.11. Other Environments .....                                                      | 75  |
| 4.11.1. Oracle Java Cloud Service .....                                             | 75  |
| 5. Client API .....                                                                 | 76  |
| 5.1. Uniform Interface Constraint .....                                             | 76  |
| 5.2. Ease of use and reusing JAX-RS artifacts .....                                 | 77  |
| 5.3. Overview of the Client API .....                                               | 78  |
| 5.3.1. Getting started with the client API .....                                    | 78  |
| 5.3.2. Creating and configuring a Client instance .....                             | 78  |
| 5.3.3. Targeting a web resource .....                                               | 80  |
| 5.3.4. Identifying resource on WebTarget .....                                      | 80  |
| 5.3.5. Invoking a HTTP request .....                                                | 81  |
| 5.3.6. Example summary .....                                                        | 82  |
| 5.3.7. Setting ExecutorService and ScheduledExecutorService .....                   | 83  |
| 5.4. Java instances and types for representations .....                             | 84  |
| 5.4.1. Adding support for new representations .....                                 | 84  |
| 5.5. Client Transport Connectors .....                                              | 84  |
| 5.5.1. The default <code>HttpURLConnection</code> .....                             | 86  |
| 5.5.2. Client Connectors Properties .....                                           | 88  |
| 5.5.3. Applying additional settings to Connectors .....                             | 88  |
| 5.6. Using client request and response filters .....                                | 89  |
| 5.7. Closing connections .....                                                      | 89  |
| 5.8. Injections into client providers .....                                         | 90  |
| 5.9. Securing a Client .....                                                        | 90  |
| 5.9.1. Http Authentication Support .....                                            | 92  |
| 5.9.2. Server Name Indication (SNI) Support .....                                   | 93  |
| 5.10. <code>InvocationInterceptors</code> .....                                     | 93  |
| 5.10.1. <code>PreInvocationInterceptor</code> .....                                 | 93  |
| 5.10.2. <code>PostInvocationInterceptor</code> .....                                | 94  |
| 5.11. <code>InvocationBuilderListener</code> .....                                  | 95  |
| 5.12. Header <code>Expect:100-continue</code> support .....                         | 96  |
| 6. Reactive JAX-RS Client API .....                                                 | 97  |
| 6.1. Motivation for Reactive Client Extension .....                                 | 97  |
| 6.2. Usage and Extension Modules .....                                              | 103 |
| 6.3. Supported Reactive Libraries .....                                             | 104 |
| 6.3.1. <code>RxJava (Observable)</code> .....                                       | 105 |
| 6.3.2. <code>RxJava (Flowable)</code> .....                                         | 106 |
| 6.3.3. <code>Guava (ListenableFuture and Futures)</code> .....                      | 107 |
| 6.4. Implementing Support for Custom Reactive Libraries (SPI) .....                 | 108 |
| 7. Representations and Responses .....                                              | 110 |
| 7.1. Representations and Java Types .....                                           | 110 |
| 7.2. Building Responses .....                                                       | 111 |
| 7.3. <code>WebApplicationException</code> and Mapping Exceptions to Responses ..... | 112 |
| 7.4. Conditional GETs and Returning 304 (Not Modified) Responses .....              | 114 |
| 8. JAX-RS Entity Providers .....                                                    | 116 |
| 8.1. Introduction .....                                                             | 116 |
| 8.2. How to Write Custom Entity Providers .....                                     | 116 |
| 8.2.1. <code>MessageBodyWriter</code> .....                                         | 117 |
| 8.2.2. <code>MessageBodyReader</code> .....                                         | 121 |
| 8.3. Entity Provider Selection .....                                                | 123 |

|                                                        |     |
|--------------------------------------------------------|-----|
| 8.4. Jersey <code>MessageBodyWorkers</code> API .....  | 126 |
| 8.5. Default Jersey Entity Providers .....             | 127 |
| 9. Support for Common Media Type Representations ..... | 129 |
| 9.1. JSON .....                                        | 129 |
| 9.1.1. Approaches to JSON Support .....                | 129 |
| 9.1.2. MOXy .....                                      | 132 |
| 9.1.3. Java API for JSON Processing (JSON-P) .....     | 134 |
| 9.1.4. Jackson (2.x) .....                             | 136 |
| 9.1.5. Jettison .....                                  | 138 |
| 9.1.6. @JSONP - JSON with Padding Support .....        | 142 |
| 9.1.7. Java API for JSON Binding (JSON-B) .....        | 144 |
| 9.1.8. Properties for configuring JSON providers ..... | 145 |
| 9.2. XML .....                                         | 145 |
| 9.2.1. Low level XML support .....                     | 145 |
| 9.2.2. Getting started with JAXB .....                 | 146 |
| 9.2.3. POJOs .....                                     | 148 |
| 9.2.4. Using custom JAXBContext .....                  | 149 |
| 9.2.5. MOXy .....                                      | 149 |
| 9.2.6. Properties for configuring XML providers .....  | 150 |
| 9.3. Multipart .....                                   | 150 |
| 9.3.1. Overview .....                                  | 150 |
| 9.3.2. Client .....                                    | 151 |
| 9.3.3. Server .....                                    | 154 |
| 10. Filters and Interceptors .....                     | 158 |
| 10.1. Introduction .....                               | 158 |
| 10.2. Filters .....                                    | 158 |
| 10.2.1. Server filters .....                           | 158 |
| 10.2.2. Client filters .....                           | 161 |
| 10.3. Interceptors .....                               | 161 |
| 10.4. Filter and interceptor execution order .....     | 163 |
| 10.5. Name binding .....                               | 165 |
| 10.6. Dynamic binding .....                            | 166 |
| 10.7. Priorities .....                                 | 167 |
| 11. Asynchronous Services and Clients .....            | 169 |
| 11.1. Asynchronous Server API .....                    | 169 |
| 11.1.1. Asynchronous Server-side Callbacks .....       | 171 |
| 11.1.2. Chunked Output .....                           | 172 |
| 11.2. Client API .....                                 | 174 |
| 11.2.1. Asynchronous Client Callbacks .....            | 175 |
| 11.2.2. Chunked input .....                            | 176 |
| 12. URIs and Links .....                               | 178 |
| 12.1. Building URIs .....                              | 178 |
| 12.2. Resolve and Relativize .....                     | 179 |
| 12.3. Link .....                                       | 179 |
| 13. Declarative Hyperlinking .....                     | 181 |
| 13.1. Dependency .....                                 | 181 |
| 13.2. Links in Representations .....                   | 181 |
| 13.3. List of Link Injection .....                     | 182 |
| 13.4. Links from Resources .....                       | 182 |
| 13.5. Binding Template Parameters .....                | 182 |
| 13.6. Conditional Link Injection .....                 | 183 |
| 13.7. Link Headers .....                               | 183 |
| 13.8. Prevent Recursive Injection .....                | 184 |
| 13.9. Meta-annotation support .....                    | 184 |

|                                                                                 |     |
|---------------------------------------------------------------------------------|-----|
| 13.10. Configure and register .....                                             | 185 |
| 14. Programmatic API for Building Resources .....                               | 186 |
| 14.1. Introduction .....                                                        | 186 |
| 14.2. Programmatic Hello World example .....                                    | 186 |
| 14.2.1. Deployment of programmatic resources .....                              | 188 |
| 14.3. Additional examples .....                                                 | 189 |
| 14.4. Model processors .....                                                    | 190 |
| 15. Jersey configuration .....                                                  | 192 |
| 15.1. Jersey default configuration provider .....                               | 192 |
| 15.2. Micro profile configuration provider .....                                | 192 |
| 16. Micrometer - application observability facade .....                         | 194 |
| 16.1. Integration into Jersey .....                                             | 194 |
| 17. Server-Sent Events (SSE) Support .....                                      | 196 |
| 17.1. What are Server-Sent Events .....                                         | 196 |
| 17.2. When to use Server-Sent Events .....                                      | 197 |
| 17.3. Server-Sent Events API .....                                              | 197 |
| 17.4. Implementing SSE support in a JAX-RS resource (with JAX-RS SSE API) ..... | 198 |
| 17.4.1. Simple SSE resource method .....                                        | 198 |
| 17.4.2. Broadcasting with Jersey SSE .....                                      | 200 |
| 17.5. Consuming SSE events within Jersey clients .....                          | 202 |
| 17.5.1. <code>SseEventSource</code> reconnect support .....                     | 204 |
| 17.6. Jersey-specific Server-Sent Events API .....                              | 204 |
| 17.6.1. Implementing SSE support in a JAX-RS resource .....                     | 205 |
| 17.6.2. Consuming SSE events with Jersey clients .....                          | 208 |
| 18. Security .....                                                              | 212 |
| 18.1. Securing server .....                                                     | 212 |
| 18.1.1. <code>SecurityContext</code> .....                                      | 212 |
| 18.1.2. Authorization - securing resources .....                                | 213 |
| 18.2. Client Security .....                                                     | 215 |
| 18.3. OAuth Support .....                                                       | 215 |
| 18.3.1. OAuth 1 .....                                                           | 217 |
| 18.3.2. OAuth 2 Support .....                                                   | 221 |
| 19. WADL Support .....                                                          | 224 |
| 19.1. WADL introduction .....                                                   | 224 |
| 19.2. Configuration .....                                                       | 232 |
| 19.3. Extended WADL support .....                                               | 233 |
| 20. Bean Validation Support .....                                               | 234 |
| 20.1. Bean Validation Dependencies .....                                        | 234 |
| 20.2. Enabling Bean Validation in Jersey .....                                  | 234 |
| 20.3. Configuring Bean Validation Support .....                                 | 235 |
| 20.4. Validating JAX-RS resources and methods .....                             | 237 |
| 20.4.1. Constraint Annotations .....                                            | 237 |
| 20.4.2. Annotation constraints and Validators .....                             | 239 |
| 20.4.3. Entity Validation .....                                                 | 240 |
| 20.4.4. Annotation Inheritance .....                                            | 241 |
| 20.5. <code>@ValidateOnExecution</code> .....                                   | 241 |
| 20.6. Injecting .....                                                           | 242 |
| 20.7. Error Reporting .....                                                     | 243 |
| 20.7.1. <code>ValidationError</code> .....                                      | 244 |
| 20.8. Example .....                                                             | 246 |
| 21. Entity Data Filtering .....                                                 | 247 |
| 21.1. Enabling and configuring Entity Filtering in your application .....       | 247 |
| 21.2. Components used to describe Entity Filtering concepts .....               | 249 |
| 21.3. Using custom annotations to filter entities .....                         | 251 |

|                                                                                                           |     |
|-----------------------------------------------------------------------------------------------------------|-----|
| 21.3.1. Server-side Entity Filtering .....                                                                | 253 |
| 21.3.2. Client-side Entity Filtering .....                                                                | 255 |
| 21.4. Role-based Entity Filtering using ( <code>jakarta.annotation.security</code> )<br>annotations ..... | 256 |
| 21.5. Entity Filtering based on dynamic and configurable query parameters .....                           | 257 |
| 21.6. Defining custom handling for entity-filtering annotations .....                                     | 257 |
| 21.7. Supporting Entity Data Filtering in custom entity providers or frameworks .....                     | 258 |
| 21.8. Modules with support for Entity Data Filtering .....                                                | 259 |
| 21.9. Examples .....                                                                                      | 259 |
| 22. MVC Templates .....                                                                                   | 261 |
| 22.1. Viewable .....                                                                                      | 261 |
| 22.2. <code>@Template</code> .....                                                                        | 262 |
| 22.2.1. Annotating Resource methods .....                                                                 | 262 |
| 22.2.2. Annotating Resource classes .....                                                                 | 262 |
| 22.3. Absolute vs. Relative template reference .....                                                      | 263 |
| 22.3.1. Relative template reference .....                                                                 | 263 |
| 22.3.2. Absolute template reference .....                                                                 | 264 |
| 22.4. Handling errors with MVC .....                                                                      | 264 |
| 22.4.1. MVC & Bean Validation .....                                                                       | 265 |
| 22.5. Registration and Configuration .....                                                                | 266 |
| 22.6. Supported templating engines .....                                                                  | 267 |
| 22.6.1. Mustache .....                                                                                    | 267 |
| 22.6.2. Freemarker .....                                                                                  | 268 |
| 22.6.3. JSP .....                                                                                         | 269 |
| 22.7. Writing Custom Templating Engines .....                                                             | 271 |
| 22.8. Other Examples .....                                                                                | 272 |
| 23. Logging .....                                                                                         | 273 |
| 23.1. Logging traffic .....                                                                               | 273 |
| 23.1.1. Introduction .....                                                                                | 273 |
| 23.1.2. Configuration and registering .....                                                               | 273 |
| 24. Monitoring and Diagnostics .....                                                                      | 276 |
| 24.1. Monitoring Jersey Applications .....                                                                | 276 |
| 24.1.1. Introduction .....                                                                                | 276 |
| 24.1.2. Event Listeners .....                                                                             | 277 |
| 24.2. Tracing Support .....                                                                               | 286 |
| 24.2.1. Configuration options .....                                                                       | 286 |
| 24.2.2. Tracing Log .....                                                                                 | 287 |
| 24.2.3. Configuring tracing support via HTTP request headers .....                                        | 287 |
| 24.2.4. Format of the HTTP response headers .....                                                         | 287 |
| 24.2.5. Tracing Examples .....                                                                            | 288 |
| 25. Custom Injection and Lifecycle Management .....                                                       | 290 |
| 25.1. <code>InjectionManager</code> .....                                                                 | 290 |
| 25.2. Implementing Custom Injection Provider .....                                                        | 291 |
| 25.2.1. Using HK2 classes .....                                                                           | 291 |
| 25.2.2. Injection Provider Using Jersey API .....                                                         | 293 |
| 25.3. Defining Custom Injection Annotation .....                                                          | 294 |
| 25.3.1. Custom Injection Annotation using HK2 .....                                                       | 295 |
| 25.3.2. Custom Injection Annotation using Jersey <code>InjectionResolver</code> .....                     | 296 |
| 25.4. Custom Life Cycle Management .....                                                                  | 298 |
| 26. Jersey CDI Container Agnostic Support .....                                                           | 302 |
| 26.1. Introduction .....                                                                                  | 302 |
| 26.2. Containers Known to Work With Jersey CDI Support .....                                              | 302 |
| 26.3. Request Scope Binding .....                                                                         | 302 |
| 26.4. Jersey Weld SE Support .....                                                                        | 303 |

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| 27. GraalVM native-image generation .....                                  | 304 |
| 27.1. Modules with GraalVM native image support .....                      | 304 |
| 27.2. HelloWorld native image generation .....                             | 304 |
| 27.3. What's under the cover .....                                         | 305 |
| 28. Jersey Test Framework .....                                            | 306 |
| 28.1. Basics .....                                                         | 306 |
| 28.2. Supported Containers .....                                           | 307 |
| 28.3. Running TestNG Tests .....                                           | 308 |
| 28.4. Advanced features .....                                              | 311 |
| 28.4.1. JerseyTest Features .....                                          | 311 |
| 28.4.2. External container .....                                           | 311 |
| 28.4.3. Test Client configuration .....                                    | 311 |
| 28.4.4. Accessing the logged test records programmatically .....           | 312 |
| 28.5. Parallel Testing with Jersey Test Framework .....                    | 312 |
| 29. Building and Testing Jersey .....                                      | 314 |
| 29.1. Checking Out the Source .....                                        | 314 |
| 29.2. Building the Source .....                                            | 314 |
| 29.3. Testing .....                                                        | 314 |
| 29.4. Using NetBeans .....                                                 | 315 |
| 30. Migration Guide .....                                                  | 316 |
| 30.1. Migrating from Jersey 3.x to 4.0.x. ....                             | 316 |
| 30.1.1. Breaking Changes .....                                             | 316 |
| 30.1.2. Application servers for Jersey .....                               | 316 |
| A. Configuration Properties .....                                          | 317 |
| A.1. Common (client/server) configuration properties .....                 | 317 |
| A.2. Server configuration properties .....                                 | 322 |
| A.3. SeBootstrap and WebServer related configuration properties .....      | 331 |
| A.4. Servlet configuration properties .....                                | 332 |
| A.5. Client configuration properties .....                                 | 333 |
| A.6. Jersey configuration properties for message & entity processing ..... | 339 |
| A.7. Apache 5 HTTP client configuration properties .....                   | 340 |
| A.8. Helidon HTTP client configuration properties .....                    | 344 |
| A.9. JDK HTTP client configuration properties .....                        | 344 |
| A.10. Jetty HTTP client configuration properties .....                     | 346 |
| A.11. Netty HTTP client configuration properties .....                     | 348 |
| A.12. Java Net HTTP client configuration properties .....                  | 350 |
| A.13. Multipart configuration properties .....                             | 351 |



---

## List of Figures

|                                                                                 |     |
|---------------------------------------------------------------------------------|-----|
| 6.1. Travel Agency Orchestration Service .....                                  | 98  |
| 6.2. Time consumed to create a response for the client – synchronous way .....  | 99  |
| 6.3. Time consumed to create a response for the client – asynchronous way ..... | 101 |

---

## List of Tables

|                                                                                                                                             |     |
|---------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 2.1. Jersey 4.0.x JDK compatibility .....                                                                                                   | 12  |
| 2.2. Jersey Core .....                                                                                                                      | 15  |
| 2.3. Jersey Containers .....                                                                                                                | 15  |
| 2.4. Jersey Connectors .....                                                                                                                | 17  |
| 2.5. Jersey Media .....                                                                                                                     | 18  |
| 2.6. Jersey Extensions .....                                                                                                                | 19  |
| 2.7. Jersey Test Framework .....                                                                                                            | 23  |
| 2.8. Jersey Test Framework Providers .....                                                                                                  | 26  |
| 2.9. Jersey Glassfish Bundles .....                                                                                                         | 28  |
| 2.10. Security .....                                                                                                                        | 28  |
| 2.11. Jersey Examples .....                                                                                                                 | 28  |
| 3.1. Resource scopes .....                                                                                                                  | 50  |
| 3.2. Overview of injection types .....                                                                                                      | 52  |
| 4.1. Servlet 3 Pluggability Overview .....                                                                                                  | 69  |
| 5.1. List of Jersey Connectors .....                                                                                                        | 85  |
| 9.1. Default property values for MOXy MessageBodyReader<T> / MessageBodyWriter<T> .....                                                     | 134 |
| A.1. List of common configuration properties .....                                                                                          | 317 |
| A.2. List of server configuration properties .....                                                                                          | 322 |
| A.3. List of SeBootstrap and WebServer configuration properties .....                                                                       | 331 |
| A.4. List of servlet configuration properties .....                                                                                         | 332 |
| A.5. List of client configuration properties .....                                                                                          | 333 |
| A.6. List of Apache HTTP client configuration properties .....                                                                              | 339 |
| A.7. List of Apache 5 HTTP client configuration properties .....                                                                            | 341 |
| A.8. List of Helidon HTTP client configuration properties .....                                                                             | 344 |
| A.9. List of Jdk HTTP client configuration properties .....                                                                                 | 344 |
| A.10. List of Jetty HTTP client configuration properties .....                                                                              | 346 |
| A.11. List of Netty HTTP client configuration properties .....                                                                              | 348 |
| A.12. List of Java Net HTTP client configuration properties .....                                                                           | 350 |
| A.13. List of multipart configuration properties settable in the<br>MultiPartProperties.MULTI_PART_CONFIG_RESOURCE configuration file. .... | 351 |

---

## List of Examples

|                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------|----|
| 3.1. Simple hello world root resource class .....                                                          | 38 |
| 3.2. Specifying URI path parameter .....                                                                   | 39 |
| 3.3. PUT method .....                                                                                      | 40 |
| 3.4. Specifying output MIME type .....                                                                     | 40 |
| 3.5. Using multiple output MIME types .....                                                                | 41 |
| 3.6. Server-side content negotiation .....                                                                 | 41 |
| 3.7. Specifying input MIME type .....                                                                      | 42 |
| 3.8. Query parameters .....                                                                                | 42 |
| 3.9. Custom Java type for consuming request parameters .....                                               | 43 |
| 3.10. Processing POSTed HTML form .....                                                                    | 44 |
| 3.11. Obtaining general map of URI path and/or query parameters .....                                      | 44 |
| 3.12. Obtaining general map of header parameters .....                                                     | 44 |
| 3.13. Obtaining general map of form parameters .....                                                       | 45 |
| 3.14. Example of the bean which will be used as <code>@BeanParam</code> .....                              | 45 |
| 3.15. Injection of <code>MyBeanParam</code> as a method parameter: .....                                   | 45 |
| 3.16. Injection of more beans into one resource methods: .....                                             | 46 |
| 3.17. Sub-resource methods .....                                                                           | 46 |
| 3.18. Sub-resource locators .....                                                                          | 47 |
| 3.19. Sub-resource locators with empty path .....                                                          | 48 |
| 3.20. Sub-resource locators returning sub-type .....                                                       | 48 |
| 3.21. Sub-resource locators created from classes .....                                                     | 49 |
| 3.22. Sub-resource locators returning resource model .....                                                 | 49 |
| 3.23. Injection .....                                                                                      | 50 |
| 3.24. Wrong injection into a singleton scope .....                                                         | 51 |
| 3.25. Injection of proxies into singleton .....                                                            | 52 |
| 3.26. Example of possible injections .....                                                                 | 52 |
| 4.1. Deployment agnostic application model .....                                                           | 54 |
| 4.2. Reusing Jersey implementation in your custom application model .....                                  | 55 |
| 4.3. Registering SPI implementations using <code>ResourceConfig</code> .....                               | 58 |
| 4.4. Registering SPI implementations using <code>ResourceConfig</code> subclass .....                      | 58 |
| 4.5. Using Jersey with JDK HTTP Server .....                                                               | 59 |
| 4.6. Using Jersey with Grizzly HTTP Server .....                                                           | 59 |
| 4.7. Using Jersey with the Simple framework .....                                                          | 60 |
| 4.8. Using Jersey with Jetty HTTP Server .....                                                             | 60 |
| 4.9. Using Jersey with Netty HTTP Server .....                                                             | 61 |
| 4.10. Hooking up Jersey as a Servlet .....                                                                 | 63 |
| 4.11. Hooking up Jersey as a Servlet Filter .....                                                          | 63 |
| 4.12. Configuring Jersey container Servlet or Filter to use custom <code>Application</code> subclass ..... | 64 |
| 4.13. Configuring Jersey container Servlet or Filter to use package scanning .....                         | 65 |
| 4.14. Configuring Jersey container Servlet or Filter to use a list of classes .....                        | 65 |
| 4.15. Deployment of a JAX-RS application using <code>@ApplicationPath</code> with Servlet 5.0 .....        | 66 |
| 4.16. Configuration of maven-war-plugin to ignore missing <code>web.xml</code> .....                       | 66 |
| 4.17. Deployment of a JAX-RS application using <code>web.xml</code> with Servlet 5.0 .....                 | 66 |
| 4.18. <code>web.xml</code> of a JAX-RS application without an <code>Application</code> subclass .....      | 67 |
| 4.19. ....                                                                                                 | 68 |
| 4.20. ....                                                                                                 | 73 |
| 5.1. POST request with form parameters .....                                                               | 77 |
| 5.2. Using JAX-RS Client API .....                                                                         | 82 |
| 5.3. Using JAX-RS Client API fluently .....                                                                | 83 |
| 5.4. Setting JAX-RS Client <code>ExecutorService</code> .....                                              | 83 |
| 5.5. Sending restricted headers with <code>HttpURLConnection</code> .....                                  | 87 |

|                                                                                             |     |
|---------------------------------------------------------------------------------------------|-----|
| 5.6. Closing connections .....                                                              | 89  |
| 5.7. InjectionManagerClientProvider example .....                                           | 90  |
| 6.1. Excerpt from a synchronous approach while implementing the orchestration layer .....   | 98  |
| 6.2. Excerpt from an asynchronous approach while implementing the orchestration layer ..... | 100 |
| 6.3. Excerpt from a reactive approach while implementing the orchestration layer .....      | 102 |
| 6.4. Synchronous invocation of HTTP requests .....                                          | 103 |
| 6.5. Asynchronous invocation of HTTP requests .....                                         | 103 |
| 6.6. Reactive invocation of HTTP requests .....                                             | 103 |
| 6.7. Creating JAX-RS Client with RxJava reactive extension .....                            | 105 |
| 6.8. Obtaining Observable<Response> from Jersey/RxJava Client .....                         | 105 |
| 6.9. Creating JAX-RS Client with RxJava2 reactive extension .....                           | 106 |
| 6.10. Obtaining Flowable<Response> from Jersey/RxJava Client .....                          | 106 |
| 6.11. Creating Jersey/Guava Client .....                                                    | 107 |
| 6.12. Obtaining ListenableFuture<Response> from Jersey/Guava Client .....                   | 107 |
| 6.13. Extending RxInvoker .....                                                             | 108 |
| 6.14. Extending RxInvokerProvider .....                                                     | 109 |
| 7.1. Using File with a specific media type to produce a response .....                      | 111 |
| 7.2. Returning 201 status code and adding Location header in response to POST request ..... | 112 |
| 7.3. Adding an entity body to a custom response .....                                       | 112 |
| 7.4. Throwing exceptions to control response .....                                          | 112 |
| 7.5. Application specific exception implementation .....                                    | 113 |
| 7.6. Mapping generic exceptions to responses .....                                          | 113 |
| 7.7. Conditional GET support .....                                                          | 114 |
| 8.1. Example resource class .....                                                           | 116 |
| 8.2. MyBean entity class .....                                                              | 117 |
| 8.3. MessageBodyWriter example .....                                                        | 118 |
| 8.4. Example of assignment of annotations to a response entity .....                        | 119 |
| 8.5. Client code testing MyBeanMessageBodyWriter .....                                      | 120 |
| 8.6. Result of MyBeanMessageBodyWriter test .....                                           | 121 |
| 8.7. MessageBodyReader example .....                                                        | 121 |
| 8.8. Testing MyBeanMessageBodyReader .....                                                  | 122 |
| 8.9. Result of testing MyBeanMessageBodyReader .....                                        | 123 |
| 8.10. MessageBodyReader registered on a JAX-RS client .....                                 | 123 |
| 8.11. Result of client code execution .....                                                 | 123 |
| 8.12. Usage of MessageBodyWorkers interface .....                                           | 126 |
| 9.1. Simple JAXB bean implementation .....                                                  | 130 |
| 9.2. JAXB bean used to generate JSON representation .....                                   | 130 |
| 9.3. Tweaking JSON format using JAXB .....                                                  | 130 |
| 9.4. JAXB bean creation .....                                                               | 131 |
| 9.5. Constructing a JsonObject (JSON-Processing) .....                                      | 131 |
| 9.6. Constructing a JsonObject (Jettison) .....                                             | 131 |
| 9.7. MoxyJsonConfig - Setting properties. ....                                              | 133 |
| 9.8. Creating ContextResolver<MoxyJsonConfig> .....                                         | 133 |
| 9.9. Setting properties for MOXY providers into Configurable .....                          | 133 |
| 9.10. Building client with MOXY JSON feature enabled. ....                                  | 134 |
| 9.11. Creating JAX-RS application with MOXY JSON feature enabled. ....                      | 134 |
| 9.12. Building client with JSON-Processing JSON feature enabled. ....                       | 135 |
| 9.13. Creating JAX-RS application with JSON-Processing JSON feature enabled. ....           | 135 |
| 9.14. ContextResolver<ObjectMapper> .....                                                   | 137 |
| 9.15. Building client with Jackson JSON feature enabled. ....                               | 137 |
| 9.16. Creating JAX-RS application with Jackson JSON feature enabled. ....                   | 137 |
| 9.17. JAXB beans for JSON supported notations description, simple address bean .....        | 138 |
| 9.18. JAXB beans for JSON supported notations description, contact bean .....               | 139 |
| 9.19. JAXB beans for JSON supported notations description, initialization .....             | 139 |

|                                                                                                                   |     |
|-------------------------------------------------------------------------------------------------------------------|-----|
| 9.20. XML namespace to JSON mapping configuration for Jettison based mapped notation .....                        | 140 |
| 9.21. JSON expression with XML namespaces mapped into JSON .....                                                  | 140 |
| 9.22. JSON Array configuration for Jettison based mapped notation .....                                           | 140 |
| 9.23. JSON expression with JSON arrays explicitly configured via Jersey .....                                     | 140 |
| 9.24. JSON expression produced using badgerfish notation .....                                                    | 141 |
| 9.25. ContextResolver<ObjectMapper> .....                                                                         | 141 |
| 9.26. Building client with Jettison JSON feature enabled. ....                                                    | 142 |
| 9.27. Creating JAX-RS application with Jettison JSON feature enabled. ....                                        | 142 |
| 9.28. Simplest case of using @JSONP .....                                                                         | 142 |
| 9.29. JAXBBean for @JSONP example .....                                                                           | 142 |
| 9.30. Example of @JSONP with configured parameters. ....                                                          | 143 |
| 9.31. ContextResolver<Jsonb> .....                                                                                | 144 |
| 9.32. Register the feature and ContextResolver<Jsonb> .....                                                       | 145 |
| 9.33. Low level XML test - methods added to HelloWorldResource.java .....                                         | 145 |
| 9.34. Planet class .....                                                                                          | 146 |
| 9.35. Resource class .....                                                                                        | 146 |
| 9.36. Method for consuming Planet .....                                                                           | 147 |
| 9.37. Resource class - JAXBElement .....                                                                          | 148 |
| 9.38. Client side - JAXBElement .....                                                                             | 148 |
| 9.39. PlanetJAXBContextProvider .....                                                                             | 149 |
| 9.40. Using Provider with JAX-RS client .....                                                                     | 149 |
| 9.41. Add jersey-media-moxy dependency. ....                                                                      | 150 |
| 9.42. Register the MoxyXmlFeature class. ....                                                                     | 150 |
| 9.43. Configure and register an MoxyXmlFeature instance. ....                                                     | 150 |
| 9.44. MultiPart entity .....                                                                                      | 152 |
| 9.45. MultiPart entity in HTTP message. ....                                                                      | 152 |
| 9.46. FormDataMultiPart entity .....                                                                              | 152 |
| 9.47. FormDataMultiPart entity in HTTP message. ....                                                              | 153 |
| 9.48. Multipart - sending files. ....                                                                             | 153 |
| 9.49. Using EntityPart.Builder for building an Entity .....                                                       | 154 |
| 9.50. EntityPart - sending files. ....                                                                            | 154 |
| 9.51. Resource method using MultiPart as input parameter / return value. ....                                     | 155 |
| 9.52. Use of @FormDataParam annotation .....                                                                      | 156 |
| 9.53. Use of @FormParam annotation with EntityPart InputStream and String types<br>and returning a Response ..... | 157 |
| 9.54. Receiving a List of EntityParts .....                                                                       | 157 |
| 9.55. Returning a List of EntityParts .....                                                                       | 157 |
| 10.1. Container response filter .....                                                                             | 158 |
| 10.2. Container request filter .....                                                                              | 159 |
| 10.3. Pre-matching request filter .....                                                                           | 160 |
| 10.4. Client request filter .....                                                                                 | 161 |
| 10.5. GZIP writer interceptor .....                                                                               | 162 |
| 10.6. GZIP reader interceptor .....                                                                               | 163 |
| 10.7. @NameBinding example .....                                                                                  | 165 |
| 10.8. Dynamic binding example .....                                                                               | 166 |
| 10.9. Priorities example .....                                                                                    | 168 |
| 11.1. Simple async resource .....                                                                                 | 169 |
| 11.2. Simple async method with timeout .....                                                                      | 170 |
| 11.3. CompletionCallback example .....                                                                            | 171 |
| 11.4. ChunkedOutput example .....                                                                                 | 173 |
| 11.5. Simple client async invocation .....                                                                        | 174 |
| 11.6. Simple client fluent async invocation .....                                                                 | 174 |
| 11.7. Client async callback .....                                                                                 | 175 |
| 11.8. Client async callback for specific entity .....                                                             | 175 |

|                                                                                                |     |
|------------------------------------------------------------------------------------------------|-----|
| 11.9. ChunkedInput example .....                                                               | 176 |
| 12.1. URI building .....                                                                       | 178 |
| 12.2. Building URIs using query parameters .....                                               | 179 |
| 13.1. Creating JAX-RS application with Declarative Linking feature enabled. ....               | 185 |
| 14.1. A standard resource class .....                                                          | 186 |
| 14.2. A programmatic resource .....                                                            | 187 |
| 14.3. A programmatic resource .....                                                            | 188 |
| 14.4. A programmatic resource .....                                                            | 189 |
| 14.5. A programmatic resource .....                                                            | 189 |
| 14.6. A programmatic resource .....                                                            | 190 |
| 16.1. Annotated Micrometer resource methods .....                                              | 194 |
| 17.1. Adding the SSE dependency .....                                                          | 197 |
| 17.2. Simple SSE resource method .....                                                         | 198 |
| 17.3. Broadcasting SSE messages (with JAX-RS 3.0 API) .....                                    | 200 |
| 17.4. Consuming SSE events with SseEventSource .....                                           | 202 |
| 17.5. SseEventSource subscribe() methods .....                                                 | 203 |
| 17.6. Add jersey-media-sse dependency. ....                                                    | 205 |
| 17.7. Simple SSE resource method .....                                                         | 205 |
| 17.8. Broadcasting SSE messages .....                                                          | 207 |
| 17.9. Registering EventListener with EventSource .....                                         | 209 |
| 17.10. Overriding EventSource.onEvent(InboundEvent) method .....                               | 211 |
| 18.1. Using SecurityContext for a Resource Selection .....                                     | 212 |
| 18.2. Injecting SecurityContext into a singleton resource .....                                | 212 |
| 18.3. Securing resources using web.xml .....                                                   | 213 |
| 18.4. Registering RolesAllowedDynamicFeature using ResourceConfig .....                        | 214 |
| 18.5. Registering RolesAllowedDynamicFeature by extending ResourceConfig .....                 | 214 |
| 18.6. Applying jakarta.annotation.security to JAX-RS resource methods. ....                    | 214 |
| 18.7. Build the authorization flow utility .....                                               | 219 |
| 18.8. Perform the OAuth Authorization Flow .....                                               | 219 |
| 18.9. Authenticated requests .....                                                             | 220 |
| 18.10. Build feature from Access Token .....                                                   | 220 |
| 18.11. Specifying Access Token on a Request. ....                                              | 221 |
| 18.12. Creating Public/Private RSA-SHA1 keys .....                                             | 221 |
| 18.13. Building OAuth 2 Authorization Flow. ....                                               | 223 |
| 19.1. A simple WADL example - JAX-RS resource definition .....                                 | 224 |
| 19.2. A simple WADL example - WADL content .....                                               | 225 |
| 19.3. OPTIONS method returning WADL .....                                                      | 228 |
| 19.4. More complex WADL example - JAX-RS resource definition .....                             | 229 |
| 19.5. More complex WADL example - WADL content .....                                           | 230 |
| 20.1. Configuring Jersey specific properties for Bean Validation. ....                         | 235 |
| 20.2. Using ValidationConfig to configure Validator. ....                                      | 236 |
| 20.3. Constraint annotations on input parameters .....                                         | 237 |
| 20.4. Constraint annotations on fields .....                                                   | 238 |
| 20.5. Constraint annotations on class .....                                                    | 239 |
| 20.6. Definition of a constraint annotation .....                                              | 239 |
| 20.7. Validator implementation. ....                                                           | 239 |
| 20.8. Entity validation .....                                                                  | 240 |
| 20.9. Entity validation 2 .....                                                                | 240 |
| 20.10. Response entity validation .....                                                        | 241 |
| 20.11. Validate getter on execution .....                                                      | 241 |
| 20.12. Injecting UriInfo into a ConstraintValidator .....                                      | 242 |
| 20.13. Support for injecting Jersey's resources/providers via ConstraintValidatorFactory. .... | 243 |
| 20.14. ValidationException to text/plain .....                                                 | 245 |
| 20.15. ValidationException to text/html .....                                                  | 245 |

|                                                                                                                     |     |
|---------------------------------------------------------------------------------------------------------------------|-----|
| 20.16. <code>ValidationError</code> to <code>application/xml</code> .....                                           | 245 |
| 20.17. <code>ValidationError</code> to <code>application/json</code> .....                                          | 246 |
| 21.1. Registering and configuring entity-filtering feature on server. ....                                          | 248 |
| 21.2. Registering and configuring entity-filtering feature with security annotations on server. ....                | 248 |
| 21.3. Registering and configuring entity-filtering feature based on dynamic and configurable query parameters. .... | 249 |
| 21.4. Registering and configuring entity-filtering feature on client. ....                                          | 249 |
| 21.5. <code>Project</code> .....                                                                                    | 249 |
| 21.6. <code>User</code> .....                                                                                       | 250 |
| 21.7. <code>Task</code> .....                                                                                       | 250 |
| 21.8. <code>ProjectsResource</code> .....                                                                           | 250 |
| 21.9. <code>ProjectDetailedView</code> .....                                                                        | 251 |
| 21.10. <code>Annotated Project</code> .....                                                                         | 252 |
| 21.11. <code>Annotated User</code> .....                                                                            | 252 |
| 21.12. <code>Annotated Task</code> .....                                                                            | 253 |
| 21.13. <code>ProjectsResource</code> - Response entity-filtering annotations .....                                  | 254 |
| 21.14. <code>ProjectsResource</code> - Entity-filtering annotations on methods .....                                | 254 |
| 21.15. Client - Request entity-filtering annotations .....                                                          | 255 |
| 21.16. Client - Request entity-filtering annotations .....                                                          | 255 |
| 21.17. Sever - Query Parameter driven entity-filtering .....                                                        | 257 |
| 21.18. ....                                                                                                         | 257 |
| 21.19. Entity-filtering annotation with custom meaning .....                                                        | 258 |
| 21.20. Entity Data Filtering support in MOXy JSON binding provider .....                                            | 258 |
| 22.1. Using <code>Viewable</code> in a resource class .....                                                         | 261 |
| 22.2. Using <code>@Template</code> on a resource method .....                                                       | 262 |
| 22.3. Using <code>@Template</code> on a resource class .....                                                        | 262 |
| 22.4. Using absolute path to template in <code>Viewable</code> .....                                                | 264 |
| 22.5. Using <code>@ErrorTemplate</code> on a resource method .....                                                  | 265 |
| 22.6. Using <code>@ErrorTemplate</code> with Bean Validation .....                                                  | 265 |
| 22.7. Iterating through <code>ValidationError</code> in JSP .....                                                   | 265 |
| 22.8. Registering <code>MvcFeature</code> .....                                                                     | 266 |
| 22.9. Registering <code>FreemarkerMvcFeature</code> .....                                                           | 266 |
| 22.10. Setting <code>MvcFeature.TEMPLATE_BASE_PATH</code> value in <code>ResourceConfig</code> .....                | 267 |
| 22.11. Setting <code>FreemarkerMvcProperties.TEMPLATE_BASE_PATH</code> value in <code>web.xml</code> ...            | 267 |
| 22.12. Including JSP page into JSP page .....                                                                       | 270 |
| 22.13. Custom <code>TemplateProcessor</code> .....                                                                  | 271 |
| 22.14. Registering custom <code>TemplateProcessor</code> .....                                                      | 271 |
| 23.1. Logging on the client side .....                                                                              | 275 |
| 23.2. Register <code>LoggingFeature</code> via constructor .....                                                    | 275 |
| 23.3. Register <code>LoggingFeature</code> class .....                                                              | 275 |
| 24.1. Application event listener .....                                                                              | 277 |
| 24.2. Request event listener .....                                                                                  | 277 |
| 24.3. Event listener test resource .....                                                                            | 278 |
| 24.4. Injecting <code>MonitoringStatistics</code> .....                                                             | 280 |
| 24.5. Summary level messages .....                                                                                  | 288 |
| 24.6. On demand request, snippet of MVC JSP forwarding .....                                                        | 288 |
| 26.1. Bootstrapping Jersey application with Weld support on Grizzly .....                                           | 303 |

---

# Preface

This is user guide for Jersey 4.0.0. We are trying to keep it up to date as we add new features. When reading the user guide, please consult also our Jersey API documentation [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/index.html>] as an additional source of information about Jersey features and API.

If you would like to contribute to the guide or have questions on things not covered in our docs, please contact us at [jersey-dev@eclipse.org](mailto:jersey-dev@eclipse.org) [<mailto:jersey-dev@eclipse.org>]. Similarly, in case you spot any errors in the Jersey documentation, please report them by filing a new issue in our Jersey Issue Tracker [<https://github.com/eclipse-ee4j/jersey/issues>] under `docs` component. Please make sure to specify the version of the Jersey User Guide where the error has been spotted by selecting the proper value for the `Affected Version` field.

## Text formatting conventions

First mention of any Jersey and JAX-RS API component in a section links to the API documentation of the referenced component. Any sub-sequent mentions of the component in the same chapter are rendered using a `monospaced` font.

*Emphasised font* is used to call attention to a newly introduce concept, when it first occurs in the text.

In some of the code listings, certain lines are too long to be displayed on one line for the available page width. In such case, the lines that exceed the available page width are broken up into multiple lines using a `'\'` at the end of each line to indicate that a break has been introduced to fit the line in the page. For example:

```
This is an overly long line that \  
might not fit the available page \  
width and had to be broken into \  
multiple lines.
```

```
This line fits the page width.
```

Should read as:

```
This is an overly long line that might not fit the available page width and had to
```

```
This line fits the page width.
```



---

# Chapter 1. Getting Started

This chapter provides a quick introduction on how to get started building RESTful services using Jersey. The example described here uses the lightweight Grizzly HTTP server. At the end of this chapter you will see how to implement equivalent functionality as a JavaEE web application you can deploy on any servlet container supporting Servlet 5 and higher.

## 1.1. Creating a New Project from Maven Archetype

Jersey project is built using Apache Maven [<http://maven.apache.org/>] software project build and management tool. All modules produced as part of Jersey project build are pushed to the Central Maven Repository [<https://search.maven.org/>]. Therefore it is very convenient to work with Jersey for any Maven-based project as all the released (non-SNAPSHOT) Jersey dependencies are readily available without a need to configure a special maven repository to consume the Jersey modules.

### Note

In case you want to depend on the latest SNAPSHOT versions of Jersey modules, the following repository configuration needs to be added to your Maven project pom:

```
<snapshotRepository>
  <id>ossrh</id>
  <name>Sonatype Nexus Snapshots</name>
  <url>https://jakarta.oss.sonatype.org/content/repositories/snapshots/</url>
</snapshotRepository>
```

Since starting from a Maven project is the most convenient way for working with Jersey, let's now have a look at this approach. We will now create a new Jersey project that runs on top of a Grizzly [<https://javaee.github.io/grizzly/>] container. We will use a Jersey-provided maven archetype. To create the project, execute the following Maven command in the directory where the new project should reside:

```
mvn archetype:generate -DarchetypeArtifactId=jersey-quickstart-grizzly2 \
-DarchetypeGroupId=org.glassfish.jersey.archetypes -DinteractiveMode=false \
-DgroupId=com.example -DartifactId=simple-service -Dpackage=com.example \
-DarchetypeVersion=4.0.0
```

Feel free to adjust the `groupId`, `package` and/or `artifactId` of your new project. Alternatively, you can change it by updating the new project `pom.xml` once it gets generated.

## 1.2. Exploring the Newly Created Project

Once the project generation from a Jersey maven archetype is successfully finished, you should see the new `simple-service` project directory created in your current location. The directory contains a standard Maven project structure:

Project build and management configuration is described in the `pom.xml` located in the project root directory.

Project sources are located under `src/main/java`.

Project test sources are located under `src/test/java`.

There are 2 classes in the project source directory in the `com.example` package. The `Main` class is responsible for bootstrapping the Grizzly container as well as configuring and deploying the project's JAX-

RS application to the container. Another class in the same package is `MyResource` class, that contains implementation of a simple JAX-RS resource. It looks like this:

```
1 package com.example;
2
3 import jakarta.ws.rs.GET;
4 import jakarta.ws.rs.Path;
5 import jakarta.ws.rs.Produces;
6 import jakarta.ws.rs.core.MediaType;
7
8 /**
9  * Root resource (exposed at "myresource" path)
10  */
11 @Path("myresource")
12 public class MyResource {
13
14     /**
15      * Method handling HTTP GET requests. The returned object will be sent
16      * to the client as "text/plain" media type.
17      *
18      * @return String that will be returned as a text/plain response.
19      */
20     @GET
21     @Produces(MediaType.TEXT_PLAIN)
22     public String getIt() {
23         return "Got it!";
24     }
25 }
```

A JAX-RS resource is an annotated POJO that provides so-called *resource methods* that are able to handle HTTP requests for URI paths that the resource is bound to. See Chapter 3, *JAX-RS Application, Resources and Sub-Resources* for a complete guide to JAX-RS resources. In our case, the resource exposes a single resource method that is able to handle HTTP GET requests, is bound to `/myresource` URI path and can produce responses with response message content represented in `"text/plain"` media type. In this version, the resource returns the same `"Got it!"` response to all client requests.

The last piece of code that has been generated in this skeleton project is a `MyResourceTest` unit test class that is located in the same `com.example` package as the `MyResource` class, however, this unit test class is placed into the maven project test source directory `src/test/java` (certain code comments and JUnit imports have been excluded for brevity):

```
1 package com.example;
2
3 import jakarta.ws.rs.client.Client;
4 import jakarta.ws.rs.client.ClientBuilder;
5 import jakarta.ws.rs.client.WebTarget;
6
7 import org.glassfish.grizzly.http.server.HttpServer;
8
9 ...
10
11 public class MyResourceTest {
12
13     private HttpServer server;
14     private WebTarget target;
```

```
15
16     @BeforeEach
17     public void setUp() throws Exception {
18         server = Main.startServer();
19
20         Client c = ClientBuilder.newClient();
21         target = c.target(Main.BASE_URI);
22     }
23
24     @AfterEach
25     public void tearDown() throws Exception {
26         server.stop();
27     }
28
29     /**
30      * Test to see that the message "Got it!" is sent in the response.
31      */
32     @Test
33     public void testGetIt() {
34         String responseMsg = target.path("myresource").request().get(String.class);
35         assertEquals("Got it!", responseMsg);
36     }
37 }
```

In this unit test, a Grizzly container is first started and server application is deployed in the test `setUp()` method by a static call to `Main.startServer()`. Next, JAX-RS client components are created in the same test set-up method. First a new JAX-RS client instance `c` is built and then a JAX-RS web target component pointing to the context root of our application deployed at `http://localhost:8080/myapp/` (a value of `Main.BASE_URI` constant) is stored into a `target` field of the unit test class. This field is then used in the actual unit test method (`testGetIt()`).

In the `testGetIt()` method a fluent JAX-RS Client API is used to connect to and send a HTTP GET request to the `MyResource` JAX-RS resource class listening on `/myresource` URI. As part of the same fluent JAX-RS API method invocation chain, a response is read as a Java `String` type. On the second line in the test method, the response content string returned from the server is compared with the expected phrase in the test assertion. To learn more about using JAX-RS Client API, please see the Chapter 5, *Client API* chapter.

## 1.3. Running the Project

Now that we have seen the content of the project, let's try to test-run it. To do this, we need to invoke following command on the command line:

```
mvn clean test
```

This will compile the project and run the project unit tests. We should see a similar output that informs about a successful build once the build is finished:

Results :

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

```
[INFO] -----
[INFO] BUILD SUCCESS
```

```
[INFO] -----
[INFO] Total time: 34.527s
[INFO] Finished at: Sun May 26 19:26:24 CEST 2020
[INFO] Final Memory: 17M/490M
[INFO] -----
```

Now that we have verified that the project compiles and that the unit test passes, we can execute the application in a standalone mode. To do this, run the following maven command:

```
mvn exec:java
```

The application starts and you should soon see the following notification in your console:

```
May 26, 2020 8:08:45 PM org.glassfish.grizzly.http.server.NetworkListener start
INFO: Started listener bound to [localhost:8080]
May 26, 2020 8:08:45 PM org.glassfish.grizzly.http.server.HttpServer start
INFO: [HttpServer] Started.
Jersey app started with WADL available at http://localhost:8080/myapp/application.
Hit enter to stop it...
```

This informs you that the application has been started and its WADL descriptor is available at `http://localhost:8080/myapp/application.wadl` URL. You can retrieve the WADL content by executing a `curl http://localhost:8080/myapp/application.wadl` command in your console or by typing the WADL URL into your favorite browser. You should get back an XML document in describing your deployed RESTful application in a WADL format. To learn more about working with WADL, check the Chapter 19, *WADL Support* chapter.

The last thing we should try before concluding this section is to see if we can communicate with our resource deployed at `/myresource` path. We can again either type the resource URL in the browser or we can use `curl`:

```
$ curl http://localhost:8080/myapp/myresource
Got it!
```

As we can see, the `curl` command returned with the `Got it!` message that was sent by our resource. We can also ask `curl` to provide more information about the response, for example we can let it display all response headers by using the `-i` switch:

```
curl -i http://localhost:8080/myapp/myresource
HTTP/1.1 200 OK
Content-Type: text/plain
Date: Sun, 26 May 2020 18:27:19 GMT
Content-Length: 7
```

```
Got it!
```

Here we see the whole content of the response message that our Jersey/JAX-RS application returned, including all the HTTP headers. Notice the `Content-Type: text/plain` header that was derived from the value of `@Produces [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html]` annotation attached to the `MyResource` class.

In case you want to see even more details about the communication between our `curl` client and our resource running on Jersey in a Grizzly I/O container, feel free to try other various options and switches that `curl` provides. For example, this last command will make `curl` output a lot of additional information about the whole communication:

```
$ curl -v http://localhost:8080/myapp/myresource
```

```
* Trying 127.0.0.1:8080...
* TCP_NODELAY set
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /myapp/myresource HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Content-Type: text/plain
< Date: Sun, 26 May 2020 18:29:18 GMT
< Content-Length: 7
<
* Connection #0 to host localhost left intact
Got it!
```

## 1.4. Creating a JavaEE Web Application

To create a Web Application that can be packaged as WAR and deployed in a Servlet container follow a similar process to the one described in Section 1.1, “Creating a New Project from Maven Archetype”. In addition to the Grizzly-based archetype, Jersey provides also a Maven archetype for creating web application skeletons. To create the new web application skeleton project, execute the following Maven command in the directory where the new project should reside:

```
mvn archetype:generate -DarchetypeArtifactId=jersey-quickstart-webapp \
    -DarchetypeGroupId=org.glassfish.jersey.archetypes -DinteractiveMode=false \
    -DgroupId=com.example -DartifactId=simple-service-webapp -DpackageName=com.example \
    -DarchetypeVersion=4.0.0
```

As with the Grizzly based project, feel free to adjust the `groupId`, `packageName` and/or `artifactId` of your new web application project. Alternatively, you can change it by updating the new project `pom.xml` once it gets generated.

Once the project generation from a Jersey maven archetype is successfully finished, you should see the new `simple-service-webapp` project directory created in your current location. The directory contains a standard Maven project structure, similar to the `simple-service` project content we have seen earlier, except it is extended with an additional web application specific content:

Project build and management configuration is described in the `pom.xml` located in the project root directory.

Project sources are located under `src/main/java`.

Project resources are located under `src/main/resources`.

Project web application files are located under `src/main/webapp`.

The project contains the same `MyResource` JAX-RS resource class. It does not contain any unit tests as well as it does not contain a `Main` class that was used to setup Grizzly container in the previous project. Instead, it contains the standard Java/Jakarta EE web application `web.xml` deployment descriptor under `src/main/webapp/WEB-INF`. The last component in the project is an `index.jsp` page that serves as a client for the `MyResource` resource class that is packaged and deployed with the application.

To compile and package the application into a WAR, invoke the following maven command in your console:

```
mvn clean package
```

A successful build output will produce an output similar to the one below:

Results :

Tests run: 0, Failures: 0, Errors: 0, Skipped: 0

```
[INFO]
[INFO] --- maven-war-plugin:2.2:war (default-war) @ simple-service-webapp ---
[INFO] Packaging webapp
[INFO] Assembling webapp [simple-service-webapp] in
[.../simple-service-webapp/target/simple-service-webapp]
[INFO] Processing war project
[INFO] Copying webapp resources [.../simple-service-webapp/src/main/webapp]
[INFO] Webapp assembled in [117 msecs]
[INFO] Building war: [.../simple-service-webapp/target/simple-service-webapp.war]
[INFO] WEB-INF/web.xml already added, skipping
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.419 s
[INFO] Finished at: 2020-11-25T09:34:59+01:00
[INFO] -----
```

Now you are ready to take the packaged WAR (located under `./target/simple-service-webapp.war`) and deploy it to a Servlet container of your choice.

### Important

To deploy a Jersey application, with full set of advanced features (such as JAX-RS 2.0 Async Support) you will need a Servlet 5.0 or later compliant container.

## 1.5. Creating a Web Application that can be deployed on Heroku

To create a Web Application that can be either packaged as WAR and deployed in a Servlet container or that can be pushed and deployed on Heroku [<https://www.heroku.com>] the process is very similar to the one described in Section 1.4, “Creating a JavaEE Web Application”. To create the new web application skeleton project, execute the following Maven command in the directory where the new project should reside:

```
mvn archetype:generate -DarchetypeArtifactId=jersey-heroku-webapp \
-DarchetypeGroupId=org.glassfish.jersey.archetypes -DinteractiveMo
-DgroupId=com.example -DartifactId=simple-heroku-webapp -Dpackage=
-DarchetypeVersion=4.0.0
```

Adjust the `groupId`, `package` and/or `artifactId` of your new web application project to your needs or, alternatively, you can change it by updating the new project `pom.xml` once it gets generated.

Once the project generation from a Jersey maven archetype is successfully finished, you should see the new `simple-heroku-webapp` project directory created in your current location. The directory contains a standard Maven project structure:

Project build and management configuration is described in the `pom.xml` located in the project root directory.

Project sources are located under `src/main/java`.

Project resources are located under `src/main/resources`.

Project web application files are located under `src/main/webapp`.

Project test-sources (based on JerseyTest [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html>]) are located under `src/test/java`.

Heroku system properties (OpenJDK version) are defined in `system.properties`.

Lists of the process types in an application for Heroku is in `Procfile`.

The project contains one JAX-RS resource class, `MyResource`, and one resource method which returns simple text message. To make sure the resource is properly tested there is also a end-to-end test-case in `MyResourceTest` (the test is based on JerseyTest [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html>] from our Chapter 28, *Jersey Test Framework*). Similarly to `simple-service-webapp`, the project contains the standard Java/Jakarta EE web application `web.xml` deployment descriptor under `src/main/webapp/WEB-INF` since the goal is to deploy the application in a Servlet container (the application will run in Jetty on Heroku).

To compile and package the application into a WAR, invoke the following maven command in your console:

```
mvn clean package
```

A successful build output will produce an output similar to the one below:

Results :

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

```
[INFO]
[INFO] --- maven-war-plugin:2.2:war (default-war) @ simple-heroku-webapp ---
[INFO] Packaging webapp
[INFO] Assembling webapp [simple-heroku-webapp] in [.../simple-heroku-webapp/target]
[INFO] Processing war project
[INFO] Copying webapp resources [.../simple-heroku-webapp/src/main/webapp]
[INFO] Webapp assembled in [109 msecs]
[INFO] Building war: [.../simple-heroku-webapp/target/simple-heroku-webapp.war]
[INFO] WEB-INF/web.xml already added, skipping
[INFO]
[INFO] --- maven-dependency-plugin:2.8:copy-dependencies (copy-dependencies) @ simple-heroku-webapp ---
[INFO] Copying jakarta.validation-api-3.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying javassist-3.25.0-GA.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jersey-client-3.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jersey-hk2-3.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jetty-util-11.0.0.beta3.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jakarta.annotation-api-2.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jersey-common-3.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jersey-container-servlet-core-3.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jakarta.activation-2.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jetty-webapp-11.0.0.beta3.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying osgi-resource-locator-1.0.3.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jersey-container-servlet-3.0.0.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jetty-io-11.0.0.beta3.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying slf4j-api-2.0.0-alpha1.jar to ../simple-heroku-webapp/target/dependencies
[INFO] Copying jetty-server-11.0.0.beta3.jar to ../simple-heroku-webapp/target/dependencies
```

```
[INFO] Copying jakarta.ws.rs-api-3.0.0.jar to ../simple-heroku-webapp/target/d
[INFO] Copying jetty-servlet-11.0.0.beta3.jar to ../simple-heroku-webapp/targe
[INFO] Copying hk2-locator-3.0.0-RC1.jar to ../simple-heroku-webapp/target/dep
[INFO] Copying hk2-utils-3.0.0-RC1.jar to ../simple-heroku-webapp/target/dep
[INFO] Copying jetty-xml-11.0.0.beta3.jar to ../simple-heroku-webapp/target/de
[INFO] Copying hk2-api-3.0.0-RC1.jar to ../simple-heroku-webapp/target/depende
[INFO] Copying jetty-security-11.0.0.beta3.jar to ../simple-heroku-webapp/targ
[INFO] Copying jetty-http-11.0.0.beta3.jar to ../simple-heroku-webapp/target/d
[INFO] Copying jersey-server-3.0.0.jar to ../simple-heroku-webapp/target/dep
[INFO] Copying jakarta.inject-api-2.0.0.jar to ../simple-heroku-webapp/target/
[INFO] Copying jetty-jakarta-servlet-api-5.0.1.jar to ../simple-heroku-webapp/
[INFO] Copying aopalliance-repackaged-3.0.0-RC1.jar to ../simple-heroku-webapp
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 12.567 s
[INFO] Finished at: 2021-01-05T15:16:05+01:00
[INFO] -----
```

Now that you know everything went as expected you are ready to:

- make some changes in your project,
- take the packaged WAR (located under `./target/simple-service-webapp.war`) and deploy it to a Servlet container of your choice, or
- Section 1.5.1, “Deploy it on Heroku”

## Tip

If you want to make some changes to your application you can run the application locally by simply running `mvn clean package jetty:run` (which starts the embedded Jetty server) or by `java -cp target/classes:target/dependency/* com.example.heroku.Main` (this is how Jetty is started on Heroku).

### 1.5.1. Deploy it on Heroku

We won't go into details how to create an account on Heroku [<https://www.heroku.com>] and setup the Heroku tools on your machine. You can find a lot of information in this article: [Getting Started with Java on Heroku](https://devcenter.heroku.com/articles/getting-started-with-java) [<https://devcenter.heroku.com/articles/getting-started-with-java>]. Instead, we'll take a look at the steps needed after your environment is ready.

The first step is to create a Git repository from your project:

```
$ git init
Initialized empty Git repository in ../../simple-heroku-webapp/.git/
```

Then, create a Heroku [<https://www.heroku.com>] instance and add a remote reference to your Git repository:

```
$ heroku create
Creating simple-heroku-webapp... done, stack is cedar
http://simple-heroku-webapp.herokuapp.com/ | git@heroku.com:simple-heroku-weba
Git remote heroku added
```



## Note

The name of the instance is changed in the output to `simple-heroku-webapp`. Your will be named more like `tranquil-basin-4744`.

Add and commit files to your Git repository:

```
$ git add src/ pom.xml Procfile system.properties

$ git commit -a -m "initial commit"
[master (root-commit) e2b58e3] initial commit
7 files changed, 221 insertions(+)
create mode 100644 Procfile
create mode 100644 pom.xml
create mode 100644 src/main/java/com/example/MyResource.java
create mode 100644 src/main/java/com/example/heroku/Main.java
create mode 100644 src/main/webapp/WEB-INF/web.xml
create mode 100644 src/test/java/com/example/MyResourceTest.java
create mode 100644 system.properties
```

Push changes to Heroku:

```
$ git push heroku master
Counting objects: 21, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (11/11), done.
Writing objects: 100% (21/21), 3.73 KiB | 0 bytes/s, done.
Total 21 (delta 0), reused 0 (delta 0)

-----> Java app detected
-----> Installing OpenJDK 1.8... done
-----> Installing Maven 3.6.3... done
-----> Installing settings.xml... done
-----> executing /app/tmp/cache/.maven/bin/mvn -B -Duser.home=/tmp/build_992cc
[INFO] Scanning for projects...
[INFO]
[INFO] -----
[INFO] Building simple-heroku-webapp 1.0-SNAPSHOT
[INFO] -----
[INFO] --- maven-clean-plugin:2.4.1:clean (default-clean) @ simple-heroku-webapp ---
[INFO] --- maven-resources-plugin:2.4.3:resources (default-resources) @ simple-heroku-webapp ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /tmp/build_992cc747-26d6-4800-bdb1-ad...
[INFO] --- maven-compiler-plugin:2.5.1:compile (default-compile) @ simple-heroku-webapp ---
[INFO] Compiling 2 source files to /tmp/build_992cc747-26d6-4800-bdb1-ad...
[INFO] --- maven-resources-plugin:2.4.3:testResources (default-testResources) @ simple-heroku-webapp ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /tmp/build_992cc747-26d6-4800-bdb1-ad...
[INFO] --- maven-compiler-plugin:2.5.1:testCompile (default-testCompile) @ simple-heroku-webapp ---
[INFO] Compiling 1 source file to /tmp/build_992cc747-26d6-4800-bdb1-ad...
```

```
[INFO]
[INFO] --- maven-surefire-plugin:2.7.2:test (default-test) @ simple-her
[INFO] Tests are skipped.
[INFO]
[INFO] --- maven-war-plugin:2.1.1:war (default-war) @ simple-heroku-web
[INFO] Packaging webapp
[INFO] Assembling webapp [simple-heroku-webapp] in [/tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target]
[INFO] Processing war project
[INFO] Copying webapp resources [/tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target]
[INFO] Webapp assembled in [88 msecs]
[INFO] Building war: /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/webapp.war
[INFO] WEB-INF/web.xml already added, skipping
[INFO]
[INFO] --- maven-dependency-plugin:2.1:copy-dependencies (copy-dependencies) @ simple-heroku-web
[INFO] Copying guava-14.0.1.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jakarta.annotation-api-1.2.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying validation-api-1.1.0.Final.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jakarta.ws.rs-api-2.0.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-http-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-io-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-security-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-server-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-servlet-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-util-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-webapp-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jetty-xml-9.0.6.v20130930.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jakarta.servlet-3.0.0.v201112011016.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying hk2-api-2.2.0-b21.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying hk2-locator-2.2.0-b21.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying hk2-utils-2.2.0-b21.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying osgi-resource-locator-1.0.1.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying asm-all-repackaged-2.2.0-b21.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying cglib-2.2.0-b21.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jakarta.inject-2.2.0-b21.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jersey-container-servlet-2.5.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jersey-container-servlet-core-2.5.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jersey-client-2.5.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jersey-common-2.5.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO] Copying jersey-server-2.5.jar to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/dependencies
[INFO]
[INFO] --- maven-install-plugin:2.3.1:install (default-install) @ simple-heroku-web
[INFO] Installing /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/target/webapp.war to /tmp/build_992cc747-26d6-4800-bdb1-add47b9583cd/pom.xml
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 45.861s
[INFO] Finished at: Mon Dec 09 19:51:34 UTC 2013
[INFO] Final Memory: 17M/514M
[INFO]
-----> Discovering process types
Procfile declares types -> web

-----> Compiled slug size: 75.9MB
```

```
-----> Launching... done, v6
         http://simple-heroku-webapp.herokuapp.com deployed to Heroku

To git@heroku.com:simple-heroku-webapp.git
* [new branch]      master -> master
```

Now you can access your application at, for example: <http://simple-heroku-webapp.herokuapp.com/myresource>

## 1.6. Exploring Other Jersey Examples

In the sections above, we have covered an approach how to get dirty with Jersey quickly. Please consult the other sections of the Jersey User Guide to learn more about Jersey and JAX-RS. Even though we try our best to cover as much as possible in the User Guide, there is always a chance that you would not be able to get a full answer to the problem you are solving. In that case, consider diving in our examples that provide additional tips and hints to the features you may want to use in your projects.

Jersey codebase contains a number of useful examples on how to use various JAX-RS and Jersey features. Feel free to browse through the code of individual Jersey Examples [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples>] in the Jersey source repository. For off-line browsing, you can also download a bundle with all the examples from here [<https://repo1.maven.org/maven2/org/glassfish/jersey/bundles/jersey-examples/4.0.0/>].

---

# Chapter 2. Modules and dependencies

## 2.1. Java SE Compatibility

4.0 branch:

- This user guide refers only to version 4.0.x of Jersey, its compatibility is described below.
- Since version 4.0.0 all Jersey components are compiled with Java SE 17 target. It means, that you will need at least Java SE 11 to be able to compile and run your application which uses the latest Jersey 4.0.x.
- Please see Chapter 30, *Migration Guide* for additional compatibility information.

**Table 2.1. Jersey 4.0.x JDK compatibility**

| Jersey version | JDK min version | JDK max version |
|----------------|-----------------|-----------------|
| 4.0.0          | 17              | 23              |

### 2.1.1. Virtual Threads and Thread Factories

With JDK 21 and above, Jersey has the ability to use virtual threads instead of the `CachedThreadPool` in the internal `ExecutorServices`. Jersey also has the ability to specify the backing `ThreadFactory` for the default `ExecutorServices` (the default `ExecutorServices` can be overridden by the `ExecutorServiceProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/spi/ExecutorServiceProvider.html>] SPI).

To enable virtual threads and/or specify the `ThreadFactory`, use `CommonProperties.USE_VIRTUAL_THREADS` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#USE\\_VIRTUAL\\_THREADS](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#USE_VIRTUAL_THREADS)] and/or `CommonProperties.THREAD_FACTORY` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#THREAD\\_FACTORY](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#THREAD_FACTORY)] properties, respectively. See also the Section A.1, “Common (client/server) configuration properties” in appendix for property details.

## 2.2. Introduction to Jersey dependencies

Jersey is built, assembled and installed using Apache Maven [<http://maven.apache.org/>]. Non-snapshot Jersey releases are deployed to the Central Maven Repository [<https://search.maven.org/>]. Jersey is also being deployed to Sonatype Maven repositories [<https://oss.sonatype.org/>], which contain also Jersey SNAPSHOT versions. In case you would want to test the latest development builds check out the Sonatype Snapshots Maven repository [<https://oss.sonatype.org/content/repositories/snapshots/org/glassfish/jersey>].

An application that uses Jersey and depends on Jersey modules is in turn required to also include a set of 3rd party modules that Jersey modules depend on. Jersey is designed as a pluggable component architecture and different applications can therefore require different sets of Jersey modules. This also means that a set of external Jersey dependencies required to be included in the application dependencies may vary in each application based on the Jersey modules that are being used by the application.

Developers using Maven or a Maven-aware build system in their projects are likely to find it easier to include and manage dependencies of their applications compared to developers using ant or other build

systems that are not compatible with Maven. This document will explain to both maven and non-maven developers how to depend on Jersey modules in their application. Ant developers are likely to find the Ant Tasks for Maven [<http://maven.apache.org/ant-tasks/index.html>] very useful.

## 2.3. Common Jersey Use Cases

### 2.3.1. Servlet based application on Glassfish

If you are using Glassfish application server, you don't need to package anything with your application, everything is already included. You just need to declare (provided) dependency on JAX-RS API to be able to compile your application.

```
<dependency>
  <groupId>jakarta.ws.rs</groupId>
  <artifactId>jakarta.ws.rs-api</artifactId>
  <version>4.0.0</version>
  <scope>provided</scope>
</dependency>
```

If you are using any Jersey specific feature, you will need to depend on Jersey directly.

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet</artifactId>
  <version>4.0.0</version>
  <scope>provided</scope>
</dependency>
<!-- if you are using Jersey client specific features without the server side -->
<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-client</artifactId>
  <version>4.0.0</version>
  <scope>provided</scope>
</dependency>
```

### 2.3.2. Servlet based server-side application

Following dependencies apply to application server (servlet containers) without any integrated JAX-RS implementation. Then application needs to include JAX-RS API and Jersey implementation in deployed application.

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet</artifactId>
  <version>4.0.0</version>
</dependency>
<!-- Required only when you are using JAX-RS Client -->
<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-client</artifactId>
  <version>4.0.0</version>
</dependency>
```

### 2.3.3. Client application on JDK

Applications running on plain JDK using only client part of JAX-RS specification need to depend only on client. There are various additional modules which can be added, like for example grizzly or apache or jetty connector (see dependencies snipped below). Jersey client runs by default with plain JDK (using `URLConnection`). See Chapter 5, *Client API*, for more details.

```
<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-client</artifactId>
  <version>4.0.0</version>
</dependency>
```

Currently available connectors:

```
<dependency>
  <groupId>org.glassfish.jersey.connectors</groupId>
  <artifactId>jersey-grizzly-connector</artifactId>
  <version>4.0.0</version>
</dependency>
```

```
<dependency>
  <groupId>org.glassfish.jersey.connectors</groupId>
  <artifactId>jersey-apache5-connector</artifactId>
  <version>4.0.0</version>
</dependency>
```

```
<dependency>
  <groupId>org.glassfish.jersey.connectors</groupId>
  <artifactId>jersey-jetty-connector</artifactId>
  <version>4.0.0</version>
</dependency>
```

### 2.3.4. Server-side application on supported containers

Apart for a standard JAX-RS Servlet-based deployment that works with any Servlet container that supports Servlet 5 and higher, Jersey provides support for programmatic deployment to the following containers: Grizzly 3 (HTTP and Servlet), JDK Http server, Simple Http server and Jetty Http server (requires JDK 11+). This chapter presents only required maven dependencies, more information can be found in Chapter 4, *Application Deployment and Runtime Environments*.

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-grizzly2-http</artifactId>
  <version>4.0.0</version>
</dependency>
```

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-grizzly2-servlet</artifactId>
  <version>4.0.0</version>
</dependency>
```

```
<dependency>
```

```

    <groupId>org.glassfish.jersey.containers</groupId>
    <artifactId>jersey-container-jdk-http</artifactId>
    <version>4.0.0</version>
  </dependency>

  <dependency>
    <groupId>org.glassfish.jersey.containers</groupId>
    <artifactId>jersey-container-jetty-http</artifactId>
    <version>4.0.0</version>
  </dependency>

  <dependency>
    <groupId>org.glassfish.jersey.containers</groupId>
    <artifactId>jersey-container-jetty-servlet</artifactId>
    <version>4.0.0</version>
  </dependency>

```

## 2.4. List of modules

The following chapters provide an overview of all Jersey modules and their dependencies with links to the respective binaries (follow a link on a module name to get complete set of downloadable dependencies).

**Table 2.2. Jersey Core**

| Jersey Core                                                                                                                                                                                                                                         |                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| jersey-client [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/jersey-client/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/jersey-client/dependencies.html</a> ] | Jersey core client implementation |
| jersey-common [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/jersey-common/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/jersey-common/dependencies.html</a> ] | Jersey core common packages       |
| jersey-server [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/jersey-server/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/jersey-server/dependencies.html</a> ] | Jersey core server implementation |

**Table 2.3. Jersey Containers**

| Jersey Containers                                                                                                                 |                           |
|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| jersey-container-grizzly2-http [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/">https://eclipse-ee4j.github.io/</a> ] | Grizzly 2 Http Container. |

| Jersey Containers                                                                                                                                                                                        |                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| project-info/4.0.0/<br>jersey/project/jersey-<br>container-grizzly2-http/<br>dependencies.html]                                                                                                          |                                            |
| jersey-container-<br>grizzly2-servlet<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/jersey-container-<br>grizzly2-servlet/<br>dependencies.html] | Grizzly 2 Servlet Container.               |
| jersey-container-<br>jdk-http [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>container-jdk-http/<br>dependencies.html]                    | JDK Http Container                         |
| jersey-container-<br>jetty-http [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>container-jetty-http/<br>dependencies.html]                | Jetty Http Container (for JDK 17+)         |
| jersey-container-<br>jetty-servlet [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>container-jetty-servlet/<br>dependencies.html]          | Jetty Servlet Container                    |
| jersey-container-<br>netty-http [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>container-netty-http/<br>dependencies.html]                | Netty Http Container.                      |
| jersey-container-<br>servlet [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/                                                                                            | Jersey core Servlet 3.x/5.x implementation |



| Jersey Containers                                                                                                                                                 |                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| jersey/project/jersey-container-servlet/dependencies.html]                                                                                                        |                                                                          |
| jersey-container-servlet-core [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-container-servlet-core/dependencies.html] | Jersey core Servlet 2.x way implementation with Jakarta EE 9 adjustments |
| jersey-gf-ejb [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-gf-ejb/dependencies.html]                         | Jersey EJB for GlassFish integration                                     |

**Table 2.4. Jersey Connectors**

| Jersey Connectors                                                                                                                                       |                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| jersey-apache5-connector [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-apache5-connector/dependencies.html] | Jersey Client Transport via Apache 5      |
| jersey-grizzly-connector [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-grizzly-connector/dependencies.html] | Jersey Client Transport via Grizzly       |
| jersey-helidon-connector [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-helidon-connector/dependencies.html] | Jersey Client Transport via Helidon       |
| jersey-jdk-connector [https://eclipse-ee4j.github.io/jersey.github.io/                                                                                  | Jersey Client Transport via JDK connector |

| Jersey Connectors                                                                                                                                                              |                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| project-info/4.0.0/<br>jersey/project/jersey-<br>jdk-connector/<br>dependencies.html]                                                                                          |                                   |
| jersey-jetty-connector<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>jetty-connector/<br>dependencies.html] | Jersey Client Transport via Jetty |
| jersey-netty-connector<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>netty-connector/<br>dependencies.html] | Jersey Client Transport via Netty |

**Table 2.5. Jersey Media**

| Jersey Media                                                                                                                                                                          |                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| jersey-media-jaxb<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/jersey-media-<br>jaxb/dependencies.html]                      | JAX-RS features based upon JAX-B.                          |
| jersey-media-json-<br>binding [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>media-json-binding/<br>dependencies.html] | Jersey JSON-B support module.                              |
| jersey-media-json-<br>jackson [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>media-json-jackson/<br>dependencies.html] | Jersey JSON Jackson (2.x) entity providers support module. |
| jersey-media-json-<br>jettison [https://<br>eclipse-ee4j.github.io/                                                                                                                   | Jersey JSON Jettison entity providers support module.      |

| Jersey Media                                                                                                                                                                                |                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>media-json-jettison/<br>dependencies.html]                                                                            |                                                                                 |
| jersey-media-json-<br>processing [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>media-json-processing/<br>dependencies.html] | Jersey Jakarta JSON Processing entity providers support proxy module.           |
| jersey-media-kryo<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>jersey-media-kryo/<br>dependencies.html]                        | Jersey/JAX-RS Message Body Writer and Reader using Kryo serialization framework |
| jersey-media-moxy<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>jersey-media-moxy/<br>dependencies.html]                        | Jersey JSON entity providers support module based on EclipseLink MOXy.          |
| jersey-media-<br>multipart [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>media-multipart/<br>dependencies.html]             | Jersey Multipart entity providers support module.                               |
| jersey-media-sse<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/jersey-media-<br>sse/dependencies.html]                              | Jersey Server Sent Events entity providers support module.                      |

**Table 2.6. Jersey Extensions**

| Jersey Extensions                           |                                                                              |
|---------------------------------------------|------------------------------------------------------------------------------|
| jersey-bean-validation<br>[https://eclipse- | Jersey extension module providing support for Bean Validation (JSR-349) API. |

| Jersey Extensions                                                                                                                                                                                                        |                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>bean-validation/<br>dependencies.html]                                                                                          |                                                                   |
| jersey-cdi1x [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>project/jersey-cdi1x/<br>dependencies.html]                                                          | Jersey CDI 1.1 integration                                        |
| jersey-cdi1x-ban-<br>custom-hk2-binding<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/project/<br>jersey-cdi1x-ban-<br>custom-hk2-binding/<br>dependencies.html] | Jersey CDI integration - this module disables custom HK2 bindings |
| jersey-cdi1x-servlet<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/project/<br>jersey-cdi1x-servlet/<br>dependencies.html]                                       | Jersey CDI 1.x Servlet Support                                    |
| jersey-cdi1x-<br>transaction [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/project/jersey-<br>cdi1x-transaction/<br>dependencies.html]                              | Jersey CDI 1.x Transactional Support                              |
| jersey-cdi1x-<br>validation [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/project/jersey-<br>cdi1x-validation/<br>dependencies.html]                                | Jersey CDI 1.x Bean Validation Support                            |
| jersey-declarative-<br>linking [https://<br>eclipse-ee4j.github.io/                                                                                                                                                      | Jersey support for declarative hyperlinking.                      |

| Jersey Extensions                                                                                                                                                                     |                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>declarative-linking/<br>dependencies.html]                                                                      |                                                                                                                            |
| jersey-entity-<br>filtering [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>entity-filtering/<br>dependencies.html]     | Jersey extension module providing support for Entity Data Filtering.                                                       |
| jersey-metainf-<br>services [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>metainf-services/<br>dependencies.html]     | Jersey extension module enabling automatic registration of JAX-RS providers (MBW/ MBR/EM) via META-INF/services mechanism. |
| jersey-micrometer<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>jersey-micrometer/<br>dependencies.html]                  | Jersey extension module providing support for Micrometer.                                                                  |
| jersey-mp-config<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/project/<br>jersey-mp-config/<br>dependencies.html]            | Jersey extension module providing support for MicroProfile Configuration.                                                  |
| jersey-mp-rest-<br>client [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/project/<br>jersey-mp-rest-client/<br>dependencies.html] | Jersey extension module providing support for MicroProfile REST Client.                                                    |
| jersey-mvc [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/                                                                                    | Jersey extension module providing support for MVC.                                                                         |

| Jersey Extensions                                                                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| project/jersey-mvc/dependencies.html]                                                                                                                       |                                                                                    |
| jersey-mvc-bean-validation [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-bean-validation/dependencies.html] | Jersey extension module providing support for Bean Validation in MVC.              |
| jersey-mvc-freemarker [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-freemarker/dependencies.html]           | Jersey extension module providing support for Freemarker templates.                |
| jersey-mvc-jsp [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-jsp/dependencies.html]                         | Jersey extension module providing support for JSP templates.                       |
| jersey-mvc-mustache [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-mustache/dependencies.html]               | Jersey extension module providing support for Mustache templates.                  |
| jersey-mvc-thymeleaf [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-thymeleaf/dependencies.html]             | Jersey extension module providing support for Thymeleaf templates.                 |
| jersey-proxy-client [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-proxy-client/dependencies.html]               | Jersey extension module providing support for (proxy-based) high-level client API. |

| Jersey Extensions                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey-rx-client-guava [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-guava/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-guava/dependencies.html</a> ]       | Jersey Reactive Client - Guava (ListenableFuture) provider.                                                                                                                                        |
| jersey-rx-client-rxjava [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-rxjava/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-rxjava/dependencies.html</a> ]    | Jersey Reactive Client - RxJava (Observable) provider.                                                                                                                                             |
| jersey-rx-client-rxjava2 [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-rxjava2/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-rxjava2/dependencies.html</a> ] | Jersey Reactive Client - RxJava2 (Flowable) provider.                                                                                                                                              |
| jersey-wadl-doclet [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-wadl-doclet/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-wadl-doclet/dependencies.html</a> ]                   | A doclet that generates a resourcedoc xml file: this file contains the javadoc documentation of resource classes, so that this can be used for extending generated wadl with useful documentation. |
| jersey-weld2-se [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-weld2-se/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-weld2-se/dependencies.html</a> ]                            | WELD 2.x SE support                                                                                                                                                                                |

**Table 2.7. Jersey Test Framework**

| Jersey Test Framework                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| container-runner-maven-plugin [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/</a> ] | The container runner maven plugin provides means to start and stop a container (currently Tomcat 10 and Glassfish 6 are supported). To deploy an application to this container or to repetitively redeploy and test an application in the container. |

| Jersey Test Framework                                                                                                                                                                         |                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| container-runner-maven-plugin/dependencies.html]                                                                                                                                              |                                            |
| custom-enforcer-rules [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/custom-enforcer-rules/dependencies.html]                                     | Jersey test framework Maven projects       |
| jersey-test-framework-core [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-test-framework-core/dependencies.html]                                   | Jersey Test Framework Core                 |
| jersey-test-framework-provider-bundle [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-bundle/dependencies.html]     | Jersey Test Framework Providers Bundle     |
| jersey-test-framework-provider-external [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-external/dependencies.html] | Jersey Test Framework - External container |
| jersey-test-framework-provider-grizzly2 [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-grizzly2/dependencies.html] | Jersey Test Framework - Grizzly2 container |



| <b>Jersey Test Framework</b>                                                                                                                                                                                                                                                                                                                                         |                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| jersey-test-framework-provider-inmemory<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-inmemory/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-inmemory/dependencies.html</a> ] | Jersey Test Framework - InMemory container    |
| jersey-test-framework-provider-jdk-http<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-jdk-http/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-jdk-http/dependencies.html</a> ] | Jersey Test Framework - JDK HTTP container    |
| jersey-test-framework-provider-jetty [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-jetty/dependencies.html]                                                                                                                                                                              | Jersey Test Framework - Jetty HTTP container  |
| jersey-test-framework-provider-netty [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-netty/dependencies.html]                                                                                                                                                                              | Jersey Test Framework - Netty container       |
| jersey-test-framework-provider-simple<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-simple/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-simple/dependencies.html</a> ]       | Jersey Test Framework - Simple HTTP container |
| jersey-test-framework-util [https://eclipse-                                                                                                                                                                                                                                                                                                                         | Jersey Test Framework Utils                   |

| Jersey Test Framework                                                                                                                                                     |                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/jersey-<br>test-framework-util/<br>dependencies.html]                                       |                                        |
| memleak-test-<br>common [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>memleak-test-common/<br>dependencies.html] | Jersey test framework umbrella project |

**Table 2.8. Jersey Test Framework Providers**

| Jersey Test Framework Providers                                                                                                                                                                                                  |                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| jersey-test-framework-<br>provider-bundle<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/project/<br>jersey-test-framework-<br>provider-bundle/<br>dependencies.html]     | Jersey Test Framework Providers Bundle     |
| jersey-test-framework-<br>provider-external<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/project/<br>jersey-test-framework-<br>provider-external/<br>dependencies.html] | Jersey Test Framework - External container |
| jersey-test-framework-<br>provider-grizzly2<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/project/<br>jersey-test-framework-<br>provider-grizzly2/<br>dependencies.html] | Jersey Test Framework - Grizzly2 container |
| jersey-test-framework-<br>provider-inmemory                                                                                                                                                                                      | Jersey Test Framework - InMemory container |

| Jersey Test Framework Providers                                                                                                                                                                                                                                                                                                                                      |                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-inmemory/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-inmemory/dependencies.html</a> ]                                            |                                                            |
| jersey-test-framework-provider-jdk-http<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-jdk-http/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-jdk-http/dependencies.html</a> ] | Jersey Test Framework - JDK HTTP container                 |
| jersey-test-framework-provider-jetty [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-jetty/dependencies.html]                                                                                                                                                                              | Jersey Test Framework - Jetty HTTP container (for JDK 11+) |
| jersey-test-framework-provider-netty [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-netty/dependencies.html]                                                                                                                                                                              | Jersey Test Framework - Netty container                    |
| jersey-test-framework-provider-simple<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-simple/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-test-framework-provider-simple/dependencies.html</a> ]       | Jersey Test Framework - Simple HTTP container              |

**Table 2.9. Jersey Glassfish Bundles**

| Jersey Glassfish Bundles                                                                                                                  |                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| jersey-gf-ejb [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-gf-ejb/dependencies.html] | Jersey EJB for GlassFish integration |

**Table 2.10. Security**

| Security                                                                                                                                |                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| oauth1-client [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/oauth1-client/dependencies.html]       | Module that adds an OAuth 1 support to Jersey client. |
| oauth1-server [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/oauth1-server/dependencies.html]       | Module that adds an OAuth 1 support to Jersey server  |
| oauth1-signature [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/oauth1-signature/dependencies.html] | OAuth1 signature module                               |
| oauth2-client [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/oauth2-client/dependencies.html]       | Module that adds an OAuth 2 support to Jersey client  |

**Table 2.11. Jersey Examples**

| Jersey Examples                                                                                                             |                                        |
|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| assemblies [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/assemblies/dependencies.html] | Jersey examples shared assembly types. |
| bookstore-webapp [https://eclipse-                                                                                          | Jersey MVC Bookstore example.          |

| Jersey Examples                                                                                                                                                                 |                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/webapp-<br>example-parent/<br>bookstore-webapp/<br>dependencies.html]                             |                                                     |
| cdi-webapp [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>webapp-example-<br>parent/cdi-webapp/<br>dependencies.html]   | Jersey CDI example.                                 |
| clipboard [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/clipboard/<br>dependencies.html]                                   | Jersey clipboard example.                           |
| clipboard-<br>programmatic [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/clipboard-<br>programmatic/<br>dependencies.html] | Jersey programmatic resource API clipboard example. |
| declarative-linking<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>declarative-linking/<br>dependencies.html]        | Declarative Hyperlinking - Jersey Sample            |
| entity-filtering [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/entity-filtering/<br>dependencies.html]                     | Jersey Entity Data Filtering Example.               |
| entity-filtering-<br>security [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/entity-                                        | Jersey Entity Data Filtering Security Example.      |

| Jersey Examples                                                                                                                                                 |                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| filtering-security/dependencies.html]                                                                                                                           |                                                        |
| entity-filtering-selectable [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/entity-filtering-selectable/dependencies.html]   | Jersey Entity Data Filtering Selectable Example.       |
| exception-mapping [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/exception-mapping/dependencies.html]                       | Jersey example showing exception mappers in action.    |
| freemarker-webapp [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/freemarker-webapp/dependencies.html] | Jersey Freemarker example.                             |
| groovy [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/groovy/dependencies.html]                                             | Groovy Jersey                                          |
| helloworld [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/helloworld/dependencies.html]                                     | Jersey annotated resource class "Hello world" example. |
| helloworld-benchmark [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/helloworld-benchmark/dependencies.html]                 | Jersey "Hello World" benchmark example.                |
| helloworld-cdi2-se [https://eclipse-                                                                                                                            | Jersey "Hello world" example with CDI 2 SE.            |

| Jersey Examples                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/helloworld-cdi2-<br>se/dependencies.html]                                                                        |                                                                                                                                                                                                                                                                                                                         |
| helloworld-netty<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>helloworld-netty/<br>dependencies.html]                             | Jersey "Hello world" example on Netty container.                                                                                                                                                                                                                                                                        |
| helloworld-<br>programmatic [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/helloworld-<br>programmatic/<br>dependencies.html]              | Jersey programmatic resource API "Hello world" example.                                                                                                                                                                                                                                                                 |
| helloworld-pure-<br>jax-rs [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>helloworld-pure-jax-rs/<br>dependencies.html]                | Example using only the standard JAX-RS API's and the lightweight HTTP server<br>bundled in JDK.                                                                                                                                                                                                                         |
| helloworld-webapp<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/webapp-<br>example-parent/<br>helloworld-webapp/<br>dependencies.html] | Jersey annotated resource class "Hello world" example.                                                                                                                                                                                                                                                                  |
| helloworld-weld<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>helloworld-weld/<br>dependencies.html]                               | Jersey annotated resource class "Hello world" example with Weld support.                                                                                                                                                                                                                                                |
| http-patch [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/                                                                                             | Jersey example for implementing generic PATCH support via JAX-RS reader interceptor.<br>Taken from Gerard Davison's blog entry: <a href="http://kingsfleet.blogspot.co.uk/2014/02/transparent-patch-support-in-jax-rs-20.html">http://kingsfleet.blogspot.co.uk/2014/02/transparent-patch-support-in-jax-rs-20.html</a> |

| Jersey Examples                                                                                                                                                           |                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| project/http-patch/dependencies.html]                                                                                                                                     |                                                       |
| http-trace [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/http-trace/dependencies.html]                                               | Jersey HTTP TRACE support example.                    |
| https-clientserver-grizzly [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/https-clientserver-grizzly/dependencies.html]               | Jersey HTTPS Client/Server example on Grizzly.        |
| https-server-glassfish [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/https-server-glassfish/dependencies.html] | Jersey HTTPS server on GlassFish example.             |
| java8-webapp [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/java8-webapp/dependencies.html]                     | Java 8 Types WebApp Example.                          |
| jaxb [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jaxb/dependencies.html]                                                           | Jersey JAXB example.                                  |
| jaxrs-types-injection [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jaxrs-types-injection/dependencies.html]                         | Jersey JAX-RS types injection example.                |
| jersey-ejb [https://eclipse-ee4j.github.io/                                                                                                                               | Jersey Web Application (Servlet) examples parent POM. |



| Jersey Examples                                                                                                                                                                                           |                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/webapp-<br>example-parent/jersey-<br>ejb/dependencies.html]                                                                                    |                                         |
| json-binding-<br>webapp [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/json-<br>binding-webapp/<br>dependencies.html]                                 | Jersey JSON Binding example.            |
| json-jackson [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/json-jackson/<br>dependencies.html]                                                       | Jersey JSON with Jackson example.       |
| json-jettison [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/json-jettison/<br>dependencies.html]                                                     | Jersey JSON with Jettison JAXB example. |
| json-moxy [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/json-moxy/<br>dependencies.html]                                                             | Jersey JSON with MOXy example.          |
| json-processing-<br>webapp [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/webapp-<br>example-parent/json-<br>processing-webapp/<br>dependencies.html] | Jersey Jakarta JSON Processing example. |
| json-with-padding<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>json-with-padding/<br>dependencies.html]                                      | Jersey JSON with Padding example.       |

| Jersey Examples                                                                                                                                                                                                                                                                                                                                              |                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| managed-beans-webapp [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/managed-beans-webapp/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/managed-beans-webapp/dependencies.html</a> ]                         | Jersey Managed Beans Web Application Example.         |
| managed-client [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/managed-client/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/managed-client/dependencies.html</a> ]                                                                                       | Jersey managed client example.                        |
| managed-client-simple-webapp [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/managed-client-simple-webapp/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/managed-client-simple-webapp/dependencies.html</a> ] | Jersey Web Application (Servlet) examples parent POM. |
| managed-client-webapp [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/managed-client-webapp/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/managed-client-webapp/dependencies.html</a> ]                      | Jersey managed client web application example.        |
| multipart-webapp [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/multipart-webapp/dependencies.html">https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/multipart-webapp/dependencies.html</a> ]                                     | Jersey Multipart example.                             |
| open-tracing [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/">https://eclipse-ee4j.github.io/jersey/</a> ]                                                                                                                                                                                                             | Jersey OpenTracing example                            |

| Jersey Examples                                                                                                                                                                             |                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| project/open-tracing/dependencies.html]                                                                                                                                                     |                                                                                       |
| rx-client-webapp [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/webapp-example-parent/rx-client-webapp/dependencies.html]                               | Jersey Reactive Client WebApp Example.                                                |
| server-async [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/server-async/dependencies.html]                                                             | Jersey JAX-RS asynchronous server-side example.                                       |
| server-async-managed [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/server-async-managed/dependencies.html]                                             | Jersey JAX-RS asynchronous server-side example with custom Jersey executor providers. |
| server-async-standalone [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/server-async-standalone/dependencies.html]                                       | Standalone Jersey JAX-RS asynchronous server-side processing example.                 |
| server-async-standalone-client [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/server-async-standalone/server-async-standalone-client/dependencies.html] | Standalone Jersey JAX-RS asynchronous server-side processing example client.          |
| server-async-standalone-webapp [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/server-                                                                   | Standalone Jersey JAX-RS asynchronous server-side processing example web application. |

| Jersey Examples                                                                                                                                                                                                         |                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| async-standalone/<br>server-async-<br>standalone-webapp/<br>dependencies.html]                                                                                                                                          |                                                                         |
| server-sent-events-<br>jaxrs [https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/server-<br>sent-events-jaxrs/<br>dependencies.html]                                     | Jersey JAX-RS 2.1/3.0 Server-Sent Events example.                       |
| server-sent-events-<br>jersey [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/server-<br>sent-events-jersey/<br>dependencies.html]                                   | Jersey Server-Sent Events example.                                      |
| servlet3-webapp<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/webapp-<br>example-parent/<br>servlet3-webapp/<br>dependencies.html]                              | Jersey Servlet 3 example with missing servlet-class in the web.xml file |
| simple-console<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/simple-console/<br>dependencies.html]                                                              | Jersey Simple Console example                                           |
| sse-item-store-jaxrs-<br>webapp [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/<br>webapp-example-<br>parent/sse-item-<br>store-jaxrs-webapp/<br>dependencies.html] | Jersey JAX-RS 2.1/3.0 SSE API-based item store example.                 |
| sse-item-store-jersey-<br>webapp [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/                                                                                                                              | Jersey SSE API-based item store example.                                |

| Jersey Examples                                                                                                                                                                       |                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| project-info/4.0.0/<br>jersey/project/<br>webapp-example-<br>parent/sse-item-<br>store-jersey-webapp/<br>dependencies.html]                                                           |                                                       |
| sse-twitter-aggregator<br>[https://eclipse-<br>ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/sse-<br>twitter-aggregator/<br>dependencies.html]        | Jersey SSE Twitter Message Aggregator Example.        |
| system-properties-<br>example [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/system-<br>properties-example/<br>dependencies.html] | Jersey system properties example.                     |
| webapp-example-<br>parent [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/<br>project-info/4.0.0/<br>jersey/project/webapp-<br>example-parent/<br>dependencies.html]         | Jersey Web Application (Servlet) examples parent POM. |
| xml-moxy [https://<br>eclipse-ee4j.github.io/<br>jersey.github.io/project-<br>info/4.0.0/jersey/<br>project/xml-moxy/<br>dependencies.html]                                           | Jersey XML MOXy example.                              |

---

# Chapter 3. JAX-RS Application, Resources and Sub-Resources

This chapter presents an overview of the core JAX-RS concepts - resources and sub-resources.

The JAX-RS 4.0.0 JavaDoc can be found online here [<https://github.com/eclipse-ee4j/jaxrs-api/apidocs/index.html>].

The JAX-RS 4.0.0 specification draft can be found online here [<https://jakarta.ee/specifications/restful-ws/3.0/jakarta-restful-ws-spec-3.0.pdf>].

## 3.1. Root Resource Classes

*Root resource classes* are POJOs (Plain Old Java Objects) that are annotated with `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] have at least one method annotated with `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] or a resource method designator annotation such as `@GET` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html>], `@PUT` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PUT.html>], `@POST` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/POST.html>], `@DELETE` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/DELETE.html>]. Resource methods are methods of a resource class annotated with a resource method designator. This section shows how to use Jersey to annotate Java objects to create RESTful web services.

The following code example is a very simple example of a root resource class using JAX-RS annotations. The example code shown here is from one of the samples that ships with Jersey, the zip file of which can be found in the maven repository here [<https://repo1.maven.org/maven2/org/glassfish/jersey/examples/helloworld/4.0.0/>].

### Example 3.1. Simple hello world root resource class

```
1 package org.glassfish.jersey.examples.helloworld;
2
3 import jakarta.ws.rs.GET;
4 import jakarta.ws.rs.Path;
5 import jakarta.ws.rs.Produces;
6
7 @Path("helloworld")
8 public class HelloWorldResource {
9     public static final String CLICHED_MESSAGE = "Hello World!";
10
11     @GET
12     @Produces("text/plain")
13     public String getHello() {
14         return CLICHED_MESSAGE;
15     }
16 }
```

Let's look at some of the JAX-RS annotations used in this example.

### 3.1.1. @Path

The `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] annotation's value is a relative URI path. In the example above, the Java class will be hosted at the URI path `/helloWorld`. This is an extremely simple use of the `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] annotation. What makes JAX-RS so useful is that you can embed variables in the URIs.

*URI path templates* are URIs with variables embedded within the URI syntax. These variables are substituted at runtime in order for a resource to respond to a request based on the substituted URI. Variables are denoted by curly braces. For example, look at the following `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] annotation:

```
@Path("/users/{username}")
```

In this type of example, a user will be prompted to enter their name, and then a Jersey web service configured to respond to requests to this URI path template will respond. For example, if the user entered their username as "Galileo", the web service will respond to the following URL: `http://example.com/users/Galileo`

To obtain the value of the username variable the `@PathParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PathParam.html>] may be used on method parameter of a request method, for example:

#### Example 3.2. Specifying URI path parameter

```
1 @Path("/users/{username}")
2 public class UserResource {
3
4     @GET
5     @Produces("text/xml")
6     public String getUser(@PathParam("username") String userName) {
7         ...
8     }
9 }
```

If it is required that a user name must only consist of lower and upper case numeric characters then it is possible to declare a particular regular expression, which overrides the default regular expression, `"[/]+"`, for example:

```
@Path("users/{username: [a-zA-Z][a-zA-Z_0-9]*}")
```

In this type of example the username variable will only match user names that begin with one upper or lower case letter and zero or more alpha numeric characters and the underscore character. If a user name does not match that a 404 (Not Found) response will occur.

A `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] value may or may not begin with a `'/'`, it makes no difference. Likewise, by default, a `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] value may or may not end in a `'/'`, it makes no difference, and thus request URLs that end or do not end in a `'/'` will both be matched.

### 3.1.2. @GET, @PUT, @POST, @DELETE, ... (HTTP Methods)

`@GET` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html>], `@PUT` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PUT.html>], `@POST` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/POST.html>],

`rest/apidocs/4.0.0/jakarta/ws/rs/POST.html]`, `@DELETE` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/DELETE.html>] and `@HEAD` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/HEAD.html>] are *resource method designator* annotations defined by JAX-RS and which correspond to the similarly named HTTP methods. In the example above, the annotated Java method will process HTTP GET requests. The behavior of a resource is determined by which of the HTTP methods the resource is responding to.

The following example is an extract from the storage service sample that shows the use of the PUT method to create or update a storage container:

### Example 3.3. PUT method

```
1 @PUT
2 public Response putContainer() {
3     System.out.println("PUT CONTAINER " + container);
4
5     URI uri = uriInfo.getAbsolutePath();
6     Container c = new Container(container, uri.toString());
7
8     Response r;
9     if (!MemoryStore.MS.hasContainer(c)) {
10         r = Response.created(uri).build();
11     } else {
12         r = Response.noContent().build();
13     }
14
15     MemoryStore.MS.createContainer(c);
16     return r;
17 }
```

By default the JAX-RS runtime will automatically support the methods HEAD and OPTIONS, if not explicitly implemented. For HEAD the runtime will invoke the implemented GET method (if present) and ignore the response entity (if set). A response returned for the OPTIONS method depends on the requested media type defined in the 'Accept' header. The OPTIONS method can return a response with a set of supported resource methods in the 'Allow' header or return a WADL [<https://javaee.github.io/wadl/>] document. See wadl section for more information.

## 3.1.3. @Produces

The `@Produces` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html>] annotation is used to specify the MIME media types of representations a resource can produce and send back to the client. In this example, the Java method will produce representations identified by the MIME media type "text/plain". `@Produces` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html>] can be applied at both the class and method levels. Here's an example:

### Example 3.4. Specifying output MIME type

```
1 @Path("/myResource")
2 @Produces("text/plain")
3 public class SomeResource {
4     @GET
5     public String doGetAsPlainText() {
6         ...
7     }
```



```
8
9     @GET
10    @Produces("text/html")
11    public String doGetAsHtml() {
12        ...
13    }
14 }
```

The `doGetAsPlainText` method defaults to the MIME type of the `@Produces` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html] annotation at the class level. The `doGetAsHtml` method's `@Produces` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html] annotation overrides the class-level `@Produces` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html] setting, and specifies that the method can produce HTML rather than plain text.

If a resource class is capable of producing more than one MIME media type then the resource method chosen will correspond to the most acceptable media type as declared by the client. More specifically the Accept header of the HTTP request declares what is most acceptable. For example if the Accept header is "Accept: text/plain" then the `doGetAsPlainText` method will be invoked. Alternatively if the Accept header is "Accept: text/plain;q=0.9, text/html", which declares that the client can accept media types of "text/plain" and "text/html" but prefers the latter, then the `doGetAsHtml` method will be invoked.

More than one media type may be declared in the same `@Produces` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html] declaration, for example:

### Example 3.5. Using multiple output MIME types

```
1 @GET
2 @Produces({"application/xml", "application/json"})
3 public String doGetAsXmlOrJson() {
4     ...
5 }
```

The `doGetAsXmlOrJson` method will get invoked if either of the media types "application/xml" and "application/json" are acceptable. If both are equally acceptable then the former will be chosen because it occurs first.

Optionally, server can also specify the quality factor for individual media types. These are considered if several are equally acceptable by the client. For example:

### Example 3.6. Server-side content negotiation

```
1 @GET
2 @Produces({"application/xml; qs=0.9", "application/json"})
3 public String doGetAsXmlOrJson() {
4     ...
5 }
```

In the above sample, if client accepts both "application/xml" and "application/json" (equally), then a server always sends "application/json", since "application/xml" has a lower quality factor.

The examples above refer explicitly to MIME media types for clarity. It is possible to refer to constant values, which may reduce typographical errors, see the constant field values of `MediaType` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/MediaType.html].

### 3.1.4. @Consumes

The `@Consumes` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Consumes.html>] annotation is used to specify the MIME media types of representations that can be consumed by a resource. The above example can be modified to set the cliched message as follows:

#### Example 3.7. Specifying input MIME type

```
1 @POST
2 @Consumes("text/plain")
3 public void postClichedMessage(String message) {
4     // Store the message
5 }
```

In this example, the Java method will consume representations identified by the MIME media type "text/plain". Notice that the resource method returns void. This means no representation is returned and response with a status code of 204 (No Content) will be returned to the client.

`@Consumes` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Consumes.html>] can be applied at both the class and the method levels and more than one media type may be declared in the same `@Consumes` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Consumes.html>] declaration.

## 3.2. Parameter Annotations (@\*Param)

Parameters of a resource method may be annotated with parameter-based annotations to extract information from a request. One of the previous examples presented the use of `@PathParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PathParam.html>] to extract a path parameter from the path component of the request URL that matched the path declared in `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>].

`@QueryParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/QueryParam.html>] is used to extract query parameters from the Query component of the request URL. The following example is an extract from the sparklines sample:

#### Example 3.8. Query parameters

```
1 @Path("smooth")
2 @GET
3 public Response smooth(
4     @DefaultValue("2") @QueryParam("step") int step,
5     @DefaultValue("true") @QueryParam("min-m") boolean hasMin,
6     @DefaultValue("true") @QueryParam("max-m") boolean hasMax,
7     @DefaultValue("true") @QueryParam("last-m") boolean hasLast,
8     @DefaultValue("blue") @QueryParam("min-color") ColorParam minColor,
9     @DefaultValue("green") @QueryParam("max-color") ColorParam maxColor,
10    @DefaultValue("red") @QueryParam("last-color") ColorParam lastColor) {
11    ...
12 }
```

If a query parameter "step" exists in the query component of the request URI then the "step" value will be extracted and parsed as a 32 bit signed integer and assigned to the step method parameter. If "step" does not exist then a default value of 2, as declared in the `@DefaultValue` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/DefaultValue.html>] annotation, will be assigned to the step method parameter.

If the "step" value cannot be parsed as a 32 bit signed integer then a HTTP 404 (Not Found) response is returned. User defined Java types such as `ColorParam` may be used, which as implemented as follows:

### Example 3.9. Custom Java type for consuming request parameters

```
1 public class ColorParam extends Color {
2
3     public ColorParam(String s) {
4         super(getRGB(s));
5     }
6
7     private static int getRGB(String s) {
8         if (s.charAt(0) == '#') {
9             try {
10                 Color c = Color.decode("0x" + s.substring(1));
11                 return c.getRGB();
12             } catch (NumberFormatException e) {
13                 throw new WebApplicationException(400);
14             }
15         } else {
16             try {
17                 Field f = Color.class.getField(s);
18                 return ((Color)f.get(null)).getRGB();
19             } catch (Exception e) {
20                 throw new WebApplicationException(400);
21             }
22         }
23     }
24 }
```

In general the Java type of the method parameter may:

1. Be a primitive type;
2. Have a constructor that accepts a single `String` argument;
3. Have a static method named `valueOf` or `fromString` that accepts a single `String` argument (see, for example, `Integer.valueOf(String)` and `java.util.UUID.fromString(String)`);
4. Have a registered implementation of `jakarta.ws.rs.ext.ParamConverterProvider` JAX-RS extension SPI that returns a `jakarta.ws.rs.ext.ParamConverter` instance capable of a "from string" conversion for the type. or
5. Be `List<T>`, `Set<T>` or `SortedSet<T>`, where `T` satisfies 2 or 3 above. The resulting collection is read-only.

Sometimes parameters may contain more than one value for the same name. If this is the case then types in 5) may be used to obtain all values.

If the `@DefaultValue` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/DefaultValue.html>] is not used in conjunction with `@QueryParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/QueryParam.html>] and the query parameter is not present in the request then value will be an empty collection for `List`, `Set` or `SortedSet`, null for other object types, and the Java-defined default for primitive types.

The `@PathParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PathParam.html>] and the other parameter-based annotations, `@MatrixParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/MatrixParam.html>], `@HeaderParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/HeaderParam.html>], `@CookieParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/CookieParam.html>], `@FormParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/FormParam.html>] obey the same rules as `@QueryParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/QueryParam.html>]. `@MatrixParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/MatrixParam.html>] extracts information from URL path segments. `@HeaderParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/HeaderParam.html>] extracts information from the HTTP headers. `@CookieParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/CookieParam.html>] extracts information from the cookies declared in cookie related HTTP headers.

`@FormParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/FormParam.html>] is slightly special because it extracts information from a request representation that is of the MIME media type `"application/x-www-form-urlencoded"` and conforms to the encoding specified by HTML forms, as described here. This parameter is very useful for extracting information that is POSTed by HTML forms, for example the following extracts the form parameter named `"name"` from the POSTed form data:

### Example 3.10. Processing POSTed HTML form

```
1 @POST
2 @Consumes("application/x-www-form-urlencoded")
3 public void post(@FormParam("name") String name) {
4     // Store the message
5 }
```

If it is necessary to obtain a general map of parameter name to values then, for query and path parameters it is possible to do the following:

### Example 3.11. Obtaining general map of URI path and/or query parameters

```
1 @GET
2 public String get(@Context UriInfo ui) {
3     MultivaluedMap<String, String> queryParams = ui.getQueryParameters();
4     MultivaluedMap<String, String> pathParams = ui.getPathParameters();
5 }
```

For header and cookie parameters the following:

### Example 3.12. Obtaining general map of header parameters

```
1 @GET
2 public String get(@Context HttpHeaders hh) {
3     MultivaluedMap<String, String> headerParams = hh.getRequestHeaders();
4     Map<String, Cookie> pathParams = hh.getCookies();
5 }
```

For quicker getting of values for a known header name there is a shortcut for `hh.getRequestHeaders().get(name)` which is `hh.getRequestHeader(name)`.

In general `@Context` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>] can be used to obtain contextual Java types related to the request or response.

Because form parameters (unlike others) are part of the message entity, it is possible to do the following:

### Example 3.13. Obtaining general map of form parameters

```
1 @POST
2 @Consumes("application/x-www-form-urlencoded")
3 public void post(MultivaluedMap<String, String> formParams) {
4     // Store the message
5 }
```

I.e. you don't need to use the `@Context` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>] annotation.

Another kind of injection is the `@BeanParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/BeanParam.html>] which allows to inject the parameters described above into a single bean. A bean annotated with `@BeanParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/BeanParam.html>] containing any fields and appropriate `*param` annotation (like `@PathParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PathParam.html>]) will be initialized with corresponding request values in expected way as if these fields were in the resource class. Then instead of injecting request values like path param into a constructor parameters or class fields the `@BeanParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/BeanParam.html>] can be used to inject such a bean into a resource or resource method. The `@BeanParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/BeanParam.html>] is used this way to aggregate more request parameters into a single bean.

### Example 3.14. Example of the bean which will be used as `@BeanParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/BeanParam.html>]

```
1 public class MyBeanParam {
2     @PathParam("p")
3     private String pathParam;
4
5     @MatrixParam("m")
6     @Encoded
7     @DefaultValue("default")
8     private String matrixParam;
9
10    @HeaderParam("header")
11    private String headerParam;
12
13    private String queryParams;
14
15    public MyBeanParam(@QueryParam("q") String queryParams) {
16        this.queryParam = queryParams;
17    }
18
19    public String getPathParam() {
20        return pathParam;
21    }
22    ...
23 }
```

### Example 3.15. Injection of `MyBeanParam` as a method parameter:

```
1 @POST
2 public void post(@BeanParam MyBeanParam beanParam, String entity) {
3     final String pathParam = beanParam.getPathParam(); // contains injected pa
```

```
4      ...
5  }
```

The example shows aggregation of injections `@PathParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PathParam.html], `@QueryParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/QueryParam.html], `@MatrixParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/MatrixParam.html] and `@HeaderParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/HeaderParam.html] into one single bean. The rules for injections inside the bean are the same as described above for these injections. The `@DefaultValue` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/DefaultValue.html] is used to define the default value for matrix parameter `matrixParam`. Also the `@Encoded` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Encoded.html] annotation has the same behaviour as if it were used for injection in the resource method directly. Injecting the bean parameter into `@Singleton` resource class fields is not allowed (injections into method parameter must be used instead).

`@BeanParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/BeanParam.html] can contain all parameters injections (`@PathParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/PathParam.html], `@QueryParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/QueryParam.html], `@MatrixParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/MatrixParam.html], `@HeaderParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/HeaderParam.html], `@CookieParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/CookieParam.html], `@FormParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/FormParam.html]). More beans can be injected into one resource or method parameters even if they inject the same request values. For example the following is possible:

### Example 3.16. Injection of more beans into one resource methods:

```
1 @POST
2 public void post(@BeanParam MyBeanParam beanParam, @BeanParam AnotherBean anot
3 String entity) {
4     // beanParam.getPathParam() == pathParam
5     ...
6 }
```

## 3.3. Sub-resources

`@Path` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html] may be used on classes and such classes are referred to as root resource classes. `@Path` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html] may also be used on methods of root resource classes. This enables common functionality for a number of resources to be grouped together and potentially reused.

The first way `@Path` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html] may be used is on resource methods and such methods are referred to as *sub-resource methods*. The following example shows the method signatures for a root resource class from the jmaki-backend sample:

### Example 3.17. Sub-resource methods

```
1 @Singleton
2 @Path("/printers")
3 public class PrintersResource {
4
5     @GET
6     @Produces({"application/json", "application/xml"})
7     public WebResourceList getMyResources() { ... }
```

```
8
9  @GET @Path("/list")
10 @Produces({"application/json", "application/xml"})
11 public WebResourceList getListOfPrinters() { ... }
12
13 @GET @Path("/jMakiTable")
14 @Produces("application/json")
15 public PrinterTableModel getTable() { ... }
16
17 @GET @Path("/jMakiTree")
18 @Produces("application/json")
19 public TreeModel getTree() { ... }
20
21 @GET @Path("/ids/{printerid}")
22 @Produces({"application/json", "application/xml"})
23 public Printer getPrinter(@PathParam("printerid") String printerId) { ... }
24
25 @PUT @Path("/ids/{printerid}")
26 @Consumes({"application/json", "application/xml"})
27 public void putPrinter(@PathParam("printerid") String printerId, Printer p) { ... }
28
29 @DELETE @Path("/ids/{printerid}")
30 public void deletePrinter(@PathParam("printerid") String printerId) { ... }
31 }
```

If the path of the request URL is "printers" then the resource methods not annotated with `@Path` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html] will be selected. If the request path of the request URL is "printers/list" then first the root resource class will be matched and then the sub-resource methods that match "list" will be selected, which in this case is the sub-resource method `getListOfPrinters`. So, in this example hierarchical matching on the path of the request URL is performed.

The second way `@Path` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html] may be used is on methods **not** annotated with resource method designators such as `@GET` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html] or `@POST` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/POST.html]. Such methods are referred to as *sub-resource locators*. The following example shows the method signatures for a root resource class and a resource class from the optimistic-concurrency sample:

### Example 3.18. Sub-resource locators

```
1 @Path("/item")
2 public class ItemResource {
3     @Context UriInfo uriInfo;
4
5     @Path("content")
6     public ItemContentResource getItemContentResource() {
7         return new ItemContentResource();
8     }
9
10    @GET
11    @Produces("application/xml")
12    public Item get() { ... }
13 }
```

```
14 }
15
16 public class ItemContentResource {
17
18     @GET
19     public Response get() { ... }
20
21     @PUT
22     @Path("{version}")
23     public void put(@PathParam("version") int version,
24                   @Context HttpHeaders headers,
25                   byte[] in) {
26         ...
27     }
28 }
```

The root resource class `ItemResource` contains the sub-resource locator method `getItemContentResource` that returns a new resource class. If the path of the request URL is "item/content" then first of all the root resource will be matched, then the sub-resource locator will be matched and invoked, which returns an instance of the `ItemContentResource` resource class. Sub-resource locators enable reuse of resource classes. A method can be annotated with the `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] annotation with empty path (`@Path("/")` or `@Path("")`) which means that the sub resource locator is matched for the path of the enclosing resource (without sub-resource path).

### Example 3.19. Sub-resource locators with empty path

```
1 @Path("/item")
2 public class ItemResource {
3
4     @Path("/")
5     public ItemContentResource getItemContentResource() {
6         return new ItemContentResource();
7     }
8 }
```

In the example above the sub-resource locator method `getItemContentResource` is matched for example for request path `/item/locator` or even for only `/item`.

In addition the processing of resource classes returned by sub-resource locators is performed at runtime thus it is possible to support polymorphism. A sub-resource locator may return different sub-types depending on the request (for example a sub-resource locator could return different sub-types dependent on the role of the principle that is authenticated). So for example the following sub resource locator is valid:

### Example 3.20. Sub-resource locators returning sub-type

```
1 @Path("/item")
2 public class ItemResource {
3
4     @Path("/")
5     public Object getItemContentResource() {
6         return new AnyResource();
7     }
8 }
```



Note that the runtime will not manage the life-cycle or perform any field injection onto instances returned from sub-resource locator methods. This is because the runtime does not know what the life-cycle of the instance is. If it is required that the runtime manages the sub-resources as standard resources the `Class` should be returned as shown in the following example:

### Example 3.21. Sub-resource locators created from classes

```
1 import jakarta.inject.Singleton;
2
3 @Path("/item")
4 public class ItemResource {
5     @Path("content")
6     public Class<ItemContentSingletonResource> getItemContentResource() {
7         return ItemContentSingletonResource.class;
8     }
9 }
10
11 @Singleton
12 public class ItemContentSingletonResource {
13     // this class is managed in the singleton life cycle
14 }
```

JAX-RS resources are managed in per-request scope by default which means that new resource is created for each request. In this example the `jakarta.inject.Singleton` annotation says that the resource will be managed as singleton and not in request scope. The sub-resource locator method returns a class which means that the runtime will managed the resource instance and its life-cycle. If the method would return instance instead, the `Singleton` annotation would have no effect and the returned instance would be used.

The sub resource locator can also return a *programmatic resource model*. See resource builder section for information of how the programmatic resource model is constructed. The following example shows very simple resource returned from the sub-resource locator method.

### Example 3.22. Sub-resource locators returning resource model

```
1 import org.glassfish.jersey.server.model.Resource;
2
3 @Path("/item")
4 public class ItemResource {
5
6     @Path("content")
7     public Resource getItemContentResource() {
8         return Resource.from(ItemContentSingletonResource.class);
9     }
10 }
```

The code above has exactly the same effect as previous example. `Resource` is a resource simple resource constructed from `ItemContentSingletonResource`. More complex programmatic resource can be returned as long they are valid resources.

## 3.4. Life-cycle of Root Resource Classes

By default the life-cycle of root resource classes is per-request which, namely that a new instance of a root resource class is created every time the request URI path matches the root resource. This makes for a

very natural programming model where constructors and fields can be utilized (as in the previous section showing the constructor of the `SparklinesResource` class) without concern for multiple concurrent requests to the same resource.

In general this is unlikely to be a cause of performance issues. Class construction and garbage collection of JVMs has vastly improved over the years and many objects will be created and discarded to serve and process the HTTP request and return the HTTP response.

Instances of singleton root resource classes can be declared by an instance of `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>].

Jersey supports two further life-cycles using Jersey specific annotations.

**Table 3.1. Resource scopes**

| Scope            | Annotation                            | Annotation full class name                                   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------|---------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request scope    | <code>@RequestScoped</code> (or none) | <code>org.glassfish.jersey.provider.sse.RequestScoped</code> | <b>Default if no annotation is present.</b><br>In this scope the resource instance is created for each new request and used for processing of this request. If the resource is used more than one time in the request processing, always the same instance will be used. This can happen when a resource is a sub resource and is returned more times during the matching. In this situation only one instance will serve the requests.                                                                                                                                                                                                                                                                   |
| Per-lookup scope | <code>@PerLookup</code>               | <code>org.glassfish.hk2.api.PerLookup</code>                 | In this scope the resource instance is created every time it is needed for the processing even it handles the same request.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Singleton        | <code>@Singleton</code>               | <code>jakarta.inject.Singleton</code>                        | In this scope there is only one instance per jax-rs application. Singleton resource can be either annotated with <code>@Singleton</code> and its class can be registered using the instance of <code>Application</code> [ <a href="https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html">https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html</a> ]. You can also create singletons by registering singleton instances into <code>Application</code> [ <a href="https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html">https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html</a> ]. |

## 3.5. Rules of Injection

Previous sections have presented examples of annotated types, mostly annotated method parameters but also annotated fields of a class, for the injection of values onto those types.

This section presents the rules of injection of values on annotated types. Injection can be performed on fields, constructor parameters, resource/sub-resource/sub-resource locator method parameters and bean setter methods. The following presents an example of all such injection cases:

### Example 3.23. Injection

```
1 @Path("{id:\\d+}")
2 public class InjectedResource {
3     // Injection onto field
4     @DefaultValue("q") @QueryParam("p")
```

```
5     private String p;
6
7     // Injection onto constructor parameter
8     public InjectedResource(@PathParam("id") int id) { ... }
9
10    // Injection onto resource method parameter
11    @GET
12    public String get(@Context UriInfo ui) { ... }
13
14    // Injection onto sub-resource resource method parameter
15    @Path("sub-id")
16    @GET
17    public String get(@PathParam("sub-id") String id) { ... }
18
19    // Injection onto sub-resource locator method parameter
20    @Path("sub-id")
21    public SubResource getSubResource(@PathParam("sub-id") String id) { ... }
22
23    // Injection using bean setter method
24    @HeaderParam("X-header")
25    public void setHeader(String header) { ... }
26 }
```

There are some restrictions when injecting on to resource classes with a life-cycle of singleton scope. In such cases the class fields or constructor parameters cannot be injected with request specific parameters. So, for example the following is not allowed.

### Example 3.24. Wrong injection into a singleton scope

```
1 @Path("resource")
2 @Singleton
3 public static class MySingletonResource {
4
5     @QueryParam("query")
6     String param; // WRONG: initialization of application will fail as you can
7                   // inject request specific parameters into a singleton resou
8
9     @GET
10    public String get() {
11        return "query param: " + param;
12    }
13 }
```

The example above will cause validation failure during application initialization as singleton resources cannot inject request specific parameters. The same example would fail if the query parameter would be injected into constructor parameter of such a singleton. In other words, if you wish one resource instance to server more requests (in the same time) it cannot be bound to a specific request parameter.

The exception exists for specific request objects which can injected even into constructor or class fields. For these objects the runtime will inject proxies which are able to simultaneously server more request. These request objects are `HttpHeaders`, `Request`, `UriInfo`, `SecurityContext`. These proxies can be injected using the `@Context` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>] annotation. The following example shows injection of proxies into the singleton resource class.

### Example 3.25. Injection of proxies into singleton

```
1 @Path("resource")
2 @Singleton
3 public static class MySingletonResource {
4     @Context
5     Request request; // this is ok: the proxy of Request will be injected into
6
7     public MySingletonResource(@Context SecurityContext securityContext) {
8         // this is ok too: the proxy of SecurityContext will be injected
9     }
10
11     @GET
12     public String get() {
13         return "query param: " + param;
14     }
15 }
```

To summarize the injection can be done into the following constructs:

**Table 3.2. Overview of injection types**

| Java construct         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Class fields           | Inject value directly into the field of the class. The field can be private and must not be final. Cannot be used in Singleton scope except proxiable types mentioned above.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Constructor parameters | The constructor will be invoked with injected values. If more constructors exists the one with the most injectable parameters will be invoked. Cannot be used in Singleton scope except proxiable types mentioned above.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Resource methods       | The resource methods (these annotated with @GET, @POST, ...) can contain parameters that can be injected when the resource method is executed. Can be used in any scope.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Sub resource locators  | The sub resource locators (methods annotated with @Path but not @GET, @POST, ...) can contain parameters that can be injected when the resource method is executed. Can be used in any scope.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Setter methods         | Instead of injecting values directly into field the value can be injected into the setter method which will initialize the field. This injection can be used only with @Context [ <a href="https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html">https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html</a> ] annotation. This means it cannot be used for example for injecting of query params but it can be used for injections of request. The setters will be called after the object creation and only once. The name of the method does not necessary have a setter pattern. Cannot be used in Singleton scope except proxiable types mentioned above. |

The following example shows all possible java constructs into which the values can be injected.

### Example 3.26. Example of possible injections

```
1 @Path("resource")
2 public static class SummaryOfInjectionsResource {
3     @QueryParam("query")
4     String param; // injection into a class field
5
6
7     @GET
```

```
8     public String get(@QueryParam("query") String methodQueryParam) {
9         // injection into a resource method parameter
10        return "query param: " + param;
11    }
12
13    @Path("sub-resource-locator")
14    public Class<SubResource> subResourceLocator(@QueryParam("query") String s
15        // injection into a sub resource locator parameter
16        return SubResource.class;
17    }
18
19    public SummaryOfInjectionsResource(@QueryParam("query") String constructor
20        // injection into a constructor parameter
21    }
22
23
24    @Context
25    public void setRequest(Request request) {
26        // injection into a setter method
27        System.out.println(request != null);
28    }
29 }
30
31 public static class SubResource {
32     @GET
33     public String get() {
34         return "sub resource";
35     }
36 }
```

The `@FormParam` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/FormParam.html] annotation is special and may only be utilized on resource and sub-resource methods. This is because it extracts information from a request entity.

## 3.6. Use of @Context

Previous sections have introduced the use of `@Context` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html]. Chapter "Context" in the JAX-RS specification presents all the standard JAX-RS Java types that may be used with `@Context` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html].

When deploying a JAX-RS application using servlet then `ServletConfig` [https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/ServletConfig.html], `ServletContext` [https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/ServletContext.html], `HttpServletRequest` [https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/HttpServletRequest.html] and `HttpServletResponse` [https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/HttpServletResponse.html] are available using `@Context` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html].

## 3.7. Programmatic resource model

Resources can be constructed from classes or instances but also can be constructed from a programmatic resource model. Every resource created from resource classes can also be constructed using the programmatic resource builder api. See resource builder section for more information.

---

# Chapter 4. Application Deployment and Runtime Environments

## 4.1. Introduction

This chapter is an overview of various server-side environments currently capable of running JAX-RS applications on top of Jersey server runtime. Jersey supports wide range of server environments from lightweight http containers up to full-fledged Java/Jakarta EE servers. Jersey applications can also run in an OSGi runtime. The way how the application is published depends on whether the application shall run in a Java SE environment or within a container.

### Note

This chapter is focused on server-side Jersey deployment models. The Jersey client runtime does not have any specific container requirements and runs in plain Java SE 8 or higher runtime.

## 4.2. JAX-RS Application Model

JAX-RS provides a deployment agnostic abstract class `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] for declaring root resource and provider classes, and root resource and provider singleton instances. A Web service may extend this class to declare root resource and provider classes. For example,

### Example 4.1. Deployment agnostic application model

```
1 public class MyApplication extends Application {
2     @Override
3     public Set<Class<?>> getClasses() {
4         Set<Class<?>> s = new HashSet<Class<?>>();
5         s.add(HelloWorldResource.class);
6         return s;
7     }
8 }
```

Alternatively it is possible to reuse `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>] - Jersey's own implementations of `Application` class. This class can either be directly instantiated and then configured or it can be extended and the configuration code placed into the constructor of the extending class. The approach typically depends on the chosen deployment runtime.

Compared to `Application`, the `ResourceConfig` provides advanced capabilities to simplify registration of JAX-RS components, such as scanning for root resource and provider classes in a provided classpath or a set of package names etc. All JAX-RS component classes that are either manually registered or found during scanning are automatically added to the set of classes that are returned by `getClasses`. For example, the following application class that extends from `ResourceConfig` scans during deployment for JAX-RS components in packages `org.foo.rest` and `org.bar.rest`:

### Note

*Package scanning* ignores an inheritance and therefore `@Path` annotation on parent classes and interfaces will be ignored. These classes won't be registered as the JAX-RS component classes.

### Example 4.2. Reusing Jersey implementation in your custom application model

```
1 public class MyApplication extends ResourceConfig {
2     public MyApplication() {
3         packages("org.foo.rest;org.bar.rest");
4     }
5 }
```

#### Note

Later in this chapter, the term *Application subclass* is frequently used. Whenever used, this term refers to the JAX-RS Application Model explained above.

## 4.3. Auto-Discoverable Features

By default Jersey 3.x does not implicitly register any extension features from the modules available on the classpath, unless explicitly stated otherwise in the documentation of each particular extension. Users are expected to explicitly register the extension Feature [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html]s using their `Application` subclass. Since Jersey 3.1.0 all features which extends Feature [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html] and are listed as SPI of the `Feature` interface are automatically registered (not required to be registered within `Application` subclass). For server side Jersey 3.1.0 also registers SPIs for `DynamicFeature` interface. For a few Jersey provided modules however there is no need to explicitly register their extension Features as these are discovered and registered in the Configuration [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configuration.html] (on client/server) automatically by Jersey runtime whenever the modules implementing these features are present on the classpath of the deployed JAX-RS application. The modules that are automatically discovered include:

- JSON binding feature from `jersey-media-moxy`
- `jersey-media-json-processing`
- `jersey-bean-validation`

Besides these modules there are also few features/providers present in `jersey-server` module that are discovered by this mechanism and their availability is affected by Jersey auto-discovery support configuration (see Section 4.3.1, “Configuring Feature Auto-discovery Mechanism”), namely:

- `WadlFeature` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/wadl/WadlFeature.html] - enables WADL processing.
- `UriConnegFilter` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/filter/UriConnegFilter.html] - a URI-based content negotiation filter.

Almost all Jersey auto-discovery implementations have `AutoDiscoverable.DEFAULT_PRIORITY` set.

#### Note

Auto discovery functionality is in Jersey supported by implementing an internal `AutoDiscoverable` Jersey SPI. This interface is not public at the moment, and is subject to change in the future, so be careful when trying to use it.

### 4.3.1. Configuring Feature Auto-discovery Mechanism

The mechanism of feature auto-discovery in Jersey that described above is enabled by default. It can be disabled by using special (common/server/client) properties:

#### Common auto discovery properties

- `CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE\\_AUTO\\_DISCOVERY\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE)]

When set, disables auto discovery globally on client/server.

- `CommonProperties.JSON_PROCESSING_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON\\_PROCESSING\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_PROCESSING_FEATURE_DISABLE)]

When set, disables configuration of Json Processing (JSR-353) feature.

- `CommonProperties.MOXY_JSON_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#MOXY\\_JSON\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#MOXY_JSON_FEATURE_DISABLE)]

When set, disables configuration of MOXy Json feature.

For each of these properties there is a client/server counterpart that is only honored by the Jersey client or server runtime respectively (see `ClientProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html>]/`ServerProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html>]). When set, each of these client/server specific auto-discovery related properties overrides the value of the related common property.

#### Note

In case an auto-discoverable mechanism (in general or for a specific feature) is disabled, then all the features, components and/or properties, registered by default using the auto-discovery mechanism have to be registered manually.

## 4.4. Feature and Dynamic Feature SPI automatic registration

Since Jersey 3.1.0 there is a new specification requirement to automatically discover and register classes that implement `Feature` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html>] and `DynamicFeature` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/DynamicFeature.html>] interfaces. This is being done via SPI search thus all those classes in order to be discovered automatically shall be listed as SPIs for given interfaces. This automatic registration can also be disabled by `jakarta.ws.rs.loadServices` (which is new specification property) to be set to false. This is also valid for `CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE\\_AUTO\\_DISCOVERY\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE)] lookup.



## 4.5. Configuring the Classpath Scanning

Jersey uses a common Java Service Provider mechanism to obtain all service implementations. It means that Jersey scans the whole class path to find appropriate `META-INF/services/` files. The class path scanning may be time consuming. The more jar or war files on the classpath the longer the scanning time. In use cases where you need to save every millisecond of application bootstrap time, you may typically want to disable the services provider lookup in Jersey.

### List of SPIs recognized by Jersey

- `Feature` (server, client) - Features are being registered similarly to `AutoDiscoverable` feature. In addition property `jakarta.ws.rs.loadServices` is checked before registration.
- `AutoDiscoverable` (server, client) - it means if you disable service loading the `AutoDiscoverable` feature is automatically disabled too
- `ForcedAutoDiscoverable` (server, client) - Jersey always looks for these auto discoverable features even if the service loading is disabled
- `HeaderDelegateProvider` (server, client)
- `DynamicFeature` (server) - Similar to `Feature` registration but only for server.
- `ComponentProvider` (server)
- `ContainerProvider` (server)
- `AsyncContextDelegateProvider` (server/Servlet)

### List of additional SPIs recognized by Jersey in case the `metainf-services` module is on the classpath

- `MessageBodyReader` (server, client)
- `MessageBodyWriter` (server, client)
- `ExceptionMapper` (server, client)

Since it is possible to configure all SPI implementation classes or instances manually in your `Application` subclass, disabling services lookup in Jersey does not affect any functionality of Jersey core modules and extensions and can save dozens of ms during application initialization in exchange for a more verbose application configuration code.

The services lookup in Jersey (enabled by default) can be disabled via a dedicated `CommonProperties.METAINF_SERVICES_LOOKUP_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#METAINF\\_SERVICES\\_LOOKUP\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#METAINF_SERVICES_LOOKUP_DISABLE)] property. There is a client/server counter-part that only disables the feature on the client or server respectively: `ClientProperties.METAINF_SERVICES_LOOKUP_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#METAINF\\_SERVICES\\_LOOKUP\\_DISABLE/](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#METAINF_SERVICES_LOOKUP_DISABLE/)] `ServerProperties.METAINF_SERVICES_LOOKUP_DISABLE` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/>]

`ServerProperties.html#METAINF_SERVICES_LOOKUP_DISABLE`]. As in all other cases, the client/server specific properties overrides the value of the related common property, when set.

For example, following code snippet disables service provider lookup and manually registers implementations of different JAX-RS and Jersey provider types (ContainerRequestFilter [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerRequestFilter.html>], Feature [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html>], ComponentProvider [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/spi/ComponentProvider.html>] and ContainerProvider [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/spi/ContainerProvider.html>]):

#### Example 4.3. Registering SPI implementations using ResourceConfig

```
1 ResourceConfig resourceConfig = new ResourceConfig(MyResource.class);
2 resourceConfig.register(org.glassfish.jersey.server.filter.UriConnegFilter.class);
3 resourceConfig.register(org.glassfish.jersey.server.validation.ValidationFeature.class);
4 resourceConfig.register(org.glassfish.jersey.server.spring.SpringComponentProvider.class);
5 resourceConfig.register(org.glassfish.jersey.grizzly2.httpserver.GrizzlyHttpContainerProvider.class);
6 resourceConfig.property(ServerProperties.METAINF_SERVICES_LOOKUP_DISABLE, true);
```

Similarly, in scenarios where the deployment model requires extending the `Application` subclass (e.g. in all Servlet container deployments), the following code could be used to achieve the same application configuration:

#### Example 4.4. Registering SPI implementations using ResourceConfig subclass

```
1 public class MyApplication extends ResourceConfig {
2     public MyApplication() {
3         register(org.glassfish.jersey.server.filter.UriConnegFilter.class);
4         register(org.glassfish.jersey.server.validation.ValidationFeature.class);
5         register(org.glassfish.jersey.server.spring.SpringComponentProvider.class);
6         register(org.glassfish.jersey.grizzly2.httpserver.GrizzlyHttpContainerProvider.class);
7         property(ServerProperties.METAINF_SERVICES_LOOKUP_DISABLE, true);
8     }
9 }
```

## 4.6. Java SE Deployment Environments

### 4.6.1. HTTP servers

Java based HTTP servers represent a minimalistic and flexible way of deploying Jersey application. The HTTP servers are usually embedded in the application and configured and started programmatically. In general, Jersey container for a specific HTTP server provides a custom factory method that returns a correctly initialized HTTP server instance.

#### 4.6.1.1. JDK Http Server

Starting with Java SE 6, Java runtime ships with a built-in lightweight HTTP server. Jersey offers integration with this Java SE HTTP server through the `jersey-container-jdk-http` container extension module. Instead of creating the `HttpServer` [<http://docs.oracle.com/javase/6/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/HttpServer.html>] instance directly, use the `createHttpServer()`

method of `JdkHttpServerFactory` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdkhttp/JdkHttpServerFactory.html>], which creates the `HttpServer` instance configured as a Jersey container and initialized with the supplied `Application` subclass.

Creating new Jersey-enabled jdk http server is as easy as:

#### Example 4.5. Using Jersey with JDK HTTP Server

```
1 URI baseUrl = UriBuilder.fromUri("http://localhost/").port(9998).build();
2 ResourceConfig config = new ResourceConfig(MyResource.class);
3 HttpServer server = JdkHttpServerFactory.createHttpServer(baseUrl, config);
```

A JDK HTTP Container dependency needs to be added:

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-jdk-http</artifactId>
  <version>4.0.0</version>
</dependency>
```

### 4.6.1.2. Grizzly HTTP Server

Grizzly [<https://javaee.github.io/grizzly/>] is a multi-protocol framework built on top of Java NIO [<http://docs.oracle.com/javase/6/docs/api/java/nio/package-summary.html>]. Grizzly aims to simplify development of robust and scalable servers. Jersey provides a container extension module that enables support for using Grizzly as a plain vanilla HTTP container that runs JAX-RS applications. Starting a Grizzly server to run a JAX-RS or Jersey application is one of the most lightweight and easy ways how to expose a functional RESTful services application.

Grizzly HTTP container supports injection of Grizzly-specific `org.glassfish.grizzly.http.server.Request` and `org.glassfish.grizzly.http.server.Response` instances into JAX-RS and Jersey application resources and providers. However, since Grizzly `Request` is not proxiable, the injection of Grizzly `Request` into singleton (by default) JAX-RS / Jersey providers is only possible via `jakarta.inject.Provider` instance. (Grizzly `Response` does not suffer the same restriction.)

#### Example 4.6. Using Jersey with Grizzly HTTP Server

```
1 URI baseUrl = UriBuilder.fromUri("http://localhost/").port(9998).build();
2 ResourceConfig config = new ResourceConfig(MyResource.class);
3 HttpServer server = GrizzlyHttpServerFactory.createHttpServer(baseUrl, config);
```

The container extension module dependency to be added is:

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-grizzly2-http</artifactId>
  <version>4.0.0</version>
</dependency>
```

#### Note

Jersey uses Grizzly extensively in the project unit and end-to-end tests via test framework.

### 4.6.1.3. Simple server

Simple [<http://www.simpleframework.org/>] is a framework which allows developers to create a HTTP server instance and embed it within an application. Again, creating the server instance is achieved by calling a factory method from the `jersey-container-simple-http` container extension module.

Simple framework HTTP container supports injection of Simple framework-specific `org.simpleframework.http.Request` and `org.simpleframework.http.Response` instances into JAX-RS and Jersey application resources and providers.

#### Example 4.7. Using Jersey with the Simple framework

```
1 URI baseUri = UriBuilder.fromUri("http://localhost/").port(9998).build();
2   ResourceConfig config = new ResourceConfig(MyResource.class);
3   SimpleContainer server = SimpleContainerFactory.create(baseUri, config);
```

The necessary container extension module dependency in this case is:

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-simple-http</artifactId>
  <version>4.0.0</version>
</dependency>
```

#### Note

Simple framework HTTP container does not support deployment on context paths other than root path ("/"). Non-root context path is ignored during deployment.

### 4.6.1.4. Jetty HTTP Server

Jetty is a popular Servlet container and HTTP server. We will not look into Jetty's capabilities as a Servlet container (although we are using it in our tests and examples), because there is nothing specific to Jetty when using a Servlet-based deployment model, which is extensively described later in our Section 4.8, "Servlet-based Deployment" section. We will here only focus on describing how to use Jetty's HTTP server.

Jetty HTTP container supports injection of Jetty-specific `org.eclipse.jetty.server.Request` and `org.eclipse.jetty.server.Response` instances into JAX-RS and Jersey application resources and providers. However, since Jetty HTTP `Request` is not proxiable, the injection of Jetty `Request` into singleton (by default) JAX-RS / Jersey providers is only possible via `jakarta.inject.Provider` instance. (Jetty `Response` does not suffer the same restriction.)

#### Example 4.8. Using Jersey with Jetty HTTP Server

```
1 URI baseUri = UriBuilder.fromUri("http://localhost/").port(9998).build();
2 ResourceConfig config = new ResourceConfig(MyResource.class);
3 Server server = JettyHttpContainerFactory.createServer(baseUri, config);
```

And, of course, we add the necessary container extension module dependency:

```
1 <dependency>
2   <groupId>org.glassfish.jersey.containers</groupId>
3   <artifactId>jersey-container-jetty-http</artifactId>
4   <version>4.0.0</version>
```

```
5 </dependency>
```

### Note

Jetty HTTP container does not support deployment on context paths other than root path ("/").  
Non-root context path is ignored during deployment.

## 4.6.1.5. Netty HTTP Server

Netty is a NIO client server framework which enables quick and easy development of network applications such as protocol servers and clients. Jersey supports Netty as a container and as a client connector - this chapter will present how to use the container.

### Example 4.9. Using Jersey with Netty HTTP Server

```
1 URI baseUrl = UriBuilder.fromUri("http://localhost/").port(9998).build();
2 ResourceConfig resourceConfig = new ResourceConfig(HelloWorldResource.class);
3 Channel server = NettyHttpContainerProvider.createServer(baseUrl, resourceConfig);
```

And, of course, we add the necessary container extension module dependency:

```
1 <dependency>
2     <groupId>org.glassfish.jersey.containers</groupId>
3     <artifactId>jersey-container-netty-http</artifactId>
4     <version>4.0.0</version>
5 </dependency>
```

### Note

Netty HTTP container does not support deployment on context paths other than root path ("/").  
Non-root context path is ignored during deployment.

## 4.6.2. Jakarta REST Bootstrap API

Jakarta REST 3.1 comes with a new API for starting an application in Java SE environment. This Bootstrap API is mainly represented by `SeBootstrap` interface. The Jakarta REST application is started as follows:

```
Application application = new MyApplication();
SeBootstrap.Configuration.Builder configBuilder = SeBootstrap.Configuration.builder();
CompletionStage<SeBootstrap.Instance> completionStage = SeBootstrap.start(application, configBuilder);
```

Later, when the SE application is no longer needed, it can be shutdown as follows:

```
CompletionStage<SeBootstrap.Instance> completionStage = ...
SeBootstrap.Instance instance = completionStage().get();
instance.stop();
```

The `SeBootstrap.Configuration` allows for configuring the Jersey runtime. The Jakarta REST 3.1 allows for configuring the HTTP port, the protocol (HTTP), the hostname, the root path, and SSL. The `SeBootstrap` is configured as follows:

```
SeBootstrap.Configuration.Builder configBuilder = SeBootstrap.Configuration.builder();
configBuilder.property(SeBootstrap.Configuration.PROTOCOL, "HTTP")
    .property(SeBootstrap.Configuration.HOST, "localhost")
```

```
.property(SeBootstrap.Configuration.PORT, 1234)  
.property(SeBootstrap.Configuration.ROOT_PATH, "/root/path");
```

The `SeBootstrap` deployment is backed up by an HTTP server described in Section 4.6.1, “HTTP servers”. If multiple Jersey container modules are on the classpath, the first found is used.

### 4.6.3. Jersey WebServer SPI

Jersey `WebServer` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/spi/WebServer.html>] and `WebServerProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/spi/WebServerProvider.html>] are SPI interfaces similar to `ContainerProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/spi/ContainerProvider.html>] but they are used for the SE deployment. They serve as a bridge between Jersey containers and `SeBootstrap` API. The Jakarta REST application can be started as follows:

```
Application application = new MyApplication();  
SeBootstrap.Configuration.Builder configBuilder = SeBootstrap.Configuration.builder();  
WebServer webServer = WebServerFactory.createServer(WebServer.class, application,
```

Later, when the SE application is no longer needed, it can be shutdown as follows:

```
WebServer webServer = ...  
webServer.stop();
```

`WebServerFactory` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/WebServerFactory.html>] is used to automatically choose among available implementations on a classpath. If there are multiple implementations available, the first found is used. The user can choose the implementation of a `WebServer` by a concrete `WebServer` subclass, for instance:

```
WebServer webServer = WebServerFactory.createServer(GrizzlyHttpServer.class, appli
```

Another way to choose the `WebServer` implementation is by the `WebServerProvider` implementation:

```
WebServer webServer = GrizzlyHttpServerProvider.createServer(WebServer.class, appl
```

For additional customization of the `WebServer` settings, see Section A.3, “`SeBootstrap` and `WebServer` related configuration properties”.

#### Warning

When the port is set to -1, the default ports are used. Unlike the default ports used by the Section 4.6.1, “HTTP servers”, the `SeBootstrap` API and `WebServer` SPI use default ports 8080 and 8443, respectively.

When the port is set to 0, the implementation scans for a free port. The privileged ports are skipped, and the scanning starts with port 1024.

Using the restricted ports can be ensured either by setting directly the port, or by setting `ServerProperties.WEBSERVER_ALLOW_PRIVILEGED_PORTS` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#WEBSERVER\\_ALLOW\\_PRIVILEGED\\_PORTS](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#WEBSERVER_ALLOW_PRIVILEGED_PORTS)] property to true in the `SeBootstrap.Configuration`

## 4.7. Creating programmatic JAX-RS endpoint

JAX-RS specification also defines the ability to programmatically create a JAX-RS application endpoint (i.e. container) for any instance of a `Application` subclass. For example, Jersey supports creation of Grizzly [<http://grizzly.java.net/>] `HttpHandler` instance as follows:

```
HttpHandler endpoint = RuntimeDelegate.getInstance()  
    .createEndpoint(new MyApplication(), HttpHandler.class);
```

Once the Grizzly `HttpHandler` endpoint is created, it can be used for in-process deployment to a specific base URL.

## 4.8. Servlet-based Deployment

In a Servlet container, JAX-RS defines multiple deployment options depending on the Servlet API version supported by the Servlet container. Following sections describe these options in detail.

### 4.8.1. Servlet 2.x way

Jersey integrates with any Servlet containers supporting at least Servlet 2.5 specification. Running on a Servlet container that supports Servlet API 5.0 it's required to adjust this approach to jakartified Servlet API. This includes Jakarta EE 9 namespaces which is applied since the 5.x Servlet API version. In this section we will focus on the basic deployment models available in any Servlet 2.5 container.

Using Servlet 2.x way, you have to explicitly declare the Jersey container Servlet in your Web application's `web.xml` deployment descriptor file.

#### Example 4.10. Hooking up Jersey as a Servlet

```
1 <web-app>  
2         <servlet>  
3             <servlet-name>MyApplication</servlet-name>  
4             <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>  
5             <init-param>  
6                 ...  
7             </init-param>  
8         </servlet>  
9         ...  
10        <servlet-mapping>  
11            <servlet-name>MyApplication</servlet-name>  
12            <url-pattern>/myApp/*</url-pattern>  
13        </servlet-mapping>  
14        ...  
15    </web-app>
```

Alternatively, you can register Jersey container as a *filter*:

#### Example 4.11. Hooking up Jersey as a Servlet Filter

```
1 <web-app>  
2         <filter>  
3             <filter-name>MyApplication</filter-name>
```

```
4          <filter-class>org.glassfish.jersey.servlet.ServletCont
5          <init-param>
6          ...
7          </init-param>
8          </filter>
9          ...
10         <filter-mapping>
11         <filter-name>MyApplication</filter-name>
12         <url-pattern>/myApp/*</url-pattern>
13         </filter-mapping>
14         ...
15         </web-app>
```

### Important

Since pure Servlet 2.x way deployment does not provide a way how to programmatically read the filter mappings in order to make application with filter work correctly, the context path of the app needs to be defined using init parameter `jersey.config.servlet.filter.contextPath` for `jersey-container-servlet-core`. Or `jersey-container-servlet` shall be used.

The content of the `<init-param>` element will vary depending on the way you decide to configure Jersey resources.

## 4.8.1.1. Custom Application subclass

If you extend the Application [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] class to provide the list of relevant root resource classes (`getClasses()`) and singletons (`getSingletons()`), i.e. your JAX-RS application model, you then need to register it in your web application `web.xml` deployment descriptor using a Servlet or Servlet filter initialization parameter with a name of `jakarta.ws.rs.Application` [sic] as follows:

### Example 4.12. Configuring Jersey container Servlet or Filter to use custom Application subclass

```
1 <init-param>
2           <param-name>jakarta.ws.rs.Application</param-name>
3           <param-value>org.foo.MyApplication</param-value>
4           </init-param>
```

Jersey will consider all the classes returned by `getClasses()` and `getSingletons()` methods of your Application implementation.

### Note

The name of the configuration property as defined by JAX-RS specification is indeed `jakarta.ws.rs.Application` and not `jakarta.ws.rs.core.Application` as one might expect.

## 4.8.1.2. Jersey package scanning

If there is no configuration properties to be set and deployed application consists only from resources and providers stored in particular packages, you can instruct Jersey to scan these packages and register any found resources and providers automatically:



### Example 4.13. Configuring Jersey container Servlet or Filter to use package scanning

```
1 <init-param>
2                               <param-name>jersey.config.server.provider.packages
3                               <param-value>
4                               org.foo.myresources,org.bar.otherresources
5                               </param-value>
6                               </init-param>
7                               <init-param>
8                               <param-name>jersey.config.server.provider.scanning
9                               <param-value>>false</param-value>
10                              </init-param>
```

Jersey will automatically discover the resources and providers in the selected packages. You can also decide whether Jersey should recursively scan also sub-packages by setting the `jersey.config.server.provider.scanning.recursive` property. The default value is `true`, i.e. the recursive scanning of sub-packages is enabled.

#### 4.8.1.3. Selecting concrete resource and provider classes

While the above-mentioned package scanning is useful esp. for development and testing, you may want to have a little bit more control when it comes to production deployment in terms of being able to enumerate specific resource and provider classes. In Jersey it is possible to achieve this even without a need to implement a custom `Application` subclass. The specific resource and provider fully-qualified class names can be provided in a comma-separated value of `jersey.config.server.provider.classnames` initialization parameter.

### Example 4.14. Configuring Jersey container Servlet or Filter to use a list of classes

```
1 <init-param>
2                               <param-name>jersey.config.server.provider.classnames</
3                               <param-value>
4                               org.foo.myresources.MyDogResource,
5                               org.bar.otherresources.MyCatResource
6                               </param-value>
7                               </init-param>
```

#### Note

All of the techniques that have been described in this section also apply to Servlet containers that support Servlet API 5.0 with jakartified adjustments. Newer Servlet specifications only give you additional features, deployment options and more flexibility.

## 4.8.2. Servlet 5.x Container

### 4.8.2.1. Descriptor-less deployment

There are multiple deployment options in the Servlet 5.0 container for a JAX-RS application defined by implementing a custom `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] subclass. For simple deployments, no `web.xml` is necessary at all. Instead, an `@ApplicationPath` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ApplicationPath.html>] annotation can be used to annotate the custom `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/>]

`jakarta/ws/rs/core/Application.html`] subclass and define the base application URI for all JAX-RS resources configured in the application:

**Example 4.15. Deployment of a JAX-RS application using `@ApplicationPath` with Servlet 5.0**

```
1 @ApplicationPath("resources")
2 public class MyApplication extends ResourceConfig {
3     public MyApplication() {
4         packages("org.foo.rest;org.bar.rest");
5     }
6 }
```

**Note**

There are many other convenience methods in the `ResourceConfig` that can be used in the constructor of your custom subclass to configure your JAX-RS application, see `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>] API documentation for more details.

In case you are not providing `web.xml` deployment descriptor for your maven-based web application project, you need to configure your `maven-war-plugin` to ignore the missing `web.xml` file by setting `failOnMissingWebXml` [<http://maven.apache.org/plugins/maven-war-plugin/war-mojo.html#failOnMissingWebXml>] configuration property to `false` in your project `pom.xml` file:

**Example 4.16. Configuration of `maven-war-plugin` to ignore missing `web.xml`**

```
1 <plugins>
2     ...
3     <plugin>
4         <groupId>org.apache.maven.plugins</groupId>
5         <artifactId>maven-war-plugin</artifactId>
6         <version>2.3</version>
7         <configuration>
8             <failOnMissingWebXml>false</failOnMissingWebXml>
9         </configuration>
10    </plugin>
11    ...
12 </plugins>
```

## 4.8.2.2. Deployment using `web.xml` descriptor

Another Servlet 5.x container deployment model is to declare the JAX-RS application details in the `web.xml`. This is typically suitable for more complex deployments, e.g. when security model needs to be properly defined or when additional initialization parameters have to be passed to Jersey runtime. JAX-RS 1.1 and later specifies that a fully qualified name of the class that implements `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] may be used in the definition of a `<servlet-name>` element as part of your application's `web.xml` deployment descriptor.

Following example illustrates this approach:

**Example 4.17. Deployment of a JAX-RS application using `web.xml` with Servlet 5.0**

```
1 <web-app>
```

```
2    <servlet>
3        <servlet-name>org.foo.rest.MyApplication</servlet-name>
4    </servlet>
5    ...
6    <servlet-mapping>
7        <servlet-name>org.foo.rest.MyApplication</servlet-name>
8        <url-pattern>/resources</url-pattern>
9    </servlet-mapping>
10   ...
11 </web-app>
```

Note that the `<servlet-class>` element is omitted from the Servlet declaration. This is a correct declaration utilizing the Servlet 5.0 extension mechanism described in detail in the Section 4.8.2.3, “Servlet Pluggability Mechanism” section. Also note that `<servlet-mapping>` is used in the example to define the base resource URI.

### 4.8.2.3. Servlet Pluggability Mechanism

Servlet framework pluggability mechanism is a feature introduced with Servlet 3.0 specification. It simplifies the configuration of various frameworks built on top of Servlets. Instead of having one `web.xml` file working as a central point for all the configuration options, it is possible to modularize the deployment descriptor by using the concept of so-called web fragments - several specific and focused `web.xml` files. A set of web fragments basically builds up the final deployment descriptor. This mechanism also provides SPI hooks that enable web frameworks to register themselves in the Servlet container or customize the Servlet container deployment process in some other way. This section describes how JAX-RS and Jersey leverage the Servlet pluggability mechanism.

#### 4.8.2.3.1. JAX-RS application without an `Application` subclass

If no `Application` (or `ResourceConfig`) subclass is present, Jersey will dynamically add a Jersey container Servlet and set its name to `jakarta.ws.rs.core.Application`. The web application path will be scanned and all the root resource classes (the classes annotated with `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] annotation) as well as any providers that are annotated with `@Provider` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/Provider.html>] annotation packaged with the application will be automatically registered in the JAX-RS application. The web application has to be packaged with a deployment descriptor specifying at least the mapping for the added `jakarta.ws.rs.core.Application` Servlet:

**Example 4.18. `web.xml` of a JAX-RS application without an `Application` subclass**

```
1 <web-app version="5.0"
2     xmlns="https://jakarta.ee/xml/ns/jakartaee"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
4
5     <!-- Servlet declaration can be omitted in which case
6         it would be automatically added by Jersey -->
7     <servlet>
8         <servlet-name>jakarta.ws.rs.core.Application</servlet-name>
9     </servlet>
10
11     <servlet-mapping>
12         <servlet-name>jakarta.ws.rs.core.Application</servlet-name>
13         <url-pattern>/myresources/*</url-pattern>
14     </servlet-mapping>
```

15 </web-app>

#### 4.8.2.3.2. JAX-RS application with a custom Application subclass

When a custom `Application` subclass is provided, in such case the Jersey server runtime behavior depends on whether or not there is a `Servlet` defined to handle the application subclass.

If the `web.xml` contains a `Servlet` definition, that has an initialization parameter `jakarta.ws.rs.Application` whose value is the fully qualified name of the `Application` subclass, Jersey does not perform any additional steps in such case.

If no such `Servlet` is defined to handle the custom `Application` subclass, Jersey dynamically adds a `Servlet` with a fully qualified name equal to the name of the provided `Application` subclass. To define the mapping for the added `Servlet`, you can either annotate the custom `Application` subclass with an `@ApplicationPath` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ApplicationPath.html>] annotation (Jersey will use the annotation value appended with `/*` to automatically define the mapping for the `Servlet`), or specify the mapping for the `Servlet` in the `web.xml` descriptor directly.

In the following example, let's assume that the JAX-RS application is defined using a custom `Application` subclass named `org.example.MyApplication`. Then the `web.xml` file could have the following structure:

##### Example 4.19.

```
1 <web-app version="5.0"
2   xmlns="https://jakarta.ee/xml/ns/jakartaee"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
4
5   <!-- Servlet declaration can be omitted in which case
6       it would be automatically added by Jersey -->
7   <servlet>
8       <servlet-name>org.example.MyApplication</servlet-name>
9   </servlet>
10
11   <!-- Servlet mapping can be omitted in case the Application subclass
12       is annotated with @ApplicationPath annotation; in such case
13       the mapping would be automatically added by Jersey -->
14   <servlet-mapping>
15       <servlet-name>org.example.MyApplication</servlet-name>
16       <url-pattern>/myresources/*</url-pattern>
17   </servlet-mapping>
18 </web-app>
```

### Note

If your custom `Application` subclass is packaged in the `war`, it defines which resources will be taken into account.

- If both `getClasses()` and `getSingletons()` methods return an empty collection, then ALL the root resource classes and providers packaged in the web application archive will be used, Jersey will automatically discover them by scanning the `.war` file.
- If any of the two mentioned methods - `getClasses()` or `getSingletons()` returns a non-empty collection, only those classes and/or singletons will be published in the JAX-RS application.

**Table 4.1. Servlet 3 Pluggability Overview**

| Condition                                            | Jersey action | Servlet Name                                | web.xml                                                                                           |
|------------------------------------------------------|---------------|---------------------------------------------|---------------------------------------------------------------------------------------------------|
| No Application subclass                              | Adds Servlet  | <code>jakarta.ws.rs.core.Application</code> | Servlet mapping is required                                                                       |
| Application subclass handled by existing Servlet     | No action     | Already defined                             | Not required                                                                                      |
| Application subclass NOT handled by existing Servlet | Adds Servlet  | FQN of the Application subclass             | if no <code>@ApplicationPath</code> on the Application subclass, then Servlet mapping is required |

### 4.8.3. Jersey Servlet container modules

Jersey uses its own `ServletContainer` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/servlet/ServletContainer.html>] implementation of Servlet and Servlet Filter API to integrate with Servlet containers. As any Jakartified JAX-RS runtime (JAX-RS spec 3.0), Jersey provides support for Servlet containers that support Servlet specification version 5.0 or higher is required.

When deploying to a Servlet container, Jersey application is typically packaged as a `.war` file. As with any other Servlet application, JAX-RS application classes are packaged in `WEB-INF/classes` or `WEB-INF/lib` and required application libraries are located in `WEB-INF/lib`. For more details, please refer to the Servlet Specification (Servlet 5 spec [<https://jakarta.ee/specifications/servlet/5.0/>]).

Jersey provides two Servlet modules. The first module is the Jersey core Servlet module that provides the core Servlet integration support and is required in any Servlet 5 or higher container:

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet-core</artifactId>
</dependency>
```

To support additional Servlet 5.x deployment modes and asynchronous JAX-RS resource programming model, an additional Jersey module is required:

```
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet</artifactId>
</dependency>
```

The `jersey-container-servlet` module depends on `jersey-container-servlet-core` module, therefore when it is used, it is not necessary to explicitly declare the `jersey-container-servlet-core` dependency.

Note that in simple cases, you don't need to provide the deployment descriptor (`web.xml`) and can use the `@ApplicationPath` annotation, as described in ??? section.

## 4.9. Jakarta EE Platform

This section describes, how you can publish Jersey JAX-RS resources as various Jakarta EE platform elements. JAX-RS and Jersey give you wide choice of possibilities and it is up to your taste (and design of your application), what Jakarta EE technology you decide to use for the management of your resources.

## 4.9.1. Managed Beans

Jersey supports the use of Jakarta EE Managed beans as root resource classes, providers as well as `Application` subclasses.

In the code below, you can find an example of a bean, that uses a managed-bean interceptor defined as a JAX-RS bean. The bean is used to intercept calls to the resource method `getIt()`:

```
1 @ManagedBean
2 @Path("/managedbean")
3 public class ManagedBeanResource {
4
5     public static class MyInterceptor {
6         @AroundInvoke
7         public String around(InvocationContext ctx) throws Exception {
8             System.out.println("around() called");
9             return (String) ctx.proceed();
10        }
11    }
12
13    @GET
14    @Produces("text/plain")
15    @Interceptors(MyInterceptor.class)
16    public String getIt() {
17        return "Hi managed bean!";
18    }
19 }
```

## 4.9.2. Context and Dependency Injection (CDI)

CDI beans can be used as Jersey root resource classes, providers as well as `Application` subclasses. Providers and `Application` subclasses have to be singleton or application scoped.

The next example shows a usage of a CDI bean as a JAX-RS root resource class. We assume, that CDI has been enabled. The code snippet uses the type-safe dependency injection provided in CDI by using another bean (`MyOtherCdiBean`):

```
1 @Path("/cdibean")
2 public class CdiBeanResource {
3     @Inject MyOtherCdiBean bean; // CDI injected bean
4
5     @GET
6     @Produces("text/plain")
7     public String getIt() {
8         return bean.getIt();
9     }
10 }
```

The above works naturally inside any Java/Jakarta EE compliant AS container. In Jersey version 2.15, container agnostic CDI support was introduced. This feature allows you to publish CDI based JAX-RS resources also in other containers. Jersey cdi-webapp example shows Jersey/CDI integration in Grizzly HTTP and Apache Tomcat server. Detailed description of Jersey CDI support outside of a fully fledged Java/Jakarta EE application container could be found in Chapter 26, *Jersey CDI Container Agnostic Support*.

### 4.9.3. Enterprise Java Beans (EJB)

Stateless and Singleton Session beans can be used as Jersey root resource classes, providers and/or `Application` subclasses. You can choose from annotating the methods in the EJB's local interface or directly the method in an interface-less EJB POJO. JAX-RS specifications requires its implementors to discover EJBs by inspecting annotations on classes (or local interfaces), but not in the deployment descriptors (`ejb-jar.xml`). As such, to keep your JAX-RS application portable, do not override EJB annotations or provide any additional meta-data in the deployment descriptor file.

Following example consists of a stateless EJB and a local interface used in Jersey:

```
1 @Local
2 public interface LocalEjb {
3     @GET
4     @Produces("text/plain")
5     public String getIt();
6 }
7
8 @Stateless
9 @Path("/stateless")
10 public class StatelessEjbResource implements LocalEjb {
11     @Override
12     public String getIt() {
13         return "Hi Stateless!";
14     }
15 }
```

#### Note

Please note that Jersey currently does not support deployment of JAX-RS applications packaged as standalone EJB modules (`ejb-jars`). To use EJBs as JAX-RS resources, the EJBs need to be packaged either directly in a WAR or in an EAR that contains at least one WAR. This is to ensure Servlet container initialization that is necessary for bootstrapping of the Jersey runtime.

### 4.9.4. Jakarta EE Servers

#### 4.9.4.1. GlassFish Application Server

As explained in 2.3.1, you don't need to add any specific dependencies on GlassFish, Jersey is already packaged within GlassFish. You only need to add the `provided`-scoped dependencies to your project to be able to compile it. At runtime, GlassFish will make sure that your application has access to the Jersey libraries.

Started with version 2.7, Jersey allows injecting Jersey specific types into CDI enabled JAX-RS components using the `@jakarta.inject.Inject` annotation. This covers also custom HK2 bindings, that are configured as part of Jersey application. The feature specifically enables usage of Jersey monitoring statistics (provided that the statistic feature is turned on) in CDI environment, where injection is the only mean to get access to monitoring data.

Since both CDI and HK2 use the same injection annotation, Jersey could get confused in certain cases, which could lead to nasty runtime issues. The get better control over what Jersey evaluates as HK2 injection, end-users could take advantage of newly introduced, `Hk2CustomBoundTypesProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/>]

[org.glassfish/jersey/ext/cdi1x/spi/Hk2CustomBoundTypesProvider.html](http://org.glassfish/jersey/ext/cdi1x/spi/Hk2CustomBoundTypesProvider.html)], SPI. Please see the linked javadoc to get detailed information on how to use the SPI in your application.

#### 4.9.4.2. Oracle WebLogic Server

For now Oracle WebLogic Server does not support Jakartified Jersey 3.x deployment (for deployment of prior versions of Jersey please refer to corresponding manual/user guide)

In 10.3.x, a set of pre-built shared libraries were delivered with WebLogic Server to support Jersey 1.9 and 1.1.5.1 Java API for RESTful Web Services (JAX-RS) Reference Implementations (RIs). In 12.2.x, WebLogic Server supports Jersey 2.21.x (JAX-RS 2.0 RI) by default. To use the pre-built shared libraries of 10.3.x, you needed to register them with the WebLogic Server instance, and modify the `web.xml` and `weblogic.xml` deployment descriptors to use the Jersey servlet and reference the shared libraries, respectively. In 12.2.x, as WebLogic Server supports Jersey 2.21.x (JAX-RS 2.0 RI) by default, registration as a shared library with WebLogic Server is no longer required. Please read through the [Upgrading a 10.3.x RESTful Web Service \(JAX-RS\) to 12.2.x](https://docs.oracle.com/middleware/12213/wls/WLUPG/upgrade_ws.htm#WLUPG331) [https://docs.oracle.com/middleware/12213/wls/WLUPG/upgrade\_ws.htm#WLUPG331]

#### 4.9.4.3. Other Application Servers

Third party Java/Jakarta EE application servers usually ship with a JAX-RS implementation. If you want to use Jersey instead of the default JAX-RS provider, you need to add Jersey libraries to your classpath and disable the default JAX-RS provider in the container.

In general, Jersey will be deployed as a Servlet and the resources can be deployed in various ways, as described in this section. However, the exact steps will vary from vendor to vendor.

## 4.10. OSGi

OSGi support has been added to the Jersey version 1.2. Since then, you should be able to utilize standard OSGi means to run Jersey based web applications in OSGi runtime as described in the OSGi Service Platform Enterprise Specification. Jersey is currently compatible with OSGi 4.2.0, the specification could be downloaded from the OSGi 4.2.0 Download Site [http://www.osgi.org/Download/Release4V42].

The two supported ways of running an OSGi web application are:

- WAB (Web Application Bundle)
- HTTP Service

WAB is in fact just an OSGified WAR archive. HTTP Service feature allows you to publish Jakarta EE Servlets in the OSGi runtime.

Two examples were added to the Jersey distribution to depict the above mentioned features and show how to use them with Jersey:

- WAB Example [https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/osgi-helloworld-webapp]
- HTTP Service example [https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/osgi-http-service]

Both examples are multi-module maven projects and both consist of an application OSGi bundle module and a test module. The tests are based on the PAX Exam [http://ops4j1.jira.com/wiki/display/PAXEXAM3/Pax+Exam] framework. Both OSGi examples also include a readme file containing instructions how to manually run the example applications using Apache Felix [https://felix.apache.org/] framework.



The rest of the chapter describes how to run the above mentioned examples on GlassFish 6 application server.

### 4.10.1. Enabling the OSGi shell in Glassfish

Since GlassFish utilizes Apache Felix, an OSGi runtime comes out of the box with GlassFish. However, for security reasons, the OSGi shell has been turned off. You can however explicitly enable it either by starting GlassFish the `asadmin` console and creating a Java system property `glassfish.osgi.start.level.final` and setting its value to 3:

#### Example 4.20.

Start the admin console:

```
1
2 ~/glassfish/bin$ ./asadmin
3 Use "exit" to exit and "help" for online help.
4 asadmin>
```

You can check the actual value of the java property (loaded from the configuration file):

```
26
27 asadmin> list-jvm-options
28 ...
29 -Dglassfish.osgi.start.level.final=2
30 ...
```

And change the value by typing:

```
26
27 asadmin> create-jvm-options --target server -Dglassfish.osgi.start.level.fina
```

The second option is to change the value in the `osgi.properties` configuration file:

```
1
2 # Final start level of OSGi framework. This is used by GlassFish launcher code
3 # to set the start level of the OSGi framework once server is up and running s
4 # optional services can start. The initial start level of framework is control
5 # the standard framework property called org.osgi.framework.startlevel.beginni
6 glassfish.osgi.start.level.final=3
```

You can then execute the Felix shell commands by typing `osgi <felix_command>` in the `asadmin` console. For example:

```
1
2 asadmin> osgi lb
3 ... list of bundles ...
```

or launching the shell using `osgi-shell` command in the admin console (the domain must be started, otherwise the osgi shell won't launch):

```
1
2 asadmin> osgi-shell
3 Use "exit" to exit and "help" for online help.
4 gogo$
```

and execute the osgi commands directly (without the "osgi" prefix):

```
1
2 gogo$ lb
3 ... list of bundles ...
```

## 4.10.2. WAB Example

As mentioned above, WAB is just an OSGi-fied WAR archive. Besides the usual OSGi headers it must in addition contain a special header, `Web-ContextPath`, specifying the web application context path. Our WAB has (beside some other) the following headers present in the manifest:

```
1 Web-ContextPath: helloworld
2 Webapp-Context: helloworld
3 Bundle-ClassPath: WEB-INF/classes
```

Here, the second header is ignored by GlassFish, but may be required by other containers not fully compliant with the OSGi Enterprise Specification mentioned above. The third manifest header worth mentioning is the `Bundle-ClassPath` specifying where to find the application Java classes within the bundle archive. More about manifest headers in OSGi can be found in the OSGi Wiki [[http://wiki.osgi.org/wiki/Category:Manifest\\_Header](http://wiki.osgi.org/wiki/Category:Manifest_Header)].

For more detailed information on the example please see the WAB Example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/osgi-helloworld-webapp>] source code. This example does not package into a single `war` file. Instead a `war` and a set of additional `jar`s is produced during the build. See the next example to see how to deploy OSGi based Jersey application to GlassFish.

## 4.10.3. HTTP Service Example

### Note

When deploying an OSGi HTTP Service example to GlassFish, please make sure the OSGi HTTP Service bundle is installed on your GlassFish instance.

You can directly install and activate the Jersey application bundle. In case of our example, you can either install the example bundle stored locally (and alternatively build from Jersey sources):

1) Build (optional)

```
1
2 examples$ cd osgi-http-service/bundle
3 bundle$ mvn clean package
```

You can also get the binary readily compiled from Java.net Maven Repository [<https://maven.java.net/content/repositories/releases/org/glassfish/jersey/examples/osgi-http-service/bundle/4.0.0>].

2) Install into OSGi runtime:

```
1
2 gogo$ install file:///path/to/file/bundle.jar
3 Bundle ID: 303
```

or install it directly from the maven repository:

```
1
```

```
2 gogo$ install https://repo1.maven.org/maven2/org/glassfish/jersey/examples/osgi
3 Bundle ID: 303
```

Make sure to replace `<version>` with an appropriate version number. Which one is appropriate depends on the specific GlassFish 6.x version you are using. The version of the bundle cannot be higher than the version of Jersey integrated in your GlassFish 6.x server. Jersey bundles declare dependencies on other bundles at the OSGi level and those dependencies are version-sensitive. If you use example bundle from let's say version 3.0.0-RC2, but Glassfish has Jersey 3.0.0-M1, dependencies will not be satisfied and bundle will not start. If this happens, the error will look something like this:

```
1 gogo$ lb
2 ...
3 303 | Installed | 1| jersey-examples-osgi-http-service-bundle (2.5.0.SNAPS
4 gogo$ start 303
5
6 org.osgi.framework.BundleException: Unresolved constraint in bundle
7 org.glassfish.jersey.examples.osgi-http-service.bundle [303]: Unable to resolv
8 [303.0] osgi.wiring.package; (&(osgi.wiring.package=org.glassfish.jersey.servl
9 (version>=3.0.0.RC2))
10
11 gogo$
```

In the opposite scenario (example bundle version 3.0.0-M1 and Glassfish Jersey version higher), everything should work fine.

Also, if you build GlassFish from the main trunk sources and use the example from most recent Jersey release, you will most likely be able to run the examples from the latest Jersey release, as Jersey team typically integrates all newly released versions of Jersey immediately into GlassFish.

As a final step, start the bundle:

```
1 gogo$ start 303
```

Again, the Bundle ID (in our case 303) has to be replaced by the correct one returned from the `install` command.

The example app should now be up and running. You can access it on <http://localhost:8080/osgi/jersey-http-service/status> [<http://localhost:8080/osgi/jersey-http-service/status>]. Please see HTTP Service example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/osgi-http-service>] source code for more details on the example.

## 4.11. Other Environments

### 4.11.1. Oracle Java Cloud Service

As Oracle Public Cloud is based on WebLogic server, the same applies as in the paragraph about WebLogic deployment (see Section 4.9.4.2, “Oracle WebLogic Server”). More on developing applications for Oracle Java Cloud Service can be found in this guide [[http://docs.oracle.com/cloud/131/developer\\_services/CSJSU/java-develop.htm#BABHDAJH](http://docs.oracle.com/cloud/131/developer_services/CSJSU/java-develop.htm#BABHDAJH)].

---

# Chapter 5. Client API

This section introduces the JAX-RS Client API, which is a fluent Java based API for communication with RESTful Web services. This standard API that is also part of Jakarta EE 9 is designed to make it very easy to consume a Web service exposed via HTTP protocol and enables developers to concisely and efficiently implement portable client-side solutions that leverage existing and well established client-side HTTP connector implementations.

The JAX-RS client API can be utilized to consume any Web service exposed on top of a HTTP protocol or its extension (e.g. WebDAV), and is not restricted to services implemented using JAX-RS. Yet, developers familiar with JAX-RS should find the client API complementary to their services, especially if the client API is utilized by those services themselves, or to test those services. The JAX-RS client API finds inspiration in the proprietary Jersey 1.x Client API and developers familiar with the Jersey 1.x Client API should find it easy to understand all the concepts introduced in the new JAX-RS Client API.

The goals of the client API are threefold:

1. Encapsulate a key constraint of the REST architectural style, namely the Uniform Interface Constraint and associated data elements, as client-side Java artifacts;
2. Make it as easy to consume RESTful Web services exposed over HTTP, same as the JAX-RS server-side API makes it easy to develop RESTful Web services; and
3. Share common concepts and extensibility points of the JAX-RS API between the server and the client side programming models.

As an extension to the standard JAX-RS Client API, the Jersey Client API supports a pluggable architecture to enable the use of different underlying HTTP client Connector [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/Connector.html>] implementations. Several such implementations are currently provided with Jersey. We have a default client connector using `Http(s)URLConnection` supplied with the JDK as well as connector implementations based on Apache HTTP Client, Jetty HTTP client and Grizzly Asynchronous Client.

## 5.1. Uniform Interface Constraint

The uniform interface constraint bounds the architecture of RESTful Web services so that a client, such as a browser, can utilize the same interface to communicate with any service. This is a very powerful concept in software engineering that makes Web-based search engines and service mash-ups possible. It induces properties such as:

1. simplicity, the architecture is easier to understand and maintain; and
2. evolvability or loose coupling, clients and services can evolve over time perhaps in new and unexpected ways, while retaining backwards compatibility.

Further constraints are required:

1. every resource is identified by a URI;
2. a client interacts with the resource via HTTP requests and responses using a fixed set of HTTP methods;
3. one or more representations can be returned and are identified by media types; and
4. the contents of which can link to further resources.

The above process repeated over and over again should be familiar to anyone who has used a browser to fill in HTML forms and follow links. That same process is applicable to non-browser based clients.

Many existing Java-based client APIs, such as the Apache HTTP client API or `URLConnection` supplied with the JDK place too much focus on the Client-Server constraint for the exchanges of request and responses rather than a resource, identified by a URI, and the use of a fixed set of HTTP methods.

A resource in the JAX-RS client API is an instance of the Java class `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>], and encapsulates an URI. The fixed set of HTTP methods can be invoked based on the `WebTarget`. The representations are Java types, instances of which, may contain links that new instances of `WebTarget` may be created from.

## 5.2. Ease of use and reusing JAX-RS artifacts

Since a JAX-RS component is represented as an annotated Java type, it makes it easy to configure, pass around and inject in ways that are not so intuitive or possible with other client-side APIs. The Jersey Client API reuses many aspects of the JAX-RS and the Jersey implementation such as:

1. URI building using `UriBuilder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriBuilder.html>] and `UriTemplate` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/uri/UriTemplate.html>] to safely build URIs;
2. Built-in support for Java types of representations such as `byte[]`, `String`, `Number`, `Boolean`, `Character`, `InputStream`, `java.io.Reader`, `File`, `DataSource`, JAXB beans as well as additional Jersey-specific JSON and Multi Part [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/package-summary.html>] support.
3. Using the fluent builder-style API pattern to make it easier to construct requests.

Some APIs, like the Apache HTTP Client or `URLConnection` [<http://docs.oracle.com/javase/6/docs/api/java/net/URLConnection.html>] can be rather hard to use and/or require too much code to do something relatively simple, especially when the client needs to understand different payload representations. This is why the Jersey implementation of JAX-RS Client API provides support for wrapping `URLConnection` and the Apache HTTP client. Thus it is possible to get the benefits of the established JAX-RS implementations and features while getting the ease of use benefit of the simple design of the JAX-RS client API. For example, with a low-level HTTP client library, sending a POST request with a bunch of typed HTML form parameters and receiving a response de-serialized into a JAXB bean is not straightforward at all. With the new JAX-RS Client API supported by Jersey this task is very easy:

### Example 5.1. POST request with form parameters

```
1 Client client = ClientBuilder.newClient();
2 WebTarget target = client.target("http://localhost:9998").path("resource");
3
4 Form form = new Form();
5 form.param("x", "foo");
6 form.param("y", "bar");
7
8 MyJAXBBean bean =
9 target.request(MediaType.APPLICATION_JSON_TYPE)
10    .post(Entity.entity(form, MediaType.APPLICATION_FORM_URLENCODED_TYPE),
11         MyJAXBBean.class);
```

In the Example 5.1, “POST request with form parameters” a new `WebTarget` instance is created using a new `Client` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>] instance first, next a `Form` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Form.html>] instance is created with two form parameters. Once ready, the `Form` instance is `POST`ed to the target resource. First, the acceptable media type is specified in the `request(...)` method. Then in the `post(...)` method, a call to a static method on JAX-RS Entity [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Entity.html>] is made to construct the request entity instance and attach the proper content media type to the form entity that is being sent. The second parameter in the `post(...)` method specifies the Java type of the response entity that should be returned from the method in case of a successful response. In this case an instance of JAXB bean is requested to be returned on success. The Jersey client API takes care of selecting the proper `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>] for the serialization of the `Form` instance, invoking the `POST` request and producing and de-serialization of the response message payload into an instance of a JAXB bean using a proper `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>].

If the code above had to be written using `URLConnection`, the developer would have to write custom code to serialize the form data that are sent within the `POST` request and de-serialize the response input stream into a JAXB bean. Additionally, more code would have to be written to make it easy to reuse the logic when communicating with the same resource “`http://localhost:8080/resource`” that is represented by the JAX-RS `WebTarget` instance in our example.

## 5.3. Overview of the Client API

### 5.3.1. Getting started with the client API

Refer to the dependencies for details on the dependencies when using the Jersey JAX-RS Client support.

You may also want to use a custom `Connector` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/Connector.html>] implementation. In such case you would need to include additional dependencies on the module(s) containing the custom client connector that you want to use. See section “Configuring custom Connectors” about how to use and configure a custom Jersey client transport `Connector`.

### 5.3.2. Creating and configuring a Client instance

JAX-RS Client API is designed to allow fluent programming model. This means, a construction of a `Client` instance, from which a `WebTarget` is created, from which a request `Invocation` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Invocation.html>] is built and invoked can be chained in a single “flow” of invocations. The individual steps of the flow will be shown in the following sections. To utilize the client API it is first necessary to build an instance of a `Client` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>] using one of the static `ClientBuilder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientBuilder.html>] factory methods. Here’s the most simple example:

```
Client client = ClientBuilder.newClient();
```

The `ClientBuilder` is a JAX-RS API used to create new instances of `Client`. In a slightly more advanced scenarios, `ClientBuilder` can be used to configure additional client instance properties, such as a SSL transport settings, if needed (see ??? below).

A `Client` instance can be configured during creation by passing a `ClientConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/>]

`client/ClientConfig.html`] to the `newClient(Configurable) ClientBuilder` factory method. `ClientConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientConfig.html>] implements `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] and therefore it offers methods to register providers (e.g. features or individual entity providers, filters or interceptors) and setup properties. The following code shows a registration of custom client filters:

```
1 ClientConfig clientConfig = new ClientConfig();
2 clientConfig.register(MyClientResponseFilter.class);
3 clientConfig.register(new AnotherClientFilter());
4 Client client = ClientBuilder.newClient(clientConfig);
```

In the example, filters are registered using the `ClientConfig.register(...)` method. There are multiple overloaded versions of the method that support registration of feature and provider classes or instances. Once a `ClientConfig` instance is configured, it can be passed to the `ClientBuilder` to create a pre-configured `Client` instance.

Note that the Jersey `ClientConfig` supports the fluent API model of `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>]. With that the code that configures a new client instance can be also written using a more compact style as shown below.

```
1
2 Client client = ClientBuilder.newClient(new ClientConfig()
3     .register(MyClientResponseFilter.class)
4     .register(new AnotherClientFilter()));
```

The ability to leverage this compact pattern is inherent to all JAX-RS and Jersey Client API components.

Since `Client` implements `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] interface too, it can be configured further even after it has been created. Important is to mention that any configuration change done on a `Client` instance will not influence the `ClientConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientConfig.html>] instance that was used to provide the initial `Client` instance configuration at the instance creation time. The next piece of code shows a configuration of an existing `Client` instance.

```
1 client.register(ThirdClientFilter.class);
```

Similarly to earlier examples, since `Client.register(...)` method supports the fluent API style, multiple client instance configuration calls can be chained:

```
1 client.register(FilterA.class)
2     .register(new FilterB())
3     .property("my-property", true);
```

To get the current configuration of the `Client` instance a `getConfiguration()` method can be used.

```
1 ClientConfig clientConfig = new ClientConfig();
2 clientConfig.register(MyClientResponseFilter.class);
3 clientConfig.register(new AnotherClientFilter());
4 Client client = ClientBuilder.newClient(clientConfig);
5 client.register(ThirdClientFilter.class);
6 Configuration newConfiguration = client.getConfiguration();
```

In the code, an additional `MyClientResponseFilter` class and `AnotherClientFilter` instance are registered in the `clientConfig`. The `clientConfig` is then used to construct a new `Client` instance. The `ThirdClientFilter` is added separately to the constructed `Client` instance. This

does not influence the configuration represented by the original `clientConfig`. In the last step a `newConfiguration` is retrieved from the `client`. This configuration contains all three registered filters while the original `clientConfig` instance still contains only two filters. Unlike `clientConfig` created separately, the `newConfiguration` retrieved from the `client` instance represents a live client configuration view. Any additional configuration changes made to the `client` instance are also reflected in the `newConfiguration`. So, `newConfiguration` is really a view of the `client` configuration and not a configuration state copy. These principles are important in the client API and will be used in the following sections too. For example, you can construct a common base configuration for all clients (in our case it would be `clientConfig`) and then reuse this common configuration instance to configure multiple `client` instances that can be further specialized. Similarly, you can use an existing `client` instance configuration to configure another client instance without having to worry about any side effects in the original `client` instance.

### 5.3.3. Targeting a web resource

Once you have a `Client` instance you can create a `WebTarget` from it.

```
1 WebTarget webTarget = client.target("http://example.com/rest");
```

A `Client` contains several `target(...)` methods that allow for creation of `WebTarget` instance. In this case we're using `target(String uri)` version. The `uri` passed to the method as a `String` is the URI of the targeted web resource. In more complex scenarios it could be the context root URI of the whole RESTful application, from which `WebTarget` instances representing individual resource targets can be derived and individually configured. This is possible, because JAX-RS `WebTarget` also implements `Configurable`:

```
1 WebTarget webTarget = client.target("http://example.com/rest");
2 webTarget.register(FilterForExampleCom.class);
```

The configuration principles used in JAX-RS client API apply to `WebTarget` as well. Each `WebTarget` instance inherits a configuration from its parent (either a client or another web target) and can be further custom-configured without affecting the configuration of the parent component. In this case, the `FilterForExampleCom` will be registered only in the `webTarget` and not in `client`. So, the `client` can still be used to create new `WebTarget` instances pointing at other URIs using just the common client configuration, which `FilterForExampleCom` filter is not part of.

### 5.3.4. Identifying resource on WebTarget

Let's assume we have a `webTarget` pointing at `"http://example.com/rest"` URI that represents a context root of a RESTful application and there is a resource exposed on the URI `"http://example.com/rest/resource"`. As already mentioned, a `WebTarget` instance can be used to derive other web targets. Use the following code to define a path to the resource.

```
1 WebTarget resourceWebTarget = webTarget.path("resource");
```

The `resourceWebTarget` now points to the resource on URI `"http://example.com/rest/resource"`. Again if we configure the `resourceWebTarget` with a filter specific to the resource, it will not influence the original `webTarget` instance. However, the filter `FilterForExampleCom` registration will still be inherited by the `resourceWebTarget` as it has been created from `webTarget`. This mechanism allows you to share the common configuration of related resources (typically hosted under the same URI root, in our case represented by the `webTarget` instance), while allowing for further configuration specialization based on the specific requirements of each individual resource. The same configuration principles of inheritance (to allow common config propagation) and decoupling (to allow individual config customization) applies to all components in JAX-RS Client API discussed below.



Let's say there is a sub resource on the path "<http://example.com/rest/resource/helloworld>". You can derive a `WebTarget` for this resource simply by:

```
1 WebTarget helloworldWebTarget = resourceWebTarget.path("helloworld");
```

Let's assume that the `helloworld` resource accepts a query param for GET requests which defines the greeting message. The next code snippet shows a code that creates a new `WebTarget` with the query param defined.

```
1 WebTarget helloworldWebTargetWithQueryParam =  
2     helloworldWebTarget.queryParam("greeting", "Hi World!");
```

Please note that apart from methods that can derive new `WebTarget` instance based on a URI path or query parameters, the JAX-RS `WebTarget` API contains also methods for working with matrix parameters too.

### 5.3.5. Invoking a HTTP request

Let's now focus on invoking a GET HTTP request on the created web targets. To start building a new HTTP request invocation, we need to create a new `Invocation.Builder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Invocation.Builder.html>].

```
1 Invocation.Builder invocationBuilder =  
2     helloworldWebTargetWithQueryParam.request(MediaType.TEXT_PLAIN_TYPE);  
3 invocationBuilder.header("some-header", "true");
```

A new invocation builder instance is created using one of the `request(...)` methods that are available on `WebTarget`. A couple of these methods accept parameters that let you define the media type of the representation requested to be returned from the resource. Here we are saying that we request a "`text/plain`" type. This tells Jersey to add a `Accept: text/plain` HTTP header to our request.

The `invocationBuilder` is used to setup request specific parameters. Here we can setup headers for the request or for example cookie parameters. In our example we set up a "`some-header`" header to value `true`.

Once finished with request customizations, it's time to invoke the request. We have two options now. We can use the `Invocation.Builder` to build a generic `Invocation` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Invocation.html>] instance that will be invoked some time later. Using `Invocation` we will be able to e.g. set additional request properties which are properties in a batch of several requests and use the generic JAX-RS `Invocation` API to invoke the batch of requests without actually knowing all the details (such as request HTTP method, configuration etc.). Any properties set on an invocation instance can be read during the request processing. For example, in a custom `ClientRequestFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientRequestFilter.html>] you can call `getProperty()` method on the supplied `ClientRequestContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientRequestContext.html>] to read a request property. Note that these request properties are different from the configuration properties set on `Configurable`. As mentioned earlier, an `Invocation` instance provides generic invocation API to invoke the HTTP request it represents either synchronously or asynchronously. See the Chapter 11, *Asynchronous Services and Clients* for more information on asynchronous invocations.

In case you do not want to do any batch processing on your HTTP request invocations prior to invoking them, there is another, more convenient approach that you can use to invoke your requests directly from an `Invocation.Builder` instance. This approach is demonstrated in the next Java code listing.

```
1 Response response = invocationBuilder.get();
```

While short, the code in the example performs multiple actions. First, it will build the request from the `invocationBuilder`. The URI of request will be `http://example.com/rest/resource/helloworld?greeting="Hi%20World!"` and the request will contain some-header: true and Accept: text/plain headers. The request will then pass through all configured request filters ( `AnotherClientFilter`, `ThirdClientFilter` and `FilterForExampleCom`). Once processed by the filters, the request will be sent to the remote resource. Let's say the resource then returns an HTTP 200 message with a plain text response content that contains the value sent in the request `greeting` query parameter. Now we can observe the returned response:

```
1 System.out.println(response.getStatus());
2 System.out.println(response.readEntity(String.class));
```

which will produce the following output to the console:

```
200
Hi World!
```

As we can see, the request was successfully processed (code 200) and returned an entity (representation) is "Hi World!". Note that since we have configured a `MyClientResponseFilter` in the resource target, when `response.readEntity(String.class)` gets called, the response returned from the remote endpoint is passed through the response filter chain (including the `MyClientResponseFilter`) and entity interceptor chain and at last a proper `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] is located to read the response content bytes from the response stream into a Java `String` instance. Check Chapter 10, *Filters and Interceptors* to learn more about request and response filters and entity interceptors.

Imagine now that you would like to invoke a POST request but without any query parameters. You would just use the `helloworldWebTarget` instance created earlier and call the `post()` instead of `get()`.

```
1 Response postResponse =
2     helloworldWebTarget.request(MediaType.TEXT_PLAIN_TYPE)
3     .post(Entity.entity("A string entity to be POSTed", MediaType.
```

## 5.3.6. Example summary

The following code puts together the pieces used in the earlier examples.

### Example 5.2. Using JAX-RS Client API

```
1 ClientConfig clientConfig = new ClientConfig();
2 clientConfig.register(MyClientResponseFilter.class);
3 clientConfig.register(new AnotherClientFilter());
4
5 Client client = ClientBuilder.newClient(clientConfig);
6 client.register(ThirdClientFilter.class);
7
8 WebTarget webTarget = client.target("http://example.com/rest");
9 webTarget.register(FilterForExampleCom.class);
10 WebTarget resourceWebTarget = webTarget.path("resource");
11 WebTarget helloworldWebTarget = resourceWebTarget.path("helloworld");
12 WebTarget helloworldWebTargetWithQueryParam =
13     helloworldWebTarget.queryParam("greeting", "Hi World!");
14
15 Invocation.Builder invocationBuilder =
```

```
16      helloworldWebTargetWithQueryParam.request(MediaType.TEXT_PLAIN_TYPE);
17 invocationBuilder.header("some-header", "true");
18
19 Response response = invocationBuilder.get();
20 System.out.println(response.getStatus());
21 System.out.println(response.readEntity(String.class));
```

Now we can try to leverage the fluent API style to write this code in a more compact way.

### Example 5.3. Using JAX-RS Client API fluently

```
1 Client client = ClientBuilder.newClient(new ClientConfig()
2     .register(MyClientResponseFilter.class)
3     .register(new AnotherClientFilter()));
4
5 String entity = client.target("http://example.com/rest")
6     .register(FilterForExampleCom.class)
7     .path("resource/helloworld")
8     .QueryParam("greeting", "Hi World!")
9     .request(MediaType.TEXT_PLAIN_TYPE)
10    .header("some-header", "true")
11    .get(String.class);
```

The code above does the same thing except it skips the generic `Response` processing and directly requests an entity in the last `get(String.class)` method call. This shortcut method let's you specify that (in case the response was returned successfully with a HTTP 2xx status code) the response entity should be returned as Java `String` type. This compact example demonstrates another advantage of the JAX-RS client API. The fluency of JAX-RS Client API is convenient especially with simple use cases. Here is another a very simple GET request returning a `String` representation (entity):

```
1 String responseEntity = ClientBuilder.newClient()
2     .target("http://example.com").path("resource/rest")
3     .request().get(String.class);
```

## 5.3.7. Setting `ExecutorService` and `ScheduledExecutorService`

Some client invocations, like asynchronous or reactive, could lead to a need to start a new thread. This is being done on provided `ExecutorService` or `ScheduledExecutorService`. `ClientBuilder` has two methods, which can be used to define them: `executorService(ExecutorService)` and `scheduledExecutorService(ScheduledExecutorService)`. When specified, all invocations which do require running on another thread, should be executed using provided services.

Default values do depend on the environment - in Java/Jakarta EE container, it has to be `ManagedExecutorService` [<https://jakarta.ee/specifications/jakarta.enterprise/concurrent/ManagedExecutorService.html>] and `ManagedScheduledExecutorService` [<https://jakarta.ee/specifications/jakarta.enterprise/concurrent/ManagedScheduledExecutorService.html>], for Java SE it would be `ForkJoinPool.commonPool` for `Executor` service and something undefined for `Scheduled` executor service.

### Example 5.4. Setting JAX-RS Client `ExecutorService`

```
1 ExecutorService myExecutorService = Executors.newCachedThreadPool();
```

```
2 Client client = ClientBuilder.newBuilder().executorService(myExecutorService).
```

## 5.4. Java instances and types for representations

All the Java types and representations supported by default on the Jersey server side for requests and responses are also supported on the client side. For example, to process a response entity (or representation) as a stream of bytes use `InputStream` as follows:

```
InputStream in = response.readEntity(InputStream.class);

... // Read from the stream

in.close();
```

Note that it is important to close the stream after processing so that resources are freed up.

To POST a file use a `File` instance as follows:

```
File f = ...

...

webTarget.request().post(Entity.entity(f, MediaType.TEXT_PLAIN_TYPE));
```

### 5.4.1. Adding support for new representations

The support for new application-defined representations as Java types requires the implementation of the same JAX-RS entity provider extension interfaces as for the server side JAX-RS API, namely `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] and `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>] respectively, for request and response entities (or inbound and outbound representations).

Classes or implementations of the provider-based interfaces need to be registered as providers within the JAX-RS or Jersey Client API components that implement `Configurable` contract (`ClientBuilder`, `Client`, `WebTarget` or `ClientConfig`), as was shown in the earlier sections. Some media types are provided in the form of JAX-RS Feature [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html>] a concept that allows the extension providers to group together multiple different extension providers and/or configuration properties in order to simplify the registration and configuration of the provided feature by the end users. For example, `MoxyJsonFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonFeature.html>] can be register to enable and configure JSON binding support via `MOXy` library.

## 5.5. Client Transport Connectors

By default, the transport layer in Jersey is provided by `HttpURLConnection`. This transport is implemented in Jersey via `HttpURLConnectionProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/HttpURLConnection.html>] that implements Jersey-specific `Connector` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/>]

jersey/client/spi/Connector.html] SPI. You can implement and/or register your own `Connector` instance to the Jersey `Client` implementation, that will replace the default `URLConnection`-based transport layer. Jersey provides several alternative client transport connector implementations that are ready-to-use.

**Table 5.1. List of Jersey Connectors**

| Transport framework       | Jersey Connector implementation                                                                                                                                                                                                                                                                                                             | Maven dependency                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Grizzly NIO framework     | GrizzlyConnectorProvider<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/grizzly/connector/GrizzlyConnectorProvider.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/grizzly/connector/GrizzlyConnectorProvider.html</a> ]     | org.glassfish.jersey.connectors:jersey-grizzly-connector |
| Apache 5 HTTP client      | Apache5ConnectorProvider<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/apache5/connector/Apache5ConnectorProvider.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/apache5/connector/Apache5ConnectorProvider.html</a> ]     | org.glassfish.jersey.connectors:jersey-apache5-connector |
| Helidon HTTP client       | HelidonConnectorProvider<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/helidon/connector/HelidonConnectorProvider.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/helidon/connector/HelidonConnectorProvider.html</a> ]     | org.glassfish.jersey.connectors:jersey-helidon-connector |
| Jetty HTTP client         | JettyConnectorProvider<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyConnectorProvider.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyConnectorProvider.html</a> ]               | org.glassfish.jersey.connectors:jersey-jetty-connector   |
| Netty NIO framework       | NettyConnectorProvider<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyConnectorProvider.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyConnectorProvider.html</a> ]               | org.glassfish.jersey.connectors:jersey-netty-connector   |
| JDK NIO client            | JdkConnectorProvider<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProvider.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProvider.html</a> ]                         | org.glassfish.jersey.connectors:jersey-jdk-connector     |
| Java java.net.http client | JavaNetHttpConnectorProvider<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpConnectorProvider.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpConnectorProvider.html</a> ] | org.glassfish.jersey.connectors:jersey-jnh-connector     |

As indicated earlier, `Connector` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/Connector.html>] and `ConnectorProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/ConnectorProvider.html>] contracts are Jersey-specific extension APIs that would only work with Jersey and as such are not part of JAX-RS. Following example shows how to setup the custom Grizzly Asynchronous HTTP Client based `ConnectorProvider` in a Jersey client instance:

```
1 ClientConfig clientConfig = new ClientConfig();
2         clientConfig.connectorProvider(new GrizzlyConnectorProvider())
3         Client client = ClientBuilder.newClient(clientConfig);
```

`Client` accepts as a constructor argument a `Configurable` instance. Jersey implementation of the `Configurable` provider for the client is `ClientConfig`. By using the Jersey `ClientConfig` you can configure the custom `ConnectorProvider` into the `ClientConfig`. The `GrizzlyConnectorProvider` is used as a custom connector provider in the example above. Please note that the connector provider cannot be registered as a provider using `Configurable.register(...)`. Also, please note that in this API has changed since Jersey 2.5, where the `ConnectorProvider` SPI has been introduced in order to decouple client initialization from the connector instantiation. Starting with Jersey 2.5 it is therefore not possible to directly register `Connector` instances in the Jersey `ClientConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientConfig.html>]. The new `ConnectorProvider` SPI must be used instead to configure a custom client-side transport connector.

A `ConnectorProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/ConnectorProvider.html>] can also be set by a property on a `ClientBuilder` starting with Jersey 2.40. The following example shows how to setup the custom Grizzly Asynchronous HTTP Client based `ConnectorProvider` in a Jersey client instance:

```
1 Client client = ClientBuilder.newBuilder()
2         .property(ClientProperties.CONNECTOR_PROVIDER, "org.glassfish.
3         .build();
```

For more information about the property see Appendix A, *Configuration Properties*.

## Warning

Be aware of using other than default `Connector` implementation. There is an issue handling HTTP headers in `WriterInterceptor` or `MessageBodyWriter<T>`. If you need to change header fields do not use neither `ApacheConnectorProvider` nor `GrizzlyConnectorProvider` nor `JettyConnectorProvider` (for asynchronous requests). Other older version connectors can be affected by this issue as well. The issue for example applies to Jersey Multipart feature that also modifies HTTP headers.

## 5.5.1. The default `HttpURLConnection`

The default connector is the most advanced connector, and it supports the most features Jersey has to offer. However, there are a few limitations coming from the `HttpURLConnection`

One limitation is in the variety of HTTP methods supported by the `HttpURLConnection`, since only the original HTTP/1.1 methods are supported. For instance, HTTP Patch method is not supported. See property `HttpUrlConnector.SET_METHOD_WORKAROUND` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/HttpUrlConnectorProvider.html#SET\\_METHOD\\_WORKAROUND](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/HttpUrlConnectorProvider.html#SET_METHOD_WORKAROUND)] in the Appendix ??? for a possible workaround.

Also, in the default transport connector, there are some restrictions on the headers, that can be sent in the default configuration. `URLConnectionProvider` uses `URLConnection` as an underlying connection implementation. This JDK class by default restricts the use of following headers:

- `Access-Control-Request-Headers`
- `Access-Control-Request-Method`
- `Connection` (with one exception - `Connection` header with value `Closed` is allowed by default)
- `Content-Length`
- `Content-Transfer-Encoding`
- `Host`
- `Keep-Alive`
- `Origin`
- `Trailer`
- `Transfer-Encoding`
- `Upgrade`
- `Via`
- all the headers starting with `Sec-`

The underlying connection can be configured to permit all headers to be sent, however this behaviour can be changed only by setting the system property `sun.net.http.allowRestrictedHeaders`.

### Example 5.5. Sending restricted headers with `URLConnection`

```
1
2         Client client = ClientBuilder.newClient();
3         System.setProperty("sun.net.http.allowRestrictedHeader
4
5         Response response = client.target(yourUri).path(yourPa
6         header("Origin", "http://example.com").
7         header("Access-Control-Request-Method", "POST").
8         get();
9
```

### Warning

Internally, the `URLConnection` instances are pooled, so (un)setting the property after already creating a target typically does not have any effect. The property influences all the connections *created* after the property has been (un)set, but there is no guarantee, that your request will use a connection created after the property change.

In a simple environment, setting the property before creating the first target is sufficient, but in complex environments (such as application servers), where some poolable connections might exist before your application even bootstraps, this approach is not 100% reliable and

we recommend using a different client transport connector, such as Apache Connector. These limitations have to be considered especially when invoking *CORS* (Cross Origin Resource Sharing) requests.

The limited configurability of the `URLConnection` is another aspect to consider. For details, see *Java Networking Properties*, for instance property `http.maxConnections`.

## 5.5.2. Client Connectors Properties

For each Connector [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/Connector.html>] a property file defining properties tweaking the actual Connector exists. There are *ClientProperties* [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html>] properties that apply to the default `HttpUrlConnectorProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/HttpUrlConnector.html>]. Many of the properties are also understood by various Connectors. Each of the *ConnectorProvider* defines properties that apply to it.

Moreover, each Connector [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/Connector.html>] supports a list of properties that are unique for the Connector [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/Connector.html>]. The list of the client connector properties can be found in the *Appendix A, Configuration Properties*.

## 5.5.3. Applying additional settings to Connectors

It is not possible to provide all the settings the underlying HTTP Clients support. For Apache HTTP Client, and for Jetty HTTP Client, it is possible to access directly the HTTP Client classes and invoke setter methods there.

### 5.5.3.1. Apache 5 HttpClientBuilder Configuration

For Apache 5 Connector, an *Apache5HttpClientBuilderConfigurator* [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/apache5/connector/Apache5HttpClientBuilderConfigurator.html>] SPI allows for invoking methods of `org.apache.hc.client5.http.impl.classic.HttpClientBuilder`, such as `setDefaultCredentialsProvider`:

```
1
2         org.apache.hc.client5.http.auth.CredentialsStore crede
3         credentialsProvider.setCredentials(
4             new org.apache.hc.client5.http.auth.AuthScope("loc
5             new org.apache.hc.client5.http.auth.UsernamePasswo
6         );
7         Apache5HttpClientBuilderConfigurator apache5HttpClient
8             return httpClientBuilder.setDefaultCredentialsProv
9         };
10
11         ClientConfig cc = new ClientConfig();
12         cc.register(apache5HttpClientBuilderConfigurator);
13         cc.connectorProvider(new Apache5ConnectorProvider());
14         Client client = ClientBuilder.newClient(cc);
15         ...
16
```



### 5.5.3.2. Jetty HttpClient Configuration

For Jetty Connector, an `JettyHttpClientSupplier` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyHttpClientSupplier.html>] SPI allows for providing a configured instance of `org.eclipse.jetty.client.HttpClient`:

```
1
2             HttpClient httpClient = new HttpClient(...);
3             ClientConfig clientConfig = new ClientConfig()
4                 .connectorProvider(new JettyConnectorProvider())
5                 .register(new JettyHttpClientSupplier(httpClient))
6             Client client = ClientBuilder.newClient(clientConfig);
7             ...
8
```

## 5.6. Using client request and response filters

Filtering requests and responses can provide useful lower-level concept focused on a certain independent aspect or domain that is decoupled from the application layer of building and sending requests, and processing responses. Filters can read/modify the request URI, headers and entity or read/modify the response status, headers and entity.

Jersey contains the following useful client-side filters (and features registering filters) that you may want to use in your applications:

`CsrfProtectionFilter` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/filter/CsrfProtectionFilter.html>]: Cross-site request forgery protection filter (adds `X-Requested-By` to each state changing request).

`EncodingFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/filter/EncodingFeature.html>]: Feature that registers encoding filter which use registered `ContentEncoder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/spi/ContentEncoder.html>]s to encode and decode the communication. The encoding/decoding is performed in interceptor (you don't need to register this interceptor). Check the javadoc of the `EncodingFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/filter/EncodingFeature.html>] in order to use it.

`HttpAuthenticationFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/authentication/HttpAuthenticationFeature.html>]: HTTP Authentication Feature (see authentication below).

Note that these features are provided by Jersey, but since they use and implement JAX-RS API, the features should be portable and run in any JAX-RS implementation, not just Jersey. See Chapter 10, *Filters and Interceptors* chapter for more information on filters and interceptors.

## 5.7. Closing connections

The underlying connections are opened for each request and closed after the response is received and entity is processed (entity is read). See the following example:

### Example 5.6. Closing connections

```
1 final WebTarget target = ... some web target
2 Response response = target.path("resource").request().get();
3 System.out.println("Connection is still open.");
4 System.out.println("string response: " + response.readEntity(String.class));
```

```
5 System.out.println("Now the connection is closed.");
```

If you don't read the entity, then you need to close the response manually by `response.close()`. Also if the entity is read into an `InputStream` [<http://docs.oracle.com/javase/6/docs/api/java/io/InputStream.html>] (by `response.readEntity(InputStream.class)`), the connection stays open until you finish reading from the `InputStream`. In that case, the `InputStream` or the `Response` should be closed manually at the end of reading from `InputStream`.

## 5.8. Injections into client providers

In some cases you might need to inject some custom types into your client provider instance. JAX-RS types do not need to be injected as they are passed as arguments into API methods. Injections into client providers (filters, interceptor) are possible as long as the provider is registered as a class. If the provider is registered as an instance then runtime will not inject the provider. The reason is that this provider instance might be registered into multiple client configurations. For example one instance of `ClientRequestFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientRequestFilter.html>] can be registered to two `Client` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>].

To solve injection of a custom type into a client provider instance use `InjectionManagerClientProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/InjectionManagerClientProvider.html>] to extract `InjectionManager` which can return the required injection. The following example shows how to utilize `InjectionManagerClientProvider`:

### Example 5.7. `InjectionManagerClientProvider` example

```
1 public static class MyRequestFilter implements ClientRequestFilter {
2     // this injection does not work as filter is registered as an instance:
3     // @Inject
4     // private MyInjectedService service;
5
6     @Override
7     public void filter(ClientRequestContext requestContext) throws IOException {
8         // use InjectionManagerClientProvider to extract InjectionManager from
9         final InjectionManager injectionManager = InjectionManagerClientProvid
10
11         // and ask for MyInjectedService:
12         final MyInjectedService service = injectionManager.getInstance(MyInjec
13
14         final String name = service.getName();
15         ...
16     }
17 }
```

For more information see javadoc of `InjectionManagerClientProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/InjectionManagerClientProvider.html>] (and javadoc of `InjectionManagerProvider` which supports common Jakarta REST components).

## 5.9. Securing a Client

This section describes how to setup SSL configuration on Jersey client (using JAX-RS API). The SSL configuration is setup in `ClientBuilder` [<https://jakartaee.github.io/rest/>]

`apidocs/4.0.0/jakarta/ws/rs/client/ClientBuilder.html`]. The client builder contains methods for definition of `KeyStore` [<http://docs.oracle.com/javase/6/docs/api/java/security/KeyStore.html>], `TrustStore` [<http://docs.oracle.com/javase/6/docs/api/java/security/TrustStore.html>] or entire `SslContext` [<http://docs.oracle.com/javase/6/docs/api/javax/net/ssl/SslContext.html>]. See the following example:

```
1 SslContext ssl = ... your configured SSL context;
2 Client client = ClientBuilder.newBuilder().sslContext(ssl).build();
3 Response response = client.target("https://example.com/resource").request().get();
```

The example above shows how to setup a custom `SslContext` to the `ClientBuilder`. Creating a `SslContext` can be more difficult as you might need to init instance properly with the protocol, `KeyStore`, `TrustStore`, etc. Jersey offers a utility `SslConfigurator` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/SslConfigurator.html>] class that can be used to setup the `SslContext`. The `SslConfigurator` can be configured based on standardized system properties for SSL configuration, so for example you can configure the `KeyStore` file name using a environment variable `javax.net.ssl.keyStore` and `SslConfigurator` will use such a variable to setup the `SslContext`. See javadoc of `SslConfigurator` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/SslConfigurator.html>] for more details. The following code shows how a `SslConfigurator` can be used to create a custom SSL context.

```
1 SslConfigurator sslConfig = SslConfigurator.newInstance()
2     .trustStoreFile("./truststore_client")
3     .trustStorePassword("secret-password-for-truststore")
4     .keyStoreFile("./keystore_client")
5     .keyPassword("secret-password-for-keystore");
6
7 SslContext sslContext = sslConfig.createSSLContext();
8 Client client = ClientBuilder.newBuilder().sslContext(sslContext).build();
```

Note that you can also setup `KeyStore` and `TrustStore` directly on a `ClientBuilder` instance without wrapping them into the `SslContext`. However, if you setup a `SslContext` it will override any previously defined `KeyStore` and `TrustStore` settings. `ClientBuilder` also offers a method for defining a custom `HostnameVerifier` [<http://docs.oracle.com/javase/6/docs/api/javax/net/ssl/HostnameVerifier.html>] implementation. `HostnameVerifier` implementations are invoked when default host URL verification fails.

## Important

A behaviour of `HostnameVerifier` [<http://docs.oracle.com/javase/6/docs/api/javax/net/ssl/HostnameVerifier.html>] is dependent on an http client implementation. `URLConnectionProvider` and `ApacheConnectorProvider` work properly, that means that after the unsuccessful URL verification `HostnameVerifier` is called and by means of it is possible to revalidate URL using a custom implementation of `HostnameVerifier` and go on in a handshake processing. `JettyConnectorProvider` and `GrizzlyConnectorProvider` provide only host URL verification and throw a `CertificateException` without any possibility to use custom `HostnameVerifier`. Moreover, in case of `JettyConnectorProvider` and `Jetty11ConnectorProvider` there are the properties `JettyClientProperties.ENABLE_SSL_HOSTNAME_VERIFICATION` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyClientProperties.html#ENABLE\\_SSL\\_HOSTNAME\\_VERIFICATION](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyClientProperties.html#ENABLE_SSL_HOSTNAME_VERIFICATION)] and `Jetty11ClientProperties.ENABLE_SSL_HOSTNAME_VERIFICATION` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty11/connector/Jetty11ClientProperties.html#ENABLE\\_SSL\\_HOSTNAME\\_VERIFICATION](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty11/connector/Jetty11ClientProperties.html#ENABLE_SSL_HOSTNAME_VERIFICATION)] to disable an entire host URL verification mechanism in a handshake.

## Important

Note that to utilize HTTP with SSL it is necessary to utilize the “https” scheme.

Currently the default connector provider `URLConnectionProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/URLConnectionProvider.html>] provides connectors based on `URLConnection` which implement support for SSL defined by JAX-RS configuration discussed in this example.

## 5.9.1. Http Authentication Support

Jersey supports Basic and Digest HTTP Authentication.

### Important

In version of Jersey 3.x both authentication methods are provided by single `Feature` `HttpAuthenticationFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/authentication/HttpAuthenticationFeature.html>]. For migration of older applications: `org.glassfish.jersey.client.filter.HttpBasicAuthFilter` and `org.glassfish.jersey.client.filter.HttpDigestAuthFilter` shall be replaced by those two authentication methods.

In order to enable http authentication support in Jersey client register the `HttpAuthenticationFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/authentication/HttpAuthenticationFeature.html>]. This feature can provide both authentication methods, digest and basic. Feature can work in the following modes:

- **BASIC**: Basic preemptive authentication. In preemptive mode the authentication information is send always with each HTTP request. This mode is more usual than the following non-preemptive mode (if you require BASIC authentication you will probably use this preemptive mode). This mode must be combined with usage of SSL/TLS as the password is send only BASE64 encoded.
- **BASIC NON-PREEMPTIVE**: Basic non-preemptive authentication. In non-preemptive mode the authentication information is added only when server refuses the request with 401 status code and then the request is repeated with authentication information. This mode has negative impact on the performance. The advantage is that it does not send credentials when they are not needed. This mode must be combined with usage of SSL/TLS as the password is send only BASE64 encoded.
- **DIGEST**: Http digest authentication. Does not require usage of SSL/TLS.
- **UNIVERSAL**: Combination of basic and digest authentication. The feature works in non-preemptive mode which means that it sends requests without authentication information. If 401 status code is returned, the request is repeated and an appropriate authentication is used based on the authentication requested in the response (defined in `WWW-Authenticate` HTTP header). The feature remembers which authentication requests were successful for given URI and next time tries to preemptively authenticate against this URI with latest successful authentication method.

To initialize the feature use static methods and builder of this feature. Example of building the feature in Basic authentication mode:

```
1 HttpAuthenticationFeature feature = HttpAuthenticationFeature.basic("user", "s
```

Example of building the feature in basic non-preemptive mode:

```
1 HttpAuthenticationFeature feature = HttpAuthenticationFeature.basicBuilder()  
2   .nonPreemptive().credentials("user", "superSecretPassword").build();
```

You can also build the feature without any default credentials:

```
1 HttpAuthenticationFeature feature = HttpAuthenticationFeature.basicBuilder().b
```

In this case you need to supply username and password for each request using request properties:

```
1 Response response = client.target("http://localhost:8080/rest/homer/contact").  
2   .property(HTTP_AUTHENTICATION_BASIC_USERNAME, "homer")  
3   .property(HTTP_AUTHENTICATION_BASIC_PASSWORD, "p1swd745").get();
```

This allows you to reuse the same client for authenticating with many different credentials.

See javadoc of the `HttpAuthenticationFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/authentication/HttpAuthenticationFeature.html>] for more details.

## 5.9.2. Server Name Indication (SNI) Support

When using SSL/TLS protocols for the connection, `SNIHostName` is set automatically based on the host name in the HTTPS request.

There might be use-cases where the `SNIHostName` is required to be set for other host than the host specified in the HTTPS request. For those cases, when the HTTP header `Host` is set, the `SNIHostName` is set for the host specified in the `Host` header. Note that only `Apache Connector`, `JDK Connector`, `Netty connector`, and the default `HttpURLConnection` do support this feature.

Sometimes, it may be required the `SNIHostName` is not set, or it differs from the HTTP `Host` header. In that case, the `ClientProperties.SNI_HOST_NAME` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#SNI\\_HOST\\_NAME](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#SNI_HOST_NAME)] property can be utilized. The property sets the host name to be used for calculating the `javax.net.ssl.SNIHostName`. The property takes precedence over the HTTP `Host` header.

When the host name in the the `ClientProperties.SNI_HOST_NAME` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#SNI\\_HOST\\_NAME](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#SNI_HOST_NAME)] property matches the HTTP request host, the `SNIHostName` is not set, and the HTTP `Host` header is not used for setting the `SNIHostName`. Turning the SNI off allows for Domain Fronting.

## 5.10. InvocationInterceptors

Suppose a case that the start of the request is to be logged and even measured. This can be done by `ClientRequestFilter`, which is usually invoked before the request is wired on the network. However, the filter may be called as a last of the filters in the chain. Sure, it can have the highest priority, but the other filters can have the very same priority! Some long-running operations can be performed before the measuring can actually start. Even worse, the filter may even be skipped from the chain by the previous `#abortWith!`

### 5.10.1. PreInvocationInterceptor

For this, `PreInvocationInterceptor`, the code that executes before the `ClientRequestFilters` are invoked, has been added to the client request chain. Jersey ensures all the

interceptors are invoked with each request. The interceptor contains a single `#beforeRequest` method, which corresponds to `ClientRequestFilter`:

```
/**
 * The method invoked before the request starts.
 * @param requestContext the request context shared with
 * ClientRequestFilter.
 */
void beforeRequest(ClientRequestContext requestContext);
```

Note that only a single `#abortWith` is allowed in all `PreInvocationInterceptors`, otherwise an `IllegalStateException` is thrown. All the exceptions accumulated in `PreInvocationInterceptors` are thrown in a single `Exception`, available through `#getSuppressed()`.

## 5.10.2. PostInvocationInterceptor

Similarly, `ClientResponseFilter` seems to be a good place where the total time of the HTTP request can be measured, but similarly to `ClientRequestFilter`, the response filter may not be invoked at all. For this, `PostInvocationInterceptor` has been introduced. Jersey runtime ensures that every `PostInvocationInterceptor` is called. Since an exception can occur during the HTTP request, `PostInvocationInterceptor` comes with two methods:

```
/**
 * The method is invoked after a request when no
 * is thrown, or the Throwables are resolved
 * by previous PostInvocationInterceptor.
 *
 * @param requestContext the request context.
 * @param responseContext the response context
 * of the original Response or response context
 * defined by the new resolving Response.
 */
void afterRequest(ClientRequestContext requestContext, ClientResponse response);

/**
 * The method is invoked after a Throwable is caught
 * during the client request chain processing.
 *
 * @param requestContext the request context.
 * @param exceptionContext the context available to handle the
 * caught Throwables.
 */
void onException(ClientRequestContext requestContext, ExceptionContext exceptionContext);
```

The `#afterRequest` method is executed when no exception has been thrown during the HTTP request, `#onException` method is executed if the exception has been thrown during the request. It is possible to set a response in `#onException`, and the consecutive `PostInvocationInterceptor` will execute its `#afterRequest` method. The measuring example can look as follows, then:

```
2         String response = ClientBuilder.newClient().target("path")
3             .register(new PreInvocationInterceptor() {
4                 @Override
5                 public void beforeRequest(ClientRequestContext requestContext) {
6                     startTime = System.currentTimeMillis();
7                 }
8             })
9             .register(new PostInvocationInterceptor() {
10                @Override
11                public void afterRequest(ClientRequestContext requestContext) {
12                    logDuration(System.currentTimeMillis() - startTime);
13                }
14                @Override
15                public void onException(ClientRequestContext requestContext, Throwable throwable) {
16                    logDuration(System.currentTimeMillis() - startTime);
17                }
18            })
19             .request().get().readEntity(String.class);
20
```

## 5.11. InvocationBuilderListener

InvocationBuilderListener is an interface that is inspired by Microprofile REST Client RestClientBuilderListener and it contains a single method:

```
/**
 * Whenever an Invocation.Builder is created, (i.e. when
 * WebTarget#request() is called, this method would be invoked.
 *
 * @param context the updated InvocationBuilderContext.
 */
void onNewBuilder(InvocationBuilderContext context);
```

InvocationBuilderContext a subset of methods of the Invocation.Builder. It can be used to call the default values of the Invocation.Builder. Since it is invoked at the time Invocation.Builder is instantiated, any consequent calls of the Invocation.Builder's methods will replace the defaults set by the InvocationBuilderListener. For instance, if all the HTTP requests should contain a custom HTTP header, there can be created a feature that would be registered on the client:

```
1
2     public static class MyFeature implements Feature {
3         @Override
4         public boolean configure(FeatureContext context) {
5             context.register(
6                 (InvocationBuilderListener)(l)->
7                 l.getHeaders().add("MY_HEADER", "MY_VALUE")
8             );
9             return true;
10        }
11    }
12
```

## 5.12. Header Expect:100-continue support

This section describes support of Expect:100-continue in Jersey client using Expect100Continue feature. Jersey client supports given header for default JDK HTTP connector only.

### *Jersey client Expect100Continue feature*

Since Jersey 2.32 it is possible to send Expect:100-continue header from Jersey client. Feature shall be registered in client using (for example)

```
1
2         target(RESOURCE_PATH).register(Expect100ContinueFeature.basic()
3
```

Note that registration can be done in several ways: with basic settings, and with custom settings:

```
1
2         target(RESOURCE_PATH).register(Expect100ContinueFeature.withCu
3
```

Basic registration means that default sending threshold will be used. Value of the default threshold is

```
1
2         DEFAULT_EXPECT_100_CONTINUE_THRESHOLD_SIZE = 65536L;
3
```

Threshold is used to determine allowed size of request after which 100-continue header shall be sent before sending request itself.

### *Environment properties configuration*

Previous paragraph described programmatic way of configuration. However the Expect100Continue feature can be configured using environment variables as well.

Since Jersey client can be influenced through environment variables, there are two variables which come since Jersey 2.32:

```
-Djersey.config.client.request.expect.100.continue.processing=true
-Djersey.config.client.request.expect.100.continue.threshold.size=
```

First variable can be used to forbid the Expect (100-continue) header be sent at all even though it is registered as described in the previous paragraph. If this property is not provided (or true) and the Expect100Continue feature is registered, sending of the Expect header is enabled.

The second property defines (or modifies) threshold size. So, if the Expect100Continue feature is registered using basic (default threshold size) parameters, value of the threshold can be modified using this property. This is valid for custom threshold as well - when the Expect100Continue feature is registered using withCustomThreshold method its value can be modified anyway by the environment property `jersey.config.client.request.expect.100.continue.threshold.size`.

### **Important**

In other words this variable has precedence over any programmatically set value of the threshold.



---

# Chapter 6. Reactive JAX-RS Client API

## Warning

Jersey 2.26 (JAX-RS 2.1 implementation) dropped Jersey-proprietary API in favor of JAX-RS 2.1 Reactive Client API. For Jersey 3.x this approach is still valid.

Reactive client extension is quite a generic API allowing end users to utilize the popular reactive programming model when using JAX-RS Client. The API is designed to be extensible, so any existing reactive framework can integrate with it and there is build in support for CompletionStage. Along with describing the API itself, this section also covers existing extension modules and provides hints to implement a custom extension if needed.

If you are not familiar with the JAX-RS Client API, it is recommended that you see Chapter 5, *Client API* where the basics of JAX-RS Client API along with some advanced techniques are described.

## 6.1. Motivation for Reactive Client Extension

### The Problem

Imagine a travel agency whose information system consists of multiple basic services. These services might be built using different technologies (JMS, EJB, WS, ...). For simplicity we presume that the services can be consumed using REST interface via HTTP method calls (e.g. using a JAX-RS Client). We also presume that the basic services we need to work with are:

- *Customers service* – provides information about customers of the travel agency.
- *Destinations service* – provides a list of visited and recommended destinations for an authenticated customer.
- *Weather service* – provides weather forecast for a given destination.
- *Quoting service* – provides price calculation for a customer to travel to a recommended destination.

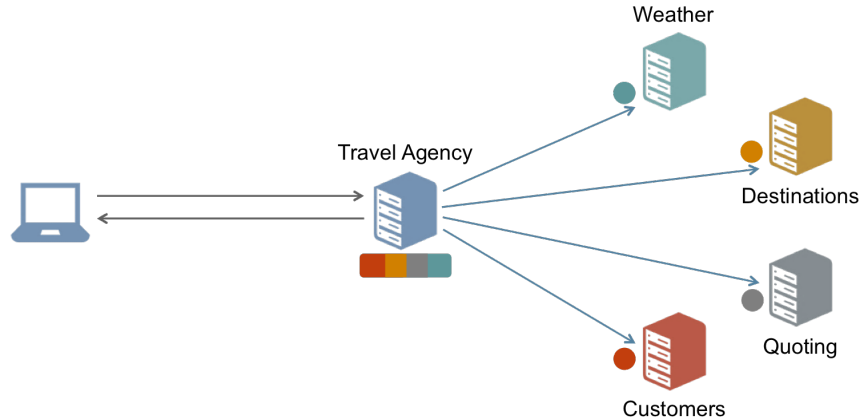
The task is to create a publicly available feature that would, for an authenticated user, display a list of 10 last visited places and also display a list of 10 new recommended destinations including weather forecast and price calculations for the user. Notice that some of the requests (to retrieve data) depend on results of previous requests. E.g. getting recommended destinations depends on obtaining information about the authenticated user first. Obtaining weather forecast depends on destination information, etc. This relationship between some of the requests is an important part of the problem and an area where you can take a real advantage of the reactive programming model.

One way how to obtain data is to make multiple HTTP method calls from the client (e.g. mobile device) to all services involved and combine the retrieved data on the client. However, since the basic services are available in the internal network only we'd rather create a public orchestration layer instead of exposing all internal services to the outside world. The orchestration layer would expose only the desired operations of the basic services to the public. To limit traffic and achieve lower latency we'd like to return all the necessary information to the client in a single response.

The orchestration layer is illustrated in the Figure 6.1. The layer accepts requests from the outside and is responsible of invoking multiple requests to the internal services. When responses from the internal

services are available in the orchestration layer they're combined into a single response that is sent back to the client.

**Figure 6.1. Travel Agency Orchestration Service**



The next sections describe various approaches (using JAX-RS Client) how the orchestration layer can be implemented.

## A Naive Approach

The simplest way to implement the orchestration layer is to use synchronous approach. For this purpose we can use JAX-RS Client Sync API (see Example 6.1, “Excerpt from a synchronous approach while implementing the orchestration layer”). The implementation is simple to do, easy to read and straightforward to debug.

### Example 6.1. Excerpt from a synchronous approach while implementing the orchestration layer

```

1 final WebTarget destination = ...;
2 final WebTarget forecast = ...;
3
4 // Obtain recommended destinations.
5 List<Destination> recommended = Collections.emptyList();
6 try {
7     recommended = destination.path("recommended").request()
8         // Identify the user.
9         .header("Rx-User", "Sync")
10        // Return a list of destinations.
11        .get(new GenericType<List<Destination>>() {});
12 } catch (final Throwable throwable) {
13     errors.offer("Recommended: " + throwable.getMessage());
14 }
15
16 // Forecasts. (depend on recommended destinations)
17 final Map<String, Forecast> forecasts = new HashMap<>();
18 for (final Destination dest : recommended) {
19     try {
20         forecasts.put(dest.getDestination(),

```

```

21         forecast.resolveTemplate("destination", dest.getDestination())
22     } catch (final Throwable throwable) {
23         errors.offer("Forecast: " + throwable.getMessage());
24     }
25 }

```

The downside of this approach is its slowness. You need to sequentially process all the independent requests which means that you're wasting resources. You are needlessly blocking threads, that could be otherwise used for some real work.

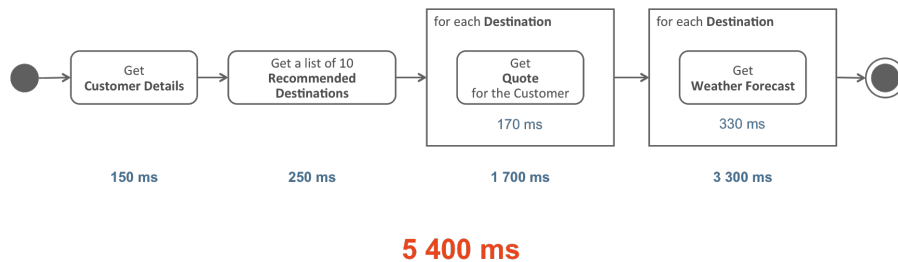
If you take a closer look at the example you can notice that at the moment when all the recommended destinations are available for further processing we try to obtain forecasts for these destinations. Obtaining a weather forecast can be done only for a single destination with a single request, so we need to make 10 requests to the *Forecast service* to get all the destinations covered. In a synchronous way this means getting the forecasts one-by-one. When one response with a forecast arrives we can send another request to obtain another one. This takes time. The whole process of constructing a response for the client can be seen in Figure 6.2.

Let's try to quantify this with assigning an approximate time to every request we make to the internal services. This way we can easily compute the time needed to complete a response for the client. For example, obtaining

- *Customer details* takes 150 ms
- *Recommended destinations* takes 250 ms
- *Price calculation for a customer and destination* takes 170 ms (each)
- *Weather forecast for a destination* takes 330 ms (each)

When summed up, 5400 ms is approximately needed to construct a response for the client.

**Figure 6.2. Time consumed to create a response for the client – synchronous way**



Synchronous approach is better to use for lower number of requests (where the accumulated time doesn't matter that much) or for a single request that depends on the result of previous operations.

## Optimized Approach

The amount of time needed by the synchronous approach can be lowered by invoking independent requests in parallel. We're going to use JAX-RS Client Async API to illustrate this approach. The implementation in this case is slightly more difficult to get right because of the nested callbacks and the need to wait at some points for the moment when all partial responses are ready to be processed. The implementation is also a little bit harder to debug and maintain. The nested calls are causing a lot of complexity here. An

example of concrete Java code following the asynchronous approach can be seen in Example 6.2, “Excerpt from an asynchronous approach while implementing the orchestration layer”.

**Example 6.2. Excerpt from an asynchronous approach while implementing the orchestration layer**

```
1 final WebTarget destination = ...;
2 final WebTarget forecast = ...;
3
4 // Obtain recommended destinations. (does not depend on visited ones)
5 destination.path("recommended").request()
6     // Identify the user.
7     .header("Rx-User", "Async")
8     // Async invoker.
9     .async()
10    // Return a list of destinations.
11    .get(new InvocationCallback<List<Destination>>() {
12        @Override
13        public void completed(final List<Destination> recommended) {
14            final CountDownLatch innerLatch = new CountDownLatch(recommend
15
16            // Forecasts. (depend on recommended destinations)
17            final Map<String, Forecast> forecasts = Collections.synchroniz
18            for (final Destination dest : recommended) {
19                forecast.resolveTemplate("destination", dest.getDestinatio
20                    .async()
21                    .get(new InvocationCallback<Forecast>() {
22                        @Override
23                        public void completed(final Forecast forecast) {
24                            forecasts.put(dest.getDestination(), forec
25                            innerLatch.countDown();
26                        }
27
28                        @Override
29                        public void failed(final Throwable throwable) {
30                            errors.offer("Forecast: " + throwable.getM
31                            innerLatch.countDown();
32                        }
33                    });
34            }
35
36            // Have to wait here for dependent requests ...
37            try {
38                if (!innerLatch.await(10, TimeUnit.SECONDS)) {
39                    errors.offer("Inner: Waiting for requests to complete
40                }
41            } catch (final InterruptedException e) {
42                errors.offer("Inner: Waiting for requests to complete has
43            }
44
45            // Continue with processing.
46        }
47
48        @Override
```

```

49         public void failed(final Throwable throwable) {
50             errors.offer("Recommended: " + throwable.getMessage());
51         }
52     });

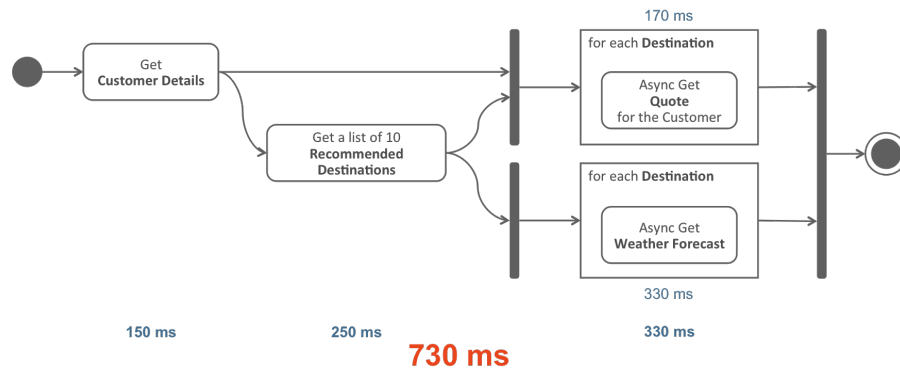
```

The example is a bit more complicated from the first glance. We provided an `InvocationCallback` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/InvocationCallback.html] to `async get` method. One of the callback methods (`completed` or `failed`) is called when the request finishes. This is a pretty convenient way to handle async invocations when no nested calls are present. Since we have some nested calls (obtaining weather forecasts) we needed to introduce a `CountDownLatch` [http://docs.oracle.com/javase/6/docs/api/java/util/concurrent/CountDownLatch.html] synchronization primitive as we use asynchronous approach in obtaining the weather forecasts as well. The latch is decreased every time a request, to the *Forecasts service*, completes successfully or fails. This indicates that the request actually finished and it is a signal for us that we can continue with processing (otherwise we wouldn't have all required data to construct the response for the client). This additional synchronization is something that was not present when taking the synchronous approach, but it is needed here.

Also the error processing can not be written as it could be in an ideal case. The error handling is scattered in too many places within the code, that it is quite difficult to create a comprehensive response for the client.

On the other hand taking asynchronous approach leads to code that is as fast as it gets. The resources are used optimally (no waiting threads) to achieve quick response time. The whole process of constructing the response for the client can be seen in Figure 6.3. It only took 730 ms instead of 5400 ms which we encountered in the previous approach.

**Figure 6.3. Time consumed to create a response for the client – asynchronous way**



As you can guess, this approach, even with all its benefits, is the one that is really hard to implement, debug and maintain. It's a safe bet when you have many independent calls to make but it gets uglier with an increasing number of nested calls.

## Reactive Approach

Reactive approach is a way out of the so-called *Callback Hell* which you can encounter when dealing with Java's `Futures` or invocation callbacks. Reactive approach is based on a data-flow concept and the execution model propagate changes through the flow. An example of a single item in the data-flow chain can be a JAX-RS Client HTTP method call. When the JAX-RS request finishes then the next item (or the user code) in the data-flow chain is notified about the continuation, completion or error in the chain. You're more describing what should be done next than how the next action in the chain should be triggered.

The other important part here is that the data-flows are composable. You can compose/transform multiple flows into the resulting one and apply more operations on the result.

An example of this approach can be seen in Example 6.3, “Excerpt from a reactive approach while implementing the orchestration layer”. The APIs would be described in more detail in the next sections.

**Example 6.3. Excerpt from a reactive approach while implementing the orchestration layer**

```
1 final WebTarget destination = ...;
2 final WebTarget forecast = ...;
3
4 // Recommended places.
5 CompletionStage<List<Destination>> recommended =
6     destination.path("recommended")
7         .request()
8         // Identify the user.
9         .header("Rx-User", "CompletionStage")
10        // Reactive invoker.
11        .rx()
12        // Return a list of destinations.
13        .get(new GenericType<List<Destination>>() {})
14        .exceptionally(throwable -> {
15            errors.offer("Recommended: " + throwable.getMessage());
16            return Collections.emptyList();
17        });
18
19 // get Forecast for recommended destinations.
20 return recommended.thenCompose(destinations -> {
21
22     List<CompletionStage<Recommendation>> recommendations = destinations.stream()
23         // For each destination, obtain a weather forecast ...
24         .map(destination -> {
25             final CompletionStage<Forecast> forecastResult =
26                 forecast.resolveTemplate("destination", destination.getDestination())
27                     .request().rx().get(Forecast.class)
28                     .exceptionally(throwable -> {
29                         errors.offer("Forecast: " + throwable.getMessage());
30                         return new Forecast(destination.getDestination(), "N/A");
31                     });
32
33             //noinspection unchecked
34             return CompletableFuture.completedFuture(new Recommendation(destination, forecastResult));
35         })
36         .collect(Collectors.toList());
37
38     // Transform List<CompletionStage<Recommendation>> to CompletionStage<List<Recommendation>>
39     return sequence(recommendations);
40 });
```

As you can see the code achieves the same work as the previous two examples. It's more readable than the pure asynchronous approach even though it's equally fast. It's as easy to read and implement as the synchronous approach. The error processing is also better handled in this way than in the asynchronous approach.

When dealing with a large amount of requests (that depend on each other) and when you need to compose/combine the results of these requests, the reactive programming model is the right technique to use.

## 6.2. Usage and Extension Modules

Reactive Client API is part of the JAX-RS specification since version 2.1.

When you compare synchronous invocation of HTTP calls ( Example 6.4, “Synchronous invocation of HTTP requests”)

### Example 6.4. Synchronous invocation of HTTP requests

```
1 Response response = ClientBuilder.newClient()  
2     .target("http://example.com/resource")  
3     .request()  
4     .get();
```

with asynchronous invocation (Example 6.5, “Asynchronous invocation of HTTP requests”)

### Example 6.5. Asynchronous invocation of HTTP requests

```
1 Future<Response> response = ClientBuilder.newClient()  
2     .target("http://example.com/resource")  
3     .request()  
4     .async()  
5     .get();
```

it is apparent how to pretty conveniently modify the way how a request is invoked (from sync to async) only by calling `async` method on an `Invocation.Builder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Invocation.Builder.html>].

Naturally, it'd be nice to copy the same pattern to allow invoking requests in a reactive way. Just instead of `async` you'd call `rx` on an extension of `Invocation.Builder`, like in Example 6.6, “Reactive invocation of HTTP requests”.

### Example 6.6. Reactive invocation of HTTP requests

```
1 CompletionStage<Response> response = ClientBuilder.newClient()  
2     .target("http://example.com/resource")  
3     .request()  
4     .rx()  
5     .get();
```

The first reactive interface in the invocation chain is `RxInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvoker.html>] which is very similar to `SyncInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/SyncInvoker.html>] and `AsyncInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/AsyncInvoker.html>]. It contains all methods present in the two latter JAX-RS interfaces but the `RxInvoker` interface is more generic, so that it can be extended and used in particular implementations taking advantage of various reactive libraries. Extending this new interface in a particular implementation also preserves type safety which means that you're not losing type information when a HTTP method call returns an object that you want to process further.

The method `"rx()"` in the example above is perfect example of that principle. It returns `CompletionStageRxInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/>]

`client/CompletionStageRxInvoker.html`], which extends `RxInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvoker.html>].

As a user of the Reactive Client API you only need to keep in mind that you won't be working with `RxInvoker` directly. You'd rather be working with an extension of this interface created for a particular implementation and you don't need to be bothered much with why are things designed the way they are.

## Note

To see how the `RxInvoker` should be extended, refer to Section 6.4, “Implementing Support for Custom Reactive Libraries (SPI)”.

The important thing to notice here is that an extension of `RxInvoker` holds the type information and the Reactive Client needs to know about this type to properly propagate it among the method calls you'll be making. This is the reason why other interfaces (described below) are parametrized with this type.

In order to extend the API to be used with other reactive frameworks, `RxInvokerProvider` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvokerProvider.html>] needs to be registered into the Client runtime:

```
1 Client client = ClientBuilder.newClient();
2 client.register(RxFlowableInvokerProvider.class);
3
4 Flowable<String> responseFlowable =
5     client.target("http://eclipse-ee4j.github.io/jersey")
6         .request()
7         .rx(RxFlowableInvoker.class)
8         .get(String.class);
9
10 String responseString = responseFlowable.blockingFirst();
```

## Dependencies

JAX-RS mandates support for `CompletionStage`, which doesn't required any other dependency and can be used out of the box.

To add support for a particular library, see the Section 6.3, “Supported Reactive Libraries”.

## Note

If you're not using Maven (or other dependency management tool) make sure to add also all the transitive dependencies of Jersey client module and any other extensions (when used) on the class-path.

## 6.3. Supported Reactive Libraries

There are already some available reactive (or reactive-like) libraries out there and Jersey brings support for some of them out of the box. Jersey currently supports:

- RxJava (Observable)
- RxJava (Flowable)
- Guava (ListenableFuture and Futures)



### 6.3.1. RxJava – Observable

RxJava [<https://github.com/ReactiveX/RxJava>], contributed by Netflix, is probably the most advanced reactive library for Java at the moment. It's used for composing asynchronous and event-based programs by using observable sequences. It uses the observer pattern [[http://en.wikipedia.org/wiki/Observer\\_pattern](http://en.wikipedia.org/wiki/Observer_pattern)] to support these sequences of data/events via its Observable [<http://reactivex.io/RxJava/javadoc/io/reactivex/Observable.html>] entry point class which implements the Reactive Pattern. **Observable** is actually the parameter type in the RxJava's extension of RxInvoker [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvoker.html>], called RxObservableInvoker [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/rx/rxjava/RxObservableInvoker.html>]. This means that the return type of HTTP method calls is **Observable** in this case (accordingly parametrized).

Requests are by default invoked at the moment when a subscriber is subscribed to an observable (it's a cold **Observable**). If not said otherwise a separate thread (JAX-RS Async Client requests) is used to obtain data. This behavior can be overridden by providing an `ExecutorService` [<http://docs.oracle.com/javase/6/docs/api/java/util/concurrent/ExecutorService.html>] when a reactive **Client** is created.

#### Usage

The extensibility is built-in JAX-RS Client API, so there are no special dependencies on Jersey Client API other than the extension itself.

#### Example 6.7. Creating JAX-RS Client with RxJava reactive extension

```
1 // New Client
2 Client client = ClientBuilder.newClient();
3 client.register(RxObservableInvokerProvider.class);
```

An example of obtaining **Observable** with JAX-RS **Response** from a remote service can be seen in Example 6.8, “Obtaining **Observable<Response>** from Jersey/RxJava Client”.

#### Example 6.8. Obtaining **Observable<Response>** from Jersey/RxJava Client

```
1 Observable<Response> observable = RxObservable.newClient()
2                                     .target("http://example.co
3                                     .request()
4                                     .rx(RxObservableInvoker.cl
5                                     .get();
```

#### Dependencies

The RxJava support is available as an extension module in Jersey. For Maven users, simply add the following dependency to your `pom.xml`:

```
<dependency>
  <groupId>org.glassfish.jersey.ext.rx</groupId>
  <artifactId>jersey-rx-client-rxjava</artifactId>
  <version>4.0.0</version>
</dependency>
```

After this step you can use the extended client right away. The dependency transitively adds the following dependencies to your class-path as well: `io.reactivex:rxjava`.

## Note

If you're not using Maven (or other dependency management tool) make sure to add also all the transitive dependencies of this extension module (see `jersey-rx-client-rxjava` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-rxjava/dependencies.html>]) on the class-path.

## 6.3.2. RxJava – Flowable

RxJava [<https://github.com/ReactiveX/RxJava>], contributed by Netflix, is probably the most advanced reactive library for Java at the moment. It's used for composing asynchronous and event-based programs by using observable sequences. It uses the observer pattern [[http://en.wikipedia.org/wiki/Observer\\_pattern](http://en.wikipedia.org/wiki/Observer_pattern)] to support these sequences of data/events via its `Flowable` [<http://reactivex.io/RxJava/javadoc/io/reactivex/Flowable.html>] entry point class which implements the Reactive Pattern. `Flowable` is actually the parameter type in the RxJava's extension of `RxInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvoker.html>], called `RxFlowableInvoker` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/rx/rxjava2/RxFlowableInvoker.html>]. This means that the return type of HTTP method calls is `Flowable` in this case (accordingly parametrized).

Requests are by default invoked at the moment when a subscriber is subscribed to a flowable (it's a cold `Flowable`). If not said otherwise a separate thread (JAX-RS Async Client requests) is used to obtain data. This behavior can be overridden by providing an `ExecutorService` [<http://docs.oracle.com/javase/6/docs/api/java/util/concurrent/ExecutorService.html>] when a reactive `Client` is created.

## Usage

The extensibility is built-in JAX-RS Client API, so there are no special dependencies on Jersey Client API other than the extension itself.

### Example 6.9. Creating JAX-RS Client with RxJava2 reactive extension

```
1 // New Client
2 Client client = ClientBuilder.newClient();
3 client.register(RxFlowableInvokerProvider.class);
```

An example of obtaining `Flowable` with JAX-RS `Response` from a remote service can be seen in Example 6.8, “Obtaining `Observable<Response>` from Jersey/RxJava Client”.

### Example 6.10. Obtaining `Flowable<Response>` from Jersey/RxJava Client

```
1 Flowable<Response> observable = RxObservable.newClient()
2                               .target("http://example.com/resource")
3                               .request()
4                               .rx(RxFlowableInvoker.class)
5                               .get();
6
```

## Dependencies

The RxJava support is available as an extension module in Jersey. For Maven users, simply add the following dependency to your `pom.xml`:

```
<dependency>
```

```
<groupId>org.glassfish.jersey.ext.rx</groupId>
<artifactId>jersey-rx-client-rxjava2</artifactId>
<version>4.0.0</version>
</dependency>
```

After this step you can use the extended client right away. The dependency transitively adds the following dependencies to your class-path as well: `io.reactivex:rxjava2`.

### Note

If you're not using Maven (or other dependency management tool) make sure to add also all the transitive dependencies of this extension module (see `jersey-rx-client-rxjava2` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-rxjava2/dependencies.html>]) on the class-path.

## 6.3.3. Guava – `ListenableFuture` and `Futures`

Guava [<https://guava.dev/>], contributed by Google, also contains a type, `ListenableFuture` [<https://guava.dev/releases/33.4.8-jre/api/docs/com/google/common/util/concurrent/ListenableFuture.html>], which can be decorated with listeners that are notified when the future completes. The `ListenableFuture` can be combined with `Futures` [<https://guava.dev/releases/33.4.8-jre/api/docs/com/google/common/util/concurrent/Futures.html>] to achieve asynchronous/event-based completion aware processing. `ListenableFuture` is the parameter type in the Guava's extension of `RxInvoker`, called `RxListenableFutureInvoker` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/rx/guava/RxListenableFutureInvoker.html>]. This means that the return type of HTTP method calls is `ListenableFuture` in this case (accordingly parametrized).

Requests are by default invoked immediately. If not said otherwise the `Executors#newCachedThreadPool()` [<http://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Executors.html#newCachedThreadPool-->] pool is used to obtain a thread which processed the request. This behavior can be overridden by providing a `ExecutorService` [<http://docs.oracle.com/javase/6/docs/api/java/util/concurrent/ExecutorService.html>] when a `Client` is created.

## Usage

The extensibility is built-in JAX-RS Client API, so there are no special dependencies on Jersey Client API other than the extension itself.

### Example 6.11. Creating Jersey/Guava Client

```
1 // New Client
2 Client client = ClientBuilder.newClient();
3 client.register(RxListenableFutureInvokerProvider.class);
```

An example of obtaining `ListenableFuture` with JAX-RS `Response` from a remote service can be seen in Example 6.12, “Obtaining `ListenableFuture<Response>` from Jersey/Guava Client”.

### Example 6.12. Obtaining `ListenableFuture<Response>` from Jersey/Guava Client

```
1
2 ListenableFuture<Response> response = client.target("http://eclipse-ee4j.githubu
3                                     .request()
4                                     .rx(RxListenableFutureInvoker.class)
5                                     .get();
```

## Dependencies

The Reactive Jersey Client with Guava support is available as an extension module in Jersey. For Maven users, simply add the following dependency to your `pom.xml`:

```
<dependency>
  <groupId>org.glassfish.jersey.ext.rx</groupId>
  <artifactId>jersey-rx-client-guava</artifactId>
  <version>4.0.0</version>
</dependency>
```

After this step you can use the extended client right away. The dependency transitively adds the following dependencies to your class-path as well: `com.google.guava:guava`.

### Note

If you're not using Maven (or other dependency management tool) make sure to add also all the transitive dependencies of this extension module (see `jersey-rx-client-guava` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/project/jersey-rx-client-guava/dependencies.html>]) on the class-path.

## 6.4. Implementing Support for Custom Reactive Libraries (SPI)

In case you want to bring support for some other library providing Reactive Programming Model into your application you can extend functionality of Reactive JAX-RS Client by implementing `RxInvokerProvider` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvokerProvider.html>], registering that implementation into the client runtime and then using `rx(Class<T>)` in your code.

### Implement `RxInvoker` and `RxInvokerProvider` interfaces

The first step when implementing support for another reactive library is to implement `RxInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvoker.html>]. JAX-RS API itself contains one implementation, which will be used as an example: `CompletionStageRxInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/CompletionStageRxInvoker.html>].

#### Example 6.13. Extending `RxInvoker`

```
1 public interface CompletionStageRxInvoker extends RxInvoker<CompletionStage> {
2     @Override
3     public CompletionStage<Response> get();
4
5     @Override
6     public <T> CompletionStage<T> get(Class<T> responseType);
7
8     // ...
9 }
```

The important fact to notice is that the generic parameter of `RxInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvoker.html>] is `CompletionStage` [<http://docs.oracle.com/javase/8/>]

[docs/api/java/util/concurrent/CompletionStage.html](https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/CompletionStage.html)] and also that the return type is overridden to be always `CompletionStage` [<http://docs.oracle.com/javase/8/docs/api/java/util/concurrent/CompletionStage.html>] with some generic param (Response [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>]; or T).

After having the extended `RxInvoker` interface, the implementor has to provide `RxInvokerProvider` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/RxInvokerProvider.html>], which will be registered as an provider to a client instance.

### Example 6.14. Extending `RxInvokerProvider`

```
1 public static class CompletionStageRxInvokerProvider implements RxInvokerProvi
2     @Override
3     public boolean isProviderFor(Class<?> clazz) {
4         return CompletionStage.class.equals(clazz);
5     }
6
7     @Override
8     public CompletionStageRxInvoker getRxInvoker(SyncInvoker syncInvoker, Exec
9         return new CompletionStageRxInvoker() {
10             // ...
11         };
12     }
13 }
```

## Example of using custom `RxInvokerProvider`

Considering the work above was done and the implementation of custom `RxInvoker` and `RxInvokerProvider` is available, the client code using those extensions will be:

```
1 Client client = ClientBuilder.newClient();
2 // register custom RxInvokerProvider
3 client.register(CompletionStageRxInvokerProvider.class);
4
5 CompletionStage<Response> response =
6     client.target("http://eclipse-ee4j.github.io/jersey")
7         .request()
8         .rx(CompletionStageRxInvoker.class)
9         // Now we have an instance of CompletionStageRxInvoker returned
10        // which is CompletionStageRxInvokerProvider in this particular
11        .get();
```

---

# Chapter 7. Representations and Responses

## 7.1. Representations and Java Types

Previous sections on `@Produces` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html>] and `@Consumes` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Consumes.html>] annotations referred to media type of an entity representation. Examples above depicted resource methods that could consume and/or produce String Java type for a number of different media types. This approach is easy to understand and relatively straightforward when applied to simple use cases.

To cover also other cases, handling non-textual data for example or handling data stored in the file system, etc., JAX-RS implementations are required to support also other kinds of media type conversions where additional, non-String, Java types are being utilized. Following is a short listing of the Java types that are supported out of the box with respect to supported media type:

- All media types (`/*/*`)
  - `byte[]`
  - `java.lang.String`
  - `java.io.Reader` (inbound only)
  - `java.io.File`
  - `jakarta.activation.DataSource`
  - `jakarta.ws.rs.core.StreamingOutput` (outbound only)
- XML media types (`text/xml`, `application/xml` and `application/...+xml`)
  - `javax.xml.transform.Source`
  - `jakarta.xml.bind.JAXBElement`
  - Application supplied JAXB classes (types annotated with `@XmlRootElement` [<http://docs.oracle.com/javase/6/docs/api/jakarta.xml.bind/annotation/XmlRootElement.html>] or `@XmlType` [<http://docs.oracle.com/javase/6/docs/api/jakarta.xml.bind/annotation/XmlType.html>])
- Form content (`application/x-www-form-urlencoded`)
  - `MultivaluedMap<String,String>`
- Plain text (`text/plain`)
  - `java.lang.Boolean`
  - `java.lang.Character`
  - `java.lang.Number`

Unlike method parameters that are associated with the extraction of request parameters, the method parameter associated with the representation being consumed does not require annotating. In other words the representation (entity) parameter does not require a specific 'entity' annotation. A method parameter without an annotation is an entity. A maximum of one such unannotated method parameter may exist since there may only be a maximum of one such representation sent in a request.

The representation being produced corresponds to what is returned by the resource method. For example JAX-RS makes it simple to produce images that are instance of `File` as follows:

### Example 7.1. Using `File` with a specific media type to produce a response

```
1 @GET
2 @Path("/images/{image}")
3 @Produces("image/*")
4 public Response getImage(@PathParam("image") String image) {
5     File f = new File(image);
6
7     if (!f.exists()) {
8         throw new WebApplicationException(404);
9     }
10
11     String mt = new MimetypesFileTypeMap().getContentType(f);
12     return Response.ok(f, mt).build();
13 }
```

The `File` type can also be used when consuming a representation (request entity). In that case a temporary file will be created from the incoming request entity and passed as a parameter to the resource method.

The `Content-Type` response header (if not set programmatically as described in the next section) will be automatically set based on the media types declared by `@Produces` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html>] annotation. Given the following method, the most acceptable media type is used when multiple output media types are allowed:

```
1 @GET
2 @Produces({"application/xml", "application/json"})
3 public String doGetAsXmlOrJson() {
4     ...
5 }
```

If `application/xml` is the most acceptable media type defined by the request (e.g. by header `Accept: application/xml`), then the `Content-Type` response header will be set to `application/xml`.

## 7.2. Building Responses

Sometimes it is necessary to return additional information in response to a HTTP request. Such information may be built and returned using `Response` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>] and `Response.ResponseBuilder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.ResponseBuilder.html>]. For example, a common RESTful pattern for the creation of a new resource is to support a POST request that returns a 201 (Created) status code and a `Location` header whose value is the URI to the newly created resource. This may be achieved as follows:

**Example 7.2. Returning 201 status code and adding Location header in response to POST request**

```
1 @POST
2 @Consumes("application/xml")
3 public Response post(String content) {
4     URI createdUri = ...
5     create(content);
6     return Response.created(createdUri).build();
7 }
```

In the above no representation produced is returned, this can be achieved by building an entity as part of the response as follows:

**Example 7.3. Adding an entity body to a custom response**

```
1 @POST
2 @Consumes("application/xml")
3 public Response post(String content) {
4     URI createdUri = ...
5     String createdContent = create(content);
6     return Response.created(createdUri).entity(Entity.text(createdContent)).build();
7 }
```

Response building provides other functionality such as setting the entity tag and last modified date of the representation.

## 7.3. WebApplicationException and Mapping Exceptions to Responses

Previous section shows how to return HTTP responses, that are built up programmatically. It is possible to use the very same mechanism to return HTTP errors directly, e.g. when handling exceptions in a try-catch block. However, to better align with the Java programming model, JAX-RS allows to define direct mapping of Java exceptions to HTTP error responses.

The following example shows throwing `CustomNotFoundException` from a resource method in order to return an error HTTP response to the client:

**Example 7.4. Throwing exceptions to control response**

```
1 @Path("items/{itemid}/")
2 public Item getItem(@PathParam("itemid") String itemid) {
3     Item i =.getItems().get(itemid);
4     if (i == null) {
5         throw new CustomNotFoundException("Item, " + itemid + ", is not found");
6     }
7
8     return i;
9 }
```

This exception is an application specific exception that extends `WebApplicationException` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/WebApplicationException.html>] and builds a HTTP response with the 404 status code and an optional message as the body of the response:



**Example 7.5. Application specific exception implementation**

```
1 public class CustomNotFoundException extends WebApplicationException {
2
3     /**
4      * Create a HTTP 404 (Not Found) exception.
5      */
6     public CustomNotFoundException() {
7         super(Responses.notFound().build());
8     }
9
10    /**
11     * Create a HTTP 404 (Not Found) exception.
12     * @param message the String that is the entity of the 404 response.
13     */
14    public CustomNotFoundException(String message) {
15        super(Response.status(Responses.NOT_FOUND).
16            entity(message).type("text/plain").build());
17    }
18 }
```

In other cases it may not be appropriate to throw instances of `WebApplicationException` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/WebApplicationException.html>], or classes that extend `WebApplicationException` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/WebApplicationException.html>], and instead it may be preferable to map an existing exception to a response. For such cases it is possible to use a custom exception mapping provider. The provider must implement the `ExceptionMapper<E extends Throwable>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ExceptionMapper.html>] interface. For example, the following maps the `EntityNotFoundException` [<http://docs.oracle.com/javaee/5/api/jakarta.persistence.EntityNotFoundException.html>] to a HTTP 404 (Not Found) response:

**Example 7.6. Mapping generic exceptions to responses**

```
1 @Provider
2 public class EntityNotFoundMapper implements ExceptionMapper<jakarta.persistence.EntityNotFoundException> {
3     public Response toResponse(jakarta.persistence.EntityNotFoundException ex) {
4         return Response.status(404).
5             entity(ex.getMessage()).
6             type("text/plain").
7             build();
8     }
9 }
```

The above class is annotated with `@Provider` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/Provider.html>], this declares that the class is of interest to the JAX-RS runtime. Such a class may be added to the set of classes of the Application [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] instance that is configured. When an application throws an `EntityNotFoundException` [<http://docs.oracle.com/javaee/6/api/jakarta.persistence.EntityNotFoundException.html>] the `toResponse` method of the `EntityNotFoundMapper` instance will be invoked.

Jersey supports extension of the exception mappers. These extended mappers must implement the `org.glassfish.jersey.spi.ExtendedExceptionMapper` interface. This interface additionally defines method `isMappable(Throwable)` which will be invoked by the Jersey runtime

when exception is thrown and this provider is considered as mappable based on the exception type. Using this method the provider can reject mapping of the exception before the method `toResponse` is invoked. The provider can for example check the exception parameters and based on them return false and let other provider to be chosen for the exception mapping.

Since Jersey 3.1.0 the default `ExceptionHandler` is implemented. It is required by JAX-RS 3.1 specification (Exception Mapping Providers [<https://jakarta.ee/specifications/restful-ws/3.1/jakarta-restful-ws-spec-3.1.html#exceptionmapper>]). The default behaviour of the mapper is to return a message from an exception caught and set the response status to 500 (internal server error). In case of a `WebApplicationException` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/WebApplicationException.html>] with a response that response is returned. If response inside the `WebApplicationException` is NULL the exception is being processed according to the default behaviour. The Default exception mapper is package private and can not be referenced. Its presence is required only by JAX-RS 3.1 specification. Processing of the default Exception Mapper occurs at the very end of the exception processing chain. It is invoked only if nothing else in the chain was invoked before.

## 7.4. Conditional GETs and Returning 304 (Not Modified) Responses

Conditional GETs are a great way to reduce bandwidth, and potentially improve on the server-side performance, depending on how the information used to determine conditions is calculated. A well-designed web site may for example return 304 (Not Modified) responses for many of static images it serves.

JAX-RS provides support for conditional GETs using the contextual interface `Request` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Request.html>].

The following example shows conditional GET support:

### Example 7.7. Conditional GET support

```
1 public SparklinesResource(  
2     @QueryParam("d") IntegerList data,  
3     @DefaultValue("0,100") @QueryParam("limits") Interval limits,  
4     @Context Request request,  
5     @Context UriInfo ui) {  
6     if (data == null) {  
7         throw new WebApplicationException(400);  
8     }  
9  
10    this.data = data;  
11    this.limits = limits;  
12  
13    if (!limits.contains(data)) {  
14        throw new WebApplicationException(400);  
15    }  
16  
17    this.tag = computeEntityTag(ui.getRequestUri());  
18  
19    if (request.getMethod().equals("GET")) {  
20        Response.ResponseBuilder rb = request.evaluatePreconditions(tag);  
21        if (rb != null) {  
22            throw new WebApplicationException(rb.build());  
23        }  
24    }  
25 }
```

```
24    }  
25 }
```

The constructor of the `SparklinesResource` root resource class computes an entity tag from the request URI and then calls the `request.evaluatePreconditions` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Request.html#evaluatePreconditions\(jakarta.ws.rs.core.EntityTag\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Request.html#evaluatePreconditions(jakarta.ws.rs.core.EntityTag))] with that entity tag. If a client request contains an `If-None-Match` header with a value that contains the same entity tag that was calculated then the `evaluatePreconditions` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Request.html#evaluatePreconditions\(jakarta.ws.rs.core.EntityTag\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Request.html#evaluatePreconditions(jakarta.ws.rs.core.EntityTag))] returns a pre-filled out response, with the 304 status code and entity tag set, that may be built and returned. Otherwise, `evaluatePreconditions` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Request.html#evaluatePreconditions\(jakarta.ws.rs.core.EntityTag\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Request.html#evaluatePreconditions(jakarta.ws.rs.core.EntityTag))] returns `null` and the normal response can be returned.

Notice that in this example the constructor of a resource class is used to perform actions that may otherwise have to be duplicated to be invoked for each resource method. The life cycle of resource classes is per-request which means that the resource instance is created for each request and therefore can work with request parameters and for example make changes to the request processing by throwing an exception as it is shown in this example.

---

# Chapter 8. JAX-RS Entity Providers

## 8.1. Introduction

Entity payload, if present in an received HTTP message, is passed to Jersey from an I/O container as an input stream. The stream may, for example, contain data represented as a plain text, XML or JSON document. However, in many JAX-RS components that process these inbound data, such as resource methods or client responses, the JAX-RS API user can access the inbound entity as an arbitrary Java object that is created from the content of the input stream based on the representation type information. For example, an entity created from an input stream that contains data represented as a XML document, can be converted to a custom JAXB bean. Similar concept is supported for the outbound entities. An entity returned from the resource method in the form of an arbitrary Java object can be serialized by Jersey into a container output stream as a specified representation. Of course, while JAX-RS implementations do provide default support for most common combinations of Java type and it's respective on-the-wire representation formats, JAX-RS implementations do not support the conversion described above for any arbitrary Java type and any arbitrary representation format by default. Instead, a generic extension concept is exposed in JAX-RS API to allow application-level customizations of this JAX-RS runtime to support for entity conversions. The JAX-RS extension API components that provide the user-level extensibility are typically referred to by several terms with the same meaning, such as *entity providers*, *message body providers*, *message body workers* or *message body readers and writers*. You may find all these terms used interchangeably throughout the user guide and they all refer to the same concept.

In JAX-RS extension API (or SPI - service provider interface, if you like) the concept is captured in 2 interfaces. One for handling inbound entity representation-to-Java de-serialization - `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] and the other one for handling the outbound entity Java-to-representation serialization - `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>]. A `MessageBodyReader<T>`, as the name suggests, is an extension that supports reading the message body representation from an input stream and converting the data into an instance of a specific Java type. A `MessageBodyWriter<T>` is then responsible for converting a message payload from an instance of a specific Java type into a specific representation format that is sent over the wire to the other party as part of an HTTP message exchange. Both of these providers can be used to provide message payload serialization and de-serialization support on the server as well as the client side. A message body reader or writer is always used whenever a HTTP request or response contains an entity and the entity is either requested by the application code (e.g. injected as a parameter of JAX-RS resource method or a response entity read on the client from a `Response` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>]) or has to be serialized and sent to the other party (e.g. an instance returned from a JAX-RS resource method or a request entity sent by a JAX-RS client).

## 8.2. How to Write Custom Entity Providers

A best way how to learn about entity providers is to walk through an example of writing one. Therefore we will describe here the process of implementing a custom `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>] and `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] using a practical example. Let's first setup the stage by defining a JAX-RS resource class for the server side story of our application.

### Example 8.1. Example resource class

```
1 @Path("resource")
```

```
2 public class MyResource {
3     @GET
4     @Produces("application/xml")
5     public MyBean getMyBean() {
6         return new MyBean("Hello World!", 42);
7     }
8
9     @POST
10    @Consumes("application/xml")
11    public String postMyBean(MyBean myBean) {
12        return myBean.anyString;
13    }
14 }
```

The resource class defines GET and POST resource methods. Both methods work with an entity that is an instance of `MyBean`.

The `MyBean` class is defined in the next example:

### Example 8.2. `MyBean` entity class

```
1 @XmlRootElement
2 public class MyBean {
3     @XmlElement
4     public String anyString;
5     @XmlElement
6     public int anyNumber;
7
8     public MyBean(String anyString, int anyNumber) {
9         this.anyString = anyString;
10        this.anyNumber = anyNumber;
11    }
12
13    // empty constructor needed for deserialization by JAXB
14    public MyBean() {
15    }
16
17    @Override
18    public String toString() {
19        return "MyBean{" +
20            "anyString='" + anyString + '\'' +
21            ", anyNumber=" + anyNumber +
22            '}' +
23    }
24 }
```

## 8.2.1. `MessageBodyWriter`

The `MyBean` is a JAXB-annotated POJO. In GET resource method we return the instance of `MyBean` and we would like Jersey runtime to serialize it into XML and write it as an entity body to the response output stream. We design a custom `MessageBodyWriter<T>` that can serialize this POJO into XML. See the following code sample:

## Note

Please note, that this is only a demonstration of how to write a custom entity provider. Jersey already contains default support for entity providers that can serialize JAXB beans into XML.

### Example 8.3. `MessageBodyWriter` example

```
1 @Produces("application/xml")
2 public class MyBeanMessageBodyWriter implements MessageBodyWriter<MyBean> {
3
4     @Override
5     public boolean isWriteable(Class<?> type, Type genericType,
6                               Annotation[] annotations, MediaType mediaType) {
7         return type == MyBean.class;
8     }
9
10    @Override
11    public long getSize(MyBean myBean, Class<?> type, Type genericType,
12                      Annotation[] annotations, MediaType mediaType) {
13        // deprecated by JAX-RS 2.0 and ignored by Jersey runtime
14        return -1;
15    }
16
17    @Override
18    public void writeTo(MyBean myBean,
19                       Class<?> type,
20                       Type genericType,
21                       Annotation[] annotations,
22                       MediaType mediaType,
23                       MultivaluedMap<String, Object> httpHeaders,
24                       OutputStream entityStream)
25        throws IOException, WebApplicationException {
26
27        try {
28            JAXBContext jaxbContext = JAXBContext.newInstance(MyBean.class);
29
30            // serialize the entity myBean to the entity output stream
31            jaxbContext.createMarshaller().marshal(myBean, entityStream);
32        } catch (JAXBException jaxbException) {
33            throw new ProcessingException(
34                "Error serializing a MyBean to the output stream", jaxbException);
35        }
36    }
37 }
```

The `MyBeanMessageBodyWriter` implements the `MessageBodyWriter<T>` interface that contains three methods. In the next sections we'll explore these methods more closely.

#### 8.2.1.1. `MessageBodyWriter.isWriteable`

A method `isWriteable` should return true if the `MessageBodyWriter<T>` is able to write the given type. Method does not decide only based on the Java type of the entity but also on annotations attached to the entity and the requested representation media type.

Parameters `type` and `genericType` both define the entity, where `type` is a raw Java type (for example, a `java.util.List` class) and `genericType` is a `ParameterizedType` [<http://docs.oracle.com/javase/6/docs/api/java/lang/reflect/ParameterizedType.html>] including generic information (for example `List<String>`).

Parameter `annotations` contains annotations that are either attached to the resource method and/or annotations that are attached to the entity by building response like in the following piece of code:

#### Example 8.4. Example of assignment of annotations to a response entity

```
1 @Path("resource")
2 public static class AnnotatedResource {
3
4     @GET
5     public Response get() {
6         Annotation annotation = AnnotatedResource.class
7             .getAnnotation(Path.class);
8         return Response.ok()
9             .entity("Entity", new Annotation[] {annotation}).build();
10    }
11 }
```

In the example above, the `MessageBodyWriter<T>` would get `annotations` parameter containing a JAX-RS `@GET` annotation as it annotates the resource method and also a `@Path` annotation as it is passed in the response (but not because it annotates the resource; only resource method annotations are included). In the case of `MyResource` and method `getMyBean` the annotations would contain the `@GET` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html>] and the `@Produces` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html>] annotation.

The last parameter of the `isWriteable` method is the `mediaType` which is the media type attached to the response entity by annotating the resource method with a `@Produces` annotation or the request media type specified in the JAX-RS Client API. In our example, the media type passed to providers for the resource `MyResource` and method `getMyBean` would be `"application/xml"`.

In our implementation of the `isWriteable` method, we just check that the type is `MyBean`. Please note, that this method might be executed multiple times by Jersey runtime as Jersey needs to check whether this provider can be used for a particular combination of entity Java type, media type, and attached annotations, which may be potentially a performance hog. You can limit the number of execution by properly defining the `@Produces` annotation on the `MessageBodyWriter<T>`. In our case thanks to `@Produces` annotation, the provider will be considered as writeable (and the method `isWriteable` might be executed) only if the media type of the outbound message is `"application/xml"`. Additionally, the provider will only be considered as possible candidate and its `isWriteable` method will be executed, if the generic type of the provider is either a sub class or super class of `type` parameter.

### 8.2.1.2. `MessageBodyWriter.writeTo`

Once a message body writer is selected as the most appropriate (see the Section 8.3, “Entity Provider Selection” for more details on entity provider selection), its `writeTo` method is invoked. This method receives parameters with the same meaning as in `isWriteable` as well as a few additional ones.

In addition to the parameters already introduced, the `writeTo` method defines also `httpHeaders` parameter, that contains HTTP headers associated with the outbound message.

## Note

When a `MessageBodyWriter<T>` is invoked, the headers still can be modified in this point and any modification will be reflected in the outbound HTTP message being sent. The modification of headers must however happen before a first byte is written to the supplied output stream.

Another new parameter, `myBean`, contains the entity instance to be serialized (the type of entity corresponds to generic type of `MessageBodyWriter<T>`). Related parameter `entityStream` contains the entity output stream to which the method should serialize the entity. In our case we use JAXB to marshall the entity into the `entityStream`. Note, that the `entityStream` is not closed at the end of method; the stream will be closed by Jersey.

## Important

Do not close the entity output stream in the `writeTo` method of your `MessageBodyWriter<T>` implementation.

### 8.2.1.3. `MessageBodyWriter.getSize`

The method is deprecated since JAX-RS 2.0 and Jersey 2 ignores the return value. In JAX-RS 1.0 the method could return the size of the entity that would be then used for "Content-Length" response header. In Jersey 2.0 the "Content-Length" parameter is computed automatically using an internal outbound entity buffering. For details about configuration options of outbound entity buffering see the javadoc of `MessageProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html>], property `OUTBOUND_CONTENT_LENGTH_BUFFER` which configures the size of the buffer.

## Note

You can disable the Jersey outbound entity buffering by setting the buffer size to 0.

### 8.2.1.4. Testing a `MessageBodyWriter<T>`

Before testing the `MyBeanMessageBodyWriter`, the writer must be registered as a custom JAX-RS extension provider. It should either be added to your application `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>], or returned from your custom `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] sub-class, or annotated with `@Provider` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/Provider.html>] annotation to leverage JAX-RS provider auto-discovery feature.

After registering the `MyBeanMessageBodyWriter` and `MyResource` class in our application, the request can be initiated (in this example from Client API).

#### Example 8.5. Client code testing `MyBeanMessageBodyWriter`

```
1 WebTarget webTarget = // initialize web target to the context root
2                       // of example application
3 Response response = webTarget.path("resource")
4                       .request(MediaType.APPLICATION_XML).get();
5 System.out.println(response.getStatus());
6 String myBeanXml = response.readEntity(String.class);
7 System.out.println(myBeanXml);
```



The client code initiates the GET which will be matched to the resource method `MyResource.getBean()`. The response entity is de-serialized as a `String`.

The result of console output is:

### Example 8.6. Result of `MyBeanMessageBodyWriter` test

```
200
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><myBean>
<anyString>Hello World!</anyString><anyNumber>42</anyNumber></myBean>
```

The returned status is 200 and the entity is stored in the response in a XML format. Next, we will look at how the Jersey de-serializes this XML document into a `MyBean` consumed by our POST resource method.

## 8.2.2. MessageBodyReader

In order to de-serialize the entity of `MyBean` on the server or the client, we need to implement a custom `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>].

### Note

Please note, that this is only a demonstration of how to write a custom entity provider. Jersey already contains default support for entity providers that can serialize JAXB beans into XML.

Our `MessageBodyReader<T>` implementation is listed in Example 8.7, “`MessageBodyReader` example”.

### Example 8.7. `MessageBodyReader` example

```
1 public static class MyBeanMessageBodyReader
2     implements MessageBodyReader<MyBean> {
3
4     @Override
5     public boolean isReadable(Class<?> type, Type genericType,
6         Annotation[] annotations, MediaType mediaType) {
7         return type == MyBean.class;
8     }
9
10    @Override
11    public MyBean readFrom(Class<MyBean> type,
12        Type genericType,
13        Annotation[] annotations, MediaType mediaType,
14        MultivaluedMap<String, String> httpHeaders,
15        InputStream entityStream)
16        throws IOException, WebApplicationException {
17
18        try {
19            JAXBContext jaxbContext = JAXBContext.newInstance(MyBean.class);
20            MyBean myBean = (MyBean) jaxbContext.createUnmarshaller()
21                .unmarshal(entityStream);
22            return myBean;
23        } catch (JAXBException jaxbException) {
24            throw new ProcessingException("Error deserializing a MyBean.",
25                jaxbException);
26        }
27    }
28 }
```

```
26     }  
27 }  
28 }
```

It is obvious that the `MessageBodyReader<T>` interface is similar to `MessageBodyWriter<T>`. In the next couple of sections we will explore its API methods.

### 8.2.2.1. `MessageBodyReader.isReadable`

It defines the method `isReadable()` which has a very similar meaning as method `isWritable()` in `MessageBodyWriter<T>`. The method returns `true` if it is able to de-serialize the given type. The `annotations` parameter contains annotations that are attached to the entity parameter in the resource method. In our `POST` resource method `postMyBean` the entity parameter `myBean` is not annotated, therefore no annotation will be passed to the `isReadable`. The `mediaType` parameter contains the entity media type. The media type, in our case, must be consumable by the `POST` resource method, which is specified by placing a JAX-RS `@Consumes` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Consumes.html>] annotation to the method. The resource method `postMyBean()` is annotated with `@Consumes("application/xml")`, therefore for purpose of de-serialization of entity for the `postMyBean()` method, only requests with entities represented as `"application/xml"` media type will match the method. However, this method might be executed for entity types that are sub classes or super classes of the declared generic type on the `MessageBodyReader<T>` will be also considered. It is a responsibility of the `isReadable` method to decide whether it is able to de-serialize the entity and type comparison is one of the basic decision steps.

#### Tip

In order to reduce number of `isReadable` executions, always define correctly the consumable media type(s) with the `@Consumes` annotation on your custom `MessageBodyReader<T>`.

### 8.2.2.2. `MessageBodyReader.readFrom`

The `readForm()` method gets the parameters with the same meaning as in `isReadable()`. The additional `entityStream` parameter provides a handle to the entity input stream from which the entity bytes should be read and de-serialized into a Java entity which is then returned from the method. Our `MyBeanMessageBodyReader` de-serializes the incoming XML data into an instance of `MyBean` using JAXB.

#### Important

Do not close the entity input stream in your `MessageBodyReader<T>` implementation. The stream will be automatically closed by Jersey runtime.

### 8.2.2.3. Testing a `MessageBodyWriter<T>`

Now let's send a test request using the JAX-RS Client API.

#### Example 8.8. Testing `MyBeanMessageBodyReader`

```
1 final MyBean myBean = new MyBean("posted MyBean", 11);  
2 Response response = webTarget.path("resource").request("application/xml")  
3     .post(Entity.entity(myBean, "application/xml"));  
4  
5 System.out.println(response.getStatus());  
6 final String responseEntity = response.readEntity(String.class);
```

```
7 System.out.println(responseEntity);
8
```

The console output is:

### Example 8.9. Result of testing `MyBeanMessageBodyReader`

```
200
posted MyBean
```

## 8.2.2.4. Using Entity Providers with JAX-RS Client API

Both, `MessageBodyReader<T>` and `MessageBodyWriter<T>` can be registered in a configuration of JAX-RS Client API components typically without any need to change their code. The example Example 8.10, “`MessageBodyReader` registered on a JAX-RS client” is a variation on the Example 8.5, “Client code testing `MyBeanMessageBodyWriter`” listed in one of the previous sections.

### Example 8.10. `MessageBodyReader` registered on a JAX-RS client

```
1 Client client = ClientBuilder.newBuilder()
2   .register(MyBeanMessageBodyReader.class).build();
3
4 Response response = client.target("http://example/comm/resource")
5   .request(MediaType.APPLICATION_XML).get();
6 System.out.println(response.getStatus());
7 MyBean myBean = response.readEntity(MyBean.class);
8 System.out.println(myBean);
```

The code above registers `MyBeanMessageBodyReader` to the `Client` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>] configuration using a `ClientBuilder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientBuilder.html>] which means that the provider will be used for any `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>] produced by the `client` instance.

### Note

You could also register the JAX-RS entity (and any other) providers to individual `WebTarget` instances produced by the client.

Then, using the fluent chain of method invocations, a resource target pointing to our `MyResource` is defined, a HTTP GET request is invoked. The response entity is then read as an instance of a `MyBean` type by invoking the `response.readEntity` method, that internally locates the registered `MyBeanMessageBodyReader` and uses it for entity de-serialization.

The console output for the example is:

### Example 8.11. Result of client code execution

```
200
MyBean{anyString='Hello World!', anyNumber=42}
```

## 8.3. Entity Provider Selection

Usually there are many entity providers registered on the server or client side (by default there must be at least providers mandated by the JAX-RS specification, such as providers for primitive types, byte

array, JAXB beans, etc.). JAX-RS defines an algorithm for selecting the most suitable provider for entity processing. This algorithm works with information such as entity Java type and on-the-wire media type representation of entity, and searches for the most suitable entity provider from the list of available providers based on the supported media type declared on each provider (defined by `@Produces` or `@Consumes` on the provider class) as well as based on the generic type declaration of the available providers. When a list of suitable candidate entity providers is selected and sorted based on the rules defined in JAX-RS specification, a JAX-RS runtime then it invokes `isReadable` or `isWritable` method respectively on each provider in the list until a first provider is found that returns `true`. This provider is then used to process the entity.

The following steps describe the algorithm for selecting a `MessageBodyWriter<T>` (extracted from JAX-RS with little modifications). The steps refer to the previously discussed example application. The `MessageBodyWriter<T>` is searched for purpose of deserialization of `MyBean` entity returned from the method `getMyBean`. So, *type is `MyBean` and media type `"application/xml"`*. Let's assume the runtime contains also registered providers, namely:

```
A: @Produces("application/*") with generic type <Object>
B: @Produces("*/*") with generic type <MyBean>
C: @Produces("text/plain") with generic type <MyBean>
D: @Produces("application/xml") with generic type <Object>
MyBeanMessageBodyWriter: @Produces("application/xml") with generic type
<MyBean>
```

The algorithm executed by a JAX-RS runtime to select a proper `MessageBodyWriter<T>` implementation is illustrated in Procedure 8.1, “`MessageBodyWriter<T>` Selection Algorithm”.

### Procedure 8.1. `MessageBodyWriter<T>` Selection Algorithm

1. Obtain the object that will be mapped to the message entity body. For a return type of `Response` or subclasses, the object is the value of the entity property, for other return types it is the returned object.

So in our case, for the resource method `getMyBean` the type will be `MyBean`.

2. Determine the media type of the response.

In our case, for resource method `getMyBean` annotated with `@Produces("application/xml")`, the media type will be `"application/xml"`.

3. Select the set of `MessageBodyWriter` providers that support the object and media type of the message entity body.

In our case, for entity media type `"application/xml"` and type `MyBean`, the appropriate `MessageBodyWriter<T>` will be the A, B, D and `MyBeanMessageBodyWriter`. The provider C does not define the appropriate media type. A and B are fine as their type is more generic and compatible with `"application/xml"`.

4. Sort the selected `MessageBodyWriter` providers with a primary key of generic type where providers whose generic type is the nearest superclass of the object class are sorted first and a secondary key of media type. Additionally, JAX-RS specification mandates that custom, user registered providers have to be sorted ahead of default providers provided by JAX-RS implementation. This is used as a tertiary comparison key. User providers are places prior to Jersey internal providers in to the final ordered list.

The sorted providers will be: `MyBeanMessageBodyWriter`, B, D, A.

5. Iterate through the sorted `MessageBodyWriter<T>` providers and, utilizing the `isWritable` method of each until you find a `MessageBodyWriter<T>` that returns `true`.

The first provider in the list - our `MyBeanMessageBodyWriter` returns `true` as it compares types and the types matches. If it would return `false`, the next provider `B` would be checked by invoking its `isWriteable` method.

6. If step 5 locates a suitable `MessageBodyWriter<T>` then use its `writeTo` method to map the object to the entity body.

`MyBeanMessageBodyWriter.writeTo` will be executed and it will serialize the entity.

- Otherwise, the server runtime MUST generate an `InternalServerErrorException`, a subclass of `WebApplicationException` with its status set to 500, and no entity and the client runtime MUST generate a `ProcessingException`.

We have successfully found a provider, thus no exception is generated.

## Note

JAX-RS 3.x/2.x is incompatible with JAX-RS 1.x in one step of the entity provider selection algorithm. JAX-RS 1.x defines sorting keys priorities in the Step 4 in exactly opposite order. So, in JAX-RS 1.x the keys are defined in the order: primary media type, secondary type declaration distance where custom providers have always precedence to internal providers. If you want to force Jersey to use the algorithm compatible with JAX-RS 1.x, setup the property (to `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>] or return from `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] from its `getProperties` method):

```
jersey.config.workers.legacyOrdering=true
```

Documentation of this property can be found in the javadoc of `MessageProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html>].

The algorithm for selection of `MessageBodyReader<T>` is similar, including the incompatibility between JAX-RS 3.x/2.x and JAX-RS 1.x and the property to workaround it. The algorithm is defined as follows:

### Procedure 8.2. `MessageBodyReader<T>` Selection Algorithm

1. Obtain the media type of the request. If the request does not contain a `Content-Type` header then use `application/octet-stream` media type.
2. Identify the Java type of the parameter whose value will be mapped from the entity body. The Java type on the server is the type of the entity parameter of the resource method. On the client it is the `Class` passed to `readFrom` method.
3. Select the set of available `MessageBodyReader<T>` providers that support the media type of the request.
4. Iterate through the selected `MessageBodyReader<T>` classes and, utilizing their `isReadable` method, choose the first `MessageBodyReader<T>` provider that supports the desired combination of Java type/media type/annotations parameters.
5. If Step 4 locates a suitable `MessageBodyReader<T>`, then use its `readFrom` method to map the entity body to the desired Java type.

- Otherwise, the server runtime **MUST** generate a `NotSupportedException` (HTTP 415 status code) and no entity and the client runtime **MUST** generate an instance of `ProcessingException`.

## 8.4. Jersey `MessageBodyWorkers` API

In case you need to directly work with JAX-RS entity providers, for example to serialize an entity in your resource method, filter or in a composite entity provider, you would need to perform quite a lot of steps. You would need to choose the appropriate `MessageBodyWriter<T>` based on the type, media type and other parameters. Then you would need to instantiate it, check it by `isWriteable` method and basically perform all the steps that are normally performed by Jersey (see Procedure 8.2, “`MessageBodyReader<T>` Selection Algorithm”).

To remove this burden from developers, Jersey exposes a proprietary public API that simplifies the manipulation of entity providers. The API is defined by `MessageBodyWorkers` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageBodyWorkers.html>] interface and Jersey provides an implementation that can be injected using the `@Context` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>] injection annotation. The interface declares methods for selection of most appropriate `MessageBodyReader<T>` and `MessageBodyWriter<T>` based on the rules defined in JAX-RS spec, methods for writing and reading entity that ensure proper and timely invocation of interceptors and other useful methods.

See the following example of usage of `MessageBodyWorkers`.

### Example 8.12. Usage of `MessageBodyWorkers` interface

```
1 @Path("workers")
2 public static class WorkersResource {
3
4     @Context
5     private MessageBodyWorkers workers;
6
7     @GET
8     @Produces("application/xml")
9     public String getMyBeanAsString() {
10
11         final MyBean myBean = new MyBean("Hello World!", 42);
12
13         // buffer into which myBean will be serialized
14         ByteArrayOutputStream baos = new ByteArrayOutputStream();
15
16         // get most appropriate MBW
17         final MessageBodyWriter<MyBean> messageBodyWriter =
18             workers.getMessageBodyWriter(MyBean.class, MyBean.class,
19                 new Annotation[] {}, MediaType.APPLICATION_XML_TYPE);
20
21         try {
22             // use the MBW to serialize myBean into baos
23             messageBodyWriter.writeTo(myBean,
24                 MyBean.class, MyBean.class, new Annotation[] {},
25                 MediaType.APPLICATION_XML_TYPE, new MultivaluedHashMap<String,
```

```
26         baos);
27     } catch (IOException e) {
28         throw new RuntimeException(
29             "Error while serializing MyBean.", e);
30     }
31
32     final String stringXmlOutput = baos.toString();
33     // stringXmlOutput now contains XML representation:
34     // "<?xml version='1.0' encoding='UTF-8' standalone='yes'?>
35     // <myBean><anyString>Hello World!</anyString>
36     // <anyNumber>42</anyNumber></myBean>"
37
38     return stringXmlOutput;
39 }
40 }
```

In the example a resource injects `MessageBodyWorkers` and uses it for selection of the most appropriate `MessageBodyWriter<T>`. Then the writer is utilized to serialize the entity into the buffer as XML document. The `String` content of the buffer is then returned. This will cause that Jersey will not use `MyBeanMessageBodyWriter` to serialize the entity as it is already in the `String` type (`MyBeanMessageBodyWriter` does not support `String`). Instead, a simple `String`-based `MessageBodyWriter<T>` will be chosen and it will only serialize the `String` with XML to the output entity stream by writing out the bytes of the `String`.

Of course, the code in the example does not bring any benefit as the entity could have been serialized by `MyBeanMessageBodyWriter` by Jersey as in previous examples; the purpose of the example was to show how to use `MessageBodyWorkers` in a resource method.

## 8.5. Default Jersey Entity Providers

Jersey internally contains entity providers for these types with combination of media types (in brackets):

```
byte[] (*)
String [http://docs.oracle.com/javase/6/docs/api/java/io/String.html] (*)
InputStream [http://docs.oracle.com/javase/6/docs/api/java/io/InputStream.html] (*)
Reader [http://docs.oracle.com/javase/6/docs/api/java/io/Reader.html] (*)
File [http://docs.oracle.com/javase/6/docs/api/java/io/File.html] (*)
DataSource [https://jakarta.ee/specifications/activation/2.0/apidocs/jakarta.activation/jakarta/activation/datasource] (*)
Source [http://docs.oracle.com/javase/6/docs/api/javax/xml/transform/Source.html] (text/xml, application/xml and media types of the form application/*+xml)
JAXBElement [http://docs.oracle.com/javase/6/docs/api/jakarta.xml.bind/JAXBElement.html] (text/xml, application/xml and media types of the form application/*+xml)
MultivaluedMap<K,V> [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/MultivaluedMap.html] (application/x-www-form-urlencoded)
Form [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Form.html] (application/x-www-form-urlencoded)
StreamingOutput [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/StreamingOutput.html] ((*) - this class can be used as an lightweight MessageBodyWriter<T> that can be returned from a resource method
Boolean [http://docs.oracle.com/javase/6/docs/api/java/lang/Boolean.html], Character [http://docs.oracle.com/javase/6/docs/api/java/lang/Character.html] and Number [http://docs.oracle.com/javase/6/docs/api/java/lang/Number.html] (text/plain) - corresponding primitive types supported via boxing/unboxing conversion
```

For other media type supported in jersey please see the Chapter 9, *Support for Common Media Type Representations* which describes additional Jersey entity provider extensions for serialization to JSON, XML, serialization of collections, Multi Part [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/package-summary.html>] and others.



---

# Chapter 9. Support for Common Media Type Representations

## 9.1. JSON

Jersey JSON support comes as a set of extension modules where each of these modules contains an implementation of a Feature [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html>] that needs to be registered into your Configurable [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] instance (client/server). There are multiple frameworks that provide support for JSON processing and/or JSON-to-Java binding. The modules listed below provide support for JSON representations by integrating the individual JSON frameworks into Jersey. At present, Jersey integrates with the following modules to provide JSON support:

- MOXy - JSON binding support via MOXy is a default and preferred way of supporting JSON binding in your Jersey applications since Jersey 2.0. When JSON MOXy module is on the class-path, Jersey will automatically discover the module and seamlessly enable JSON binding support via MOXy in your applications. (See Section 4.3, “Auto-Discoverable Features”.)
- Java API for JSON Processing (JSON-P)
- Jackson
- Jettison
- Java API for JSON Binding (JSON-B)

### 9.1.1. Approaches to JSON Support

Each of the aforementioned extension modules uses one or more of the three basic approaches available when working with JSON representations:

- POJO based JSON binding support
- JAXB based JSON binding support
- Low-level JSON parsing & processing support

The first method is pretty generic and allows you to map any Java Object to JSON and vice versa. The other two approaches limit you in Java types your resource methods could produce and/or consume. JAXB based approach is useful if you plan to utilize certain JAXB features and support both XML and JSON representations. The last, low-level, approach gives you the best fine-grained control over the out-coming JSON data format.

#### 9.1.1.1. POJO support

POJO support represents the easiest way to convert your Java Objects to JSON and back.

Media modules that support this approach are MOXy, Jackson, and Java API for JSON Binding (JSON-B)

#### 9.1.1.2. JAXB based JSON support

Taking this approach will save you a lot of time, if you want to easily produce/consume both JSON and XML data format. With JAXB beans you will be able to use the same Java model to generate JSON as well as XML representations. Another advantage is simplicity of working with such a model and availability

of the API in Java SE Platform. JAXB leverages annotated POJOs and these could be handled as simple Java beans.

A disadvantage of JAXB based approach could be if you need to work with a very specific JSON format. Then it might be difficult to find a proper way to get such a format produced and consumed. This is a reason why a lot of configuration options are provided, so that you can control how JAXB beans get serialized and de-serialized. The extra configuration options however requires you to learn more details about the framework you are using.

Following is a very simple example of how a JAXB bean could look like.

### Example 9.1. Simple JAXB bean implementation

```
1 @XmlRootElement
2 public class MyJaxbBean {
3     public String name;
4     public int age;
5
6     public MyJaxbBean() {} // JAXB needs this
7
8     public MyJaxbBean(String name, int age) {
9         this.name = name;
10        this.age = age;
11    }
12 }
```

Using the above JAXB bean for producing JSON data format from you resource method, is then as simple as:

### Example 9.2. JAXB bean used to generate JSON representation

```
1 @GET
2 @Produces("application/json")
3 public MyJaxbBean getMyBean() {
4     return new MyJaxbBean("Agamemnon", 32);
5 }
```

Notice, that JSON specific mime type is specified in `@Produces` annotation, and the method returns an instance of `MyJaxbBean`, which JAXB is able to process. Resulting JSON in this case would look like:

```
{"name": "Agamemnon", "age": "32"}
```

A proper use of JAXB annotations itself enables you to control output JSON format to certain extent. Specifically, renaming and omitting items is easy to do directly just by using JAXB annotations. For example, the following example depicts changes in the above mentioned `MyJaxbBean` that will result in `{"king": "Agamemnon"}` JSON output.

### Example 9.3. Tweaking JSON format using JAXB

```
1 @XmlRootElement
2 public class MyJaxbBean {
3
4     @XmlElement(name="king")
5     public String name;
6
7     @XmlTransient
8     public int age;
```

```
9
10    // several lines removed
11 }
```

Media modules that support this approach are MOXy, Jackson, Jettison

### 9.1.1.3. Low-level based JSON support

JSON Processing API is a new standard API for parsing and processing JSON structures in similar way to what SAX and StAX parsers provide for XML. The API is part of Jakarta EE 9 and later. Another such JSON parsing/processing API is provided by Jettison framework (which is also supported in jakartified environment). Both APIs provide a low-level access to producing and consuming JSON data structures. By adopting this low-level approach you would be working with `JsonObject` (or `JSONObject` respectively) and/or `JsonArray` (or `JSONArray` respectively) classes when processing your JSON data representations.

The biggest advantage of these low-level APIs is that you will gain full control over the JSON format produced and consumed. You will also be able to produce and consume very large JSON structures using streaming JSON parser/generator APIs. On the other hand, dealing with your data model objects will probably be a lot more complex, compared to the POJO or JAXB based binding approach. Differences are depicted at the following code snippets.

Let's start with JAXB-based approach.

#### Example 9.4. JAXB bean creation

```
1 MyJaxbBean myBean = new MyJaxbBean("Agamemnon", 32);
```

Above you construct a simple JAXB bean, which could be written in JSON as `{"name": "Agamemnon", "age": 32}`

Now to build an equivalent `JsonObject/JSONObject` (in terms of resulting JSON expression), you would need several more lines of code. The following example illustrates how to construct the same JSON data using the standard Jakarta EE 9 JSON-Processing API.

#### Example 9.5. Constructing a `JsonObject` (JSON-Processing)

```
1 JsonObject myObject = Json.createObjectBuilder()
2     .add("name", "Agamemnon")
3     .add("age", 32)
4     .build();
```

And at last, here's how the same work can be done with Jettison API.

#### Example 9.6. Constructing a `JSONObject` (Jettison)

```
1 JSONObject myObject = new JSONObject();
2 try {
3     myObject.put("name", "Agamemnon");
4     myObject.put("age", 32);
5 } catch (JSONException ex) {
6     LOGGER.log(Level.SEVERE, "Error ...", ex);
7 }
```

Media modules that support the low-level JSON parsing and generating approach are Java API for JSON Processing (JSON-P) and Jettison. Unless you have a strong reason for using the non-standard Jettison API, we recommend you to use the new standard Java API for JSON Processing (JSON-P) API instead.

## 9.1.2. MOXy

### 9.1.2.1. Dependency

To use MOXy as your JSON provider you need to add `jersey-media-moxy` module to your `pom.xml` file:

```
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-moxy</artifactId>
  <version>4.0.0</version>
</dependency>
```

If you're not using Maven make sure to have all needed dependencies (see `jersey-media-moxy` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-media-moxy/dependencies.html>]) on the classpath.

### 9.1.2.2. Configure and register

As stated in the Section 4.3, “Auto-Discoverable Features” as well as earlier in this chapter, MOXy media module is one of the modules where you don't need to explicitly register its `Features` (`MoxyJsonFeature`) in your client/server `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] as this feature is automatically discovered and registered when you add `jersey-media-moxy` module to your class-path.

The auto-discoverable `jersey-media-moxy` module defines a few properties that can be used to control the automatic registration of `MoxyJsonFeature` (besides the generic `CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE\\_AUTO\\_DISCOVERY\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE)] and the its client/server variants):

- `CommonProperties.MOXY_JSON_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#MOXY\\_JSON\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#MOXY_JSON_FEATURE_DISABLE)]
- `ServerProperties.MOXY_JSON_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MOXY\\_JSON\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MOXY_JSON_FEATURE_DISABLE)]
- `ClientProperties.MOXY_JSON_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#MOXY\\_JSON\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#MOXY_JSON_FEATURE_DISABLE)]

#### Note

A manual registration of any other Jersey JSON provider feature (except for Java API for JSON Processing (JSON-P)) disables the automated enabling and configuration of `MoxyJsonFeature`.

To configure `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>]s / `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>]s provided by MOXy you can simply create an instance of `MoxyJsonConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html>] and set values of needed properties. For most common

properties you can use a particular method to set the value of the property or you can use more generic methods to set the property:

- `MoxyJsonConfig#property(java.lang.String, java.lang.Object)` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html#property\(java.lang.String, java.lang.Object\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html#property(java.lang.String, java.lang.Object))] - sets a property value for both Marshaller and Unmarshaller.
- `MoxyJsonConfig#marshallerProperty(java.lang.String, java.lang.Object)` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html#marshallerProperty\(java.lang.String, java.lang.Object\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html#marshallerProperty(java.lang.String, java.lang.Object))] - sets a property value for Marshaller.
- `MoxyJsonConfig#unmarshallerProperty(java.lang.String, java.lang.Object)` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html#unmarshallerProperty\(java.lang.String, java.lang.Object\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html#unmarshallerProperty(java.lang.String, java.lang.Object))] - sets a property value for Unmarshaller.

**Example 9.7. MoxyJsonConfig** [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html>] - Setting properties.

```
final Map<String, String> namespacePrefixMapper = new HashMap<String, String>();
namespacePrefixMapper.put("http://www.w3.org/2001/XMLSchema-instance", "xsi");

final MoxyJsonConfig configuration = new MoxyJsonConfig()
    .setNamespacePrefixMapper(namespacePrefixMapper)
    .setNamespaceSeparator(':');
```

In order to make `MoxyJsonConfig` visible for MOXy you need to create and register `ContextResolver<T>` in your client/server code.

**Example 9.8. Creating ContextResolver<MoxyJsonConfig>**

```
final Map<String, String> namespacePrefixMapper = new HashMap<String, String>();
namespacePrefixMapper.put("http://www.w3.org/2001/XMLSchema-instance", "xsi");

final MoxyJsonConfig moxyJsonConfig = MoxyJsonConfig()
    .setNamespacePrefixMapper(namespacePrefixMapper)
    .setNamespaceSeparator(':');

final ContextResolver<MoxyJsonConfig> jsonConfigResolver = moxyJsonConfig.resolver
```

Another way to pass configuration properties to the underlying `MOXyJsonProvider` is to set them directly into your `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] instance (see an example below). These are overwritten by properties set into the `MoxyJsonConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/json/MoxyJsonConfig.html>].

**Example 9.9. Setting properties for MOXy providers into Configurable** [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>]

```
new ResourceConfig()
    .property(MarshallerProperties.JSON_NAMESPACE_SEPARATOR)
```

// further configuration

There are some properties for which Jersey sets the default value when `MessageBodyReader<T>` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html] / `MessageBodyWriter<T>` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html] from MOXy is used and they are:

**Table 9.1. Default property values for MOXy `MessageBodyReader<T>` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html] / `MessageBodyWriter<T>` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html]**

|                                                                                               |                           |
|-----------------------------------------------------------------------------------------------|---------------------------|
| <code>jakarta.xml.bind.Marshaller#JAXB_FORMATTED_OUTPUT</code>                                | <code>Boolean#TRUE</code> |
| <code>org.eclipse.persistence.jaxb.JAXBContextProperties#JSON_INCLUDE_ROOT</code>             | <code>Boolean#TRUE</code> |
| <code>org.eclipse.persistence.jaxb.MarshallerProperties#JSON_MARSHAL_EMPTY_COLLECTIONS</code> | <code>Boolean#TRUE</code> |
| <code>org.eclipse.persistence.jaxb.JAXBContextProperties#JSON_NAMESPACE_SEPARATOR</code>      | <code>String#DOT</code>   |

**Example 9.10. Building client with MOXy JSON feature enabled.**

```
final Client client = ClientBuilder.newBuilder()
    // The line below that registers MOXy feature can be
    // omitted if FEATURE_AUTO_DISCOVERY_DISABLE is
    // not disabled.
    .register(MoxyJsonFeature.class)
    .register(jsonConfigResolver)
    .build();
```

**Example 9.11. Creating JAX-RS application with MOXy JSON feature enabled.**

```
// Create JAX-RS application.
final Application application = new ResourceConfig()
    .packages("org.glassfish.jersey.examples.jsonmoxy")
    // The line below that registers MOXy feature can be
    // omitted if FEATURE_AUTO_DISCOVERY_DISABLE is
    // not disabled.
    .register(MoxyJsonFeature.class)
    .register(jsonConfigResolver);
```

### 9.1.2.3. Examples

Jersey provides a JSON MOXy example [https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-moxy] on how to use MOXy to consume/produce JSON.

## 9.1.3. Java API for JSON Processing (JSON-P)

### 9.1.3.1. Dependency

To use JSON-P as your JSON provider you need to add `jersey-media-json-processing` module to your `pom.xml` file:

```
<dependency>
    <groupId>org.glassfish.jersey.media</groupId>
```

```
<artifactId>jersey-media-json-processing</artifactId>
<version>4.0.0</version>
</dependency>
```

If you're not using Maven make sure to have all needed dependencies (see `jersey-media-json-processing` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-media-json-processing/dependencies.html>]) on the class-path.

### 9.1.3.2. Configure and register

As stated in Section 4.3, “Auto-Discoverable Features” JSON-Processing media module is one of the modules where you don't need to explicitly register its `Features` (`JsonProcessingFeature`) in your client/server `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] as this feature is automatically discovered and registered when you add `jersey-media-json-processing` module to your classpath.

As for the other modules, `jersey-media-json-processing` has also few properties that can affect the registration of `JsonProcessingFeature` (besides `CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE\\_AUTO\\_DISCOVERY\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE)] and the like):

- `CommonProperties.JSON_PROCESSING_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON\\_PROCESSING\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_PROCESSING_FEATURE_DISABLE)]
- `ServerProperties.JSON_PROCESSING_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#JSON\\_PROCESSING\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#JSON_PROCESSING_FEATURE_DISABLE)]
- `ClientProperties.JSON_PROCESSING_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#JSON\\_PROCESSING\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#JSON_PROCESSING_FEATURE_DISABLE)]

To configure `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>]s / `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>]s provided by JSON-P you can simply add values for supported properties into the `Configuration` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configuration.html>] instance (client/server). Currently supported are these properties:

- `JsonGenerator.PRETTY_PRINTING`  
("jakarta.json.stream.JsonGenerator.prettyPrinting")

#### Example 9.12. Building client with JSON-Processing JSON feature enabled.

```
ClientBuilder.newClient(new ClientConfig()
    // The line below that registers JSON-Processing feature can be
    // omitted if FEATURE_AUTO_DISCOVERY_DISABLE is not disabled.
    .register(JsonProcessingFeature.class)
    .property(JsonGenerator.PRETTY_PRINTING, true)
);
```

#### Example 9.13. Creating JAX-RS application with JSON-Processing JSON feature enabled.

```
// Create JAX-RS application.
```

```
final Application application = new ResourceConfig()
    // The line below that registers JSON-Processing feature can be
    // omitted if FEATURE_AUTO_DISCOVERY_DISABLE is not disabled.
    .register(JsonProcessingFeature.class)
    .packages("org.glassfish.jersey.examples.jsonp")
    .property(JsonGenerator.PRETTY_PRINTING, true);
```

### 9.1.3.3. Examples

Jersey provides a JSON Processing example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-processing-webapp>] on how to use JSON-Processing to consume/produce JSON.

## 9.1.4. Jackson (2.x)

### 9.1.4.1. Dependency

To use Jackson 2.x as your JSON provider you need to add `jersey-media-json-jackson` module to your `pom.xml` file:

```
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-json-jackson</artifactId>
  <version>4.0.0</version>
</dependency>
```

If you're not using Maven make sure to have all needed dependencies (see `jersey-media-json-jackson` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-media-json-jackson/dependencies.html>]) on the classpath.

### 9.1.4.2. Configure and register

#### Note

Note that namespace for Jackson 2.x is (`com.fasterxml.jackson`).

Jackson JSON processor could be controlled via providing a custom Jackson 2 `ObjectMapper` [<http://fasterxml.github.io/jackson-databind/javadoc/2.3.0/com/fasterxml/jackson/databind/ObjectMapper.html>] instance. This could be handy if you need to redefine the default Jackson behaviour and to fine-tune how your JSON data structures look like. Detailed description of all Jackson features is out of scope of this guide. The example below gives you a hint on how to wire your `ObjectMapper` instance into your Jersey application.

Since the 2.36 version of Jersey it is possible to filter (include/exclude) Jackson modules by properties `CommonProperties.JSON_JACKSON_DISABLED_MODULES` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON\\_JACKSON\\_DISABLED\\_MODULES](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_JACKSON_DISABLED_MODULES)] and `CommonProperties.JSON_JACKSON_ENABLED_MODULES` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON\\_JACKSON\\_ENABLED\\_MODULES](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_JACKSON_ENABLED_MODULES)] (with their client/server derivatives). If the `CommonProperties.JSON_JACKSON_ENABLED_MODULES` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON\\_JACKSON\\_ENABLED\\_MODULES](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_JACKSON_ENABLED_MODULES)] property is used, only those named modules will be used for JSON processing. On the other hand if the `CommonProperties.JSON_JACKSON_DISABLED_MODULES` [<https://eclipse-ee4j.github.io/>]



[jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON\\_JACKSON\\_DISABLED\\_MODULES](https://jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_JACKSON_DISABLED_MODULES)] property is used, those listed modules will be explicitly excluded from processing while other (not listed) will remain. Please note that the `JaxbAnnotationModule` module is always excluded from processing and this is not configurable.

In order to use Jackson as your JSON (JAXB/POJO) provider you need to register `JacksonFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jackson/JacksonFeature.html>] and a `ContextResolver<T>` for `ObjectMapper`, if needed, in your `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] (client/server).

#### Example 9.14. `ContextResolver<ObjectMapper>`

```
1 @Provider
2 public class MyObjectMapperProvider implements ContextResolver<ObjectMapper> {
3
4     final ObjectMapper defaultObjectMapper;
5
6     public MyObjectMapperProvider() {
7         defaultObjectMapper = createDefaultMapper();
8     }
9
10    @Override
11    public ObjectMapper getContext(Class<?> type) {
12        return defaultObjectMapper;
13    }
14
15    private static ObjectMapper createDefaultMapper() {
16        final ObjectMapper result = new ObjectMapper();
17        result.configure(Feature.INDENT_OUTPUT, true);
18
19        return result;
20    }
21
22    // ...
23 }
24 }
```

To view the complete example source code, see `MyObjectMapperProvider` [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-jackson/src/main/java/org/glassfish/jersey/examples/jackson/MyObjectMapperProvider.java>] class from the JSON-Jackson [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-jackson>] example.

#### Example 9.15. Building client with Jackson JSON feature enabled.

```
1 final Client client = ClientBuilder.newBuilder()
2     .register(MyObjectMapperProvider.class) // No need to register this p
3     .register(JacksonFeature.class)
4     .build();
```

#### Example 9.16. Creating JAX-RS application with Jackson JSON feature enabled.

```
1 // Create JAX-RS application.
2 final Application application = new ResourceConfig()
3     .packages("org.glassfish.jersey.examples.jackson")
```

```
4      .register(MyObjectMapperProvider.class) // No need to register this p
5      .register(JacksonFeature.class);
```

### 9.1.4.3. Examples

Jersey provides JSON Jackson (2.x) example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-jackson>] showing how to use Jackson to consume/produce JSON.

### 9.1.5. Jettison

JAXB approach for (de)serializing JSON in Jettison module provides, in addition to using pure JAXB, configuration options that could be set on an JettisonConfig [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jettison/JettisonConfig.html>] instance. The instance could be then further used to create a JettisonJaxbContext [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jettison/JettisonJaxbContext.html>], which serves as a main configuration point in this area. To pass your specialized JettisonJaxbContext to Jersey, you will finally need to implement a JAXBContext ContextResolver<T> [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ContextResolver.html>] (see below).

#### 9.1.5.1. Dependency

To use Jettison as your JSON provider you need to add jersey-media-json-jettison module to your pom.xml file:

```
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-json-jettison</artifactId>
  <version>4.0.0</version>
</dependency>
```

If you're not using Maven make sure to have all needed dependencies (see jersey-media-json-jettison [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-media-json-jettison/dependencies.html>]) on the classpath.

#### 9.1.5.2. JSON Notations

JettisonConfig allows you to use two JSON notations. Each of these notations serializes JSON in a different way. Following is a list of supported notations:

- JETTISON\_MAPPED (default notation)
- BADGERFISH

You might want to use one of these notations, when working with more complex XML documents. Namely when you deal with multiple XML namespaces in your JAXB beans.

Individual notations and their further configuration options are described below. Rather than explaining rules for mapping XML constructs into JSON, the notations will be described using a simple example. Following are JAXB beans, which will be used.

**Example 9.17. JAXB beans for JSON supported notations description, simple address bean**

```
1 @XmlElement
2 public class Address {
```

```
3     public String street;
4     public String town;
5
6     public Address(){}
7
8     public Address(String street, String town) {
9         this.street = street;
10        this.town = town;
11    }
12 }
```

**Example 9.18. JAXB beans for JSON supported notations description, contact bean**

```
1 @XmlRootElement
2 public class Contact {
3
4     public int id;
5     public String name;
6     public List<Address> addresses;
7
8     public Contact() {};
9
10    public Contact(int id, String name, List<Address> addresses) {
11        this.name = name;
12        this.id = id;
13        this.addresses =
14            (addresses != null) ? new LinkedList<Address>(addresses) : null;
15    }
16 }
```

Following text will be mainly working with a contact bean initialized with:

**Example 9.19. JAXB beans for JSON supported notations description, initialization**

```
1 Address[] addresses = {new Address("Long Street 1", "Short Village")};
2 Contact contact = new Contact(2, "Bob", Arrays.asList(addresses));
```

I.e. contact bean with id=2, name="Bob" containing a single address (street="Long Street 1", town="Short Village").

All below described configuration options are documented also in api-docs at JettisonConfig [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jettison/JettisonConfig.html>].

### 9.1.5.2.1. Jettison mapped notation

If you need to deal with various XML namespaces, you will find Jettison mapped notation pretty useful. Lets define a particular namespace for id item:

```
1 ...
2 @XmlElement(namespace="http://example.com")
3 public int id;
4 ...
```

Then you simply configure a mapping from XML namespace into JSON prefix as follows:

### Example 9.20. XML namespace to JSON mapping configuration for Jettison based mapped notation

```
1 Map<String,String> ns2json = new HashMap<String, String>();
2 ns2json.put("http://example.com", "example");
3 context = new JettisonJaxbContext(
4     JettisonConfig.mappedJettison().xml2JsonNs(ns2json).build(),
5     types);
```

Resulting JSON will look like in the example below.

### Example 9.21. JSON expression with XML namespaces mapped into JSON

```
1 {
2   "contact":{
3     "example.id":2,
4     "name":"Bob",
5     "addresses":{
6       "street":"Long Street 1",
7       "town":"Short Village"
8     }
9   }
10 }
```

Please note, that `id` item became `example.id` based on the XML namespace mapping. If you have more XML namespaces in your XML, you will need to configure appropriate mapping for all of them.

Another configurable option introduced in Jersey version 2.2 is related to serialization of JSON arrays with Jettison's mapped notation. When serializing elements representing single item lists/arrays, you might want to utilise the following Jersey configuration method to explicitly name which elements to treat as arrays no matter what the actual content is.

### Example 9.22. JSON Array configuration for Jettison based mapped notation

```
1 context = new JettisonJaxbContext(
2     JettisonConfig.mappedJettison().serializeAsArray("name").build(),
3     types);
```

Resulting JSON will look like in the example below, unimportant lines removed for sanity.

### Example 9.23. JSON expression with JSON arrays explicitly configured via Jersey

```
1 {
2   "contact":{
3     ...
4     "name":["Bob"],
5     ...
6   }
7 }
```

#### 9.1.5.2.2. Badgerfish notation

From JSON and JavaScript perspective, this notation is definitely the worst readable one. You will probably not want to use it, unless you need to make sure your JAXB beans could be flawlessly written and read back to and from JSON, without bothering with any formatting configuration, namespaces, etc.

JettisonConfig instance using `badgerfish` notation could be built with

```
JettisonConfig.badgerFish().build()
```

and the JSON output JSON will be as follows.

**Example 9.24. JSON expression produced using `badgerfish` notation**

```
1 {
2   "contact":{
3     "id":{
4       "$":"2"
5     },
6     "name":{
7       "$":"Bob"
8     },
9     "addresses":{
10      "street":{
11        "$":"Long Street 1"
12      },
13      "town":{
14        "$":"Short Village"
15      }
16    }
17  }
18 }
```

### 9.1.5.3. Configure and register

In order to use Jettison as your JSON (JAXB/POJO) provider you need to register `JettisonFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jettison/JettisonFeature.html>] and a `ContextResolver<T>` for `JAXBContext` (if needed) in your `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] (client/server).

**Example 9.25. `ContextResolver<ObjectMapper>`**

`@Provider`

```
public class JaxbContextResolver implements ContextResolver<JAXBContext> {

    private final JAXBContext context;
    private final Set<Class<?>> types;
    private final Class<?>[] cTypes = {Flights.class, FlightType.class, AircraftTy

    public JaxbContextResolver() throws Exception {
        this.types = new HashSet<Class<?>>(Arrays.asList(cTypes));
        this.context = new JettisonJaxbContext(JettisonConfig.DEFAULT, cTypes);
    }

    @Override
    public JAXBContext getContext(Class<?> objectType) {
        return (types.contains(objectType)) ? context : null;
    }
}
```

#### **Example 9.26. Building client with Jettison JSON feature enabled.**

```
final Client client = ClientBuilder.newBuilder()
    .register(JaxbContextResolver.class) // No need to register this provider
    .register(JettisonFeature.class)
    .build();
```

#### **Example 9.27. Creating JAX-RS application with Jettison JSON feature enabled.**

```
// Create JAX-RS application.
final Application application = new ResourceConfig()
    .packages("org.glassfish.jersey.examples.jettison")
    .register(JaxbContextResolver.class) // No need to register this provider
    .register(JettisonFeature.class);
```

### **9.1.5.4. Examples**

Jersey provides an JSON Jettison example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-jettison>] on how to use Jettison to consume/produce JSON.

### **9.1.6. @JSONP - JSON with Padding Support**

Jersey provides out-of-the-box support for JSONP [<http://en.wikipedia.org/wiki/JSONP>] - JSON with padding. The following conditions has to be met to take advantage of this capability:

- Resource method, which should return wrapped JSON, needs to be annotated with `@JSONP` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/JSONP.html>] annotation.
- `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>] for `application/json` media type, which also accepts the return type of the resource method, needs to be registered (see JSON section of this chapter).
- User's request has to contain `Accept` header with one of the JavaScript media types defined (see below).

Acceptable media types compatible with `@JSONP` are: `application/javascript`, `application/x-javascript`, `application/ecmascript`, `text/javascript`, `text/x-javascript`, `text/ecmascript`, `text/jscript`.

#### **Example 9.28. Simplest case of using @JSONP**

```
@GET
@JSONP
@Produces({"application/json", "application/javascript"})
public JaxbBean getSimpleJSONP() {
    return new JaxbBean("jsonp");
}
```

Assume that we have registered a JSON providers and that the `JaxbBean` looks like:

#### **Example 9.29. JaxbBean for @JSONP example**

```
@XmlRootElement
public class JaxbBean {
```

```
private String value;

public JaxbBean() {}

public JaxbBean(final String value) {
    this.value = value;
}

public String getValue() {
    return value;
}

public void setValue(final String value) {
    this.value = value;
}
}
```

When you send a GET request with Accept header set to `application/javascript` you'll get a result entity that look like:

```
callback({
    "value" : "jsonp",
})
```

There are, of course, ways to configure wrapping method of the returned entity which defaults to `callback` as you can see in the previous example. `@JSONP` has two parameters that can be configured: `callback` and `queryParam`. `callback` stands for the name of the JavaScript callback function defined by the application. The second parameter, `queryParam`, defines the name of the query parameter holding the name of the callback function to be used (if present in the request). Value of `queryParam` defaults to `__callback` so even if you do not set the name of the query parameter yourself, client can always affect the result name of the wrapping JavaScript callback method.

## Note

`queryParam` value (if set) always takes precedence over `callback` value.

Lets modify our example a little bit:

### Example 9.30. Example of `@JSONP` with configured parameters.

```
@GET
@Produces({"application/json", "application/javascript"})
@JSONP(callback = "eval", queryParam = "jsonpCallback")
public JaxbBean getSimpleJSONP() {
    return new JaxbBean("jsonp");
}
```

And make two requests:

```
curl -X GET http://localhost:8080/jsonp
```

will return

```
eval({
    "value" : "jsonp",
```

```
})
```

and the

```
curl -X GET http://localhost:8080/jsonp?jsonpCallback=alert
```

will return

```
alert({  
    "value" : "jsonp",  
})
```

**Example.** You can take a look at a provided JSON with Padding example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-with-padding>].

## 9.1.7. Java API for JSON Binding (JSON-B)

Jersey uses Yasson [<https://eclipse-ee4j.github.io/yasson/>] for JSON Binding (JSR-367) implementation.

### 9.1.7.1. Dependency

To use JSON-B as your JSON provider you need to add `jersey-media-json-binding` module to your `pom.xml` file:

```
<dependency>  
    <groupId>org.glassfish.jersey.media</groupId>  
    <artifactId>jersey-media-json-binding</artifactId>  
    <version>4.0.0</version>  
</dependency>
```

If you're not using Maven make sure to have all needed dependencies (see `jersey-media-json-binding` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-media-json-binding/dependencies.html>]) on the classpath.

### 9.1.7.2. Configure and register

As stated in Section 4.3, “Auto-Discoverable Features” JSON-Binding media module is one of the modules where you don't need to explicitly register its `Features (JsonBindingFeature)` in your client/server `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] as this feature is automatically discovered and registered when you add `jersey-media-json-binding` module to your classpath.

To use custom preconfigured JSON-B, it is simply possible to register a `ContextResolver<T>` for `Jsonb` in your `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] (client/server) and configure `JsonbConfig` [<https://jakarta.ee/specifications/jsonb/2.0/apidocs/jakarta/json/bind/JsonbConfig>].

#### Example 9.31. `ContextResolver<Jsonb>`

```
@Provider  
public class JsonbContextResolver implements ContextResolver<Jsonb> {  
  
    @Override  
    public Jsonb getContext(Class<?> type) {  
        JsonbConfig config = new JsonbConfig();
```



```
        // configure JsonbConfig
        ...
        return JsonbBuilder.create(config);
    }
}
```

### Example 9.32. Register the feature and ContextResolver<Jsonb>

```
ClientBuilder.newClient(new ClientConfig()
    // The line below that registers JSON-Binding feature can be
    // omitted if FEATURE_AUTO_DISCOVERY_DISABLE is not disabled.
    .register(JsonBindingFeature.class)
    .register(JsonbContextResolver.class)
);
```

**Example.** You can take a look at a provided JSON-B example. [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/json-binding-webapp>].

## 9.1.8. Properties for configuring JSON providers

Apart from using `ContextResolver<T>` for configuring directly the specific JSON provider classes, Jersey supports additional configuration properties prefixed by `JSON` that are available in Section A.6, “Jersey configuration properties for message & entity processing”

## 9.2. XML

As you probably already know, Jersey uses `MessageBodyWriter<T>`s and `MessageBodyReader<T>`s to parse incoming requests and create outgoing responses. Every user can create its own representation but... this is not recommended way how to do things. XML is proven standard for interchanging information, especially in web services. Jerseys supports low level data types used for direct manipulation and JAXB XML entities.

### 9.2.1. Low level XML support

Jersey currently support several low level data types: `StreamSource` [<http://docs.oracle.com/javase/7/docs/api/javax/xml/transform/stream/StreamSource.html>], `SAXSource` [<http://docs.oracle.com/javase/7/docs/api/javax/xml/transform/sax/SAXSource.html>], `DOMSource` [<http://docs.oracle.com/javase/7/docs/api/javax/xml/transform/dom/DOMSource.html>] and `Document` [<http://docs.oracle.com/javase/7/docs/api/org/w3c/dom/Document.html>]. You can use these types as the return type or as a method (resource) parameter. Lets say we want to test this feature and we have helloworld example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/helloworld>] as a starting point. All we need to do is add methods (resources) which consumes and produces XML and types mentioned above will be used.

### Example 9.33. Low level XML test - methods added to `HelloWorldResource.java`

```
1 @POST
2 @Path("StreamSource")
3 public StreamSource getStreamSource(StreamSource streamSource) {
4     return streamSource;
5 }
6
```

```
7 @POST
8 @Path("SAXSource")
9 public SAXSource getSAXSource(SAXSource saxSource) {
10     return saxSource;
11 }
12
13 @POST
14 @Path("DOMSource")
15 public DOMSource getDOMSource(DOMSource domSource) {
16     return domSource;
17 }
18
19 @POST
20 @Path("Document")
21 public Document getDocument(Document document) {
22     return document;
23 }
```

Both `MessageBodyWriter<T>` and `MessageBodyReader<T>` are used in this case, all we need is a POST request with some XML document as a request entity. To keep this as simple as possible only root element with no content will be sent: `<test />`. You can create JAX-RS client to do that or use some other tool, for example `curl`:

```
curl -v http://localhost:8080/base/helloworld/StreamSource -d "<test/>"
```

You should get exactly the same XML from our service as is present in the request; in this case, XML headers are added to response but content stays. Feel free to iterate through all resources.

## 9.2.2. Getting started with JAXB

Good start for people which already have some experience with JAXB annotations is JAXB example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/jaxb>]. You can see various use-cases there. This text is mainly meant for those who don't have prior experience with JAXB. Don't expect that all possible annotations and their combinations will be covered in this chapter, JAXB (JSR 222 implementation) [<http://jaxb.java.net>] is pretty complex and comprehensive. But if you just want to know how you can interchange XML messages with your REST service, you are looking at the right chapter.

Lets start with simple example. Lets say we have class `Planet` and service which produces "Planets".

### Example 9.34. Planet class

```
1 @XmlElement
2 public class Planet {
3     public int id;
4     public String name;
5     public double radius;
6 }
```

### Example 9.35. Resource class

```
1 @Path("planet")
2 public class Resource {
3
4     @GET
```

```
5    @Produces(MediaType.APPLICATION_XML)
6    public Planet getPlanet() {
7        final Planet planet = new Planet();
8
9        planet.id = 1;
10       planet.name = "Earth";
11       planet.radius = 1.0;
12
13       return planet;
14   }
15 }
```

You can see there is some extra annotation declared on `Planet` class, particularly `@XmlRootElement` [<https://eclipse-ee4j.github.io/jaxb-ri/docs/api/jakarta.xml.bind/annotation/XmlRootElement.html>]. This is an JAXB annotation which maps java classes to XML elements. We don't need to specify anything else, because `Planet` is very simple class and all fields are public. In this case, XML element name will be derived from the class name or you can set the name property: `@XmlRootElement(name="yourName")`.

Our resource class will respond to `GET /planet` with

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<planet>
  <id>1</id>
  <name>Earth</name>
  <radius>1.0</radius>
</planet>
```

which might be exactly what we want... or not. Or we might not really care, because we can use JAX-RS client for making requests to this resource and this is easy as:

```
Planet planet = webTarget.path("planet").request(MediaType.APP
```

There is pre-created `WebTarget` object which points to our applications context root and we simply add path (in our case its `planet`), accept header (not mandatory, but service could provide different content based on this header; for example `text/html` can be served for web browsers) and at the end we specify that we are expecting `Planet` class via `GET` request.

There may be need for not just producing XML, we might want to consume it as well.

### Example 9.36. Method for consuming Planet

```
1 @POST
2 @Consumes(MediaType.APPLICATION_XML)
3 public void setPlanet(Planet planet) {
4     System.out.println("setPlanet " + planet);
5 }
```

After valid request is made, service will print out string representation of `Planet`, which can look like `Planet{id=2, name='Mars', radius=1.51}`. With JAX-RS client you can do:

```
webTarget.path("planet").request().post(Entity.xml(planet));
```

If there is a need for some other (non default) XML representation, other JAXB annotations would need to be used. This process is usually simplified by generating java source from XML Schema which is done by `xjc` which is XML to java compiler and it is part of JAXB.

### 9.2.3. POJOs

Sometimes you can't / don't want to add JAXB annotations to source code and you still want to have resources consuming and producing XML representation of your classes. In this case, `JAXBElement` [<https://eclipse-ee4j.github.io/jaxb-ri/docs/api/jakarta.xml.bind/JAXBElement.html>] class should help you. Let's redo planet resource but this time we won't have an `@XmlRootElement` [<https://eclipse-ee4j.github.io/jaxb-ri/docs/api/jakarta.xml.bind/annotation/XmlRootElement.html>] annotation on `Planet` class.

#### Example 9.37. Resource class - `JAXBElement`

```
1 @Path("planet")
2 public class Resource {
3
4     @GET
5     @Produces(MediaType.APPLICATION_XML)
6     public JAXBElement<Planet> getPlanet() {
7         Planet planet = new Planet();
8
9         planet.id = 1;
10        planet.name = "Earth";
11        planet.radius = 1.0;
12
13        return new JAXBElement<Planet>(new QName("planet"), Planet.class, planet);
14    }
15
16    @POST
17    @Consumes(MediaType.APPLICATION_XML)
18    public void setPlanet(JAXBElement<Planet> planet) {
19        System.out.println("setPlanet " + planet.getValue());
20    }
21 }
```

As you can see, everything is little more complicated with `JAXBElement`. This is because now you need to explicitly set element name for `Planet` class XML representation. Client side is even more complicated than server side because you can't do `JAXBElement<Planet>` so JAX-RS client API provides way how to workaround it by declaring subclass of `GenericType<T>`.

#### Example 9.38. Client side - `JAXBElement`

```
1 // GET
2 GenericType<JAXBElement<Planet>> planetType = new GenericType<JAXBElement<Planet>>(){};
3
4 Planet planet = (Planet) webTarget.path("planet").request(MediaType.APPLICATION_XML).get(planetType);
5 System.out.println("### " + planet);
6
7 // POST
8 planet = new Planet();
9
10 // ...
```

```
11
12 webTarget.path("planet").post(new JAXBElement<Planet>(new QName("planet"), Pla
```

## 9.2.4. Using custom JAXBContext

In some scenarios you can take advantage of using custom JAXBContext [https://eclipse-ee4j.github.io/jaxb-ri/docs/api/jakarta.xml.bind/JAXBContext.html]. Creating JAXBContext is an expensive operation and if you already have one created, same instance can be used by Jersey. Other possible use-case for this is when you need to set some specific things to JAXBContext, for example to set a different class loader.

### Example 9.39. PlanetJAXBContextProvider

```
1 @Provider
2 public class PlanetJAXBContextProvider implements ContextResolver<JAXBContext> {
3     private JAXBContext context = null;
4
5     public JAXBContext getContext(Class<?> type) {
6         if (type != Planet.class) {
7             return null; // we don't support anything else than Planet
8         }
9
10        if (context == null) {
11            try {
12                context = JAXBContext.newInstance(Planet.class);
13            } catch (JAXBException e) {
14                // log warning/error; null will be returned which indicates th
15                // provider won't/can't be used.
16            }
17        }
18
19        return context;
20    }
21 }
```

Sample above shows simple JAXBContext creation, all you need to do is put this @Provider annotated class somewhere where Jersey can find it. Users sometimes have problems with using provider classes on client side, so just to reminder - you have to register them in the client config (client does not do anything like package scanning done by server).

### Example 9.40. Using Provider with JAX-RS client

```
1
2 ClientConfig config = new ClientConfig();
3 config.register(PlanetJAXBContextProvider.class);
4
5 Client client = ClientBuilder.newClient(config);
6
```

## 9.2.5. MOXy

If you want to use MOXy [http://www.eclipse.org/eclipselink/moxy.php] as your JAXB implementation instead of JAXB RI you have two options. You can either use the standard JAXB mechanisms

to define the `JAXBContextFactory` from which a `JAXBContext` instance would be obtained (for more on this topic, read JavaDoc on `JAXBContext` [<https://eclipse-ee4j.github.io/jaxb-ri/docs/api/jakarta.xml.bind/JAXBContext.html>]) or you can add `jersey-media-moxy` module to your project and register/configure `MoxyXmlFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/moxy/xml/MoxyXmlFeature.html>] class/instance in the `Configurable` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>].

#### Example 9.41. Add `jersey-media-moxy` dependency.

```
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-moxy</artifactId>
  <version>4.0.0</version>
</dependency>
```

#### Example 9.42. Register the `MoxyXmlFeature` class.

```
1 final ResourceConfig config = new ResourceConfig()
2   .packages("org.glassfish.jersey.examples.xmlmoxy")
3   .register(MoxyXmlFeature.class);
```

#### Example 9.43. Configure and register an `MoxyXmlFeature` instance.

```
1 // Configure Properties.
2 final Map<String, Object> properties = new HashMap<String, Object>();
3 // ...
4
5 // Obtain a ClassLoader you want to use.
6 final ClassLoader classLoader = Thread.currentThread().getContextClassLoader()
7
8 final ResourceConfig config = new ResourceConfig()
9   .packages("org.glassfish.jersey.examples.xmlmoxy")
10  .register(new MoxyXmlFeature(
11    properties,
12    classLoader,
13    true, // Flag to determine whether eclipselink-oxm.xml file should be
14    CustomClassA.class, CustomClassB.class // Classes to be bound.
15  ));
```

## 9.2.6. Properties for configuring XML providers

Apart from using `ContextResolver<T>` for configuring directly the specific JSON provider classes, Jersey supports additional configuration properties prefixed by `JAXB` and `XML` that are available in Section A.6, “Jersey configuration properties for message & entity processing”

## 9.3. Multipart

### 9.3.1. Overview

The classes in this module provide an integration of `multipart/*` request and response bodies in a JAX-RS runtime environment. The set of registered providers is leveraged, in that the content type for a

body part of such a message reuses the same `MessageBodyReader<T>/MessageBodyWriter<T>` implementations as would be used for that content type as a standalone entity.

The following list of general MIME MultiPart features is currently supported:

- The `MIME-Version: 1.0` HTTP header is included on generated responses. It is accepted, but not required, on processed requests.
- A `MessageBodyReader<T>` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html] implementation for consuming MIME MultiPart entities.
- A `MessageBodyWriter<T>` implementation for producing MIME MultiPart entities. The appropriate `@Provider` is used to serialize each body part, based on its media type.
- Optional creation of an appropriate `boundary` parameter on a generated `Content-Type` header, if not already present.

For more information refer to Multi Part [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/package-summary.html].

### 9.3.1.1. Dependency

To use multipart features you need to add `jersey-media-multipart` module to your `pom.xml` file:

```
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-multipart</artifactId>
  <version>4.0.0</version>
</dependency>
```

If you're not using Maven make sure to have all needed dependencies (see `jersey-media-multipart` [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-media-multipart/dependencies.html]) on the class-path.

### 9.3.1.2. Registration

Prior to Jersey 3.1.0, before you can use the capabilities of the `jersey-media-multipart` module in your client/server code, you need to register `MultiPartFeature` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartFeature.html]. The multipart feature is supported by Jakarta RESTful Web Services 3.1 multipart API. From Jersey 3.1.0 on, the `MultiPartFeature` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartFeature.html] is no longer required to be registered and it is registered automatically.

### 9.3.1.3. Examples

Jersey provides a Multipart Web Application Example [https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/multipart-webapp] on how to use multipart features.

## 9.3.2. Client

- Client using Jersey API
- Client using Jakarta REST API

### 9.3.2.1. Client using Jersey API

`MultiPart` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPart.html>] class (or its subclasses) can be used as an entry point to use `jersey-media-multipart` module on the client side. This class represents a MIME multipart message [[http://en.wikipedia.org/wiki/MIME#Multipart\\_messages](http://en.wikipedia.org/wiki/MIME#Multipart_messages)] and is able to hold an arbitrary number of `BodyPart` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/BodyPart.html>]. Default media type is `multipart/mixed` [<http://en.wikipedia.org/wiki/MIME#Mixed>] for `MultiPart` entity and `text/plain` for `BodyPart`.

#### Example 9.44. `MultiPart` entity

```
final MultiPart multiPartEntity = new MultiPart()
    .bodyPart(new BodyPart().entity("hello"))
    .bodyPart(new BodyPart(new JAXBBean("xml"), MediaType.APPLICATION_XML_TYPE))
    .bodyPart(new BodyPart(new JAXBBean("json"), MediaType.APPLICATION_JSON_TYPE));

final WebTarget target = // Create WebTarget.
final Response response = target
    .request()
    .post(Entity.entity(multiPartEntity, multiPartEntity.getMediaType()));
```

If you send a `multiPartEntity` to the server the entity with `Content-Type` header in HTTP message would look like:

#### Example 9.45. `MultiPart` entity in HTTP message.

```
Content-Type: multipart/mixed; boundary=Boundary_1_829077776_1369128119878

--Boundary_1_829077776_1369128119878
Content-Type: text/plain

hello
--Boundary_1_829077776_1369128119878
Content-Type: application/xml

<?xml version="1.0" encoding="UTF-8" standalone="yes"?><jaxbBean><value>xml</value>
--Boundary_1_829077776_1369128119878
Content-Type: application/json

{"value": "json"}
--Boundary_1_829077776_1369128119878--
```

When working with forms (e.g. media type `multipart/form-data`) and various fields in them, there is a more convenient class to be used - `FormDataMultiPart` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/FormDataMultiPart.html>]. It automatically sets the media type for the `FormDataMultiPart` entity to `multipart/form-data` and `Content-Disposition` header to `FormDataBodyPart` body parts.

#### Example 9.46. `FormDataMultiPart` entity

```
final FormDataMultiPart multipart = new FormDataMultiPart()
    .field("hello", "hello")
    .field("xml", new JAXBBean("xml"))
```



```
.field("json", new JAXBBean("json"), MediaType.APPLICATION_JSON_TYPE);

final WebTarget target = // Create WebTarget.
final Response response = target.request().post(Entity.entity(multipart, multipart
```

To illustrate the difference when using `FormDataMultiPart` instead of `FormDataBodyPart` you can take a look at the `FormDataMultiPart` entity from HTML message:

#### **Example 9.47. `FormDataMultiPart` entity in HTTP message.**

```
Content-Type: multipart/form-data; boundary=Boundary_1_511262261_1369143433608

--Boundary_1_511262261_1369143433608
Content-Type: text/plain
Content-Disposition: form-data; name="hello"

hello
--Boundary_1_511262261_1369143433608
Content-Type: application/xml
Content-Disposition: form-data; name="xml"

<?xml version="1.0" encoding="UTF-8" standalone="yes"?><jaxbBean><value>xml</value>
--Boundary_1_511262261_1369143433608
Content-Type: application/json
Content-Disposition: form-data; name="json"

{"value":"json"}
--Boundary_1_511262261_1369143433608--
```

A common use-case for many users is sending files from client to server. For this purpose you can use classes from `org.glassfish.jersey.media.multipart` package, such as `FileDataBodyPart` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/file/FileDataBodyPart.html>] or `StreamDataBodyPart` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/file/StreamDataBodyPart.html>].

#### **Example 9.48. Multipart - sending files.**

```
// MediaType of the body part will be derived from the file.
final FileDataBodyPart filePart = new FileDataBodyPart("my_pom", new File("pom.xml"))

final FormDataMultiPart multipart = new FormDataMultiPart()
    .field("foo", "bar")
    .bodyPart(filePart);

final WebTarget target = // Create WebTarget.
final Response response = target.request()
    .post(Entity.entity(multipart, multipart.getMediaType()));
```

### **Warning**

Do not use `ApacheConnectorProvider` nor `GrizzlyConnectorProvider` neither `JettyConnectorProvider` connector implementations with Jersey Multipart features. See [Header modification issue warning](#) for more details.

### 9.3.2.2. Client using Jakarta REST API

EntityPart [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/EntityPart.html] interface can be used as an entry point to use `jersey-media-multipart` module on the client side. This class represents multipart message is able to hold an arbitrary number of EntityPart [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/EntityPart.html]s. Default media type is multipart/form-data [http://en.wikipedia.org/wiki/MIME#Mixed].

#### Example 9.49. Using EntityPart.Builder for building an Entity

```
final List<EntityPart> multiPartEntity = new List<>();
list.add(EntityPart.withName("part-01").content("hello").build());
list.add(EntityPart.withName("part-01").content(new JAXBBean("xml")).mediaType(MediaType.APPLICATION_XML));
list.add(EntityPart.withName("part-02").content(new JAXBBean("json")).mediaType(MediaType.APPLICATION_JSON));
final GenericEntity<List<EntityPart>> genericEntity = new GenericEntity<>(list) {}
final Entity entity = Entity.entity(genericEntity, MediaType.MULTIPART_FORM_DATA_T

final WebTarget target = // Create WebTarget.
final Response response = target.request().post(entity);
```

The common use-case for many users is sending files from client to server. It is also covered by EntityPart [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/EntityPart.html].Builder.

#### Example 9.50. EntityPart - sending files.

```
// MediaType of the body part will be derived from the file.
final List<EntityPart> multiPartEntity = new List<>();
list.add(EntityPart.withFileName("file001.txt").content(Files.newInputStream(Path.of("file001.txt"))));
list.add(EntityPart.withFileName("mypom.xml").content(Files.newInputStream(Path.of("mypom.xml"))));

final GenericEntity<List<EntityPart>> genericEntity = new GenericEntity<>(list) {}
final Entity entity = Entity.entity(genericEntity, MediaType.MULTIPART_FORM_DATA_T

final WebTarget target = // Create WebTarget.
final Response response = target.request().post(entity);
```

## 9.3.3. Server

- Server using Jersey API
- Server using Jakarta REST API

### 9.3.3.1. Jersey Server API

Returning a multipart response from server to client is not much different from the parts described in the client section above. To obtain a multipart entity, sent by a client, in the application you can use two approaches:

- Injecting the whole MultiPart [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPart.html] entity.

- Injecting particular parts of a `form-data` multipart request via `@FormDataParam` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/FormDataParam.html>] annotation.

### 9.3.3.1.1. Injecting and returning the `MultiPart` entity

Working with `MultiPart` types is no different from injecting/returning other entity types. Jersey provides `MessageBodyReader<T>` for reading the request entity and injecting this entity into a method parameter of a resource method and `MessageBodyWriter<T>` for writing output entities. You can expect that either `MultiPart` or `FormDataMultiPart` (multipart/form-data media type) object to be injected into a resource method.

**Example 9.51. Resource method using `MultiPart` as input parameter / return value.**

```
@POST
@Produces("multipart/mixed")
public MultiPart post(final FormDataMultiPart multiPart) {
    return multiPart;
}
```

### 9.3.3.1.2. Injecting with `@FormDataParam`

If you just need to bind the named body part(s) of a `multipart/form-data` request entity body to a resource method parameter you can use `@FormDataParam` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/FormDataParam.html>] annotation.

This annotation in conjunction with the media type `multipart/form-data` should be used for submitting and consuming forms that contain files, non-ASCII data, and binary data.

The type of the annotated parameter can be one of the following (for more detailed description see javadoc to `@FormDataParam` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/FormDataParam.html>]):

- `FormDataBodyPart` - The value of the parameter will be the first named body part or `null` if such a named body part is not present.
- A `List` or `Collection` of `FormDataBodyPart`. The value of the parameter will be one or more named body parts with the same name or `null` if such a named body part is not present.
- `FormDataContentDisposition` - The value of the parameter will be the content disposition of the first named body part or `null` if such a named body part is not present.
- A `List` or `Collection` of `FormDataContentDisposition`. The value of the parameter will be one or more content dispositions of the named body parts with the same name or `null` if such a named body part is not present.
- A type for which a message body reader is available given the media type of the first named body part. The value of the parameter will be the result of reading using the message body reader given the type `T`, the media type of the named part, and the bytes of the named body part as input.

If there is no named part present and there is a default value present as declared by `@DefaultValue` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/DefaultValue.html>] then the media type will be set to `text/plain`. The value of the parameter will be the result of reading using the message body reader given the type `T`, the media type `text/plain`, and the UTF-8 encoded bytes of the default value as input.

If there is no message body reader available and the type `T` conforms to a type specified by `@FormParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/FormParam.html>] then processing is performed as specified by `@FormParam`, where the values of the form parameter are `String` instances produced by reading the bytes of the named body parts utilizing a message body reader for the `String` type and the media type `text/plain`.

If there is no named part present then processing is performed as specified by `@FormParam`.

### Example 9.52. Use of `@FormDataParam` annotation

```
@POST
@Consumes(MediaType.MULTIPART_FORM_DATA)
public String postForm(
    @DefaultValue("true") @FormDataParam("enabled") boolean enabled,
    @FormDataParam("data") FileData bean,
    @FormDataParam("file") InputStream file,
    @FormDataParam("file") FormDataContentDisposition fileDisposition) {

    // ...
}
```

In the example above the server consumes a `multipart/form-data` request entity body that contains one optional named body part `enabled` and two required named body parts `data` and `file`.

The optional part `enabled` is processed as a `boolean` value, if the part is absent then the value will be `true`.

The part `data` is processed as a JAXB bean and contains some meta-data about the following part.

The part `file` is a file that is uploaded, this is processed as an `InputStream`. Additional information about the file from the `Content-Disposition` header can be accessed by the parameter `fileDisposition`.

### Tip

`@FormDataParam` annotation can be also used on fields.

## 9.3.3.2. Properties for configuring the Multipart

There are multiple options that can be used when configuring the multipart. See `MultiPartProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartProperties.html>] or Section A.13, “Multipart configuration properties” for the possibilities.

The options can set in a configuration file specified by the `MultiPartProperties.MULTI_PART_CONFIG_RESOURCE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartProperties.html#MULTI\\_PART\\_CONFIG\\_RESOURCE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartProperties.html#MULTI_PART_CONFIG_RESOURCE)] property. That is the standard Java properties file.

Or the options can be set programmatically, by registering `ContextResolver<MultiPartProperties>`. For instance:

```
ResourceConfig resourceConfig = new ResourceConfig();
```

```
resourceConfig.register(new MultiPartProperties().bufferThreshold(65535).maxParts(
```

### 9.3.3.3. Server using Jakarta REST API

Using `EntityPart` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/EntityPart.html>] on the server side is similar to the client side. Jakarta REST specification allows for returning a `Response` or a `List` of `EntityParts`.

Receiving the `EntityPart` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/EntityPart.html>]s can be done either using `@FormParam` annotations and `EntityPart`, `InputStream` or `String` data-types, or using a `List` of `EntityParts`.

#### Example 9.53. Use of `@FormParam` annotation with `EntityPart` `InputStream` and `String` types and returning a `Response`

```
@POST
@Path("/postFormVarious")
public Response postFormVarious(@FormParam("name1") EntityPart part1,
                                @FormParam("name2") InputStream part2,
                                @FormParam("name3") String part3) throws IOException {
    final List<EntityPart> list = new LinkedList<>();
    list.add(EntityPart.withName(part1.getName())
        .content(part1.getContent(String.class) + new String(part2.readAllBytes()))
        .mediaType(MediaType.TEXT_PLAIN_TYPE)
        .build());
    final GenericEntity<List<EntityPart>> genericEntity = new GenericEntity<>(list);
    return Response.ok(genericEntity, MediaType.MULTIPART_FORM_DATA_TYPE).build();
}
```

#### Example 9.54. Receiving a `List` of `EntityParts`

```
@POST
@Path("/postListForm")
public String postEntityPartForm(@FormParam("part-0x") List<EntityPart> part) throws IOException {
    final String entity = part.get(0).getContent(String.class) + part.get(1).getContent(String.class);
    return entity;
}
```

#### Example 9.55. Returning a `List` of `EntityParts`

```
@GET
@Produces(MediaType.MULTIPART_FORM_DATA)
@Path("/getList")
public List<EntityPart> getList() throws IOException {
    final List<EntityPart> list = new LinkedList<>();
    list.add(EntityPart.withName("name1").content("data1").build());
    return list;
}
```

---

# Chapter 10. Filters and Interceptors

## 10.1. Introduction

This chapter describes filters, interceptors and their configuration. Filters and interceptors can be used on both sides, on the client and the server side. Filters can modify inbound and outbound requests and responses including modification of headers, entity and other request/response parameters. Interceptors are used primarily for modification of entity input and output streams. You can use interceptors for example to zip and unzip output and input entity streams.

## 10.2. Filters

Filters can be used when you want to modify any request or response parameters like headers. For example you would like to add a response header "X-Powered-By" to each generated response. Instead of adding this header in each resource method you would use a response filter to add this header.

There are filters on the server side and the client side.

Server filters:

ContainerRequestFilter                      [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerRequestFilter.html>]

ContainerResponseFilter                    [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerResponseFilter.html>]

Client filters:

ClientRequestFilter                        [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientRequestFilter.html>]

ClientResponseFilter                       [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientResponseFilter.html>]

### 10.2.1. Server filters

The following example shows a simple container response filter adding a header to each response.

#### Example 10.1. Container response filter

```
1 import java.io.IOException;
2 import jakarta.ws.rs.container.ContainerRequestContext;
3 import jakarta.ws.rs.container.ContainerResponseContext;
4 import jakarta.ws.rs.container.ContainerResponseFilter;
5 import jakarta.ws.rs.core.Response;
6
7 public class PoweredByResponseFilter implements ContainerResponseFilter {
8
9     @Override
10    public void filter(ContainerRequestContext requestContext, ContainerResponseContext responseContext)
11        throws IOException {
12
13        responseContext.getHeaders().add("X-Powered-By", "Jersey :-)");
14    }
```

15 }

In the example above the `PoweredByResponseFilter` always adds a header "X-Powered-By" to the response. The filter must inherit from the `ContainerResponseFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerResponseFilter.html>] and must be registered as a provider. The filter will be executed for every response which is in most cases after the resource method is executed. Response filters are executed even if the resource method is not run, for example when the resource method is not found and 404 "Not found" response code is returned by the Jersey runtime. In this case the filter will be executed and will process the 404 response.

The `filter()` method has two arguments, the container request and container response. The `ContainerRequestContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerRequestContext.html>] is accessible only for read only purposes as the filter is executed already in response phase. The modifications can be done in the `ContainerResponseContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerResponseContext.html>].

The following example shows the usage of a request filter.

### Example 10.2. Container request filter

```
1 import java.io.IOException;
2 import jakarta.ws.rs.container.ContainerRequestContext;
3 import jakarta.ws.rs.container.ContainerRequestFilter;
4 import jakarta.ws.rs.core.Response;
5 import jakarta.ws.rs.core.SecurityContext;
6
7 public class AuthorizationRequestFilter implements ContainerRequestFilter {
8
9     @Override
10    public void filter(ContainerRequestContext requestContext)
11        throws IOException {
12
13        final SecurityContext securityContext =
14            requestContext.getSecurityContext();
15        if (securityContext == null ||
16            !securityContext.isUserInRole("privileged")) {
17
18            requestContext.abortWith(Response
19                .status(Response.Status.UNAUTHORIZED)
20                .entity("User cannot access the resource.")
21                .build());
22        }
23    }
24 }
```

The request filter is similar to the response filter but does not have access to the `ContainerResponseContext` as no response is accessible yet. Response filter inherits from `ContainerResponseFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerResponseFilter.html>]. Request filter is executed before the resource method is run and before the response is created. The filter has possibility to manipulate the request parameters including request headers or entity.

The `AuthorizationRequestFilter` in the example checks whether the authenticated user is in the privileged role. If it is not then the request is *aborted* by calling `ContainerRequestContext.abortWith(Response response)` method. The method is

intended to be called from the request filter in situation when the request should not be processed further in the standard processing chain. When the `filter` method is finished the response passed as a parameter to the `abortWith` method is used to respond to the request. Response filters, if any are registered, will be executed and will have possibility to process the aborted response.

### 10.2.1.1. Pre-matching and post-matching filters

All the request filters shown above was implemented as post-matching filters. It means that the filters would be applied only after a suitable resource method has been selected to process the actual request i.e. after request matching happens. Request matching is the process of finding a resource method that should be executed based on the request path and other request parameters. Since post-matching request filters are invoked when a particular resource method has already been selected, such filters can not influence the resource method matching process.

To overcome the above described limitation, there is a possibility to mark a server request filter as a *pre-matching* filter, i.e. to annotate the filter class with the `@PreMatching` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/PreMatching.html>] annotation. Pre-matching filters are request filters that are executed before the request matching is started. Thanks to this, pre-matching request filters have the possibility to influence which method will be matched. Such a pre-matching request filter example is shown here:

#### Example 10.3. Pre-matching request filter

```
1 ...
2 import jakarta.ws.rs.container.ContainerRequestContext;
3 import jakarta.ws.rs.container.ContainerRequestFilter;
4 import jakarta.ws.rs.container.PreMatching;
5 ...
6
7 @PreMatching
8 public class PreMatchingFilter implements ContainerRequestFilter {
9
10     @Override
11     public void filter(ContainerRequestContext requestContext)
12         throws IOException {
13         // change all PUT methods to POST
14         if (requestContext.getMethod().equals("PUT")) {
15             requestContext.setMethod("POST");
16         }
17     }
18 }
```

The `PreMatchingFilter` is a simple pre-matching filter which changes all PUT HTTP methods to POST. This might be useful when you want to always handle these PUT and POST HTTP methods with the same Java code. After the `PreMatchingFilter` has been invoked, the rest of the request processing will behave as if the POST HTTP method was originally used. You cannot do this in post-matching filters (standard filters without `@PreMatching` annotation) as the resource method is already matched (selected). An attempt to tweak the original HTTP method in a post-matching filter would cause an `IllegalArgumentException`.

As written above, pre-matching filters can fully influence the request matching process, which means you can even modify request URI in a pre-matching filter by invoking the `setRequestUri(URI)` method of `ContainerRequestFilter` so that a different resource would be matched.

Like in post-matching filters you can abort a response in pre-matching filters too.



## 10.2.2. Client filters

Client filters are similar to container filters. The response can also be aborted in the `ClientRequestFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientRequestFilter.html>] which would cause that no request will actually be sent to the server at all. A new response is passed to the `abort` method. This response will be used and delivered as a result of the request invocation. Such a response goes through the client response filters. This is similar to what happens on the server side. The process is shown in the following example:

### Example 10.4. Client request filter

```
1 public class CheckRequestFilter implements ClientRequestFilter {
2
3     @Override
4     public void filter(ClientRequestContext requestContext)
5         throws IOException {
6         if (requestContext.getHeaders(
7             ).get("Client-Name") == null) {
8             requestContext.abortWith(
9                 Response.status(Response.Status.BAD_REQUEST)
10                    .entity("Client-Name header must be defined.")
11                    .build());
12         }
13     }
14 }
```

The `CheckRequestFilter` validates the outgoing request. It is checked for presence of a `Client-Name` header. If the header is not present the request will be aborted with a made up response with an appropriate code and message in the entity body. This will cause that the original request will not be effectively sent to the server but the actual invocation will still end up with a response as if it would be generated by the server side. If there would be any client response filter it would be executed on this response.

To summarize the workflow, for any client request invoked from the client API the client request filters (`ClientRequestFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientRequestFilter.html>]) are executed that could manipulate the request. If not aborted, the outgoing request is then physically sent over to the server side and once a response is received back from the server the client response filters (`ClientResponseFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/ClientResponseFilter.html>]) are executed that might again manipulate the returned response. Finally the response is passed back to the code that invoked the request. If the request was aborted in any client request filter then the client/server communication is skipped and the aborted response is used in the response filters.

## 10.3. Interceptors

Interceptors share a common API for the server and the client side. Whereas filters are primarily intended to manipulate request and response parameters like HTTP headers, URIs and/or HTTP methods, interceptors are intended to manipulate entities, via manipulating entity input/output streams. If you for example need to encode entity body of a client request then you could implement an interceptor to do the work for you.

There are two kinds of interceptors, `ReaderInterceptor` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ReaderInterceptor.html>] and `WriterInterceptor` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/WriterInterceptor.html>]. Reader interceptors are used to manipulate

inbound entity streams. These are the streams coming from the "wire". So, using a reader interceptor you can manipulate request entity stream on the server side (where an entity is read from the client request) and response entity stream on the client side (where an entity is read from the server response). Writer interceptors are used for cases where entity is written to the "wire" which on the server means when writing out a response entity and on the client side when writing request entity for a request to be sent out to the server. Writer and reader interceptors are executed before message body readers or writers are executed and their primary intention is to wrap the entity streams that will be used in message body reader and writers.

The following example shows a writer interceptor that enables GZIP compression of the whole entity body.

### Example 10.5. GZIP writer interceptor

```
1 public class GZIPWriterInterceptor implements WriterInterceptor {
2
3     @Override
4     public void aroundWriteTo(WriterInterceptorContext context)
5         throws IOException, WebApplicationException {
6         final OutputStream outputStream = context.getOutputStream();
7         context.setOutputStream(new GZIPOutputStream(outputStream));
8         context.proceed();
9     }
10 }
```

The interceptor gets an output stream from the `WriterInterceptorContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/WriterInterceptorContext.html>] and sets a new one which is a GZIP wrapper of the original output stream. After all interceptors are executed the output stream lastly set to the `WriterInterceptorContext` will be used for serialization of the entity. In the example above the entity bytes will be written to the `GZIPOutputStream` which will compress the stream data and write them to the original output stream. The original stream is always the stream which writes the data to the "wire". When the interceptor is used on the server, the original output stream is the stream into which writes data to the underlying server container stream that sends the response to the client.

The interceptors wrap the streams and they itself work as wrappers. This means that each interceptor is a wrapper of another interceptor and it is responsibility of each interceptor implementation to call the wrapped interceptor. This is achieved by calling the `proceed()` method on the `WriterInterceptorContext`. This method will call the next registered interceptor in the chain, so effectively this will call all remaining registered interceptors. Calling `proceed()` from the last interceptor in the chain will call the appropriate message body reader. Therefore every interceptor must call the `proceed()` method otherwise the entity would not be written. The wrapping principle is reflected also in the method name, `aroundWriteTo`, which says that the method is wrapping the writing of the entity.

The method `aroundWriteTo()` gets `WriterInterceptorContext` as a parameter. This context contains getters and setters for header parameters, request properties, entity, entity stream and other properties. These are the properties which will be passed to the final `MessageBodyWriter<T>`. Interceptors are allowed to modify all these properties. This could influence writing of an entity by `MessageBodyWriter<T>` and even selection of such a writer. By changing media type (`WriterInterceptorContext.setMediaType()`) the interceptor can cause that different message body writer will be chosen. The interceptor can also completely replace the entity if it is needed. However, for modification of headers, request properties and such, the filters are usually more preferable choice. Interceptors are executed only when there is any entity and when the entity is to be written. So, when you always want to add a new header to a response no matter what, use filters as interceptors might not be executed when no entity is present. Interceptors should modify properties only for entity serialization and deserialization purposes.

Let's now look at an example of a `ReaderInterceptor`

**Example 10.6. GZIP reader interceptor**

```
1 public class GZIPReaderInterceptor implements ReaderInterceptor {
2
3     @Override
4     public Object aroundReadFrom(ReaderInterceptorContext context)
5         throws IOException, WebApplicationException {
6         final InputStream originalInputStream = context.getInputStream();
7         context.setInputStream(new GZIPInputStream(originalInputStream));
8         return context.proceed();
9     }
10 }
```

The `GZIPReaderInterceptor` wraps the original input stream with the `GZIPInputStream`. All further reads from the entity stream will cause that data will be decompressed by this stream. The interceptor method `aroundReadFrom()` must return an entity. The entity is returned from the `proceed` method of the `ReaderInterceptorContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ReaderInterceptorContext.html>]. The `proceed` method internally calls the wrapped interceptor which must also return an entity. The `proceed` method invoked from the last interceptor in the chain calls message body reader which deserializes the entity and returns it. Every interceptor can change this entity if there is a need but in the most cases interceptors will just return the entity as returned from the `proceed` method.

As already mentioned above, interceptors should be primarily used to manipulate entity body. Similar to methods exposed by `WriterInterceptorContext` the `ReaderInterceptorContext` introduces a set of methods for modification of request/response properties like HTTP headers, URIs and/or HTTP methods (excluding getters and setters for entity as entity has not been read yet). Again the same rules as for `WriterInterceptor` applies for changing these properties (change only properties in order to influence reading of an entity).

## 10.4. Filter and interceptor execution order

Let's look closer at the context of execution of filters and interceptors. The following steps describes scenario where a JAX-RS client makes a POST request to the server. The server receives an entity and sends a response back with the same entity. GZIP reader and writer interceptors are registered on the client and the server. Also filters are registered on client and server which change the headers of request and response.

1. Client request invoked: The POST request with attached entity is built on the client and invoked.
2. ClientRequestFilters: client request filters are executed on the client and they manipulate the request headers.
3. Client `WriterInterceptor`: As the request contains an entity, writer interceptor registered on the client is executed before a `MessageBodyWriter` is executed. It wraps the entity output stream with the `GZipOutputStream`.
4. Client `MessageBodyWriter`: message body writer is executed on the client which serializes the entity into the new `GZipOutput` stream. This stream zips the data and sends it to the "wire".
5. Server: server receives a request. Data of the entity is compressed which means that pure read from the entity input stream would return compressed data.
6. Server pre-matching `ContainerRequestFilters`: `ContainerRequestFilters` are executed that can manipulate resource method matching process.

7. Server: matching: resource method matching is done.
8. Server: post-matching ContainerRequestFilters: ContainerRequestFilters post matching filters are executed. This include execution of all global filters (without name binding) and filters name-bound to the matched method.
9. Server **ReaderInterceptor**: reader interceptors are executed on the server. The **GZIPReaderInterceptor** wraps the input stream (the stream from the "wire") into the **GZipInputStream** and set it to context.
10. Server **MessageBodyReader**: server message body reader is executed and it deserializes the entity from new **GZipInputStream** (get from the context). This means the reader will read unzipped data and not the compressed data from the "wire".
11. Server resource method is executed: the deserialized entity object is passed to the matched resource method as a parameter. The method returns this entity as a response entity.
12. Server **ContainerResponseFilters** are executed: response filters are executed on the server and they manipulate the response headers. This include all global bound filters (without name binding) and all filters name-bound to the resource method.
13. Server **WriterInterceptor**: is executed on the server. It wraps the original output stream with a new **GZIPOutputStream**. The original stream is the stream that "goes to the wire" (output stream for response from the underlying server container).
14. Server **MessageBodyWriter**: message body writer is executed on the server which serializes the entity into the **GZIPOutputStream**. This stream compresses the data and writes it to the original stream which sends this compressed data back to the client.
15. Client receives the response: the response contains compressed entity data.
16. Client **ClientResponseFilters**: client response filters are executed and they manipulate the response headers.
17. Client response is returned: the `jakarta.ws.rs.core.Response` is returned from the request invocation.
18. Client code calls `response.readEntity()`: read entity is executed on the client to extract the entity from the response.
19. Client **ReaderInterceptor**: the client reader interceptor is executed when `readEntity(Class)` is called. The interceptor wraps the entity input stream with **GZIPInputStream**. This will decompress the data from the original input stream.
20. Client **MessageBodyReaders**: client message body reader is invoked which reads decompressed data from **GZIPInputStream** and deserializes the entity.
21. Client: The entity is returned from the `readEntity()`.

It is worth to mention that in the scenario above the reader and writer interceptors are invoked only if the entity is present (it does not make sense to wrap entity stream when no entity will be written). The same behaviour is there for message body readers and writers. As mentioned above, interceptors are executed before the message body reader/writer as a part of their execution and they can wrap the input/output stream before the entity is read/written. There are exceptions when interceptors are not run before message body reader/writers but this is not the case of simple scenario above. This happens for example when the entity is read many times from client response using internal buffering. Then the data are intercepted only once and kept 'decoded' in the buffer.

## 10.5. Name binding

Filters and interceptors can be *name-bound*. Name binding is a concept that allows to say to a JAX-RS runtime that a specific filter or interceptor will be executed only for a specific resource method. When a filter or an interceptor is limited only to a specific resource method we say that it is *name-bound*. Filters and interceptors that do not have such a limitation are called *global*.

Filter or interceptor can be assigned to a resource method using the `@NameBinding` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/NameBinding.html>] annotation. The annotation is used as meta annotation for other user implemented annotations that are applied to a providers and resource methods. See the following example:

### Example 10.7. `@NameBinding` example

```
1 ...
2 import java.lang.annotation.Retention;
3 import java.lang.annotation.RetentionPolicy;
4 import java.util.zip.GZIPInputStream;
5
6 import jakarta.ws.rs.GET;
7 import jakarta.ws.rs.NameBinding;
8 import jakarta.ws.rs.Path;
9 import jakarta.ws.rs.Produces;
10 ...
11
12
13 // @Compress annotation is the name binding annotation
14 @NameBinding
15 @Retention(RetentionPolicy.RUNTIME)
16 public @interface Compress {}
17
18
19 @Path("helloworld")
20 public class HelloWorldResource {
21
22     @GET
23     @Produces("text/plain")
24     public String getHello() {
25         return "Hello World!";
26     }
27
28     @GET
29     @Path("too-much-data")
30     @Compress
31     public String getVeryLongString() {
32         String str = ... // very long string
33         return str;
34     }
35 }
36
37 // interceptor will be executed only when resource methods
38 // annotated with @Compress annotation will be executed
39 @Compress
```

```
40 public class GZIPWriterInterceptor implements WriterInterceptor {
41     @Override
42     public void aroundWriteTo(WriterInterceptorContext context)
43         throws IOException, WebApplicationException {
44         final OutputStream outputStream = context.getOutputStream();
45         context.setOutputStream(new GZIPOutputStream(outputStream));
46         context.proceed();
47     }
48 }
```

The example above defines a new `@Compress` annotation which is a name binding annotation as it is annotated with `@NameBinding`. The `@Compress` is applied on the resource method `getVeryLongString()` and on the interceptor `GZIPWriterInterceptor`. The interceptor will be executed only if any resource method with such an annotation will be executed. In our example case the interceptor will be executed only for the `getVeryLongString()` method. The interceptor will not be executed for method `getHello()`. In this example the reason is probably clear. We would like to compress only long data and we do not need to compress the short response of "Hello World!".

Name binding can be applied on a resource class. In the example `HelloWorldResource` would be annotated with `@Compress`. This would mean that all resource methods will use compression in this case.

There might be many name binding annotations defined in an application. When any provider (filter or interceptor) is annotated with more than one name binding annotation, then it will be executed for resource methods which contain ALL these annotations. So, for example if our interceptor would be annotated with another name binding annotation `@GZIP` then the resource method would need to have both annotations attached, `@Compress` and `@GZIP`, otherwise the interceptor would not be executed. Based on the previous paragraph we can even use the combination when the resource method `getVeryLongString()` would be annotated with `@Compress` and resource class `HelloWorldResource` would be annotated from with `@GZIP`. This would also trigger the interceptor as annotations of resource methods are aggregated from resource method and from resource class. But this is probably just an edge case which will not be used so often.

Note that *global filters are always executed*, even for resource methods which have any name binding annotations.

## 10.6. Dynamic binding

Dynamic binding is a way how to assign filters and interceptors to the resource methods in a dynamic manner. Name binding from the previous chapter uses a static approach and changes to binding require source code change and recompilation. With dynamic binding you can implement code which defines bindings during the application initialization time. The following example shows how to implement dynamic binding.

### Example 10.8. Dynamic binding example

```
1 ...
2 import jakarta.ws.rs.core.FeatureContext;
3 import jakarta.ws.rs.container.DynamicFeature;
4 ...
5
6 @Path("helloworld")
7 public class HelloWorldResource {
8
9     @GET
```

```
10     @Produces("text/plain")
11     public String getHello() {
12         return "Hello World!";
13     }
14
15     @GET
16     @Path("too-much-data")
17     public String getVeryLongString() {
18         String str = ... // very long string
19         return str;
20     }
21 }
22
23 // This dynamic binding provider registers GZIPWriterInterceptor
24 // only for HelloWorldResource and methods that contain
25 // "VeryLongString" in their name. It will be executed during
26 // application initialization phase.
27 public class CompressionDynamicBinding implements DynamicFeature {
28
29     @Override
30     public void configure(ResourceInfo resourceInfo, FeatureContext context) {
31         if (HelloWorldResource.class.equals(resourceInfo.getResourceClass())
32             && resourceInfo.getResourceMethod()
33                 .getName().contains("VeryLongString")) {
34             context.register(GZIPWriterInterceptor.class);
35         }
36     }
37 }
```

The example contains one `HelloWorldResource` which is known from the previous name binding example. The difference is in the `getVeryLongString` method, which now does not define the `@Compress` name binding annotations. The binding is done using the provider which implements `DynamicFeature` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/DynamicFeature.html>] interface. The interface defines one `configure` method with two arguments, `ResourceInfo` and `FeatureContext`. `ResourceInfo` contains information about the resource and method to which the binding can be done. The `configure` method will be executed once for each resource method that is defined in the application. In the example above the provider will be executed twice, once for the `getHello()` method and once for `getVeryLongString()` (once the `resourceInfo` will contain information about `getHello()` method and once it will point to `getVeryLongString()`). If a dynamic binding provider wants to register any provider for the actual resource method it will do that using provided `FeatureContext` which extends JAX-RS Configurable API. All methods for registration of filter or interceptor classes or instances can be used. Such dynamically registered filters or interceptors will be bound only to the actual resource method. In the example above the `GZIPWriterInterceptor` will be bound only to the method `getVeryLongString()` which will cause that data will be compressed only for this method and not for the method `getHello()`. The code of `GZIPWriterInterceptor` is in the examples above.

Note that filters and interceptors registered using dynamic binding are only additional filters run for the resource method. If there are any name bound providers or global providers they will still be executed.

## 10.7. Priorities

In case you register more filters and interceptors you might want to define an exact order in which they should be invoked. The order can be controlled by the `@Priority` annotation defined by the

`jakarta.annotation.Priority` class. The annotation accepts an integer parameter of priority. Providers used in request processing (`ContainerRequestFilter`, `ClientRequestFilter`) as well as entity interceptors (`ReaderInterceptor`, `WriterInterceptor`) are sorted based on the priority in an ascending manner. So, a request filter with priority defined with `@Priority(1000)` will be executed before another request filter with priority defined as `@Priority(2000)`. Providers used during response processing (`ContainerResponseFilter`, `ClientResponseFilter`) are executed in the reverse order (using descending manner), so a provider with the priority defined with `@Priority(2000)` will be executed before another provider with priority defined with `@Priority(1000)`.

It's a good practice to assign a priority to filters and interceptors. Use `Priorities` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Priorities.html>] class which defines standardized priorities in JAX-RS for different usages, rather than inventing your own priorities. For example, when you write an authentication filter you would assign a priority 1000 which is the value of `Priorities.AUTHENTICATION`. The following example shows the filter from the beginning of this chapter with a priority assigned.

### Example 10.9. Priorities example

```
1 ...
2 import jakarta.annotation.Priority;
3 import jakarta.ws.rs.Priorities;
4 ...
5
6 @Priority(Priorities.HEADER_DECORATOR)
7 public class ResponseFilter implements ContainerResponseFilter {
8
9     @Override
10     public void filter(ContainerRequestContext requestContext,
11                       ContainerResponseContext responseContext)
12                       throws IOException {
13
14         responseContext.getHeaders().add("X-Powered-By", "Jersey :-)");
15     }
16 }
```

As this is a response filter and response filters are executed in the reverse order, any other filter with priority lower than 3000 (`Priorities.HEADER_DECORATOR` is 3000) will be executed after this filter. So, for example `AUTHENTICATION` filter (priority 1000) would be run after this filter.



---

# Chapter 11. Asynchronous Services and Clients

This chapter describes the usage of asynchronous API on the client and server side. The term *async* will be sometimes used interchangeably with the term *asynchronous* in this chapter.

## 11.1. Asynchronous Server API

Request processing on the server works by default in a synchronous processing mode, which means that a client connection of a request is processed in a single I/O container thread. Once the thread processing the request returns to the I/O container, the container can safely assume that the request processing is finished and that the client connection can be safely released including all the resources associated with the connection. This model is typically sufficient for processing of requests for which the processing resource method execution takes a relatively short time. However, in cases where a resource method execution is known to take a long time to compute the result, server-side asynchronous processing model should be used. In this model, the association between a request processing thread and client connection is broken. I/O container that handles incoming request may no longer assume that a client connection can be safely closed when a request processing thread returns. Instead a facility for explicitly suspending, resuming and closing client connections needs to be exposed. Note that the use of server-side asynchronous processing model will not improve the request processing time perceived by the client. It will however increase the throughput of the server, by releasing the initial request processing thread back to the I/O container while the request may still be waiting in a queue for processing or the processing may still be running on another dedicated thread. The released I/O container thread can be used to accept and process new incoming request connections.

The following example shows a simple asynchronous resource method defined using the new JAX-RS *async* API:

### Example 11.1. Simple *async* resource

```
1 @Path("/resource")
2 public class AsyncResource {
3     @GET
4     public void asyncGet(@Suspended final AsyncResponse asyncResponse) {
5
6         new Thread(new Runnable() {
7             @Override
8             public void run() {
9                 String result = veryExpensiveOperation();
10                asyncResponse.resume(result);
11            }
12
13            private String veryExpensiveOperation() {
14                // ... very expensive operation
15            }
16        }).start();
17    }
18 }
```

In the example above, a resource `AsyncResource` with one GET method `asyncGet` is defined. The `asyncGet` method injects a JAX-RS `AsyncResponse` [<https://jakartaee.github.io/>]

[rest/apidocs/4.0.0/jakarta/ws/rs/container/AsyncResponse.html](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/AsyncResponse.html)] instance using a JAX-RS `@Suspended` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/Suspended.html>] annotation. Please note that `AsyncResponse` must be injected by the `@Suspended` annotation and not by `@Context` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>] as `@Suspended` does not only inject response but also says that the method is executed in the asynchronous mode. By the `AsyncResponse` parameter into a resource method we tell the Jersey runtime that the method is supposed to be invoked using the asynchronous processing mode, that is the client connection should not be automatically closed by the underlying I/O container when the method returns. Instead, the injected `AsyncResponse` instance (that represents the suspended client request connection) will be used to explicitly send the response back to the client using some other thread. In other words, Jersey runtime knows that when the `asyncGet` method completes, the response to the client may not be ready yet and the processing must be suspended and wait to be explicitly resumed with a response once it becomes available. Note that the method `asyncGet` returns `void` in our example. This is perfectly valid in case of an asynchronous JAX-RS resource method, even for a `@GET` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html>] method, as the response is never returned directly from the resource method as its return value. Instead, the response is later returned using `AsyncResponse` instance as it is demonstrated in the example. The `asyncGet` resource method starts a new thread and exits from the method. In that state the request processing is suspended and the container thread (the one which entered the resource method) is returned back to the container's thread pool and it can process other requests. New thread started in the resource method may execute an expensive operation which might take a long time to finish. Once a result is ready it is resumed using the `resume()` method on the `AsyncResponse` instance. The resumed response is then processed in the new thread by Jersey in a same way as any other synchronous response, including execution of filters and interceptors, use of exception mappers as necessary and sending the response back to the client.

It is important to note that the asynchronous response (`asyncResponse` in the example) does not need to be resumed from the thread started from the resource method. The asynchronous response can be resumed even from different request processing thread as it is shown in the the example of the `AsyncResponse` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/AsyncResponse.html>] javadoc. In the javadoc example the async response suspended from the `GET` method is resumed later on from the `POST` method. The suspended async response is passed between requests using a static field and is resumed from the other resource method running on a different request processing thread.

Imagine now a situation when there is a long delay between two requests and you would not like to let the client wait for the response "forever" or at least for an unacceptable long time. In asynchronous processing model, occurrences of such situations should be carefully considered with client connections not being automatically closed when the processing method returns and the response needs to be resumed explicitly based on an event that may actually even never happen. To tackle these situations asynchronous *timeouts* can be used.

The following example shows the usage of timeouts:

### Example 11.2. Simple async method with timeout

```
1 @GET
2 public void asyncGetWithTimeout(@Suspended final AsyncResponse asyncResponse)
3     asyncResponse.setTimeoutHandler(new TimeoutHandler() {
4
5         @Override
6         public void handleTimeout(AsyncResponse asyncResponse) {
7             asyncResponse.resume(Response.status(Response.Status.SERVICE_UNAVAILABLE)
8                 .entity("Operation time out.").build());
9         }
10    });
```

```
11     asyncResponse.setTimeout(20, TimeUnit.SECONDS);
12
13     new Thread(new Runnable() {
14
15         @Override
16         public void run() {
17             String result = veryExpensiveOperation();
18             asyncResponse.resume(result);
19         }
20
21         private String veryExpensiveOperation() {
22             // ... very expensive operation that typically finishes within 20
23         }
24     }).start();
25 }
```

By default, there is no timeout defined on the suspended `AsyncResponse` instance. A custom timeout and timeout event handler may be defined using `setTimeoutHandler(TimeoutHandler)` and `setTimeout(long, TimeUnit)` methods. The `setTimeoutHandler(TimeoutHandler)` method defines the handler that will be invoked when timeout is reached. The handler resumes the response with the response code 503 (from `Response.Status.SERVICE_UNAVAILABLE`). A timeout interval can be also defined without specifying a custom timeout handler (using just the `setTimeout(long, TimeUnit)` method). In such case the default behaviour of Jersey runtime is to throw a `ServiceUnavailableException` that gets mapped into 503, "Service Unavailable" HTTP error response, as defined by the JAX-RS specification.

### 11.1.1. Asynchronous Server-side Callbacks

As operations in asynchronous cases might take long time and they are not always finished within a single resource method invocation, JAX-RS offers facility to register callbacks to be invoked based on suspended async response state changes. In Jersey you can register two JAX-RS callbacks:

- `CompletionCallback` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/CompletionCallback.html>] that is executed when request finishes or fails, and
- `ConnectionCallback` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ConnectionCallback.html>] executed when a connection to a client is closed or lost.

#### Example 11.3. CompletionCallback example

```
1 @Path("/resource")
2 public class AsyncResource {
3     private static int numberOfSuccessResponses = 0;
4     private static int numberOfFailures = 0;
5     private static Throwable lastException = null;
6
7     @GET
8     public void asyncGetWithTimeout(@Suspended final AsyncResponse asyncResponse) {
9         asyncResponse.register(new CompletionCallback() {
10             @Override
11             public void onComplete(Throwable throwable) {
12                 if (throwable == null) {
13                     // no throwable - the processing ended successfully
14                     // (response already written to the client)
```

```
15         numberOfSuccessResponses++;
16     } else {
17         numberOfFailures++;
18         lastException = throwable;
19     }
20 }
21 });
22
23 new Thread(new Runnable() {
24     @Override
25     public void run() {
26         String result = veryExpensiveOperation();
27         asyncResponse.resume(result);
28     }
29
30     private String veryExpensiveOperation() {
31         // ... very expensive operation
32     }
33 }).start();
34 }
35 }
```

A completion callback is registered using `register(...)` method on the `AsyncResponse` instance. A registered completion callback is bound only to the response(s) to which it has been registered. In the example the `CompletionCallback` is used to calculate successfully processed responses, failures and to store last exception. This is only a simple case demonstrating the usage of the callback. You can use completion callback to release the resources, change state of internal resources or representations or handle failures. The method has an argument `Throwable` which is set only in case of an error. Otherwise the parameter will be `null`, which means that the response was successfully written. The callback is executed only after the response is written to the client (not immediately after the response is resumed).

The `AsyncResponse register(...)` method is overloaded and offers options to register a single callback as an `Object` (in the example), as a `Class` or multiple callbacks using varargs.

As some async requests may take long time to process the client may decide to terminate its connection to the server before the response has been resumed or before it has been fully written to the client. To deal with these use cases a `ConnectionCallback` can be used. This callback will be executed only if the connection was prematurely terminated or lost while the response is being written to the back client. Note that this callback will not be invoked when a response is written successfully and the client connection is closed as expected. See javadoc of `ConnectionCallback` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ConnectionCallback.html>] for more information.

## 11.1.2. Chunked Output

Jersey offers a facility for sending response to the client in multiple more-or-less independent chunks using a *chunked output*. Each response chunk usually takes some (longer) time to prepare before sending it to the client. The most important fact about response chunks is that you want to send them to the client immediately as they become available without waiting for the remaining chunks to become available too. The first bytes of each chunked response consists of the HTTP headers that are sent to the client. As noted above, the entity of the response is then sent in chunks as they become available. Client knows that the response is going to be chunked, so it reads each chunk of the response separately, processes it, and waits for more chunks to arrive on the same connection. After some time, the server generates another response chunk and send it again to the client. Server keeps on sending response chunks until it closes the connection after sending the last chunk when the response processing is finished.

In Jersey you can use `ChunkedOutput` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ChunkedOutput.html>] to send response to a client in chunks. Chunks are strictly defined pieces of a response body can be marshalled as a separate entities using Jersey/JAX-RS `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>] providers. A chunk can be `String`, `Long` or JAXB bean serialized to XML or JSON or any other defined custom type for which a `MessageBodyWriter<T>` is available.

The resource method that returns `ChunkedOutput` informs the Jersey runtime that the response will be chunked and that the processing works asynchronously as such. You do not need to inject `AsyncResponse` to start the asynchronous processing mode in this case. Returning a `ChunkedOutput` instance from the method is enough to indicate the asynchronous processing. Response headers will be sent to a client when the resource method returns and the client will wait for the stream of chunked data which you will be able to write from different thread using the same `ChunkedOutput` instance returned from the resource method earlier. The following example demonstrates this use case:

#### Example 11.4. `ChunkedOutput` example

```
1 @Path("/resource")
2 public class AsyncResource {
3     @GET
4     public ChunkedOutput<String> getChunkedResponse() {
5         final ChunkedOutput<String> output = new ChunkedOutput<String>(String.
6
7         // Or use the builder pattern instead, which also allows to configure
8         // final ChunkedOutput<String> output = ChunkedOutput.<String>builder(
9
10        new Thread() {
11            public void run() {
12                try {
13                    String chunk;
14
15                    while ((chunk = getNextString()) != null) {
16                        output.write(chunk);
17                    }
18                } catch (IOException e) {
19                    // IOException thrown when writing the
20                    // chunks of response: should be handled
21                } finally {
22                    output.close();
23                    // simplified: IOException thrown from
24                    // this close() should be handled here...
25                }
26            }
27        }.start();
28
29        // the output will be probably returned even before
30        // a first chunk is written by the new thread
31        return output;
32    }
33
34    private String getNextString() {
35        // ... long running operation that returns
36        // next string or null if no other string is accessible
37    }
```

38 }

The example above defines a `GET` method that returns a `ChunkedOutput` instance. The generic type of `ChunkedOutput` defines the chunk types (in this case chunks are `Strings`). Before the instance is returned a new thread is started that writes individual chunks into the chunked output instance named `output`. Once the original thread returns from the resource method, Jersey runtime writes headers to the container response but does not close the client connection yet and waits for the response data to be written to the chunked `output`. New thread in a loop calls the method `getNextString()` which returns a next `String` or `null` if no other `String` exists (the method could for example load latest data from the database). Returned `Strings` are written to the chunked `output`. Such a written chunks are internally written to the container response and client can read them. At the end the chunked output is closed which determines the end of the chunked response. Please note that you must close the output explicitly in order to close the client connection as Jersey does not implicitly know when you are finished with writing the chunks.

A chunked output can be processed also from threads created from another request as it is explained in the sections above. This means that one resource method may e.g. only return a `ChunkedOutput` instance and other resource method(s) invoked from another request thread(s) can write data into the chunked output and/or close the chunked response.

## 11.2. Client API

The client API supports asynchronous processing too. Simple usage of asynchronous client API is shown in the following example:

### Example 11.5. Simple client async invocation

```
1 final AsyncInvoker asyncInvoker = target().path("http://example.com/resource/")
2   .request().async();
3 final Future<Response> responseFuture = asyncInvoker.get();
4 System.out.println("Request is being processed asynchronously.");
5 final Response response = responseFuture.get();
6   // get() waits for the response to be ready
7 System.out.println("Response received.");
```

The difference against synchronous invocation is that the http method call `get()` is not called on `SyncInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/SyncInvoker.html>] but on `AsyncInvoker` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/AsyncInvoker.html>]. The `AsyncInvoker` is returned from the call of method `Invocation.Builder.async()` as shown above. `AsyncInvoker` offers methods similar to `SyncInvoker` only these methods do not return a response synchronously. Instead a `Future<...>` representing response data is returned. These method calls also return immediately without waiting for the actual request to complete. In order to get the response of the invoked `get()` method, the `responseFuture.get()` is invoked which waits for the response to be finished (this call is blocking as defined by the Java SE `Future` contract).

Asynchronous Client API in JAX-RS is fully integrated in the fluent JAX-RS Client API flow, so that the async client-side invocations can be written fluently just like in the following example:

### Example 11.6. Simple client fluent async invocation

```
1 final Future<Response> responseFuture = target().path("http://example.com/reso
2   .request().async().get();
```

To work with asynchronous results on the client-side, all standard `Future` API facilities can be used. For example, you can use the `isDone()` method to determine whether a response has finished to avoid the use of a blocking call to `Future.get()`.

## 11.2.1. Asynchronous Client Callbacks

Similarly to the server side, in the client API you can register asynchronous callbacks too. You can use these callbacks to be notified when a response arrives instead of waiting for the response on `Future.get()` or checking the status by `Future.isDone()` in a loop. A client-side asynchronous invocation callback can be registered as shown in the following example:

### Example 11.7. Client async callback

```
1 final Future<Response> responseFuture = target().path("http://example.com/reso
2     .request().async().get(new InvocationCallback<Response>() {
3         @Override
4         public void completed(Response response) {
5             System.out.println("Response status code "
6                 + response.getStatus() + " received.");
7         }
8
9         @Override
10        public void failed(Throwable throwable) {
11            System.out.println("Invocation failed.");
12            throwable.printStackTrace();
13        }
14    });
```

The registered callback is expected to implement the `InvocationCallback` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/InvocationCallback.html>] interface that defines two methods. First method `completed(Response)` gets invoked when an invocation successfully finishes. The result response is passed as a parameter to the callback method. The second method `failed(Throwable)` is invoked in case the invocation fails and the exception describing the failure is passed to the method as a parameter. In this case since the callback generic type is `Response`, the `failed(Throwable)` method would only be invoked in case the invocation fails because of an internal client-side processing error. It would not be invoked in case a server responds with an HTTP error code, for example if the requested resource is not found on the server and HTTP 404 response code is returned. In such case `completed(Response)` callback method would be invoked and the response passed to the method would contain the returned error response with HTTP 404 error code. This is a special behavior in case the generic callback return type is `Response`. In the next example an exception is thrown (or `failed(Throwable)` method on the invocation callback is invoked) even in case a non-2xx HTTP error code is returned.

As with the synchronous client API, you can retrieve the response entity as a Java type directly without requesting a `Response` first. In case of an `InvocationCallback`, you need to set its generic type to the expected response entity type instead of using the `Response` type as demonstrated in the example below:

### Example 11.8. Client async callback for specific entity

```
1 final Future<String> entityFuture = target().path("http://example.com/resource
2     .request().async().get(new InvocationCallback<String>() {
3         @Override
4         public void completed(String response) {
5             System.out.println("Response entity '" + response + "' receive
6         }
7
8         @Override
9         public void failed(Throwable throwable) {
```

```
10         System.out.println("Invocation failed.");
11         throwable.printStackTrace();
12     }
13     });
14 System.out.println(entityFuture.get());
```

Here, the generic type of the invocation callback information is used to unmarshall the HTTP response content into a desired Java type.

### Important

Please note that in this case the method `failed(Throwable throwable)` would be invoked even for cases when a server responds with a non HTTP-2xx HTTP error code. This is because in this case the user does not have any other means of finding out that the server returned an error response.

## 11.2.2. Chunked input

In an earlier section the `ChunkedOutput` was described. It was shown how to use a chunked output on the server. In order to read chunks on the client the `ChunkedInput` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ChunkedInput.html>] can be used to complete the story.

You can, of course, process input on the client as a standard input stream but if you would like to leverage Jersey infrastructure to provide support of translating message chunk data into Java types using a `ChunkedInput` is much more straightforward. See the usage of the `ChunkedInput` in the following example:

### Example 11.9. `ChunkedInput` example

```
1 final Response response = target().path("http://example.com/resource/")
2   .request().get();
3 final ChunkedInput<String> chunkedInput =
4   response.readEntity(new GenericType<ChunkedInput<String>>() {});
5 String chunk;
6 while ((chunk = chunkedInput.read()) != null) {
7   System.out.println("Next chunk received: " + chunk);
8 }
9
```

The response is retrieved in a standard way from the server. The entity is read as a `ChunkedInput` entity. In order to do that the `GenericEntity<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/GenericEntity.html>] is used to preserve a generic information at run time. If you would not use `GenericEntity<T>`, Java language generic type erasure would cause that the generic information would get lost at compile time and an exception would be thrown at run time complaining about the missing chunk type definition.

In the next lines in the example, individual chunks are being read from the response. Chunks can come with some delay, so they will be written to the console as they come from the server. After receiving last chunk the `null` will be returned from the `read()` method. This will mean that the server has sent the last chunk and closed the connection. Note that the `read()` is a blocking operation and the invoking thread is blocked until a new chunk comes.

Writing chunks with `ChunkedOutput` is simple, you only call method `write()` which writes exactly one chunk to the output. With the input reading it is slightly more complicated. The `ChunkedInput`



does not know how to distinguish chunks in the byte stream unless being told by the developer. In order to define custom chunks boundaries, the **ChunkedInput** offers possibility to register a **ChunkParser** [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ChunkParser.html>] which reads chunks from the input stream and separates them. Jersey provides several chunk parser implementations and you can implement your own parser to separate your chunks if you need. In our example above the default parser provided by Jersey is used that separates chunks based on presence of a `\r\n` delimiting character sequence.

Each incoming input stream is firstly parsed by the **ChunkParser**, then each chunk is processed by the proper **MessageBodyReader<T>** [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>]. You can define the media type of chunks to aid the selection of a proper **MessageBodyReader<T>** in order to read chunks correctly into the requested entity types (in our case into Strings).

---

# Chapter 12. URIs and Links

## 12.1. Building URIs

A very important aspect of REST is hyperlinks, URIs, in representations that clients can use to transition the Web service to new application states (this is otherwise known as "hypermedia as the engine of application state"). HTML forms present a good example of this in practice.

Building URIs and building them safely is not easy with URI [<http://docs.oracle.com/javase/6/docs/api/java/net/URI.html>], which is why JAX-RS has the `UriBuilder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriBuilder.html>] class that makes it simple and easy to build URIs safely. `UriBuilder` can be used to build new URIs or build from existing URIs. For resource classes it is more than likely that URIs will be built from the base URI the web service is deployed at or from the request URI. The class `UriInfo` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html>] provides such information (in addition to further information, see next section).

The following example shows URI building with `UriInfo` and `UriBuilder` from the bookmark example:

### Example 12.1. URI building

```
1 @Path("/users/")
2 public class UsersResource {
3
4     @Context
5     UriInfo uriInfo;
6
7     ...
8
9     @GET
10    @Produces("application/json")
11    public JSONArray getUsersAsJsonArray() {
12        JSONArray uriArray = new JSONArray();
13        for (UserEntity userEntity : getUsers()) {
14            UriBuilder ub = uriInfo.getAbsolutePathBuilder();
15            URI userUri = ub.
16                path(userEntity.getUserId()).
17                build();
18            uriArray.put(userUri.toASCIIString());
19        }
20        return uriArray;
21    }
22 }
```

`UriInfo` is obtained using the `@Context` annotation, and in this particular example injection onto the field of the root resource class is performed, previous examples showed the use of `@Context` on resource method parameters.

`UriInfo` can be used to obtain URIs and associated `UriBuilder` instances for the following URIs: the base URI the application is deployed at; the request URI; and the absolute path URI, which is the request URI minus any query components.

The `getUsersAsJsonArray` method constructs a `JSONArray`, where each element is a URI identifying a specific user resource. The URI is built from the absolute path of the request URI

by calling `UriInfo.getAbsolutePathBuilder()` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#getAbsolutePathBuilder\(\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#getAbsolutePathBuilder())]. A new path segment is added, which is the user ID, and then the URI is built. Notice that it is not necessary to worry about the inclusion of `'/'` characters or that the user ID may contain characters that need to be percent encoded. `UriBuilder` takes care of such details.

`UriBuilder` can be used to build/replace query or matrix parameters. URI templates can also be declared, for example the following will build the URI `"http://localhost/segment?name=value"`:

### Example 12.2. Building URIs using query parameters

```
1 UriBuilder.fromUri("http://localhost/")
2   .path("{a}")
3   .QueryParam("name", "{value}")
4   .build("segment", "value");
```

## 12.2. Resolve and Relativize

JAX-RS 2.0 introduced additional URI resolution and relativization methods in the `UriBuilder`:

- `UriInfo.resolve(java.net.URI)` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#resolve\(java.net.URI\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#resolve(java.net.URI))]
- `UriInfo.relativize(java.net.URI)` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#relativize\(java.net.URI\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#relativize(java.net.URI))]
- `UriBuilder.resolveTemplate(...)` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriBuilder.html#resolveTemplate\(java.lang.String, java.lang.Object\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriBuilder.html#resolveTemplate(java.lang.String, java.lang.Object))] (various arguments)

Resolve and relativize methods in `UriInfo` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html>] are essentially counterparts to the methods listed above - `UriInfo.resolve(java.net.URI)` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#resolve\(java.net.URI\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#resolve(java.net.URI))] resolves given relative URI to an absolute URI using application context URI as the base URI; `UriInfo.relativize(java.net.URI)` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#relativize\(java.net.URI\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriInfo.html#relativize(java.net.URI))] then transforms an absolute URI to a relative one, using again the applications context URI as the base URI.

`UriBuilder` also introduces a set of methods that provide ways of resolving URI templates by replacing individual templates with a provided value(s). A short example:

```
1 final URI uri = UriBuilder.fromUri("http://{host}/{path}?q={param}")
2   .resolveTemplate("host", "localhost")
3   .resolveTemplate("path", "myApp")
4   .resolveTemplate("param", "value").build();
5
6 uri.toString(); // returns "http://localhost/myApp?q=value"
```

See the `UriBuilder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriBuilder.html>] javadoc for more details.

## 12.3. Link

JAX-RS 2.0 introduces `Link` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Link.html>] class, which serves as a representation of Web Link defined in RFC 5988 [<http://tools.ietf.org/html/>]

rfc5988]. The JAX-RS `Link` class adds API support for providing additional metadata in HTTP messages, for example, if you are consuming a REST interface of a public library, you might have a resource returning description of a single book. Then you can include links to related resources, such as a book category, author, etc. to make the produced response concise but complete at the same time. Clients are then able to query all the additional information they are interested in and are not forced to consume details they are not interested in. At the same time, this approach relieves the server resources as only the information that is truly requested is being served to the clients.

A `Link` can be serialized to an HTTP message (typically a response) as additional HTTP header (there might be multiple `Link` headers provided, thus multiple links can be served in a single message). Such HTTP header may look like:

```
Link: <http://example.com/TheBook/chapter2>; rel="prev"; title="previous chapter"
```

Producing and consuming Links with JAX-RS API is demonstrated in the following example:

```
// server side - adding links to a response:
Response r = Response.ok().
    link("http://oracle.com", "parent").
    link(new URI("http://eclipse-ee4j.github.io/jersey"), "framework").
    build();

...

// client-side processing:
final Response response = target.request().get();

URI u = response.getLink("parent").getUri();
URI u = response.getLink("framework").getUri();
```

Instances of `Link` can be also created directly by invoking one of the factory methods on the `Link` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Link.html>] API that returns a `Link.Builder` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Link.Builder.html>] that can be used to configure and produce new links.

---

# Chapter 13. Declarative Hyperlinking

RESTful APIs must be hypertext-driven [<http://roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven>]. JAX-RS currently offers UriBuilder [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/UriBuilder.html>] to simplify URI creation but Jersey adds an additional annotation-based alternative that is described here.

## Important

This API is currently under development and experimental so it is subject to change at any time.

## 13.1. Dependency

To use Declarative Linking you need to add `jersey-declarative-linking` module to your `pom.xml` file:

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-declarative-linking</artifactId>
  <version>4.0.0</version>
</dependency>
```

Additionally you will need to add the following dependencies, if you are not deploying into a container that is already including them:

```
<dependency>
  <groupId>jakarta.el</groupId>
  <artifactId>jakarta.el-api</artifactId>
  <version>5.0.1</version>
</dependency>

<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.el</artifactId>
  <version>5.0.0</version>
</dependency>
```

If you're not using Maven make sure to have all needed dependencies (see `jersey-declarative-linking` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-declarative-linking/dependencies.html>]) on the classpath.

## 13.2. Links in Representations

Links are added to representations using the `@InjectLink` annotation on entity class fields. The Jersey runtime is responsible for injecting the appropriate URI into the field prior to serialization by a message body writer. E.g. consider the following resource and entity classes:

```
@Path("widgets")
public class WidgetsResource {
    @GET
    public Widgets get() {
        return new Widgets();
    }
}
```

```
}

public class Widgets {
    @InjectLink(resource=WidgetsResource.class)
    URI u;
}
```

After a call to `WidgetsResource#get`, the Jersey runtime will inject the value `"/context/widgets"`<sup>1</sup> into the returned `Widgets` instance. If an absolute URI is desired instead of an absolute path then the annotation can be changed to `@InjectLink(resource=WidgetsResource.class, style=ABSOLUTE)`.

The above usage works nicely when there's already a URI template on a class that you want to reuse. If there's no such template available then you can use a literal value instead of a reference. E.g. the following is equivalent to the earlier example: `@InjectLink(value="widgets", style=ABSOLUTE)`.

## 13.3. List of Link Injection

You can inject multiple links into an array or a `List` collection type. E.g.:

```
@InjectLinks({@InjectLink(resource=WidgetsResource.class, rel = "self")})
List<Link> links
```

The field doesn't need to be initialized. However, if it already contains a collection with manually created links, then it will merge those with the generated links into a new collection which then replaces the field value.

## 13.4. Links from Resources

As an alternative to defining the links in the entity class, they can also be defined in the resource classes by annotating the resource methods with `@ProvideLink`. This has the benefit, that the target method is already known and doesn't need to be referenced. Other than that it has the same parameters and behaviors as `@InjectLink`. The entity classes need to have a field annotated with `@InjectLinks`, which can be empty.

The `@ProvideLink` annotation can be repeated to add links to different entities using different options. Entities are defined via the `value` property. If the entities are similar in structure they can also be declared as an array. `@ProvideLink` also works with class hierarchies, e.g., contributions defined for a superclass will also be injected into the derived classes (interfaces are not supported).

```
@ProvideLink(value = Order.class, rel = "self",
    bindings = @Binding(name = "orderId", value = "${instance.id}"))
@ProvideLink(value = PaymentConfirmation.class, rel = "order",
    bindings = @Binding(name = "orderId", value = "${instance.orderId}"))
@GET
@Path("/{orderId}")
public Response get(@PathParam("orderId") String orderId)
```

## 13.5. Binding Template Parameters

Referenced or literal templates may contain parameters. Two forms of parameters are supported:

---

<sup>1</sup> Where `/context` is the application deployment context.

- URI template parameters, e.g. `widgets/{id}` where `{id}` represents a variable part of the URI.
- EL expressions, e.g. `widgets/${instance.id}` where `${instance.id}` is an EL expression.

Parameter values can be extracted from three implicit beans:

|                       |                                                                         |
|-----------------------|-------------------------------------------------------------------------|
| <code>instance</code> | Represents the instance of the class that contains the annotated field. |
| <code>entity</code>   | Represents the entity class instance returned by the resource method.   |
| <code>resource</code> | Represents the resource class instance that returned the entity.        |

By default URI template parameter values are extracted from the implicit `instance` bean, i.e. the following two annotations are equivalent:

```
@InjectLink("widgets/{id}")
```

```
@InjectLink("widgets/${instance.id}")
```

The source for URI template parameter values can be changed using the `@Binding` annotation, E.g. the following three annotations are equivalent:

```
@InjectLink(value="widgets/{id}", bindings={
    @Binding(name="id" value="${resource.id}")
})
```

```
@InjectLink(value="widgets/{value}", bindings={
    @Binding("${resource.id}")
})
@InjectLink("widgets/${resource.id}")
```

## 13.6. Conditional Link Injection

Link value injection can be made conditional by use of the `condition` property. The value of this property is a boolean EL expression and a link will only be injected if the condition expression evaluates to true. E.g.:

```
@InjectLink(value="widgets/${instance.id}/offers",
    condition="${instance.offers}")
URI offers;
```

In the above, a URI will only be injected into the `offers` field if the `offers` property of the instance is true.

## 13.7. Link Headers

HTTP Link headers [<http://tools.ietf.org/html/rfc5988#section-5>] can also be added to responses using annotations. Instead of annotating the fields of an entity class with `@InjectLink`, you instead annotate the entity class itself with `@InjectLinks`. E.g.:

```
@InjectLinks(
    @InjectLink(value="widgets/${resource.nextId}", rel="next")
)
```

The above would insert a HTTP Link header into any response whose entity was thus annotated. The `@InjectLink` annotation contains properties that map to the parameters of the HTTP Link header. The

above specifies the link relation as `next`. All properties of the `@InjectLink` annotation may be used as described above.

Multiple link headers can be added by use of the `@InjectLinks` annotation which can contain multiple `@InjectLink` annotations.

Resource links via `@ProvideLink` are currently not supported for link headers.

## 13.8. Prevent Recursive Injection

By default, Jersey will try to recursively find all `@InjectLink` annotations in the members of your object unless this member is annotated with `@XmlTransient`. But in some cases, you might want to control which member will be introspected regardless of the `@XmlTransient` annotation. You can prevent Jersey to look into an object by adding `@InjectLinkNoFollow` to a field.

```
@InjectLinkNoFollow
Context context;
```

## 13.9. Meta-annotation support

The `@ProvideLink` annotation can be used as a meta-annotation, i.e., annotating your own annotation. This enables you to create custom annotations to reuse `@ProvideLink` configurations instead of copy pasting them on each method. There is a special marker class `ProvideLink.InheritFromAnnotation` that can be used in place of the actual entity class, this indicates that the `Class<?> value()` from the custom annotation should be used instead. Repeated annotations are currently unsupported for this feature. Also the `Class<?> value()` method must return a single class and not an array of classes.

Here is an example (getter/setter omitted for brevity) of how a meta annotation can be used. The example app uses a `Page` class as a base class for all entities that contain paged data.

```
public class Page {
    private int number;

    private int size;

    private boolean isPreviousPageAvailable;

    private boolean isNextPageAvailable;

    @InjectLinks
    private List<Link> links;
}
```

Instead of duplicating the `@ProvideLink` annotations for the next and previous links on every method, we create the following `@PageLinks` annotation.

```
@ProvideLink(value = ProvideLink.InheritFromAnnotation.class, rel = "next",
    bindings = {
        @Binding(name = "page", value = "${instance.number + 1}"),
        @Binding(name = "size", value = "${instance.size}"),
    }, condition = "${instance.nextPageAvailable}")
@ProvideLink(value = ProvideLink.InheritFromAnnotation.class, rel = "prev",
    bindings = {
```



```
        @Binding(name = "page", value = "${instance.number - 1}"),
        @Binding(name = "size", value = "${instance.size}"),
    }, condition = "${instance.previousPageAvailable}")
@Target({ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface PageLinks {
    Class<?> value();
}
```

The annotation can then be used on the resource methods with the actual entity class as `value`.

```
@PageLinks(OrderPage.class)
@GET
public Response list(@QueryParam("page") @DefaultValue("0") int page,
                    @QueryParam("size") @DefaultValue("20") int size)
```

The entity just extends from `Page` and declares its content. It is necessary to use distinct classes instead of just a generic page to have a unique target for `@ProvideLink`, otherwise every method annotated with `@ProvideLink(value=Page.class)` would contribute to the entity.

```
public class OrderPage extends Page {
    private List<Order> orders;
}
```

## 13.10. Configure and register

In order to add the Declarative Linking feature register `DeclarativeLinkingFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/linking/DeclarativeLinkingFeature.html>]

**Example 13.1. Creating JAX-RS application with Declarative Linking feature enabled.**

```
// Create JAX-RS application.
final Application application = new ResourceConfig()
    .packages("org.glassfish.jersey.examples.linking")
    .register(DeclarativeLinkingFeature.class);
```

---

# Chapter 14. Programmatic API for Building Resources

## 14.1. Introduction

A standard approach of developing JAX-RS application is to implement resource classes annotated with `@Path` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html>] with resource methods annotated with HTTP method annotations like `@GET` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html>] or `@POST` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/POST.html>] and implement needed functionality. This approach is described in the chapter JAX-RS Application, Resources and Sub-Resources [[jaxrs-resources.html](#)]. Besides this standard JAX-RS approach, Jersey offers an API for constructing resources programmatically.

Imagine a situation where a deployed JAX-RS application consists of many resource classes. These resources implement standard HTTP methods like `@GET` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html>], `@POST` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/POST.html>], `@DELETE` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/DELETE.html>]. In some situations it would be useful to add an `@OPTIONS` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/OPTIONS.html>] method which would return some kind of meta data about the deployed resource. Ideally, you would want to expose the functionality as an additional feature and you want to decide at the deploy time whether you want to add your custom `OPTIONS` method. However, when custom `OPTIONS` method are not enabled you would like to be `OPTIONS` requests handled in the standard way by JAX-RS runtime. To achieve this you would need to modify the code to add or remove custom `OPTIONS` methods before deployment. Another way would be to use programmatic API to build resource according to your needs.

Another use case of programmatic resource builder API is when you build any generic RESTful interface which depends on lot of configuration parameters or for example database structure. Your resource classes would need to have different methods, different structure for every new application deploy. You could use more solutions including approaches where your resource classes would be built using Java byte code manipulation. However, this is exactly the case when you can solve the problem cleanly with the programmatic resource builder API. Let's have a closer look at how the API can be utilized.

## 14.2. Programmatic Hello World example

Jersey Programmatic API was designed to fully support JAX-RS resource model. In other words, every resource that can be designed using standard JAX-RS approach via annotated resource classes can be also modelled using Jersey programmatic API. Let's try to build simple hello world resource using standard approach first and then using the Jersey programmatic resource builder API.

The following example shows standard JAX-RS "Hello world!" resource class.

### Example 14.1. A standard resource class

```
1
2      @Path("helloworld")
3      public class HelloWorldResource {
4
5          @GET
6          @Produces("text/plain")
```

```
7             public String getHello() {
8                 return "Hello World!";
9             }
10        }
11    }
```

This is just a simple resource class with one GET method returning "Hello World!" string that will be serialized as text/plain media type.

Now we will design this simple resource using programmatic API.

### Example 14.2. A programmatic resource

```
1
2        package org.glassfish.jersey.examples.helloworld;
3
4        import jakarta.ws.rs.container.ContainerRequestContext;
5        import jakarta.ws.rs.core.Application;
6        import jakarta.ws.rs.core.Response;
7        import org.glassfish.jersey.process.Inflector;
8        import org.glassfish.jersey.server.ResourceConfig;
9        import org.glassfish.jersey.server.model.Resource;
10       import org.glassfish.jersey.server.model.ResourceMethod;
11
12
13       public static class MyResourceConfig extends ResourceConfig {
14
15           public MyResourceConfig() {
16               final Resource.Builder resourceBuilder = Resource.Builder.newBuilder();
17               resourceBuilder.path("helloworld");
18
19               final ResourceMethod.Builder methodBuilder = resourceBuilder.methodBuilder();
20               methodBuilder.produces(MediaType.TEXT_PLAIN_TYPE)
21                   .handledBy(new Inflector<ContainerRequestContext>() {
22
23                       @Override
24                       public String apply(ContainerRequestContext context) {
25                           return "Hello World!";
26                       }
27                   });
28
29               final Resource resource = resourceBuilder.build();
30               registerResources(resource);
31           }
32       }
33
34   }
```

First, focus on the content of the `MyResourceConfig` constructor in the example. The Jersey programmatic resource model is constructed from `Resources` that contain `ResourceMethods`. In the example, a single resource would be constructed from a `Resource` containing one GET resource method that returns "Hello World!". The main entry point for building programmatic resources in Jersey is the `Resource.Builder` class. `Resource.Builder` contains just a few methods like the `path` method used in the example, which sets the name of the path. Another useful method is a `addMethod(String path)` which adds a new method to the resource builder and returns a resource method builder for the

new method. Resource method builder contains methods which set consumed and produced media type, define name bindings, timeout for asynchronous executions, etc. It is always necessary to define a resource method handler (i.e. the code of the resource method that will return "Hello World!"). There are more options how a resource method can be handled. In the example the implementation of `Inflector` is used. The Jersey `Inflector` interface defines a simple contract for transformation of a request into a response. An inflector can either return a `Response` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>] or directly an entity object, the way it is shown in the example. Another option is to setup a Java method handler using `handledBy(Class<?> handlerClass, Method method)` and pass it the chosen `java.lang.reflect.Method` instance together with the enclosing class.

A resource method model construction can be explicitly completed by invoking `build()` on the resource method builder. It is however not necessary to do so as the new resource method model will be built automatically once the parent resource is built. Once a resource model is built, it is registered into the resource config at the last line of the `MyResourceConfig` constructor in the example.

## 14.2.1. Deployment of programmatic resources

Let's now look at how a programmatic resources are deployed. The easiest way to setup your application as well as register any programmatic resources in Jersey is to use a Jersey `ResourceConfig` utility class, an extension of `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] class. If you deploy your Jersey application into a Servlet container you will need to provide a `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>] subclass that will be used for configuration. You may use a `web.xml` where you would define a `org.glassfish.jersey.servlet.ServletContainer` Servlet entry there and specify initial parameter `jakarta.ws.rs.Application` with the class name of your JAX-RS Application that you wish to deploy. In the example above, this application will be `MyResourceConfig` class. This is the reason why `MyResourceConfig` extends the `ResourceConfig` (which, if you remember extends the `jakarta.ws.rs.Application`).

The following example shows a fragment of `web.xml` that can be used to deploy the `ResourceConfig` JAX-RS application.

### Example 14.3. A programmatic resource

```
1
2
3         ...
4         <servlet>
5             <servlet-name>org.glassfish.jersey.examples.hellow
6             <servlet-class>org.glassfish.jersey.servlet.Servle
7             <init-param>
8                 <param-name>jakarta.ws.rs.Application</param-n
9                 <param-value>org.glassfish.jersey.examples.hel
10            </init-param>
11            <load-on-startup>1</load-on-startup>
12        </servlet>
13        <servlet-mapping>
14            <servlet-name>org.glassfish.jersey.examples.hellow
15            <url-pattern>/*</url-pattern>
16        </servlet-mapping>
17        ...
```

If you use another deployment options and you have accessible instance of `ResourceConfig` which you use for configuration, you can register programmatic resources directly by `registerResources()`

method called on the `ResourceConfig`. Please note that the method `registerResources()` replaces all the previously registered resources.

Because Jersey programmatic API is not a standard JAX-RS feature the `ResourceConfig` must be used to register programmatic resources as shown above. See deployment chapter for more information.

## 14.3. Additional examples

### Example 14.4. A programmatic resource

```
1
2         final Resource.Builder resourceBuilder = Resource.builder(
3         resourceBuilder.addMethod("OPTIONS")
4         .handledBy(new Inflector<ContainerRequestContext, Resp
5         @Override
6         public Response apply(ContainerRequestContext cont
7         return Response.ok("This is a response to an O
8         }
9     });
10     final Resource resource = resourceBuilder.build();
11
```

In the example above the `Resource` is built from a `HelloWorldResource` resource class. The resource model built this way contains a `GET` method producing 'text/plain' responses with "Hello World!" entity. This is quite important as it allows you to build whatever `Resource` objects based on introspecting existing JAX-RS resources and use builder API to enhance such these standard resources. In the example we used already implemented `HelloWorldResource` resource class and enhanced it by `OPTIONS` method. The `OPTIONS` method is handled by an `Inflector` which returns `Response` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>].

The following sample shows how to define sub-resource methods (methods that contains sub-path).

### Example 14.5. A programmatic resource

```
1
2         final Resource.Builder resourceBuilder = Resource.builder(
3
4         final Resource.Builder childResource = resourceBuilder.add
5         childResource.addMethod("GET").handledBy(new GetInflector(
6
7         final Resource resource = resourceBuilder.build();
8
```

Sub-resource methods are defined using so called *child resource models*. Child resource models (or child resources) are programmatic resources build in the same way as any other programmatic resource but they are registered as a child resource of a parent resource. The child resource in the example is build directly from the parent builder using method `addChildResource(String path)`. However, there is also an option to build a child resource model separately as a standard resource and then add it as a child resource to a selected parent resource. This means that child resource objects can be reused to define child resources in different parent resources (you just build a single child resource and then register it in multiple parent resources). Each child resource groups methods with the same sub-resource path. Note that a child resource cannot have any child resources as there is nothing like sub-sub-resource method concept in JAX-RS. For example if a sub resource method contains more path segments in its path (e.g. `"/root/sub/resource/method"` where `"root"` is a path of the resource and `"sub/resource/method"` is the sub resource

method path) then parent resource will have path "root" and child resource will have path "sub/resource/method" (so, there will not be any separate nested sub-resources for "sub", "resource" and "method").

See the javadocs of the resource builder API for more information. `Resource.Builder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/model/Resource.Builder.html>]

## 14.4. Model processors

Jersey gives you an option to register so called *model processor providers*. These providers are able to modify or enhance the application resource model during application deployment. This is an advanced feature and will not be needed in most use cases. However, imagine you would like to add `OPTIONS` resource method to each deployed resource as it is described at the top of this chapter. You would want to do it for every programmatic resource that is registered as well as for all standard JAX-RS resources.

To do that, you first need to register a model processor provider in your application, which implements `org.glassfish.jersey.server.model.ModelProcessor` extension contract. An example of a model processor implementation is shown here:

### Example 14.6. A programmatic resource

```
1
2         import jakarta.ws.rs.GET;
3         import jakarta.ws.rs.Path;
4         import jakarta.ws.rs.Produces;
5         import jakarta.ws.rs.container.ContainerRequestContext;
6         import jakarta.ws.rs.core.Application;
7         import jakarta.ws.rs.core.Configuration;
8         import jakarta.ws.rs.core.MediaType;
9         import jakarta.ws.rs.core.Response;
10        import jakarta.ws.rs.ext.Provider;
11
12        import org.glassfish.jersey.process.Inflector;
13        import org.glassfish.jersey.server.model.ModelProcessor;
14        import org.glassfish.jersey.server.model.Resource;
15        import org.glassfish.jersey.server.model.ResourceMethod;
16        import org.glassfish.jersey.server.model.ResourceModel;
17
18        @Provider
19        public static class MyOptionsModelProcessor implements ModelProcessor {
20
21            @Override
22            public ResourceModel processResourceModel(ResourceModel resourceModel) {
23                // we get the resource model and we want to enhance it
24                ResourceModel.Builder newResourceModelBuilder = new ResourceModel.Builder();
25                for (final Resource resource : resourceModel.getResources()) {
26                    // for each resource in the resource model we
27                    final Resource.Builder resourceBuilder = Resource.Builder.newBuilder(resource);
28
29                    // we add a new OPTIONS method to each resource
30                    // note that we should check whether the method already exists
31                    resourceBuilder.addMethod("OPTIONS")
32                        .handledBy(new Inflector<ContainerRequestContext>() {
33                            @Override
```

```

34         public String apply(ContainerRequestContext requestContext,
35                             UriInfo uriInfo,
36                             Object resource) {
37             return "On this path the resource
38                 + " methods is deployed.";
39         }
40     }
41     }).produces(MediaType.TEXT_PLAIN)
42     .extended(true); // extended means:
43     // we add to the model new resource which is a
44     // by the OPTIONS method
45     newResourceModelBuilder.addResource(resourceBuilder
46                                     .build());
47     final ResourceModel newResourceModel = newResourceModelBuilder
48     .build();
49     // and we return new model
50     return newResourceModel;
51 };
52 @Override
53 public ResourceModel processSubResource(ResourceModel resourceModel,
54     UriInfo uriInfo,
55     Object resource) {
56     // we just return the original subResourceModel which is
57     return subResourceModel;
58 }
59 }

```

The `MyOptionsModelProcessor` from the example is a relatively simple model processor which iterates over all registered resources and for all of them builds a new resource that is equal to the old resource except it is enhanced with a new `OPTIONS` method.

Note that you only need to register such a `ModelProcessor` as your custom extension provider in the same way as you would register any standard JAX-RS extension provider. During an application deployment, Jersey will look for any registered model processor and execute them. As you can see, model processors are very powerful as they can do any manipulations with the resource models. A model processor can even, for example, completely ignore the old resource model and return a new custom resource model with a single "Hello world!" resource, which would result in only the "Hello world!" resource being deployed in your application. Of course, it would not make much sense to implement such model processor, but the scenario demonstrates how powerful the model processor concept is. A better, real-life use case scenario would, for example, be to always add some custom new resource to each application that might return additional metadata about the deployed application. Or, you might want to filter out particular resources or resource methods, which is another situation where a model processor could be used successfully.

Also note that `processSubResource(ResourceModel resourceModel, UriInfo uriInfo, Object resource, Configuration configuration)` method is executed for each sub resource returned from the sub resource locator. The example is simplified and does not do any manipulation but probably in such a case you would want to enhance all sub resources with a new `OPTIONS` method handlers too.

It is important to remember that any model processor must always return valid resource model. As you might have already noticed, in the example above this important rule is not followed. If any of the resources in the original resource model would already have an `OPTIONS` method handler defined, adding another handler would cause the application fail during the deployment in the resource model validation phase. In order to retain the consistency of the final model, a model processor implementation would have to be more robust than what is shown in the example.

---

# Chapter 15. Jersey configuration

This chapter provides Jersey configuration basics which includes configuration using default configuration provider (included in Jersey by default) using system properties, and micro-profile configuration extension which allows plugging-in of configuration modules based on micro profile configuration specification.

## 15.1. Jersey default configuration provider

Since Jersey 2.29 it is possible to turn on the ability to convert the System properties into Configuration properties. That can be done by using the System property, too:

```
java -Djersey.config.allowSystemPropertiesProvider=true -DNAME=VALUE
```

Note that with the security manager turned on, write access permission is required to execute `System.getProperties()`. With insufficient permissions, the warning message is logged (with `Level.FINER`) and only `CommonProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html>], `ClientProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html>], and `ServerProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html>] properties are used, as the property names are known and `System.getProperty(name)` method can be used, which does not require the write access permission.

## 15.2. Micro profile configuration provider

Microprofile platform became very popular lately and Microprofile Config Specification [<https://microprofile.io/project/eclipse/microprofile-config>] is a recommended way in the Jakarta EE world to configure the specifications under the Jakarta EE umbrella.

Jersey 2.29 comes with support for Microprofile Config implementation such as Helidon [<https://helidon.io/>] or SmallRye [<https://smallrye.io/>]. To configure the Jersey application, the `microprofile-config.properties` file needs to be created in the `META-INF` folder. The required properties are then simply set in the `microprofile-config.properties`:

```
NAME=VALUE
```

Then Jersey Microprofile Config extension is needed to be added:

```
<dependency>
  <groupId>org.glassfish.jersey.ext.microprofile</groupId>
  <artifactId>jersey-mp-config</artifactId>
  <version>4.0.0</scope>
</dependency>
```

And the Microprofile Config implementation, such as Helidon:

```
<dependency>
```



```
<groupId>io.helidon.microprofile.config</groupId>  
<artifactId>helidon-microprofile-config</artifactId>  
<version>2.1.0</version>  
</dependency>
```

Or SmallRye:

```
<dependency>  
  <groupId>io.smallrye</groupId>  
  <artifactId>smallrye-config</artifactId>  
  <version>1.5.0</version>  
</dependency>
```

or any other suitable Microprofile Config implementation.

---

# Chapter 16. Micrometer - application observability facade

The chapter is about Micrometer integration into Jersey which comes since the version 2.41 as an extension module. Before Jersey 2.41, it was possible to integrate Micrometer with Jersey using directly Micrometer Jersey/Jetty support [<https://micrometer.io/docs/ref/jetty>]. There is also support for Jakarta EE 10 integration. The detailed documentation regarding metrics fine-tuning can be found at the Micrometer project [<https://micrometer.io/>].

## 16.1. Integration into Jersey

Since Jersey 2.41 it's possible to use an extension module in order to use Micrometer instrumentation inside your projects. The module shall be added as a dependency:

```
<dependency>
  <groupId>org.glassfish.jersey.ext.micrometer</groupId>
  <artifactId>jersey-micrometer</artifactId>
  <version>4.0.0</scope>
</dependency>
```

After the dependency is added, the Micrometer can be configured as follows:

```
final ResourceConfig resourceConfig = new ResourceConfig();
resourceConfig.register(new MetricsApplicationEventListener(
    registry,
    new DefaultJerseyTagsProvider(), "http.shared.metrics", true));
final ServletContainer servletContainer = new ServletContainer(resourceConfig);
```

the registry instance is of type `MeterRegistry` which could be `new SimpleMeterRegistry();`. Then all metrics can be accessed like `registry.get("http.shared.metrics")`. The "http.shared.metrics" string is the name of a particular registry which was registered within the `MetricsApplicationEventListener`. Micrometer supports a set of `Meter` primitives, including `Timer`, `Counter`, `Gauge`, `DistributionSummary`, `LongTaskTimer`, `FunctionCounter`, `FunctionTimer`, and `TimeGauge`. Different meter types result in a different number of time series metrics. For example, while there is a single metric that represents a `Gauge`, a `Timer` measures both the count of timed events and the total time of all timed events.

Implementing resource methods, which should be measured, several annotations can be used. The basic example demonstrates the `@Counted` annotation.

### Example 16.1. Annotated Micrometer resource methods

```
@GET
@Counted(value = COUNTER_NAME, description = COUNTER_DESCRIPTION)
@Produces(MediaType.TEXT_PLAIN)
@Path("counted")
public String getCounterMessage() {
    return "Requests to this method are counted. Use /metrics to see more";
}
```

Metrics however can be introduced using another annotations `@Timed`, or `@TimedSet` which is a set of `@Timed`.

---

# Chapter 17. Server-Sent Events (SSE) Support

## 17.1. What are Server-Sent Events

In a standard HTTP request-response scenario a client opens a connection, sends a HTTP request to the server (for example a HTTP GET request), then receives a HTTP response back and the server closes the connection once the response is fully sent/received. The initiative *always* comes from a client when the client requests all the data. In contrast, *Server-Sent Events (SSE)* is a mechanism that allows server to asynchronously push the data from the server to the client once the client-server connection is established by the client. Once the connection is established by the client, it is the server who provides the data and decides to send it to the client whenever new "chunk" of data is available. When a new data event occurs on the server, the data event is sent by the server to the client. Thus the name Server-Sent Events. Note that at high level there are more technologies working on this principle, a short overview of the technologies supporting server-to-client communication is in this list:

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Polling            | With polling a client repeatedly sends new requests to a server. If the server has no new data, then it send appropriate indication and closes the connection. The client then waits a bit and sends another request after some time (after one second, for example).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Long-polling       | With long-polling a client sends a request to a server. If the server has no new data, it just holds the connection open and waits until data is available. Once the server has data (message) for the client, it uses the connection and sends it back to the client. Then the connection is closed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Server-Sent events | SSE is similar to the long-polling mechanism, except it does not send only one message per connection. The client sends a request and server holds a connection until a new message is ready, then it sends the message back to the client while still keeping the connection open so that it can be used for another message once it becomes available. Once a new message is ready, it is sent back to the client on the same initial connection. Client processes the messages sent back from the server individually without closing the connection after processing each message. So, SSE typically reuses one connection for more messages (called events). SSE also defines a dedicated media type that describes a simple format of individual events sent from the server to the client. SSE also offers standard javascript client API implemented most modern browsers. For more information about SSE, see the SSE API specification [ <a href="https://www.w3.org/TR/eventsourcing/">https://www.w3.org/TR/eventsourcing/</a> ]. |
| WebSocket          | WebSocket technology is different from previous technologies as it provides a real full duplex connection. The initiator is again a client which sends a request to a server with a special HTTP header that informs the server that the HTTP connection may be "upgraded" to a full duplex TCP/IP WebSocket connection. If server supports WebSocket, it may choose to do so. Once a WebSocket connection is established, it can be used for bi-directional communication between the client and the server. Both client and server can then send data to the other party at will whenever it is needed. The communication on the new WebSocket connection is no longer based on HTTP protocol and can be used for example for for online gaming or any                                                                                                                                                                                                                                                                                      |

other applications that require fast exchange of small chunks of data in flowing in both directions.

## 17.2. When to use Server-Sent Events

As explained above, SSE is a technology that allows clients to subscribe to event notifications that originate on a server. Server generates new events and sends these events back to the clients subscribed to receive the notifications. In other words, SSE offers a solution for a one-way publish-subscribe model.

A good example of the use case where SSE can be used is a simple message exchange RESTful service. Clients POST new messages to the service and subscribe to receive messages from other clients. Let's call the resource `messages`. While POSTing a new message to this resource involves a typical HTTP request-response communication between a client and the `messages` resource, subscribing to receive all new message notifications would be hard and impractical to model with a sequence of standard request-response message exchanges. Using Server-sent events provides a much more practical approach here. You can use SSE to let clients subscribe to the `messages` resource via standard GET request (use a SSE client API, for example javascript API or Jersey Client SSE API) and let the server broadcast new messages to all connected clients in the form of individual events (in our case using Jersey Server SSE API). Note that with Jersey a SSE support is implemented as an usual JAX-RS resource method. There's no need to do anything special to provide a SSE support in your Jersey/JAX-RS applications, your SSE-enabled resources are a standard part of your RESTful Web application that defines the REST API of your application. The following chapters describes SSE support in Jersey in more details.

## 17.3. Server-Sent Events API

In previous JAX-RS versions, no standard API for server-sent events was defined. The SSE support bundled with Jersey was Jersey-specific. With JAX-RS 2.1 (with respect to namespace change after jakartification), situation changed and SSE API is well defined in the `jakarta.ws.rs.sse` package.

Following chapters will describe the new SSE API. For backwards compatibility reasons, the original Jersey-specific API remains valid and will be described in Section 17.6, "Jersey-specific Server-Sent Events API"

Jersey contains support for SSE for both - server and client. SSE in Jersey is implemented as an extension supporting a new media type using existing "chunked" messages support. However, in contrast to the original API, the instances of SSE related classes are not to be obtained manually by invoking constructors, nor to be directly returned from the resource methods. Actually, the implementing classes in the `jersey.media.sse.internal` package should never be needed to be imported. The only API to be used is directly in the JAX-RS package (`jakarta.ws.rs.sse`). Only builders in the API along with dependency injection should be used and provides access to the entire functionality.

In order to take advantage of the SSE support, the `jersey-media-sse` module has to be on classpath. In maven, this can be achieved by adding the dependency to the *SSE media type module*:

### Example 17.1. Adding the SSE dependency

```
1 <dependency>
2     <groupId>org.glassfish.jersey.media</groupId>
3     <artifactId>jersey-media-sse</artifactId>
4 </dependency>
```

The `Feature` defined in the module is (forced) auto-discoverable, which means having the module on classpath is sufficient, no need to further register it in the code.

## 17.4. Implementing SSE support in a JAX-RS resource (with JAX-RS SSE API)

### 17.4.1. Simple SSE resource method

#### Example 17.2. Simple SSE resource method

As mentioned above, the SSE related are not instantiated directly. In this case, Jersey takes care of the dependencies and injects the `SseEventSink` [https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/SseEventSink.html] (represents the output) and `Sse` [https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/Sse.html] (provides factory methods for other SSE related types, in this case it is used to retrieve the event builder).

```

1 ...
2 import jakarta.ws.rs.sse.Sse;
3 import jakarta.ws.rs.sse.SseEventSink;
4 import jakarta.ws.rs.sse.OutboundSseEvent;
5 ...
6
7 @Path("events")
8 public static class SseResource {
9
10     @GET
11     @Produces(MediaType.SERVER_SENT_EVENTS)
12     public void getServerSentEvents(@Context SseEventSink eventSink, @Context
13         new Thread(() -> {
14             for (int i = 0; i < 10; i++) {
15                 // ... code that waits 1 second
16                 final OutboundSseEvent event = sse.newEventBuilder()
17                     .name("message-to-client")
18                     .data(String.class, "Hello world " + i + "!")
19                     .build();
20                 eventSink.send(event);
21             }
22         }).start();
23     }
24 }
25
```

The code above defines the resource deployed on URI `"/events"`. This resource has a single `@GET` resource method which *returns void*. This is an imported difference against the original API. It is Jersey's responsibility to bind the injected `SseEventSink` to the output chain.

After the `SseEventInput` is "returned" from the method, the Jersey runtime recognizes that this is a `ChunkedOutput` extension and does not close the client connection immediately. Instead, it writes the HTTP headers to the response stream and waits for more chunks (SSE events) to be sent. At this point the client can read headers and starts listening for individual events.

In the Example 17.2, "Simple SSE resource method", the resource method creates a new thread that sends a sequence of 10 events. There is a 1 second delay between two subsequent events as indicated in a comment. Each event is represented by `jakarta.ws.rs.sse.OutboundSseEvent` type and is built with a help of a provided `Builder`. The `Builder` is obtain via the injected instance (actually, it is a singleton)

of `jakarta.ws.rs.sse.Sse` (the `newEventBuilder()` method). The `OutboundSseEvent` implementation reflects the standardized format of SSE messages and contains properties that represent `name` (for named events), `comment`, `data` or `id`. The code also sets the event data media type using the `mediaType(MediaType)` method on the `eventBuilder`. The media type, together with the data type set by the `data(Class, Object)` method (in our case `String.class`), is used for serialization of the event data. Note that the event data media type will not be written to any headers as the response `Content-type` header is already defined by the `@Produces` and set to `"text/event-stream"` using constant from the `MediaType`. The event media type is used for serialization of event data. Event data media type and Java type are used to select the proper `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>] for event data serialization and are passed to the selected writer that serializes the event data content. In our case the string `"Hello world " + i + "!"` is serialized as `"text/plain"`. In event data you can send any Java entity and associate it with any media type that you would be able to serialize with an available `MessageBodyWriter<T>`. Typically, you may want to send e.g. JSON data, so you would fill the data with a JAXB annotated bean instance and define media type to JSON.

## Note

If the event media type is not set explicitly, the `"text/plain"` media type is used by default.

Once an outbound event is ready, it can be written to the `EventSink`. At that point the event is serialized by internal `OutboundEventWriter` which uses an appropriate `MessageBodyWriter<T>` to serialize the `"Hello world " + i + "!"` string. You can send as many messages as you like. At the end of the thread execution the response is closed which also closes the connection to the client. After that, no more messages can be sent to the client on this connection. If the client would like to receive more messages, it would have to send a new request to the server to initiate a new SSE streaming connection.

A client connecting to our SSE-enabled resource will receive the following data from the entity stream:

```
event: message-to-client
data: Hello world 0!
```

```
event: message-to-client
data: Hello world 1!
```

```
event: message-to-client
data: Hello world 2!
```

```
event: message-to-client
data: Hello world 3!
```

```
event: message-to-client
data: Hello world 4!
```

```
event: message-to-client
data: Hello world 5!
```

```
event: message-to-client
data: Hello world 6!
```

```
event: message-to-client
data: Hello world 7!
```

```
event: message-to-client
data: Hello world 8!
```

```
event: message-to-client
data: Hello world 9!
```

Each message is received with a delay of one second.

## Note

If you have worked with streams in JAX-RS, you may wonder what is the difference between `ChunkedOutput` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ChunkedOutput.html>] and `StreamingOutput` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/StreamingOutput.html>].

`ChunkedOutput` is Jersey-specific API. It lets you send "chunks" of data without closing the client connection using series of convenient calls to `ChunkedOutput.write` methods that take POJO + chunk media type as an input and then use the configured JAX-RS `MessageBodyWriter<T>` providers to figure out the proper way of serializing each chunk POJO to bytes. Additionally, `ChunkedOutput` writes can be invoked multiple times on the same outbound response connection, i.e. individual chunks are written in each write, not the full response entity.

`StreamingOutput` is, on the other hand, a low level JAX-RS API that works with bytes directly. You have to implement `StreamingOutput` interface yourself. Also, its `write(OutputStream)` method will be invoked by JAX-RS runtime only once per response and the call to this method is blocking, i.e. the method is expected to write the entire entity body before returning.

## 17.4.2. Broadcasting with Jersey SSE

JAX-RS SSE API defines `SseBroadcaster` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseBroadcaster.html>] which allows to broadcast individual events to multiple clients. A simple broadcasting implementation is shown in the following example:

### Example 17.3. Broadcasting SSE messages (with JAX-RS 3.0 API)

```
1 ...
2 import jakarta.ws.rs.sse.Sse;
3 import jakarta.ws.rs.sse.SseEventSink;
4 import jakarta.ws.rs.sse.SseBroadcaster;
5 ...
6
7 @Singleton
8 @Path("broadcast")
9 public static class BroadcasterResource {
10     private Sse sse;
11     private SseBroadcaster broadcaster;
12
13     public BroadcasterResource(@Context final Sse sse) {
14         this.sse = sse;
15         this.broadcaster = sse.newBroadcaster();
16     }
17
18     @POST
```



```
19     @Produces(MediaType.TEXT_PLAIN)
20     @Consumes(MediaType.TEXT_PLAIN)
21     public String broadcastMessage(String message) {
22         final OutboundSseEvent event = sse.newEventBuilder()
23             .name("message")
24             .mediaType(MediaType.TEXT_PLAIN_TYPE)
25             .data(String.class, message)
26             .build();
27
28         broadcaster.broadcast(event);
29
30         return "Message '" + message + "' has been broadcast.";
31     }
32
33     @GET
34     @Produces(MediaType.SERVER_SENT_EVENTS)
35     public void listenToBroadcast(@Context SseEventSink eventSink) {
36         this.broadcaster.register(eventSink);
37     }
38 }
39
```

Let's explore the example together. The `BroadcasterResource` resource class is annotated with `@Singleton` [<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/singleton>] annotation which tells Jersey runtime that only a single instance of the resource class should be used to serve all the incoming requests to `/broadcast` path. This is needed as we want to keep an application-wide single reference to the private `broadcaster` field so we can use the same instance for all requests. Clients that want to listen to SSE events first send a `GET` request to the `BroadcasterResource`, that is handled by the `listenToBroadcast()` resource method. The method is injected with a new `SseEventSink` representing the connection to the requesting client and registers this `eventSink` instance with the singleton `broadcaster` by calling its `subscribe()` method. The method then, as already explained returns `void` and Jersey runtime is responsible for binding the injected `EventSink` instance so as it would have been returned from the resource method (note that really returning the `EventSink` from the resource method will cause failure) and to bind the `eventSink` instance with the requesting client and send the response HTTP headers to the client. The client connection remains open and the client is now waiting ready to receive new SSE events. All the events are written to the `eventSink` by `broadcaster` later on. This way developers can conveniently handle sending new events to all the clients that subscribe to them.

When a client wants to broadcast new message to all the clients listening on their SSE connections, it sends a `POST` request to `BroadcasterResource` resource with the message content. The method `broadcastMessage(String)` is invoked on `BroadcasterResource` resource with the message content as an input parameter. A new SSE outbound event is built in the standard way and passed to the broadcaster. The broadcaster internally invokes `write(OutboundEvent)` on all registered `EventSinks`. After that the method just returns a standard text response to the `POSTing` client to inform the client that the message was successfully broadcast. As you can see, the `broadcastMessage(String)` resource method is just a simple JAX-RS resource method.

In order to implement such a scenario, you may have noticed, that the `SseBroadcaster` is not mandatory to complete the use case. Individual `EventSinks` can be just stored in a collection and iterated over in the `broadcastMessage` method. However, the `SseBroadcaster` internally identifies and handles also client disconnects. When a client closes the connection, the broadcaster detects this and removes the stale connection from the internal collection of the registered `EventSinks` as well as it frees all the server-side resources associated with the stale connection. Additionally, the `SseBroadcaster` is implemented to be thread-safe, so that clients can connect and disconnect at any

time and `SseBroadcaster` will always broadcast messages to the most recent collection of registered and active set of clients.

## 17.5. Consuming SSE events within Jersey clients

On the client side, push programming model is used (event consumer / client) gets asynchronously notified about incoming events by subscribing custom listener to `jakarta.ws.rs.sse.SseEventSource`. This happens by invoking one of its `subscribe()` methods.

The usage of `SseEventSource` is shown in the following example.

### Example 17.4. Consuming SSE events with `SseEventSource`

```
1 import jakarta.ws.rs.sse.SseEventSource;
2 ...
3 Client client = ClientBuilder.newBuilder().build();
4 WebTarget target = client.target("http://example.com/events");
5 SseEventSource sseEventSource = SseEventSource.target(target).build();
6 sseEventSource.register(event -> System.out.println(event.getName() + "; " +
7     + event.readData(String.class)));
8 sseEventSource.open();
9
10 // do other stuff, block here and continue when done
11
12 sseEventSource.close();
13
```

In this example, the client code connects to the server where the `SseResource` from the Example 17.2, “Simple SSE resource method” is deployed. The Client [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>] instance is created (and initialized with `SseFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html>] automatically). Then the `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>] is built. In this case a request to the web target is not made directly in the code, instead, the web target instance is used to initialize a new `SseEventSource.Builder` [<https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/SseEventSource.Builder.html>] instance that is used to build a new `SseEventSource` [<https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/SseEventSource.html>]. The choice of `build()` method is important, as it tells the `SseEventSource.Builder` to create a new `SseEventSource` that is not automatically connected to the target. The connection is established only later by manually invoking the `sseEventSource.open()` method. A custom `java.util.function.Consumer<InboundSseEvent>` implementation is used to listen to and process incoming SSE events. The method `readData(Class)` says that the event data should be de-serialized from a received `InboundSseEvent` [<https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/InboundSseEvent.html>] instance into a `String` Java type. This method call internally executes `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] which de-serializes the event data. This is similar to reading an entity from the Response [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>] by `readEntity(Class)`. The method `readData` can throw a `ProcessingException` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ProcessingException.html>].

After a connection to the server is opened by calling the `open()` method on the event source, the `eventSource` starts listening to events. When an event comes, the listener will be executed by the event

source. Once the client is done with processing and does not want to receive events the connection by calling the `close()` method on the event source.

The listener from the example above will print the following output:

```
message-to-client; Hello world 0!
message-to-client; Hello world 1!
message-to-client; Hello world 2!
message-to-client; Hello world 3!
message-to-client; Hello world 4!
message-to-client; Hello world 5!
message-to-client; Hello world 6!
message-to-client; Hello world 7!
message-to-client; Hello world 8!
message-to-client; Hello world 9!
```

There are other events than the incoming data that also may occur. The `SseEventSource` for instance always signals, that it has finished processing events, or there might also be an error while processing the messages. `SseEventSource`. There are total of four overloaded `subscribe()` methods defined in the API.

### Example 17.5. `SseEventSource` `subscribe()` methods

```
1 // 1. basic one - the one we used in the example
2 void subscribe(Consumer<InboundSseEvent> onEvent);
3
4 // 2. with an error callback
5 void subscribe(Consumer<InboundSseEvent> onEvent, Consumer<Throwable> onError)
6
7 // 3. with an error callback and completion callback
8 void subscribe(Consumer<InboundSseEvent> onEvent, Consumer<Throwable> onError,
9
10 // 4. complete one - with error callback, completion callback an onSubscribe c
11 void subscribe(Consumer<SseSubscription> onSubscribe, Consumer<InboundSseEvent
12 onError,
13 Runnable
14 onComplete);
15
```

Few notes to the `subscribe()` methods:

- All the overloaded methods have the `onEvent` handler. As shown in the example, this parameter is used to consume the SSE events with data.
- Except the basic one-arg method, all the others contain an `onError` handler. In case of error, the `SseEventSource` invokes the `onError` method of all its subscribers, that registered the handler. This makes it possible to react to the error conditions in a custom manner.
- Another possible argument is the `onComplete` handler. If registered (used an appropriate `subscribe()` method, that has the `onComplete` argument), it is invoked (for all the subscribers) every time when the `SseEventSource` terminates normally. Either `onComplete` or `onError` should be called every time.
- The complete `subscribe()` method adds the `onSubscribe()` callback. This gives the subscriber a tool to manage the load and do a back-pressure by incrementally requesting only certain amount

of items. When `SseEventSource` registers a new subscriber, it calls its `onSubscribe` handler and hands over the `jakarta.ws.rs.sse.SseSubscription` instance. This class only has two methods - `request(long)` for asking for a certain amount of events (often used as `request(Long.MAX_VALUE)` when no back-pressure is needed) and `cancel()` to stop receiving further events.

- When using the full-arg version of `subscribe()`, it is the caller's responsibility to manage the amount of data it can handle. The `sseSubscription.request()` method *MUST* be called, otherwise the subscriber will not receive ANY data. Furthermore, in the current `SseEventSource` implementation, such a subscriber will block a thread and will occasionally lead to overflow of an internal buffer in `SseEventSource`. As mentioned, calling `subscription.request(Long.MAX_VALUE)`, e.g. in the registered `onSubscribe` handler is sufficient (and is also a default behaviour for all the other overloaded methods).

## 17.5.1. SseEventSource reconnect support

The `SseEventSource` [<https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/SseEventSource.html>] implementation supports automated recuperation from a connection loss, including negotiation of delivery of any missed events based on the last received SSE event `id` field value, provided this field is set by the server and the negotiation facility is supported by the server. In case of a connection loss, the last received SSE event `id` field value is sent in the `Last-Event-ID` HTTP request header as part of a new connection request sent to the SSE endpoint. Upon a receipt of such reconnect request, the SSE endpoint that supports this negotiation facility is expected to replay all missed events.

### Note

Note, that SSE lost event negotiation facility is a best-effort mechanism which does not provide any guarantee that all events would be delivered without a loss. You should therefore not rely on receiving every single event and design your client application code accordingly.

By default, when a connection to the SSE endpoint is lost, the event source will use a default delay before attempting to reconnect to the SSE endpoint. The SSE endpoint can however control the client-side retry delay by including a special `retry` field value in any event sent to the client. Jersey `SseEventSource` [<https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/SseEventSource.html>] implementation automatically tracks any received SSE event `retry` field values set by the endpoint and adjusts the reconnect delay accordingly, using the last received `retry` field value as the new reconnect delay.

In addition to handling the standard connection losses, Jersey `SseEventSource` [<https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/SseEventSource.html>] automatically deals with any HTTP 503 `Service Unavailable` responses received from the SSE endpoint, that include a `Retry-After` HTTP header with a valid value. The HTTP 503 + `Retry-After` technique is often used by HTTP endpoints as a means of connection and traffic throttling. In case a HTTP 503 + `Retry-After` response is received in return to a connection request from SSE endpoint, Jersey `SseEventSource` will automatically schedule a reconnect attempt and use the received `Retry-After` HTTP header value as a one-time override of the reconnect delay.

## 17.6. Jersey-specific Server-Sent Events API

### Important

Prior to JAX-RS 2.1, server-sent events was not standardized and was optional and implementation-specific. Jersey provided its own, specific version of SSE implementation, that remains valid and functional to achieve backwards compatibility. This implementation is a Jersey-

specific extension of JAX-RS (2.0) standard. It works with common JAX-RS resources the same way as the JAX-RS 2.1 based implementation does.

After the jakartification Jersey complies with JAX-RS 3.0 and SSE implementation may not be backward compatible due to namespace change. When the 3.x version is being used, it's required to comply with JAX-RS 3.0 SSE spec.

This chapter briefly describes the Jersey-specific support for SSE, focusing on the differences against the new SSE implementation described in ???

The API contains SSE support for both - server and client. To use the Jersey-specific SSE API, you need to add the dependency to the

In order to add support for this SSE implementation, you also need to include the dependency to the *SSE media type module* the same way as for the JAX-RS SSE implementation.

### Example 17.6. Add `jersey-media-sse` dependency.

```
1 <dependency>
2     <groupId>org.glassfish.jersey.media</groupId>
3     <artifactId>jersey-media-sse</artifactId>
4 </dependency>
```

### Note

Prior to Jersey 2.8, you had to manually register `SseFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html>] in your application. (The `SseFeature` is a feature that can be registered for both, the client and the server.) Since Jersey 2.8, the feature gets automatically discovered and registered when Jersey SSE module is put on the application's classpath. The automatic discovery and registration of SSE feature can be suppressed by setting `DISABLE_SSE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html#DISABLE\\_SSE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html#DISABLE_SSE)] property to `true`. The behavior can also be selectively suppressed in either client or server runtime by setting `DISABLE_SSE_CLIENT` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html#DISABLE\\_SSE\\_CLIENT](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html#DISABLE_SSE_CLIENT)] or `DISABLE_SSE_SERVER` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html#DISABLE\\_SSE\\_SERVER](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html#DISABLE_SSE_SERVER)] property respectively.

## 17.6.1. Implementing SSE support in a JAX-RS resource

### 17.6.1.1. Simple SSE resource method

#### Example 17.7. Simple SSE resource method

```
1 ...
2 import org.glassfish.jersey.media.sse.EventOutput;
3 import org.glassfish.jersey.media.sse.OutboundEvent;
4 import org.glassfish.jersey.media.sse.SseFeature;
5 ...
6
7 @Path("events")
8 public static class SseResource {
```

```
9
10  @GET
11  @Produces(SseFeature.SERVER_SENT_EVENTS)
12  public EventOutput getServerSentEvents() {
13      final EventOutput eventOutput = new EventOutput();
14      new Thread(new Runnable() {
15          @Override
16          public void run() {
17              try {
18                  for (int i = 0; i < 10; i++) {
19                      // ... code that waits 1 second
20                      final OutboundEvent.Builder eventBuilder = new OutboundEvent.Builder();
21                      eventBuilder.name("message-to-client");
22                      eventBuilder.data(String.class, "Hello world " + i + "!");
23                      final OutboundEvent event = eventBuilder.build();
24                      eventOutput.write(event);
25                  }
26              } catch (IOException e) {
27                  throw new RuntimeException("Error when writing the event.");
28              } finally {
29                  try {
30                      eventOutput.close();
31                  } catch (IOException ioClose) {
32                      throw new RuntimeException("Error when closing the event output.");
33                  }
34              }
35          }
36      }).start();
37      return eventOutput;
38  }
39 }
40
```

The code above defines the resource deployed on URI `"/events"`. This resource has a single `@GET` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/GET.html] resource method which returns as an entity `EventOutput` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventOutput.html] - an extension of generic Jersey `ChunkedOutput` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ChunkedOutput.html] API for output chunked message processing.

In the Example 17.7, “Simple SSE resource method”, the resource method creates a new thread that sends a sequence of 10 events. There is a 1 second delay between two subsequent events as indicated in a comment. Each event is represented by `OutboundEvent` type and is built with a help of an outbound event `Builder`. The `OutboundEvent` reflects the standardized format of SSE messages and contains properties that represent `name` (for named events), `comment`, `data` or `id`. The code also sets the event data media type using the `mediaType(MediaType)` method on the `eventBuilder`. The media type, together with the data type set by the `data(Class, Object>` method (in our case `String.class`), is used for serialization of the event data. Note that the event data media type will not be written to any headers as the response `Content-type` header is already defined by the `@Produces` and set to `"text/event-stream"` using constant from the `SseFeature`. The event media type is used for serialization of event data. Event data media type and Java type are used to select the proper `MessageBodyWriter<T>` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html] for event data serialization and are passed to the selected writer that serializes the event data content. In our case the string `"Hello world " + i + "!"` is serialized as `"text/`

plain". In event `data` you can send any Java entity and associate it with any media type that you would be able to serialize with an available `MessageBodyWriter<T>`. Typically, you may want to send e.g. JSON data, so you would fill the `data` with a JAXB annotated bean instance and define media type to JSON.

### Note

If the event media type is not set explicitly, the "text/plain" media type is used by default.

Once an outbound event is ready, it can be written to the `eventOutput`. At that point the event is serialized by internal `OutboundEventWriter` which uses an appropriate `MessageBodyWriter<T>` to serialize the "Hello world " + `i` + "!" string. You can send as many messages as you like. At the end of the thread execution the response is closed which also closes the connection to the client. After that, no more messages can be sent to the client on this connection. If the client would like to receive more messages, it would have to send a new request to the server to initiate a new SSE streaming connection.

A client connecting to our SSE-enabled resource will receive the exact same output as in the corresponding example in the JAX-RS implementation example.

```
event: message-to-client
data: Hello world 0!
```

```
event: message-to-client
data: Hello world 1!
```

...

## 17.6.1.2. Broadcasting

Jersey SSE server API defines `SseBroadcaster` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseBroadcaster.html>] which allows to broadcast individual events to multiple clients. A simple broadcasting implementation is shown in the following example:

### Example 17.8. Broadcasting SSE messages

```
1 ...
2 import org.glassfish.jersey.media.sse.SseBroadcaster;
3 ...
4
5 @Singleton
6 @Path("broadcast")
7 public static class BroadcasterResource {
8
9     private SseBroadcaster broadcaster = new SseBroadcaster();
10
11     @POST
12     @Produces(MediaType.TEXT_PLAIN)
13     @Consumes(MediaType.TEXT_PLAIN)
14     public String broadcastMessage(String message) {
15         OutboundEvent.Builder eventBuilder = new OutboundEvent.Builder();
16         OutboundEvent event = eventBuilder.name("message")
17             .mediaType(MediaType.TEXT_PLAIN_TYPE)
18             .data(String.class, message)
```

```
19         .build();
20
21     broadcaster.broadcast(event);
22     return "Message '" + message + "' has been broadcast.";
23 }
24
25 @GET
26 @Produces(SseFeature.SERVER_SENT_EVENTS)
27 public EventOutput listenToBroadcast() {
28     final EventOutput eventOutput = new EventOutput();
29     this.broadcaster.add(eventOutput);
30     return eventOutput;
31 }
32 }
33
```

The example is similar to its relevant JAX-RS counterpart. The `listenToBroadcast()` resource method creates a new `EventOutput` representing the connection to the requesting client and registers this `eventOutput` instance with the singleton `broadcaster`, using its `add(EventOutput)` method. The method then returns the `eventOutput` which causes Jersey to bind the `eventOutput` instance with the requesting client and send the response HTTP headers to the client. The client connection remains open and the client is now waiting ready to receive new SSE events. All the events are written to the `eventOutput` by `broadcaster` later on.

When a client wants to broadcast new message to all the clients listening on their SSE connections, it sends a `POST` request to `BroadcasterResource` resource with the message content. The method `broadcastMessage(String)` is invoked on `BroadcasterResource` resource with the message content as an input parameter. A new SSE outbound event is built in the standard way and passed to the broadcaster. The broadcaster internally invokes `write(OutboundEvent)` on all registered `EventOutputs`. After that the method just return a standard text response to the `POST`ing client to inform the client that the message was successfully broadcast.

## 17.6.2. Consuming SSE events with Jersey clients

On the client side, Jersey exposes APIs that support receiving and processing SSE events using two programming models:

Pull model - pulling events from a `EventInput` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventInput.html>], or

Push model - listening for asynchronous notifications of `EventSource`

The push model is similar to what is implemented in the JAX-RS SSE API. The pull model does not have a direct counterpart in the JAX-RS API and has to be implemented by the developer, if required.

### 17.6.2.1. Reading SSE events with EventInput

The events can be read on the client side from a `EventInput` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventInput.html>]. See the following code:

```
1 Client client = ClientBuilder.newBuilder()
2     .register(SseFeature.class).build();
3 WebTarget target = client.target("http://localhost:9998/events");
4
5 EventInput eventInput = target.request().get(EventInput.class);
6 while (!eventInput.isClosed()) {
7     final InboundEvent inboundEvent = eventInput.read();
8 }
```



```
8     if (inboundEvent == null) {
9         // connection has been closed
10        break;
11    }
12    System.out.println(inboundEvent.getName() + "; " + inboundEvent.readData(S
13 }
14
```

In this example, a client connects to the server where the `SseResource` from the Example 17.7, “Simple SSE resource method” is deployed. At first, a new JAX-RS/Jersey `client` instance is created with a `SseFeature` registered. Then a `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>] instance is retrieved from the `client` and is used to invoke a HTTP request. The returned response entity is directly read as a `EventInput` Java type, which is an extension of Jersey `ChunkedInput` that provides generic support for consuming chunked message payloads. The code in the example then process starts a loop to process the inbound SSE events read from the `eventInput` response stream. Each chunk read from the input is a `InboundEvent`. The method `InboundEvent.readData(Class)` provides a way for the client to indicate what Java type should be used for the event data de-serialization. In our example, individual events are de-serialized as `String` Java type instances. This method internally finds and executes a proper `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] which is the used to do the actual de-serialization. This is similar to reading an entity from the `Response` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>] by `readEntity(Class)`. The method `readData` can also throw a `ProcessingException` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ProcessingException.html>].

The `null` check on `inboundEvent` is necessary to make sure that the chunk was properly read and connection has not been closed by the server. Once the connection is closed, the loop terminates and the program completes execution. The client code produces the following console output:

```
message-to-client; Hello world 0!
message-to-client; Hello world 1!
message-to-client; Hello world 2!
message-to-client; Hello world 3!
message-to-client; Hello world 4!
message-to-client; Hello world 5!
message-to-client; Hello world 6!
message-to-client; Hello world 7!
message-to-client; Hello world 8!
message-to-client; Hello world 9!
```

### 17.6.2.2. Asynchronous SSE processing with EventSource

The main Jersey-specific SSE client API component used to read SSE events asynchronously is `EventSource` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventSource.html>]. The usage of the `EventSource` is shown on the following example.

#### Example 17.9. Registering EventListener with EventSource

```
1 Client client = ClientBuilder.newBuilder()
2     .register(SseFeature.class).build();
3 WebTarget target = client.target("http://example.com/events");
4 EventSource eventSource = EventSource.target(target).build();
5 EventListener listener = new EventListener() {
```

```
6      @Override
7      public void onEvent(InboundEvent inboundEvent) {
8          System.out.println(inboundEvent.getName() + "; " + inboundEvent.readData());
9      }
10 };
11 eventSource.register(listener, "message-to-client");
12 eventSource.open();
13 ...
14 eventSource.close();
15
```

In this example, the client code again connects to the server where the `SseResource` from the Example 17.7, “Simple SSE resource method” is deployed. The Client [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>] instance is again created and initialized with `SseFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/SseFeature.html>]. Then the `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>] is built. In this case a request to the web target is not made directly in the code, instead, the web target instance is used to initialize a new `EventSource.Builder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventSource.Builder.html>] instance that is used to build a new `EventSource`. The choice of `build()` method is important, as it tells the `EventSource.Builder` to create a new `EventSource` that is not automatically connected to the target. The connection is established only later by manually invoking the `eventSource.open()` method. A custom `EventListener` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventListener.html>] implementation is used to listen to and process incoming SSE events. The method `readData(Class)` says that the event data should be de-serialized from a received `InboundEvent` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/InboundEvent.html>] instance into a `String` Java type. This method call internally executes `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] which de-serializes the event data. This is similar to reading an entity from the `Response` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>] by `readEntity(Class)`. The method `readData` can throw a `ProcessingException` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ProcessingException.html>].

The custom event source listener is registered in the event source via `EventSource.register(EventListener, String)` method. The next method arguments define the names of the events to receive and can be omitted. If names are defined, the listener will be associated with the named events and will only be invoked for events with a name from the set of defined event names. It will not be invoked for events with any other names or for events without a name.

## Important

It is a common mistake to think that unnamed events will be processed by listeners that are registered to process events from a particular name set. That is NOT the case! Unnamed events are only processed by listeners that are not name-bound. The same limitation applied to HTML5 Javascript SSE Client API supported by modern browsers.

After a connection to the server is opened by calling the `open()` method on the event source, the `eventSource` starts listening to events. When an event named "message-to-client" comes, the listener will be executed by the event source. If any other event comes (with a name different from "message-to-client"), the registered listener is not invoked. Once the client is done with processing and does not want to receive events anymore, it closes the connection by calling the `close()` method on the event source.

The listener from the example above will print the following output:

```
message-to-client; Hello world 0!
message-to-client; Hello world 1!
message-to-client; Hello world 2!
message-to-client; Hello world 3!
message-to-client; Hello world 4!
message-to-client; Hello world 5!
message-to-client; Hello world 6!
message-to-client; Hello world 7!
message-to-client; Hello world 8!
message-to-client; Hello world 9!
```

When browsing through the Jersey SSE API documentation, you may have noticed that the `EventSource` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventSource.html>] implements `EventListener` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/sse/EventListener.html>] and provides an empty implementation for the `onEvent(InboundEvent inboundEvent)` listener method. This adds more flexibility to the Jersey client-side SSE API. Instead of defining and registering a separate event listener, in simple scenarios you can also choose to derive directly from the `EventSource` and override the empty listener method to handle the incoming events. This programming model is shown in the following example:

#### **Example 17.10. Overriding `EventSource.onEvent(InboundEvent)` method**

```
1 Client client = ClientBuilder.newBuilder()
2     .register(SseFeature.class).build();
3 WebTarget target = client.target("http://example.com/events");
4 EventSource eventSource = new EventSource(target) {
5     @Override
6     public void onEvent(InboundEvent inboundEvent) {
7         if ("message-to-client".equals(inboundEvent.getName())) {
8             System.out.println(inboundEvent.getName() + "; " + inboundEvent.re
9         }
10    }
11 };
12 ...
13 eventSource.close();
14
```

The code above is very similar to the code in Example 17.9, “Registering `EventListener` with `EventSource`”. In this example however, the `EventSource` is constructed directly using a single-parameter constructor. This way, the connection to the SSE endpoint is by default automatically opened at the event source creation. The implementation of the `EventListener` has been moved into the overridden `EventSource.onEvent(...)` method. However, this time, the listener method will be executed for all events - unnamed as well as with any `name`. Therefore the code checks the name whether it is an event with the name “message-to-client” that we want to handle. Note that you can still register additional `EventListeners` later on. The overridden method on the event source allows you to handle messages even when no additional listeners are registered yet.

##### **17.6.2.2.1. EventSource reconnect support**

Reconnect support in Jersey-specific `EventSource` works the same way as in the implementation of the JAX-RS `SseEventSource` [<https://jakartaee.github.io/rest/apidocs/2.1.6/javax/ws/rs/sse/SseEventSource.html>].

---

# Chapter 18. Security

## 18.1. Securing server

### 18.1.1. SecurityContext

Security information of a request is available by injecting a JAX-RS `SecurityContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/SecurityContext.html>] instance using `@Context` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>] annotation. The injected security context instance provides the equivalent of the functionality available on `HttpServletRequest` [<https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/httpServletRequest>] API. The injected security context depends on the actual Jersey application deployment. For example, for a Jersey application deployed in a Servlet container, the Jersey `SecurityContext` will encapsulate information from a security context retrieved from the Servlet request. In case of a Jersey application deployed on a Grizzly server, the `SecurityContext` will return information retrieved from the Grizzly request.

`SecurityContext` can be used in conjunction with sub-resource locators to return different resources based on the specific roles a user principal is included in. For example, a sub-resource locator could return a different resource if a user is a preferred customer:

#### Example 18.1. Using `SecurityContext` for a Resource Selection

```
1 @Path("basket")
2 public ShoppingBasketResource get(@Context SecurityContext sc) {
3     if (sc.isUserInRole("PreferredCustomer") {
4         return new PreferredCustomerShoppingBasketResource();
5     } else {
6         return new ShoppingBasketResource();
7     }
8 }
```

`SecurityContext` is inherently request-scoped, yet can be also injected into fields of singleton resources and JAX-RS providers. In such case the proxy of the request-scoped `SecurityContext` will be injected.

#### Example 18.2. Injecting `SecurityContext` into a singleton resource

```
1 @Path("resource")
2 @Singleton
3 public static class MyResource {
4     // Jersey will inject proxy of Security Context
5     @Context
6     SecurityContext securityContext;
7
8     @GET
9     public String getUserPrincipal() {
10         return securityContext.getUserPrincipal().getName();
11     }
12 }
```

### 18.1.1.1. Initializing Security Context with Servlets

As described above, the `SecurityContext` by default (if not overwritten by a request filter) only exposes security information from the underlying container. In the case you deploy a Jersey application in a Servlet container, you need to configure the Servlet container security aspects (`<security-constraint>`, `<auth-constraint>` and user to roles mappings) in order to be able to secure requests via calls to the JAX-RS `SecurityContext`.

### 18.1.1.2. Using Security Context in Container Request Filters

The `SecurityContext` can be directly retrieved from `ContainerRequestContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerRequestContext.html>] via `getSecurityContext()` method. You can also replace the default `SecurityContext` in a request context with a custom one using the `setSecurityContext(SecurityContext)` method. If you set a custom `SecurityContext` instance in your `ContainerRequestFilter` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/container/ContainerRequestFilter.html>], this security context instance will be used for injection into JAX-RS resource class fields. This way you can implement a custom authentication filter that may setup your own `SecurityContext` to be used. To ensure the early execution of your custom authentication request filter, set the filter priority to `AUTHENTICATION` using constants from `Priorities` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Priorities.html>]. An early execution of your authentication filter will ensure that all other filters, resources, resource methods and sub-resource locators will execute with your custom `SecurityContext` instance.

## 18.1.2. Authorization - securing resources

### 18.1.2.1. Security resources with `web.xml`

In cases where a Jersey application is deployed in a Servlet container you can rely only on the standard Java/Jakarta EE Web application security mechanisms offered by the Servlet container and configurable via application's `web.xml` descriptor. You need to define the `<security-constraint>` elements in the `web.xml` and assign roles which are able to access these resources. You can also define HTTP methods that are allowed to be executed. See the following example.

#### Example 18.3. Securing resources using `web.xml`

```
1 <security-constraint>
2   <web-resource-collection>
3     <url-pattern>/rest/admin/*</url-pattern>
4   </web-resource-collection>
5   <auth-constraint>
6     <role-name>admin</role-name>
7   </auth-constraint>
8 </security-constraint>
9 <security-constraint>
10  <web-resource-collection>
11    <url-pattern>/rest/orders/*</url-pattern>
12  </web-resource-collection>
13  <auth-constraint>
14    <role-name>customer</role-name>
15  </auth-constraint>
16 </security-constraint>
17 <login-config>
```

```
18     <auth-method>BASIC</auth-method>
19     <realm-name>my-default-realm</realm-name>
20 </login-config>
```

The example secures two kinds of URI namespaces using the HTTP Basic Authentication. `rest/admin/*` will be accessible only for user group "admin" and `rest/orders/*` will be accessible for "customer" user group. This security configuration does not use JAX-RS or Jersey features at all as it is enforced by the Servlet container even before a request reaches the Jersey application. Keeping these security constraints up to date with your JAX-RS application might not be easy as whenever you change the `@Path` [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Path.html] annotations on your resource classes you may need to update also the `web.xml` security configurations to reflect the changed JAX-RS resource paths. Therefore Jersey offers a more flexible solution based on placing standard Java/Jakarta EE security annotations directly on JAX-RS resource classes and methods.

### 18.1.2.2. Securing JAX-RS resources with standard `jakarta.annotation.security` annotations

With Jersey you can define the access to resources based on the user group using annotations. You can, for example, define that only a user group "admin" can execute specific resource method. To do that you firstly need to register `RolesAllowedDynamicFeature` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/filter/RolesAllowedDynamicFeature.html] as a provider. The following example shows how to register the feature if your deployment is based on a `ResourceConfig` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html].

#### Example 18.4. Registering `RolesAllowedDynamicFeature` using `ResourceConfig`

```
1 final ResourceConfig resourceConfig = new ResourceConfig(MyResource.class);
2 resourceConfig.register(RolesAllowedDynamicFeature.class);
3
```

Alternatively, typically when deploying your application to a Servlet container, you can implement your JAX-RS Application [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html] subclass by extending from the Jersey `ResourceConfig` and registering the `RolesAllowedDynamicFeature` in the constructor:

#### Example 18.5. Registering `RolesAllowedDynamicFeature` by extending `ResourceConfig`

```
1 public class MyApplication extends ResourceConfig {
2     public MyApplication() {
3         super(MyResource.class);
4         register(RolesAllowedDynamicFeature.class);
5     }
6 }
```

Once the feature is registered, you can use annotations from package `jakarta.annotation.security` defined by JSR-250. See the following example.

#### Example 18.6. Applying `jakarta.annotation.security` to JAX-RS resource methods.

```
1 @Path("/")
```

```
2 @PermitAll
3 public class Resource {
4     @RolesAllowed("user")
5     @GET
6     public String get() { return "GET"; }
7
8     @RolesAllowed("admin")
9     @POST
10    public String post(String content) { return content; }
11
12    @Path("sub")
13    public SubResource getSubResource() {
14        return new SubResource();
15    }
16 }
```

The resource class `Resource` defined in the example is annotated with a `@PermitAll` [<https://jakarta.ee/specifications/annotations/2.0/apidocs/jakarta.annotation/jakarta/annotation/security/permitall>] annotation. This means that all methods in the class which do not override this annotation will be permitted for all user groups (no restrictions are defined). In our example, the annotation will only apply to the `getSubResource()` method as it is the only method that does not override the annotation by defining custom role-based security settings using the `@RolesAllowed` [<https://jakarta.ee/specifications/annotations/2.0/apidocs/jakarta.annotation/jakarta/annotation/security/rolesallowed>] annotation. `@RolesAllowed` annotations present on the other methods define a role or a set of roles that are allowed to execute a particular method.

These Java/Jakarta EE security annotations are processed internally in the request filter registered using the Jersey `RolesAllowedDynamicFeature`. The roles defined in the annotations are tested against current roles set in the `SecurityContext` using the `SecurityContext.isUserInRole(String role)` method. In case the caller is not in the role specified by the annotation, the HTTP 403 (Forbidden) error response is returned.

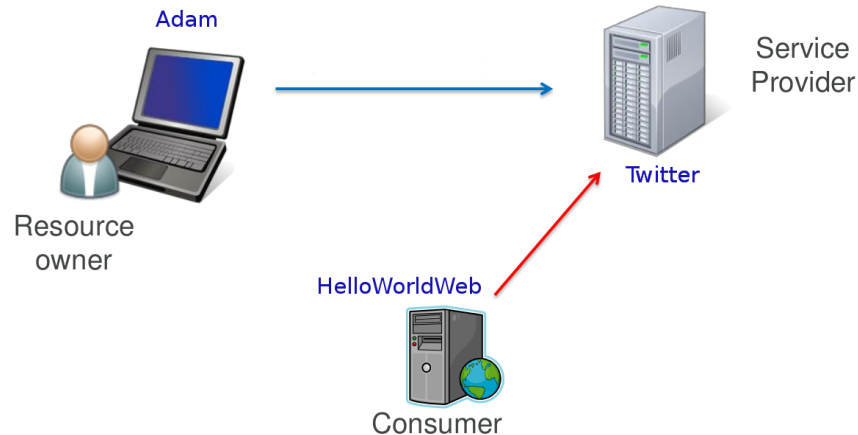
## 18.2. Client Security

For details about client security please see the Client chapter. Jersey client allows to define parameters of SSL communication using HTTPS protocol. You can also use jersey built-in authentication filters which perform *HTTP Basic Authentication* or *HTTP Digest Authentication*. See the client chapter for more details.

## 18.3. OAuth Support

OAuth is a specification that defines secure authentication model on behalf of another user. Two versions of OAuth exists at the moment - *OAuth 1* defined by OAuth 1.0 specification [<http://tools.ietf.org/html/rfc5849>] and *OAuth 2* defined by OAuth 2.0 specification [<http://tools.ietf.org/html/rfc6749>]. OAuth 2 is the latest version and it is not backward compatible with OAuth 1 specification. OAuth in general is widely used in popular social Web sites in order to grant access to a user account and associated resources for a third party consumer (application). The consumer then usually uses RESTful Web Services to access the user data. The following example describes a use case of the OAuth (similar for OAuth 1 and OAuth 2). The example is simple and probably obvious for many developers but introduces terms that are used in this documentation as well as in Jersey OAuth API documentation.

Three parties act in an OAuth scenario.



The first party represents a user, in our case Adam, who is called in the OAuth terminology a *Resource Owner*. Adam has an account on Twitter. Twitter represents the second party. This party is called a *Service Provider*. Twitter offers a web interface that Adam uses to create new tweets, read tweets of others etc. Now, Adam uses our new web site, HelloWorldWeb, which is a very simple web site that says **HELLO World** but it additionally displays the last tweet of the logged in user. To do so, our web site needs to have access to the Twitter account of Adam. Our web site is a 3rd party application that wants to connect to Twitter and get Adam's tweets. In OAuth, such party is called *Consumer*. Our Consumer would like to use Twitter's RESTful APIs to get some data associated with Adam's Twitter account. In order to solve this situation Adam could directly give his Twitter password to the HelloWorldWeb. This would however be rather unsafe, especially if we do not know much about the authors of the application. If Adam would give his password to HelloWorldWeb, he would have to deal with the associated security risks. First of all, Adam would have to fully trust HelloWorldWeb that it will not misuse the full access to his Twitter account. Next, if Adam would change his password, he would need to remember to give the new password also to the HelloWorldWeb application. And at last, if Adam would like to revoke the HelloWorldWeb's access to his Twitter account, he would need to change his password again. The OAuth protocol has been devised to address all these challenges.

With OAuth, a resource owner (Adam) grants an access to a consumer (HelloWorldWeb) without giving it his password. This access grant is achieved by a procedure called *authorization flow*. Authorization flow is out of the scope of this documentation and its description can be found in the OAuth specification linked above. The result of the authorization flow is an *Access Token* which is later used by the consumer to authenticate against the service provider. While this brief description applies to both OAuth 1 and 2, note that there are some differences in details between these two specifications.

Jersey OAuth is currently supported for the following use cases and OAuth versions:

- OAuth 1: Client (consumer) and server (service provider)
- OAuth 2: Client (consumer)

With client and server support there are two supported scenarios:

- Authorization flow
- Authentication with Access Token (support for authenticated requests)



## 18.3.1. OAuth 1

OAuth 1 protocol is based on message signatures that are calculated using specific signature methods. Signatures are quite complex and therefore are implemented in a separate module. The OAuth 1 Jersey modules are (groupId:artifactId:description):

- `org.glassfish.jersey.security:oauth1-client`: provides client OAuth 1 support for authorization flow and authentication
- `org.glassfish.jersey.security:oauth1-server`: provides server OAuth 1 support for authorization flow, SPI for token management including authentication filter.
- `org.glassfish.jersey.security:oauth1-signature` : provides support for OAuth1 request signatures. This module is a dependency of previous two modules and as such it will be implicitly included in your maven project. The module can be used as a standalone module but this will not be needed in most of the use cases. You would do that if you wanted to implement your own OAuth support and would not want to deal with implementing the complex signature algorithms.

### 18.3.1.1. Server

To add support for OAuth into your server-side application, add the following dependency to your `pom.xml`:

```
<dependency>
  <groupId>org.glassfish.jersey.security</groupId>
  <artifactId>oauth1-server</artifactId>
  <version>4.0.0</version>
</dependency>
```

Again, there is no need to add a direct dependency to the signature module, it will be transitively included.

Let's now briefly go over the most important server Jersey OAuth APIs and SPIs:

- `OAuth1ServerFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/oauth1/OAuth1ServerFeature.html>]: The feature which enables the OAuth 1 support on the server and registers `OAuth1Provider` explained in the following point.
- `OAuth1Provider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/oauth1/OAuth1Provider.html>]: Implementation of this SPI must be registered to the server runtime as a standard provider. The implementation will be used to create request and access token, get consumer by consumer key, etc. You can either implement your provider or use the default implementation provided by Jersey by `DefaultOAuth1Provider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/oauth1/DefaultOAuth1Provider.html>].
- `OAuth1ServerProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/oauth1/OAuth1ServerProperties.html>]: properties that can be used to configure the OAuth 1 support.
- `OAuth1Consumer` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/oauth1/OAuth1Consumer.html>], `OAuth1Token` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/oauth1/OAuth1Token.html>]: classes that contain consumer key, request and access tokens. You need to implement them only if you also implement the interface `OAuth1Provider`.

First step in enabling Jersey OAuth 1 support is to register a `OAuth1ServerFeature` instance initialized with an instance of `OAuth1Provider`. Additionally, you may configure the *Request Token URI* and *Access Token URI* - the endpoints accessible on the OAuth server that issue Request and Access Tokens. These endpoints are defined in the OAuth 1 specification and are contacted as part of the OAuth authorization flow.

Next, when a client initiates the OAuth authorization flow, the provided implementation of `OAuth1Provider` will be invoked as to create new tokens, get tokens and finally to store the issued Access Token. If a consumer already has a valid Access Token and makes Authenticated Requests (with OAuth 1 Authorization information in the HTTP header), the provider will be invoked to provide the `OAuth1Token` for the Access Token information in the header.

### 18.3.1.2. Client

#### Note

OAuth client support in Jersey is almost identical for OAuth 1 and OAuth 2. As such, this chapter provides useful information even for users that use OAuth 2 client support.

To add support for OAuth into your Jersey client application, add the following dependency to your `pom.xml`:

```
<dependency>
  <groupId>org.glassfish.jersey.security</groupId>
  <artifactId>oauth1-client</artifactId>
  <version>4.0.0</version>
</dependency>
```

As mentioned earlier, there is no need to add a direct dependency to the signature module, it will be transitively included.

OAuth 1 client support initially started as a code migration from Jersey 1.x. During the migration however the API of was significantly revised. The high level difference compared to Jersey 1.x OAuth client API is that the authorization flow is no longer part of a client OAuth filter. Authorization flow is now a standalone utility and can be used without a support for subsequent authenticated requests. The support for authenticated requests stays in the `ClientRequestFilter` but is not part of a public API anymore and is registered by a Jersey OAuth Feature [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html>].

The most important parts of the Jersey client OAuth API and SPI are explained here:

- `OAuth1ClientSupport` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/OAuth1ClientSupport.html>]: The main class which contains builder methods to build features that enable the OAuth 1 support. Start with this class every time you need to add any OAuth 1 support to the Jersey Client (build an Authorization flow or initialize client to perform authenticated requests). The class contains a static method that returns an instance of `OAuth1Builder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/OAuth1Builder.html>] and also the class defines request properties to influence behaviour of the authenticated request support.
- `OAuth1AuthorizationFlow` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/OAuth1AuthorizationFlow.html>]: API that allows to perform the Authorization flow against service provider. Implementation of this interface is a class that is used as a standalone utility and is not part of the JAX-RS client. In other words, this is not a feature that should be registered into the client.

- `AccessToken` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/AccessToken.html>], `ConsumerCredentials` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/ConsumerCredentials.html>]: Interfaces that define Access Token classes and Consumer Credentials. Interfaces contain getters for public keys and secret keys of token and credentials.

An example of how Jersey OAuth 1 client API is used was in the OAuth 1 Twitter Client Example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/oauth-client-twitter>] but for now it's temporally removed from examples. The following code snippets are extracted from the example and explain how to use the Jersey OAuth client API.

Before we start with any interaction with Twitter, we need to register our application on Twitter. See the example `README.TXT` file for the instructions. As a result of the registration, we get the consumer credentials that identify our application. Consumer credentials consist of `consumer key` and `consumer secret`.

As a first step in our code, we need to perform the authorization flow, where the user grants us an access to his/her Twitter client.

### Example 18.7. Build the authorization flow utility

```
1 ConsumerCredentials consumerCredentials = new ConsumerCredentials(  
2     "a846d84e68421b321a32d, "f13aed84190bc");  
3 OAuth1AuthorizationFlow authFlow = OAuth1ClientSupport.builder(consumerCredent  
4     .authorizationFlow(  
5         "http://api.twitter.com/oauth/request_token",  
6         "http://api.twitter.com/oauth/access_token",  
7         "http://api.twitter.com/oauth/authorize")  
8     .build());
```

Here we have built a `OAuth1AuthorizationFlow` utility component representing the OAuth 1 authorization flow, using `OAuth1ClientSupport` and `OAuth1Builder` API. The static `builder` method accepts mandatory parameter with `ConsumerCredentials`. These are credentials earlier issued by Twitter for our application. We have specified the Twitter OAuth endpoints where Request Token, Access Token will be retrieved and Authorization URI to which we will redirect the user in order to grant user's consent. Twitter will present an HTML page on this URI and it will ask the user whether he/she would like us to access his/her account.

Now we can proceed with the OAuth authorization flow.

### Example 18.8. Perform the OAuth Authorization Flow

```
1 String authorizationUri = authFlow.start();  
2 // here we must direct the user to authorization uri to approve  
3 // our application. The result will be verifier code (String).  
4 AccessToken accessToken = authFlow.finish(verifier);
```

In the first line, we start the authorization flow. The method internally makes a request to the `http://api.twitter.com/oauth/request_token` URL and retrieves a Request Token. Details of this request can be found in the OAuth 1 specification. It then constructs a URI to which we must redirect the user. The URI is based on Twitter's authorization URI (`http://api.twitter.com/oauth/authorize`) and contains a Request Token as a query parameter. In the Twitter example, we have a simple console application therefore we print the URL to the console and ask the user to open the URL in a browser to approve the authorization of our application. Then the user gets a verifier and enters it back to

the console. However, if our application would be a web application, we would need to return a redirection response to the user in order to redirect the user automatically to the `authorizationUri`.

Once we have a verifier, we invoke the method `finish()` on our `OAuth1AuthorizationFlow` instance, which internally sends a request to an access token service URI (`http://api.twitter.com/oauth/access_token`) and exchanges the supplied verifier for a new valid Access Token. At this point the authorization flow is finished and we can start using the retrieved `AccessToken` to make authenticated requests. We can now create an instance of an OAuth 1 client Feature [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html>] using `OAuth1ClientSupport` and pass it our `accessToken`. Another way is to use `authFlow` that already contains the information about access token to create the feature instance for us:

### Example 18.9. Authenticated requests

```
1 Feature feature = authFlow.getOAuth1Feature();
2 Client client = ClientBuilder.newBuilder()
3     .register(feature)
4     .build();
```

Once the feature is configured in the JAX-RS Client [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>] (or `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>]), all requests invoked from such `Client` (or `WebTarget`) instance will automatically include an OAuth Authorization HTTP header (that contains also the OAuth signature).

Note that if you already have a valid Access Token (for example stored in the database for each of your users), then you can skip the authorization flow steps and directly create the OAuth Feature configured to use your Access Token.

### Example 18.10. Build feature from Access Token

```
1 AccessToken storedToken = ...;
2 Feature filterFeature = OAuth1ClientSupport.builder(consumerCredentials)
3     .feature()
4     .accessToken(storedToken)
5     .build();
```

Here, the `storedToken` represents an `AccessToken` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/AccessToken.html>] that your client application keeps stored e.g. in a database.

Note that the OAuth feature builder API does not require the access token to be set. The reason for it is that you might want to build a feature which would register the internal Jersey OAuth `ClientRequestFilter` and other related providers but which would not initialize the OAuth providers with a single fixed `AccessToken` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/AccessToken.html>] instance. In such case you would need to specify a token for every single request in the request properties. Key names and API documentation of these properties can be found in `OAuth1ClientSupport`. Using this approach, you can have a single, OAuth enabled instance of a JAX-RS `Client` (or `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>]) and use it to make authenticated requests on behalf of multiple users. Note that you can use the aforementioned request properties even if the feature has been initialized with an `AccessToken` to override the default access token information for particular requests, even though it is probably not a common use case.

The following code shows how to set an access token on a single request using the Jersey OAuth properties.

**Example 18.11. Specifying Access Token on a Request.**

```
1 Response resp =
2     client.target("http://my-serviceprovider.org/rest/foo/bar")
3         .request()
4         .property(OAuth1ClientSupport.OAUTH_PROPERTY_ACCESS_TOKEN, storedToken)
5         .get();
```

`OAuth1AuthorizationFlow` internally uses a `Client` instance to communicate with the OAuth server. For this a new client instance is automatically created by default. You can supply your instance of a `Client` to be used for the authorization flow requests (for performance and/or resource management reasons) using `OAuth1Builder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/OAuth1Builder.html>] methods.

**18.3.1.2.1. Public/Private Keys for RSA-SHA1 signature method**

Follow the steps below in case the outgoing requests sent from client to server have to be signed with RSA-SHA1 signature method instead of the default one (HMAC-SHA1).

**Example 18.12. Creating Public/Private RSA-SHA1 keys**

```
1 $ # Create the private key.
2 $ openssl genrsa -out private.key 2048
3 $ # Convert the key into PKCS8 format.
4 $ openssl pkcs8 -topk8 -in private.key -nocrypt
5 $ # Extract the public key.
6 $ openssl rsa -in private.key -pubout
```

The output of the second command can be used as a consumer secret to sign the outgoing request: `new ConsumerCredentials("consumer-key", CONSUMER_PRIVATE_KEY)`. Public key obtained from the third command can be then used on the service provider to verify the signed data.

For more advanced cases (i.e. other formats of keys) a custom `OAuth1SignatureMethod` should be implemented and used.

**18.3.2. OAuth 2 Support**

At the moment Jersey supports OAuth 2 only on the client side.

**18.3.2.1. Client****Note**

Note: It is suggested to read the section Section 18.3.1.2, “Client” before this section. Support for OAuth on the client is very similar for both OAuth 1 and OAuth 2 and general principles are valid for both OAuth versions as such.

**Note**

OAuth 2 support is in a Beta [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/Beta.html>] state and as such the API is subject to change.

To add support for Jersey OAuth 2 Client API into your application, add the following dependency to your `pom.xml`:

```
<dependency>
  <groupId>org.glassfish.jersey.security</groupId>
  <artifactId>oauth2-client</artifactId>
  <version>4.0.0</version>
</dependency>
```

OAuth 2, in contrast with OAuth 1, is not a strictly defined protocol, rather a framework. OAuth 2 specification defines many extension points and it is up to service providers to implement these details and document these implementations for the service consumers. Additionally, OAuth 2 defines more than one authorization flow. The authorization flow similar to the flow from OAuth 1 is called the *Authorization Code Grant Flow*. This is the flow currently supported by Jersey (Jersey currently does not support other flows). Please refer to the OAuth 2.0 specification [<http://tools.ietf.org/html/rfc6749>] for more details about authorization flows. Another significant change compared to OAuth 1 is that OAuth 2 is not based on signatures and secret keys and therefore for most of the communication SSL needs to be used (i.e. the requests must be made through HTTPS). This means that all OAuth 2 endpoint URIs must use the `https` scheme.

Due to the fact that OAuth 2 does not define a strict protocol, it is not possible to provide a single, universal pre-configured tool interoperable with all providers. Jersey OAuth 2 APIs allows a lot of extensibility via parameters sent in each requests. Jersey currently provides two pre-configured authorization flow providers - for Google and Facebook.

The most important entry points of Jersey client OAuth 2 API and SPI are explained below:

- `OAuth2ClientSupport` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth2/OAuth2ClientSupport.html>]: The main class which contains builder methods to build features that enable the OAuth 2 support. Start with this class every time you need to add any OAuth 2 support to the Jersey Client (build an Authorization flow or initialize client to perform authenticated requests). The class contains also methods to get authorization flow utilities adjusted for Facebook or Google.
- `OAuth2CodeGrantFlow` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth2/OAuth2CodeGrantFlow.html>]: API that allows to perform the authorization flow defined as Authorization Code Grant Flow in the OAuth 2 specification. Implementation of this interface is a class that is used as a standalone utility and is not part of the JAX-RS client. In other words, this is not a feature that should be registered into the client.
- `ClientIdentifier` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth2/ClientIdentifier.html>]: Identifier of the client issued by the Service Provider for the client. Similar to `ConsumerCredentials` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth1/ConsumerCredentials.html>] from OAuth 1 client support.
- `OAuth2Parameters` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth2/OAuth2Parameters.html>]: Defines parameters that are used in requests during the authorization flow. These parameters can be used to override some of the parameters used in different authorization phases.
- `TokenResult` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/oauth2/TokenResult.html>]: Contains result of the authorization flow. One of the result values is the Access Token. It can additionally contain the expiration time of the Access Token and Refresh Token that can be used to get new Access Token.

The principle of performing the authorization flow with Jersey is similar to OAuth 1. Check the OAuth 1 Twitter Client Example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/oauth-client-twitter>]

which utilizes Jersey client support for OAuth 2 to get Google Tasks of the user. The application is a web application that uses redirection to forward the user to the authorization URI.

The following code is an example of how to build and use OAuth 2 authorization flow.

**Example 18.13. Building OAuth 2 Authorization Flow.**

```
1 OAuth2CodeGrantFlow.Builder builder =
2     OAuth2ClientSupport.authorizationCodeGrantFlowBuilder(clientId,
3     "https://example.com/oauth/authorization",
4     "https://example.com/oauth/token");
5 OAuth2CodeGrantFlow flow = builder
6     .property(OAuth2CodeGrantFlow.Phase.AUTHORIZATION, "readOnly", "true")
7     .scope("contact")
8     .build();
9 String authorizationUri = flow.start();
10
11 // Here we must redirect the user to the authorizationUri
12 // and let the user approve an access for our app.
13
14 ...
15
16 // We must handle redirection back to our web resource
17 // and extract code and state from the request
18 final TokenResult result = flow.finish(code, state);
19 System.out.println("Access Token: " + result.get());
```

In the code above we create an `OAuth2CodeGrantFlow` from an authorization URI and an access token URI. We have additionally set a `readOnly` parameter to `true` and assigned the parameter to the authorization phase. This is the way, how you can extend the standard flow with additional service provider-specific parameters. In this case, the `readOnly=true` parameter will be added as a query parameter to the authorization uri returned from the method `flow.start()`. If we would specify `ACCESS_TOKEN_REQUEST` as a phase, then the parameter would have been added to the request when `flow.finish()` is invoked. See javadocs for more information. The parameter `readOnly` is not part of the OAuth 2 specification and is used in the example for demonstration of how to configure the flow for needs of specific service providers (in this case, the `readOnly` param would be described in the service provider's documentation).

Between the calls to `flow.start()` and `flow.finish()`, a user must be redirected to the authorization URI. This means that the code will not be executed in a single method and the finish part will be invoked as a handler of redirect request back to our web from authorization URI.

---

# Chapter 19. WADL Support

## 19.1. WADL introduction

Jersey contains support for Web Application Description Language (WADL) [<https://javaee.github.io/wadl/>]. WADL is a XML description of a deployed RESTful web application. It contains model of the deployed resources, their structure, supported media types, HTTP methods and so on. In a sense, WADL is similar to the WSDL (Web Service Description Language) which describes SOAP web services. WADL is however specifically designed to describe RESTful Web resources.

### Important

Since Jersey 2.5.1 the WADL generated by default is WADL in shorter form without additional extension resources (OPTIONS methods, WADL resource). In order to get full WADL use the query parameter `detail=true`.

Let's start with the simple WADL example. In the example there is a simple `CountryResource` deployed and we request a wadl of this resource. The context root path of the application is `http://localhost:9998`.

### Example 19.1. A simple WADL example - JAX-RS resource definition

```
1 @Path("country/{id}")
2 public static class CountryResource {
3
4     private CountryService countryService;
5
6     public CountryResource() {
7         // init countryService
8     }
9
10    @GET
11    @Produces(MediaType.APPLICATION_XML)
12    public Country getCountry(@PathParam("countryId") int countryId) {
13        return countryService.getCountry(countryId);
14    }
15 }
```

The WADL of a Jersey application that contains the resource above can be requested by a HTTP GET request to `http://localhost:9998/application.wadl`. Jersey will return a response with a WADL content similar to the one in the following example:

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <application xmlns="http://wadl.dev.java.net/2009/02">
3     <doc xmlns:jersey="http://jersey.java.net/" jersey:generatedBy="Jersey: 3.
4     <grammars/>
5     <resources base="http://localhost:9998/">
6         <resource path="country/{id}">
7             <param xmlns:xs="http://www.w3.org/2001/XMLSchema" type="xs:int" s
8             <method name="GET" id="getCountry">
9                 <response>
```



```
10             <representation mediaType="application/xml"/>
11         </response>
12     </method>
13 </resource>
14 </resources>
15 </application>
```

The returned WADL is a XML that contains element `resource` with path `country/{id}`. This resource has one inner `method` element with http method as attribute, name of java method and its produced representation. This description corresponds to defined java resource. Now let's look at more complex example.

The previous WADL does not actually contain all resources exposed in our API. There are other resources that are available and are hidden in the previous WADL. The previous WADL shows only resources that are provided by the user. In the following example, the WADL is generated using query parameter `detail`: `http://localhost:9998/application.wadl?detail`. Note that usage of `http://localhost:9998/application.wadl?detail=true` is also valid. This will produce the WADL with all resource available in the application:

### Example 19.2. A simple WADL example - WADL content

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <application xmlns="http://wadl.dev.java.net/2009/02">
3     <doc xmlns:jersey="http://jersey.java.net/" jersey:generatedBy="Jersey: 3.
4     <doc xmlns:jersey="http://jersey.java.net/" jersey:hint="To get simplified
5     <grammars/>
6     <resources base="http://localhost:9998/">
7         <resource path="country/{id}">
8             <param xmlns:xs="http://www.w3.org/2001/XMLSchema" type="xs:int" s
9             <method name="GET" id="getCountry">
10                 <response>
11                     <representation mediaType="application/xml"/>
12                 </response>
13             </method>
14             <method name="OPTIONS" id="apply">
15                 <request>
16                     <representation mediaType="*/"/>
17                 </request>
18                 <response>
19                     <representation mediaType="application/vnd.sun.wadl+xml"/>
20                 </response>
21                 <jersey:extended xmlns:jersey="http://jersey.java.net/">true</
22             </method>
23             <method name="OPTIONS" id="apply">
24                 <request>
25                     <representation mediaType="*/"/>
26                 </request>
27                 <response>
28                     <representation mediaType="text/plain"/>
29                 </response>
30                 <jersey:extended xmlns:jersey="http://jersey.java.net/">true</
31             </method>
32             <method name="OPTIONS" id="apply">
33                 <request>
```

```
34         <representation mediaType="*/*/">
35     </request>
36     <response>
37         <representation mediaType="*/*/">
38     </response>
39     <jersey:extended xmlns:jersey="http://jersey.java.net/">true</
40 </method>
41 </resource>
42 <resource path="application.wadl">
43     <method name="GET" id="getWadl">
44         <response>
45             <representation mediaType="application/vnd.sun.wadl+xml"/>
46             <representation mediaType="application/xml"/>
47         </response>
48         <jersey:extended xmlns:jersey="http://jersey.java.net/">true</
49 </method>
50 <method name="OPTIONS" id="apply">
51     <request>
52         <representation mediaType="*/*/">
53     </request>
54     <response>
55         <representation mediaType="text/plain"/>
56     </response>
57     <jersey:extended xmlns:jersey="http://jersey.java.net/">true</
58 </method>
59 <method name="OPTIONS" id="apply">
60     <request>
61         <representation mediaType="*/*/">
62     </request>
63     <response>
64         <representation mediaType="*/*/">
65     </response>
66     <jersey:extended xmlns:jersey="http://jersey.java.net/">true</
67 </method>
68 <resource path="{path}">
69     <param xmlns:xs="http://www.w3.org/2001/XMLSchema" type="xs:st
70     <method name="GET" id="geExternalGrammar">
71         <response>
72             <representation mediaType="application/xml"/>
73         </response>
74         <jersey:extended xmlns:jersey="http://jersey.java.net/">tr
75     </method>
76     <method name="OPTIONS" id="apply">
77         <request>
78             <representation mediaType="*/*/">
79         </request>
80         <response>
81             <representation mediaType="text/plain"/>
82         </response>
83         <jersey:extended xmlns:jersey="http://jersey.java.net/">tr
84     </method>
85     <method name="OPTIONS" id="apply">
86         <request>
87             <representation mediaType="*/*/">
```

```
88             </request>
89             <response>
90                 <representation mediaType="*/"/>
91             </response>
92             <jersey:extended xmlns:jersey="http://jersey.java.net/">tr
93         </method>
94         <jersey:extended xmlns:jersey="http://jersey.java.net/">true</
95     </resource>
96     <jersey:extended xmlns:jersey="http://jersey.java.net/">true</jers
97 </resource>
98 </resources>
99 </application>
100
```

In the example above the returned application WADL is shown in full. WADL schema is defined by the WADL specification, so let's look at it in more details. The root WADL document element is the `application`. It contains global information about the deployed JAX-RS application. Under this element there is a nested element `resources` which contains zero or more `resource` elements. Each `resource` element describes a single deployed resource. In our example, there are only two root resources - `"country/{id}"` and `"application.wadl"`. The `"application.wadl"` resource is the resource that was just requested in order to receive the application WADL document. Even though WADL support is an additional feature in Jersey it is still a resource deployed in the resource model and therefore it is itself present in the returned WADL document. The first resource element with the `path="country/{id}"` is the element that describes our custom deployed resource. This resource contains a GET method and three OPTIONS methods. The GET method is our `getCountry()` method defined in the sample. There is a method name in the `id` attribute and `@Produces [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html]` is described in the `response/representation` WADL element. OPTIONS methods are the methods that are automatically added by Jersey to each resource. There is an OPTIONS method returning `"text/plain"` media type, that will return a response with a string entity containing the list of methods deployed on this resource (this means that instead of WADL you can use this OPTIONS method to get similar information in a textual representation). Another OPTIONS method returning `*/*` will return a response with no entity and `ALLOW` header that will contain list of methods as a String. The last OPTIONS method producing `"application/vnd.sun.wadl+xml"` returns a WADL description of the resource `"country/{id}"`. As you can see, all OPTIONS methods return information about the resource to which the HTTP OPTIONS request is made.

Second resource with a path `"application.wadl"` has, again, similar OPTIONS methods and one GET method which return this WADL. There is also a sub-resource with a path defined by path param `{path}`. This means that you can request a resource on the URI `http://localhost:9998/application.wadl/something`. This is used only to return an external grammar if there is any attached. Such an external grammar can be for example an XSD schema of the response entity which if the response entity is a JAXB bean. An external grammar support via Jersey *extended WADL support* is described in sections below.

All resource that were added in this second example into the WADL contains element `extended`. This means that this resource is not a part of a core RESTful API and is rather a helper resource. If you need to mark any your own resource are extended, annotate is with `@ExtendedResource [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/model/ExtendedResource.html]`. Note that there might be methods visible in the default simple WADL even the user has not added them. This is for example the case of MVC added methods which were added by `ModelProcessor [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/model/ModelProcessor.html]` but are still intended to be used by the client to achieve their primary use case of getting formatted data.

Let's now send a HTTP OPTIONS request to `"country/{id}"` resource using the the `curl` command:

```
curl -X OPTIONS -H "Allow: application/vnd.sun.wadl+xml" \
-v http://localhost:9998/country/15
```

We should see a WADL returned similar to this one:

### Example 19.3. OPTIONS method returning WADL

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <application xmlns="http://wadl.dev.java.net/2009/02">
3   <doc xmlns:jersey="http://jersey.java.net/"
4     jersey:generatedBy="Jersey: 3.0.0 ${buildNumber}"/>
5   <grammars/>
6   <resources base="http://localhost:9998/">
7     <resource path="country/15">
8       <method name="GET" id="getCountry">
9         <response>
10          <representation mediaType="application/xml"/>
11        </response>
12      </method>
13      <method name="OPTIONS" id="apply">
14        <request>
15          <representation mediaType="*/"/>
16        </request>
17        <response>
18          <representation mediaType="application/vnd.sun.wadl+xml"/>
19        </response>
20      </method>
21      <method name="OPTIONS" id="apply">
22        <request>
23          <representation mediaType="*/"/>
24        </request>
25        <response>
26          <representation mediaType="text/plain"/>
27        </response>
28      </method>
29      <method name="OPTIONS" id="apply">
30        <request>
31          <representation mediaType="*/"/>
32        </request>
33        <response>
34          <representation mediaType="*/"/>
35        </response>
36      </method>
37    </resource>
38  </resources>
39 </application>
```

The returned WADL document has the standard WADL structure that we saw in the WADL document returned for the whole Jersey application earlier. The main difference here is that the only `resource` is the resource to which the `OPTIONS` HTTP request was sent. The resource has now path `"country/15"` and not `"country/{id}"` as the path parameter `{id}` was already specified in the request to this concrete resource.

Another, a more complex WADL example is shown in the next example.

**Example 19.4. More complex WADL example - JAX-RS resource definition**

```
1 @Path("customer/{id}")
2 public static class CustomerResource {
3     private CustomerService customerService;
4
5     @GET
6     public Customer get(@PathParam("id") int id) {
7         return customerService.getCustomerById(id);
8     }
9
10    @PUT
11    public Customer put(Customer customer) {
12        return customerService.updateCustomer(customer);
13    }
14
15    @Path("address")
16    public CustomerAddressSubResource getCustomerAddress(@PathParam("id") int id) {
17        return new CustomerAddressSubResource(id);
18    }
19
20    @Path("additional-info")
21    public Object getAdditionalInfoSubResource(@PathParam("id") int id) {
22        return new CustomerAddressSubResource(id);
23    }
24
25 }
26
27
28 public static class CustomerAddressSubResource {
29     private final int customerId;
30     private CustomerService customerService;
31
32     public CustomerAddressSubResource(int customerId) {
33         this.customerId = customerId;
34         this.customerService = null; // init customer service here
35     }
36
37     @GET
38     public String getAddress() {
39         return customerService.getAddressForCustomer(customerId);
40     }
41
42     @PUT
43     public void updateAddress(String address) {
44         customerService.updateAddressForCustomer(customerId, address);
45     }
46
47     @GET
48     @Path("sub")
49     public String getDeliveryAddress() {
50         return customerService.getDeliveryAddressForCustomer(customerId);
51     }
52 }
```

The GET request to `http://localhost:9998/application.wadl` will return the following WADL document:

**Example 19.5. More complex WADL example - WADL content**

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <application xmlns="http://wadl.dev.java.net/2009/02">
3   <doc xmlns:jersey="http://jersey.java.net/"
4     jersey:generatedBy="Jersey: 3.0.0 ${buildNumber}"/>
5   <grammars/>
6   <resources base="http://localhost:9998/">
7     <resource path="customer/{id}">
8       <param xmlns:xs="http://www.w3.org/2001/XMLSchema"
9         type="xs:int" style="template" name="id"/>
10      <method name="GET" id="get">
11        <response/>
12      </method>
13      <method name="PUT" id="put">
14        <response/>
15      </method>
16      <method name="OPTIONS" id="apply">
17        <request>
18          <representation mediaType="*/"/>
19        </request>
20        <response>
21          <representation mediaType="application/vnd.sun.wadl+xml"/>
22        </response>
23      </method>
24      <method name="OPTIONS" id="apply">
25        <request>
26          <representation mediaType="*/"/>
27        </request>
28        <response>
29          <representation mediaType="text/plain"/>
30        </response>
31      </method>
32      <method name="OPTIONS" id="apply">
33        <request>
34          <representation mediaType="*/"/>
35        </request>
36        <response>
37          <representation mediaType="*/"/>
38        </response>
39      </method>
40      <resource path="additional-info">
41        <param xmlns:xs="http://www.w3.org/2001/XMLSchema"
42          type="xs:int" style="template" name="id"/>
43      </resource>
44      <resource path="address">
45        <param xmlns:xs="http://www.w3.org/2001/XMLSchema"
46          type="xs:int" style="template" name="id"/>
47        <method name="GET" id="getAddress">
48          <response/>
49        </method>
```

```
50         <method name="PUT" id="updateAddress"/>
51         <resource path="sub">
52             <method name="GET" id="getDeliveryAddress">
53                 <response/>
54             </method>
55         </resource>
56     </resource>
57 </resource>
58 <resource path="application.wadl">
59     <method name="GET" id="getWadl">
60         <response>
61             <representation mediaType="application/vnd.sun.wadl+xml"/>
62             <representation mediaType="application/xml"/>
63         </response>
64     </method>
65     <method name="OPTIONS" id="apply">
66         <request>
67             <representation mediaType="*/"/>
68         </request>
69         <response>
70             <representation mediaType="text/plain"/>
71         </response>
72     </method>
73     <method name="OPTIONS" id="apply">
74         <request>
75             <representation mediaType="*/"/>
76         </request>
77         <response>
78             <representation mediaType="*/"/>
79         </response>
80     </method>
81     <resource path="{path}">
82         <param xmlns:xs="http://www.w3.org/2001/XMLSchema"
83             type="xs:string" style="template" name="path"/>
84         <method name="GET" id="geExternalGrammar">
85             <response>
86                 <representation mediaType="application/xml"/>
87             </response>
88         </method>
89         <method name="OPTIONS" id="apply">
90             <request>
91                 <representation mediaType="*/"/>
92             </request>
93             <response>
94                 <representation mediaType="text/plain"/>
95             </response>
96         </method>
97         <method name="OPTIONS" id="apply">
98             <request>
99                 <representation mediaType="*/"/>
100             </request>
101             <response>
102                 <representation mediaType="*/"/>
103             </response>
```

```
104             </method>
105         </resource>
106     </resource>
107 </resources>
108 </application>
```

The `resource` with `path="customer/{id}"` is similar to the `country` resource from the previous example. There is a path parameter which identifies the customer by `id`. The resource contains 2 user-declared methods and again auto-generated `OPTIONS` methods added by Jersey. The resource declares 2 sub-resource locators which are represented in the returned WADL document as nested `resource` elements. Note that the sub-resource locator `getCustomerAddress()` returns a type `CustomerAddressSubResource` in the method declaration and also in the WADL there is a `resource` element for such a sub resource with full internal description. The second method `getAdditionalInfoSubResource()` returns only an `Object` in the method declaration. While this is correct from the JAX-RS perspective as the real returned type can be computed from a request information, it creates a problem for WADL generator because WADL is generated based on the static configuration of the JAX-RS application resources. The WADL generator does not know what type would be actually returned to a request at run time. That is the reason why the nested `resource` element with `path="additional-info"` does not contain any information about the supported resource representations.

The `CustomerAddressSubResource` sub-resource described in the nested element `<resource path="address">` does not contain an `OPTIONS` method. While these methods are in fact generated by Jersey for the sub-resource, Jersey WADL generator does not currently support adding these methods to the sub-resource description. This should be addressed in the near future. Still, there are two user-defined resource methods handling HTTP GET and PUT requests. The sub-resource method `getDeliveryAddress()` is represented as a separate nested resource with `path="sub"`. Should there be more sub-resource methods defined with `path="sub"`, then all these method descriptions would be placed into the same `resource` element. In other words, sub-resource methods are grouped in WADL as sub-resources based on their `path` value.

## 19.2. Configuration

WADL generation is enabled in Jersey by default. This means that `OPTIONS` methods are added by default to each resource and an auto-generated `/application.wadl` resource is deployed too. To override this default behavior and disable WADL generation in Jersey, setup the configuration property in your application:

```
jersey.config.server.wadl.disableWadl=true
```

This property can be setup in a `web.xml` if the Jersey application is deployed in the servlet with `web.xml` or the property can be returned from the `Application` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>]. `getProperties()`. See Deployment chapter for more information on setting the application configuration properties in various deployments.

WADL support in Jersey is implemented via `ModelProcessor` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/model/ModelProcessor.html>] extension. This implementation enhances the application resource model by adding the WADL providing resources. WADL `ModelProcessor` priority value is high (i.e. the priority is low) as it should be executed as one of the last model processors. Therefore, any `ModelProcessor` executed before will not see WADL extensions in the resource model. WADL handling resource model extensions (resources and `OPTIONS` resource methods) are not added to the application resource model if there is already a matching resource or a resource method detected in the model. In other words, if you define for example your own `OPTIONS` method that would produce `"application.wadl"` response content, this method



will not be overridden by WADL model processor. See Resource builder chapter for more information on `ModelProcessor` extension mechanism.

## 19.3. Extended WADL support

**Please note that the API of extended WADL support is going to be changed in one of the future releases of Jersey 3.x (see below).**

Jersey supports extension of WADL generation called *extended WADL*. Using the extended WADL support you can enhance the generated WADL document with additional information, such as resource method javadoc-based documentation of your REST APIs, adding general documentation, adding external grammar support, or adding any custom WADL extension information.

**Again, note that the extended WADL in Jersey 3.x is NOT the intended final version and API is going to be changed.** The existing set of features and functionality will be preserved but the APIs will be significantly re-designed to support additional use cases. This impacts mainly the APIs of `WadlGenerator` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/wadl/WadlGenerator.html>], `WadlGeneratorConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/wadl/config/WadlGeneratorConfig.html>] as well as any related classes. The API changes may impact your code if you are using a custom `WadlGenerator` or plan to implement one.

---

# Chapter 20. Bean Validation Support

Validation is a process of verifying that some data obeys one or more pre-defined constraints. This chapter describes support for Bean Validation [<http://beanvalidation.org/>] in Jersey in terms of the needed dependencies, configuration, registration and usage. For more detailed description on how JAX-RS provides native support for validating resource classes based on the Bean Validation refer to the chapter in the JAX-RS spec [<http://jcp.org/en/jsr/detail?id=339>].

## 20.1. Bean Validation Dependencies

Bean Validation support in Jersey is provided as an extension module and needs to be mentioned explicitly in your `pom.xml` file (in case of using Maven):

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-bean-validation</artifactId>
  <version>4.0.0</version>
</dependency>
```

### Note

If you're not using Maven make sure to have also all the transitive dependencies (see `jersey-bean-validation` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-bean-validation/dependencies.html>]) on the classpath.

This module depends directly on Hibernate Validator [<http://hibernate.org/validator/>] which provides a most commonly used implementation of the Bean Validation API spec.

If you want to use a different implementation of the Bean Validation API, use standard Maven mechanisms to exclude Hibernate Validator from the modules dependencies and add a dependency of your own.

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-bean-validation</artifactId>
  <version>4.0.0</version>
  <exclusions>
    <exclusion>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-validator</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

## 20.2. Enabling Bean Validation in Jersey

As stated in Section 4.3, “Auto-Discoverable Features”, Jersey Bean Validation is one of the modules where you don't need to explicitly register its Features (`ValidationFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/validation/ValidationFeature.html>]) on the server as its features are automatically discovered and registered when you add the `jersey-bean-validation` module to your classpath. There are three Jersey specific properties that could disable automatic discovery and registration of Jersey Bean Validation integration module:

- `CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE\\_AUTO\\_DISCOVERY\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE)]
- `ServerProperties.FEATURE_AUTO_DISCOVERY_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#FEATURE\\_AUTO\\_DISCOVERY\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE)]
- `ServerProperties.BV_FEATURE_DISABLE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV\\_FEATURE\\_DISABLE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV_FEATURE_DISABLE)]

## Note

Jersey does not support Bean Validation on the client at the moment.

## 20.3. Configuring Bean Validation Support

Configuration of Bean Validation support in Jersey is twofold - there are few specific properties that affects Jersey behaviour (e.g. sending validation error entities to the client) and then there is `ValidationConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/validation/ValidationConfig.html>] class that configures `Validator` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/validator>] used for validating resources in JAX-RS application.

To configure Jersey specific behaviour you can use the following properties:

`ServerProperties.BV_DISABLE_VALIDATE_ON_EXECUTABLE_OVERRIDE_CHECK` More on this is described in Section 20.5, “`@ValidateOnExecution`”.  
[[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV\\_DISABLE\\_VALIDATE\\_ON\\_EXECUTABLE\\_OVERRIDE\\_CHECK](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV_DISABLE_VALIDATE_ON_EXECUTABLE_OVERRIDE_CHECK)]

`ServerProperties.BV_SEND_ERROR_IN_RESPONSE` Enable sending validation errors in response entity to the client.  
[[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV\\_SEND\\_ERROR\\_IN\\_RESPONSE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV_SEND_ERROR_IN_RESPONSE)] More on this in Section 20.7.1, “`ValidationError`”.

### Example 20.1. Configuring Jersey specific properties for Bean Validation.

```
1 new ResourceConfig()
2     // Now you can expect validation errors to be sent to the client.
3     .property(ServerProperties.BV_SEND_ERROR_IN_RESPONSE, true)
4     // @ValidateOnExecution annotations on subclasses won't cause errors.
5     .property(ServerProperties.BV_DISABLE_VALIDATE_ON_EXECUTABLE_OVERRIDE_CHECK, true)
6     // Further configuration of ResourceConfig.
7     .register( ... );
```

Customization of the `Validator` used in validation of resource classes/methods can be done using `ValidationConfig` class and exposing it via `ContextResolver<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ContextResolver.html>] mechanism as shown in Example 20.2, “Using `ValidationConfig` to configure `Validator`.”. You can set custom instances for the following interfaces from the Bean Validation API:

- `MessageInterpolator` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/messageinterpolator>] - interpolates a given constraint violation message.
- `TraversableResolver` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/traversableresolver>] - determines if a property can be accessed by the Bean Validation provider.
- `ConstraintValidatorFactory` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/constraintvalidatorfactory>] - instantiates a `ConstraintValidator` instance based off its class. Note that by setting a custom `ConstraintValidatorFactory` you may loose injection of available resources/providers at the moment. See Section 20.6, “Injecting” how to handle this.
- `ParameterNameProvider` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/parameternameprovider>] - provides names for method and constructor parameters.

### Example 20.2. Using `ValidationConfig` to configure `Validator`.

```
/**
 * Custom configuration of validation. This configuration defines custom:
 * <ul>
 * <li>ConstraintValidationFactory - so that validators are able to inject Jer
 * <li>ParameterNameProvider - if method input parameters are invalid, this cl
 * instead of the default ones ({@code arg0, arg1, ..})</li>
 * </ul>
 */
public class ValidationConfigurationContextResolver implements ContextResolver<Val

    @Context
    private ResourceContext resourceContext;

    @Override
    public ValidationConfig getContext(final Class<?> type) {
        final ValidationConfig config = new ValidationConfig();
        config.setConstraintValidatorFactory(resourceContext.getResource(Injecting
        config.setParameterNameProvider(new CustomParameterNameProvider());
        return config;
    }

/**
 * See ContactCardTest#testAddInvalidContact.
 */
private class CustomParameterNameProvider implements ParameterNameProvider {

    private final ParameterNameProvider nameProvider;

    public CustomParameterNameProvider() {
        nameProvider = Validation.byDefaultProvider().configure().getDefaultPa
    }

    @Override
    public List<String> getParameterNames(final Constructor<?> constructor) {
        return nameProvider.getParameterNames(constructor);
    }

    @Override
```

```
        public List<String> getParameterNames(final Method method) {  
            // See ContactCardTest#testAddInvalidContact.  
            if ("addContact".equals(method.getName())) {  
                return Arrays.asList("contact");  
            }  
            return nameProvider.getParameterNames(method);  
        }  
    }  
}
```

Register this class in your app:

```
final Application application = new ResourceConfig()  
    // Validation.  
    .register(ValidationConfigurationContextResolver.class)  
    // Further configuration.  
    .register( ... );
```

### Note

This code snippet is based on CustomConfigValidationTest (from Jersey's source codes)

## 20.4. Validating JAX-RS resources and methods

JAX-RS specification states that constraint annotations are allowed in the same locations as the following annotations: `@MatrixParam`, `@QueryParam`, `@PathParam`, `@CookieParam`, `@HeaderParam` and `@Context`, *except* in class constructors and property setters. Specifically, they are allowed in resource method parameters, fields and property getters as well as resource classes, entity parameters and resource methods (return values). Jersey provides support for validation (see following sections) annotated input parameters and return value of the invoked resource method as well as validation of resource class (class constraints, field constraints) where this resource method is placed. Jersey does not support, and doesn't validate, constraints placed on constructors and Bean Validation groups (only `Default` group is supported at the moment).

### 20.4.1. Constraint Annotations

The JAX-RS Server API provides support for extracting request values and mapping them into Java fields, properties and parameters using annotations such as `@HeaderParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/HeaderParam.html>], `@QueryParam` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/QueryParam.html>], etc. It also supports mapping of the request entity bodies into Java objects via non-annotated parameters (i.e., parameters without any JAX-RS annotations).

The Bean Validation specification supports the use of *constraint annotations* as a way of declarative validating beans, method parameters and method returned values. For example, consider resource class from Example 20.3, “Constraint annotations on input parameters” augmented with constraint annotations.

#### Example 20.3. Constraint annotations on input parameters

```
@Path("/")  
class MyResourceClass {
```

```
@POST
@Consumes("application/x-www-form-urlencoded")
public void registerUser(
    @NotNull @FormParam("firstName") String firstName,
    @NotNull @FormParam("lastName") String lastName,
    @Email @FormParam("email") String email) {
    ...
}
```

The annotations `@NotNull` and `@Email` impose additional constraints on the form parameters `firstName`, `lastName` and `email`. The `@NotNull` constraint is built-in to the Bean Validation API; the `@Email` constraint is assumed to be user defined in the example above. These constraint annotations are not restricted to method parameters, they can be used in any location in which JAX-RS binding annotations are allowed with the exception of constructors and property setters.

Rather than using method parameters, the `MyResourceClass` shown above could have been written as in Example 20.4, “Constraint annotations on fields”.

#### Example 20.4. Constraint annotations on fields

```
@Path("/")
class MyResourceClass {

    @NotNull
    @FormParam("firstName")
    private String firstName;

    @NotNull
    @FormParam("lastName")
    private String lastName;

    private String email;

    @FormParam("email")
    public void setEmail(String email) {
        this.email = email;
    }

    @Email
    public String getEmail() {
        return email;
    }

    ...
}
```

Note that in this version, `firstName` and `lastName` are fields initialized via injection and `email` is a resource class property. Constraint annotations on properties are specified in their corresponding getters.

Constraint annotations are also allowed on resource classes. In addition to annotating fields and properties, an annotation can be defined for the entire class. Let us assume that `@NonEmptyNames` validates that one of the two `name` fields in `MyResourceClass` is provided. Using such an annotation, the example above can be extended to look like Example 20.5, “Constraint annotations on class”

**Example 20.5. Constraint annotations on class**

```
@Path("/")
@NonEmptyNames
class MyResourceClass {

    @NotNull
    @FormParam("firstName")
    private String firstName;

    @NotNull
    @FormParam("lastName")
    private String lastName;

    private String email;

    ...
}
```

Constraint annotations on resource classes are useful for defining cross-field and cross-property constraints.

## 20.4.2. Annotation constraints and Validators

Annotation constraints and validators are defined in accordance with the Bean Validation specification. The `@Email` annotation used in Example 20.4, “Constraint annotations on fields” is defined using the Bean Validation `@Constraint` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/constraint>] meta-annotation, see Example 20.6, “Definition of a constraint annotation”.

**Example 20.6. Definition of a constraint annotation**

```
@Target({ METHOD, FIELD, PARAMETER })
@Retention(RUNTIME)
@Constraint(validatedBy = EmailValidator.class)
public @interface Email {

    String message() default "{com.example.validation.constraints.email}";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}
```

The `@Constraint` annotation must include a reference to the validator class that will be used to validate decorated values. The `EmailValidator` class must implement `ConstraintValidator<Email, T>` where `T` is the type of values being validated, as described in Example 20.7, “Validator implementation.”.

**Example 20.7. Validator implementation.**

```
public class EmailValidator implements ConstraintValidator<Email, String> {

    public void initialize(Email email) {
        ...
    }
}
```

```
    }

    public boolean isValid(String value, ConstraintValidatorContext context) {
        ...
    }
}
```

Thus, `EmailValidator` applies to values annotated with `@Email` that are of type `String`. Validators for other Java types can be defined for the same constraint annotation.

## 20.4.3. Entity Validation

Request entity bodies can be mapped to resource method parameters. There are two ways in which these entities can be validated. If the request entity is mapped to a Java bean whose class is decorated with Bean Validation annotations, then validation can be enabled using `@Valid` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/valid>] as in Example 20.8, “Entity validation”.

### Example 20.8. Entity validation

```
@StandardUser
class User {

    @NotNull
    private String firstName;

    ...
}

@Path("/")
class MyResourceClass {

    @POST
    @Consumes("application/xml")
    public void registerUser(@Valid User user) {
        ...
    }
}
```

In this case, the validator associated with `@StandardUser` (as well as those for non-class level constraints like `@NotNull`) will be called to verify the request entity mapped to `user`.

Alternatively, a new annotation can be defined and used directly on the resource method parameter (Example 20.9, “Entity validation 2”).

### Example 20.9. Entity validation 2

```
@Path("/")
class MyResourceClass {

    @POST
    @Consumes("application/xml")
    public void registerUser(@PremiumUser User user) {
        ...
    }
}
```



```
    }  
}
```

In the example above, `@PremiumUser` rather than `@StandardUser` will be used to validate the request entity. These two ways in which validation of entities can be triggered can also be combined by including `@Valid` in the list of constraints. The presence of `@Valid` will trigger validation of *all* the constraint annotations decorating a Java bean class.

Response entity bodies returned from resource methods can be validated in a similar manner by annotating the resource method itself. To exemplify, assuming both `@StandardUser` and `@PremiumUser` are required to be checked before returning a user, the `getUser` method can be annotated as shown in Example 20.10, “Response entity validation”.

#### Example 20.10. Response entity validation

```
@Path("/")  
class MyResourceClass {  
  
    @GET  
    @Path("{id}")  
    @Produces("application/xml")  
    @Valid @PremiumUser  
    public User getUser(@PathParam("id") String id) {  
        User u = findUser(id);  
        return u;  
    }  
  
    ...  
}
```

Note that `@PremiumUser` is explicitly listed and `@StandardUser` is triggered by the presence of the `@Valid` annotation - see definition of `User` class earlier in this section.

## 20.4.4. Annotation Inheritance

The rules for inheritance of constraint annotation are defined in Bean Validation specification. It is worth noting that these rules are incompatible with those defined by JAX-RS. Generally speaking, constraint annotations in Bean Validation are cumulative (can be strengthen) across a given type hierarchy while JAX-RS annotations are inherited or, overridden and ignored.

For Bean Validation annotations Jersey follows the constraint annotation rules defined in the Bean Validation specification.

## 20.5. @ValidateOnExecution

According to Bean Validation specification, validation is enabled by default only for the so called *constrained* methods. Getter methods as defined by the Java Beans specification are not constrained methods, so they will not be validated by default. The special annotation `@ValidateOnExecution` can be used to selectively enable and disable validation. For example, you can enable validation on method `getEmail` shown in Example 20.11, “Validate getter on execution”.

#### Example 20.11. Validate getter on execution

```
@Path("/")
```

```
class MyResourceClass {  
  
    @Email  
    @ValidateOnExecution  
    public String getEmail() {  
        return email;  
    }  
  
    ...  
}
```

The default value for the `type` attribute of `@ValidateOnExecution` is `IMPLICIT` which results in method `getEmail` being validated.

### Note

According to Bean Validation specification `@ValidateOnExecution` cannot be overridden once is declared on a method (i.e. in subclass/sub-interface) and in this situations a `ValidationException` should be raised. This default behaviour can be suppressed by setting `ServerProperties.BV_DISABLE_VALIDATE_ON_EXECUTABLE_OVERRIDE_CHECK` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV\\_DISABLE\\_VALIDATE\\_ON\\_EXECUTABLE\\_OVERRIDE\\_CHECK](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV_DISABLE_VALIDATE_ON_EXECUTABLE_OVERRIDE_CHECK)] property (Jersey specific) to `true`.

## 20.6. Injecting

Jersey allows you to inject registered resources/providers into your `ConstraintValidator` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/constraintvalidator>] implementation and you can inject `Configuration` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/configuration>], `ValidatorFactory` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/validatorfactory>] and `Validator` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/validator>] as required by Bean Validation spec.

### Note

Injected `Configuration` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/configuration>], `ValidatorFactory` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/validatorfactory>] and `Validator` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/validator>] do not inherit configuration provided by `ValidationConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/validation/ValidationConfig.html>] and need to be configured manually.

Injection of JAX-RS components into `ConstraintValidators` is supported via a custom `ConstraintValidatorFactory` provided by Jersey. An example is shown in Example 20.12, “Injecting `UriInfo` into a `ConstraintValidator`”.

### Example 20.12. Injecting `UriInfo` into a `ConstraintValidator`

```
public class EmailValidator implements ConstraintValidator<Email, String> {  
  
    @Context
```

```
private UriInfo uriInfo;

public void initialize(Email email) {
    ...
}

public boolean isValid(String value, ConstraintValidatorContext context) {
    // Use UriInfo.
    ...
}
}
```

Using a custom `ConstraintValidatorFactory` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/constraintvalidatorfactory>] of your own disables registration of the one provided by Jersey and injection support for resources/providers (if needed) has to be provided by this new implementation. Example 20.13, “Support for injecting Jersey's resources/providers via `ConstraintValidatorFactory`.” shows how this can be achieved.

### **Example 20.13. Support for injecting Jersey's resources/providers via `ConstraintValidatorFactory`.**

```
public class InjectingConstraintValidatorFactory implements ConstraintValidatorFac

    @Context
    private ResourceContext resourceContext;

    @Override
    public <T extends ConstraintValidator<?, ?>> T getInstance(final Class<T> key)
        return resourceContext.getResource(key);
    }

    @Override
    public void releaseInstance(final ConstraintValidator<?, ?> instance) {
        // NOOP
    }
}
```

#### **Note**

This behaviour may likely change in one of the next version of Jersey to remove the need of manually providing support for injecting resources/providers from Jersey in your own `ConstraintValidatorFactory` implementation code.

## **20.7. Error Reporting**

Bean Validation specification defines a small hierarchy of exceptions (they all inherit from `ValidationException` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/validationexception>]) that could be thrown during initialization of validation engine or (for our case more importantly) during validation of input/output values (`ConstraintViolationException` [<https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/constraintviolationexception>]). If a thrown exception is a subclass of `ValidationException` *except* `ConstraintViolationException` then this exception is

mapped to a HTTP response with status code 500 (Internal Server Error). On the other hand, when a `ConstraintViolationException` is throw two different status code would be returned:

- 500 (Internal Server Error)

If the exception was thrown while validating a method return type.

- 400 (Bad Request)

Otherwise.

## 20.7.1. ValidationError

By default, (during mapping `ConstraintViolationExceptions`) Jersey doesn't return any entities that would include validation errors to the client. This default behaviour could be changed by enabling `ServerProperties.BV_SEND_ERROR_IN_RESPONSE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV\\_SEND\\_ERROR\\_IN\\_RESPONSE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#BV_SEND_ERROR_IN_RESPONSE)] property in your application (Example 20.1, “Configuring Jersey specific properties for Bean Validation.”). When this property is enabled then our custom `ExceptionHandler` (that is handling `ValidationExceptions`) would transform `ConstraintViolationException(s)` into `ValidationError` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/validation/ValidationError.html>](s) and set this object (collection) as the new response entity which Jersey is able to sent to the client. Four `MediaTypes` are currently supported when sending `ValidationErrors` to the client:

- `text/plain`
- `text/html`
- `application/xml`
- `application/json`

### Note

Note: You need to register one of the JSON (JAXB) providers (e.g. MOXy) to marshal validation errors to JSON.

Let's take a look at `ValidationError` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/validation/ValidationError.html>] class to see which properties are send to the client:

```
@XmlRootElement
public final class ValidationError {

    private String message;

    private String messageTemplate;

    private String path;

    private String invalidValue;
```

```
    ...
}
```

The `message` property is the interpolated error message, `messageTemplate` represents a non-interpolated error message (or key from your constraint definition e.g. `{jakarta.validation.constraints.NotNull.message}`), `path` contains information about the path in the validated object graph to the property holding invalid value and `invalidValue` is the string representation of the invalid value itself.

Here are few examples of `ValidationError` messages sent to client:

#### Example 20.14. ValidationError to text/plain

```
HTTP/1.1 500 Internal Server Error
Content-Length: 114
Content-Type: text/plain
Vary: Accept
Server: Jetty(11.0.0.beta3)
```

Contact with given ID does not exist. (path = ContactCardResource.getContact.<return value>)

#### Example 20.15. ValidationError to text/html

```
HTTP/1.1 500 Internal Server Error
Content-Length: ...
Content-Type: text/plain
Vary: Accept
Server: Jetty(11.0.0.beta3)
```

```
<div class="validation-errors">
  <div class="validation-error">
    <span class="message">Contact with given ID does not exist.</span>
    (
      <span class="path">
        <strong>path</strong>
        = ContactCardResource.getContact.<return value>
      </span>
      ,
      <span class="invalid-value">
        <strong>invalidValue</strong>
        = null
      </span>
    )
  </div>
</div>
```

#### Example 20.16. ValidationError to application/xml

```
HTTP/1.1 500 Internal Server Error
Content-Length: ...
Content-Type: text/plain
Vary: Accept
Server: Jetty(11.0.0.beta3)
```

```
<?xml version="1.0" encoding="UTF-8"?>
<validationErrors>
  <validationError>
    <message>Contact with given ID does not exist.</message>
    <messageTemplate>{contact.does.not.exist}</messageTemplate>
    <path>ContactCardResource.getContact.&lt;return value&gt;</path>
  </validationError>
</validationErrors>
```

### **Example 20.17. ValidationError to application/json**

HTTP/1.1 500 Internal Server Error  
Content-Length: 174  
Content-Type: application/json  
Vary: Accept  
Server: Jetty(11.0.0.beta3)

```
[ {
  "message" : "Contact with given ID does not exist.",
  "messageTemplate" : "{contact.does.not.exist}",
  "path" : "ContactCardResource.getContact.<return value>"
} ]
```

## **20.8. Example**

Examples are being processed to comply with Jakarta EE standards.

---

# Chapter 21. Entity Data Filtering

Support for Entity Filtering in Jersey introduces a convenient facility for reducing the amount of data exchanged over the wire between client and server without a need to create specialized data view components. The main idea behind this feature is to give you APIs that will let you to selectively filter out any non-relevant data from the marshalled object model before sending the data to the other party based on the context of the particular message exchange. This way, only the necessary or relevant portion of the data is transferred over the network with each client request or server response, without a need to create special facade models for transferring these limited subsets of the model data.

Entity filtering feature allows you to define your own entity-filtering rules for your entity classes based on the current context (e.g. matched resource method) and keep these rules in one place (directly in your domain model). With Jersey entity filtering facility it is also possible to assign security access rules to entity classes properties and property accessors.

We will first explain the main concepts and then we will explore the entity filtering feature topics from a perspective of basic use-cases,

- Section 21.3, “Using custom annotations to filter entities”
- Section 21.4, “Role-based Entity Filtering using (`jakarta.annotation.security`) annotations”
- Section 21.5, “Entity Filtering based on dynamic and configurable query parameters”

as well as some more complex ones.

- Section 21.6, “Defining custom handling for entity-filtering annotations”

## Note

Jersey entity filtering feature is supported via Jersey extension modules listed in Section 21.8, “Modules with support for Entity Data Filtering”.

## 21.1. Enabling and configuring Entity Filtering in your application

Entity Filtering support in Jersey is provided as an extension module and needs to be mentioned explicitly in your `pom.xml` file (in case of using Maven):

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-entity-filtering</artifactId>
  <version>4.0.0</version>
</dependency>
```

## Note

If you're not using Maven make sure to have also all the transitive dependencies (see `jersey-entity-filtering` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-entity-filtering/dependencies.html>]) on the classpath.

The entity-filtering extension module provides three **Features** which you can register into server/client runtime in prior to use Entity Filtering in an application:

- `EntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/EntityFilteringFeature.html>]

Filtering based on entity-filtering annotations (or i.e. external configuration file) created using `@EntityFiltering` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/EntityFiltering.html>] meta-annotation.

- `SecurityEntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/SecurityEntityFilteringFeature.html>]

Filtering based on security (`jakarta.annotation.security`) and entity-filtering annotations.

- `SelectableEntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/SelectableEntityFilteringFeature.html>]

Filtering based on dynamic and configurable query parameters.

If you want to use both entity-filtering annotations and security annotations for entity data filtering it is enough to register `SecurityEntityFilteringFeature` as this feature registers also `EntityFilteringFeature`.

Entity-filtering currently recognizes one property that can be passed into the `Configuration` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configuration.html>] instance (client/server):

- `EntityFilteringFeature.ENTITY_FILTERING_SCOPE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/EntityFilteringFeature.html#ENTITY\\_FILTERING\\_SCOPE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/EntityFilteringFeature.html#ENTITY_FILTERING_SCOPE)] -  
"`jersey.config.entityFiltering.scope`"

Defines one or more annotations that should be used as entity-filtering scope when reading/writing an entity.

## Note

Processing of entity-filtering annotations to create an entity-filtering scope is defined by following: "Request/Resource entity annotations" > "Configuration" > "Resource method/class annotations" (on server).

You can configure entity-filtering on server (basic + security examples) as follows:

### Example 21.1. Registering and configuring entity-filtering feature on server.

```
1 new ResourceConfig()
2    // Set entity-filtering scope via configuration.
3    .property(EntityFilteringFeature.ENTITY_FILTERING_SCOPE, new Annotation[])
4    // Register the EntityFilteringFeature.
5    .register(EntityFilteringFeature.class)
6    // Further configuration of ResourceConfig.
7    .register( ... );
```

### Example 21.2. Registering and configuring entity-filtering feature with security annotations on server.

```
1 new ResourceConfig()
2    // Set entity-filtering scope via configuration.
```



```
3     .property(EntityFilteringFeature.ENTITY_FILTERING_SCOPE, new Annotation[]
4     // Register the SecurityEntityFilteringFeature.
5     .register(SecurityEntityFilteringFeature.class)
6     // Further configuration of ResourceConfig.
7     .register( ... );
```

**Example 21.3. Registering and configuring entity-filtering feature based on dynamic and configurable query parameters.**

```
1 new ResourceConfig()
2     // Set query parameter name for dynamic filtering
3     .property>SelectableEntityFilteringFeature.QUERY_PARAM_NAME, "select")
4     // Register the SelectableEntityFilteringFeature.
5     .register>SelectableEntityFilteringFeature.class)
6     // Further configuration of ResourceConfig.
7     .register( ... );
```

Use similar steps to register entity-filtering on client:

**Example 21.4. Registering and configuring entity-filtering feature on client.**

```
1 final ClientConfig config = new ClientConfig()
2     // Set entity-filtering scope via configuration.
3     .property(EntityFilteringFeature.ENTITY_FILTERING_SCOPE, new Annotation[]
4     // Register the EntityFilteringFeature.
5     .register(EntityFilteringFeature.class)
6     // Further configuration of ClientConfig.
7     .register( ... );
8
9 // Create new client.
10 final Client client = ClientBuilder.newClient(config);
11
12 // Use the client.
```

## 21.2. Components used to describe Entity Filtering concepts

In the next section the entity-filtering features will be illustrated on a project-tracking application that contains three classes in its domain model and few resources (only `Project` resource will be shown in this chapter). The full source code for the example application can be found in Jersey Entity Filtering example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/entity-filtering>].

Suppose there are three domain model classes (or entities) in our model: `Project`, `User` and `Task` (getters/setter are omitted for brevity).

**Example 21.5. Project**

```
1 public class Project {
2
3     private Long id;
4
5     private String name;
```

```
6
7     private String description;
8
9     private List<Task> tasks;
10
11    private List<User> users;
12
13    // getters and setters
14 }
```

**Example 21.6. User**

```
1 public class User {
2
3     private Long id;
4
5     private String name;
6
7     private String email;
8
9     private List<Project> projects;
10
11    private List<Task> tasks;
12
13    // getters and setters
14 }
```

**Example 21.7. Task**

```
1 public class Task {
2
3     private Long id;
4
5     private String name;
6
7     private String description;
8
9     private Project project;
10
11    private User user;
12
13    // getters and setters
14 }
```

To retrieve the entities from server to client, we have created also a couple of JAX-RS resources from whose the `ProjectsResource` is shown as example.

**Example 21.8. ProjectsResource**

```
1 @Path("projects")
2 @Produces("application/json")
3 public class ProjectsResource {
4
5     @GET
```

```
6      @Path("{id}")
7      public Project getProject(@PathParam("id") final Long id) {
8          return getDetailedProject(id);
9      }
10
11      @GET
12      public List<Project> getProjects() {
13          return getDetailedProjects();
14      }
15 }
```

## 21.3. Using custom annotations to filter entities

Entity filtering via annotations is based on an `@EntityFiltering` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/EntityFiltering.html>] meta-annotation. This meta-annotation is used to identify entity-filtering annotations that can be then attached to

- domain model classes (supported on both, server and client sides), and
- resource methods / resource classes (only on server side)

An example of entity-filtering annotation applicable to a class, field or method can be seen in Example 21.9, “ProjectDetailedView” below.

### Example 21.9. ProjectDetailedView

```
1  @Target({ElementType.TYPE, ElementType.METHOD, ElementType.FIELD})
2  @Retention(RetentionPolicy.RUNTIME)
3  @Documented
4  @EntityFiltering
5  public @interface ProjectDetailedView {
6
7      /**
8       * Factory class for creating instances of {@code ProjectDetailedView} annotations.
9       */
10     public static class Factory
11         extends AnnotationLiteral<ProjectDetailedView>
12         implements ProjectDetailedView {
13
14         private Factory() {
15         }
16
17         public static ProjectDetailedView get() {
18             return new Factory();
19         }
20     }
21 }
```

Since creating annotation instances directly in Java code is not trivial, it is a good practice to provide an inner annotation `Factory` class in each custom filtering annotation, through which new instances of the annotation can be directly created. The annotation factory class can be created by extending the `HK2 AnnotationLiteral` class and implementing the annotation interface itself. It should also provide a static factory method that will create and return a new instance of the `Factory` class when invoked. Such

annotation instances can be then passed to the client and server run-times to define or override entity-filtering scopes.

By placing an entity-filtering annotation on an entity (class, fields, getters or setters) we define a so-called *entity-filtering scope* for the entity. The purpose of entity-filtering scope is to identify parts of the domain model that should be processed when the model is to be sent over the wire in a particular entity-filtering scope. We distinguish between:

- global entity-filtering scope (defined by placing filtering annotation on a class itself), and
- local entity-filtering scope (defined by placing filtering annotation on a field, getter or setter)

Unannotated members of a domain model class are automatically added to all existing global entity-filtering scopes. If there is no explicit global entity-filtering scope defined on a class a default scope is created for this class to group these members.

Creating entity-filtering scopes using custom entity-filtering annotations in domain model classes is illustrated in the following examples.

### Example 21.10. Annotated Project

```
1 public class Project {
2
3     private Long id;
4
5     private String name;
6
7     private String description;
8
9     @ProjectDetailedView
10    private List<Task> tasks;
11
12    @ProjectDetailedView
13    private List<User> users;
14
15    // getters and setters
16 }
```

### Example 21.11. Annotated User

```
1 public class User {
2
3     private Long id;
4
5     private String name;
6
7     private String email;
8
9     @UserDetailedView
10    private List<Project> projects;
11
12    @UserDetailedView
13    private List<Task> tasks;
14
15    // getters and setters
16 }
```

**Example 21.12. Annotated Task**

```
1 public class Task {
2
3     private Long id;
4
5     private String name;
6
7     private String description;
8
9     @TaskDetailedView
10    private Project project;
11
12    @TaskDetailedView
13    private User user;
14
15    // getters and setters
16 }
```

As you can see in the examples above, we have defined 3 separate scopes using `@ProjectDetailedView`, `@UserDetailedView` and `@TaskDetailedView` annotations and we have applied these scopes selectively to certain fields in the domain model classes.

Once the entity-filtering scopes are applied to the parts of a domain model, the entity filtering facility (when enabled) will check the active scopes when the model is being sent over the wire, and filter out all parts from the model for which there is no active scope set in the given context. Therefore, we need a way how to control the scopes active in any given context in order to process the model data in a certain way (e.g. expose the detailed view). We need to tell the server/client runtime which entity-filtering scopes we want to apply. There are 2 ways how to do this for client-side and 3 ways for server-side:

- Outbound client request or server response programmatically created with entity-filtering annotations that identify the scopes to be applied (available on both, client and server)
- Property identifying the applied scopes passed through Configuration [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configuration.html>] (available on both, client and server)
- Entity-filtering annotations identifying the applied scopes attached to a resource method or class (server-side only)

When the multiple approaches are combined, the priorities of calculating the applied scopes are as follows: **Entity annotations in request or response** > **Property passed through Configuration** > **Annotations applied to a resource method or class**.

In a graph of domain model objects, the entity-filtering scopes are applied to the root node as well as transitively to all the child nodes. Fields and child nodes that do not match at least a single active scope are filtered out. When the scope matching is performed, annotations applied to the domain model classes and fields are used to compute the scope for each particular component of the model. If there are no annotations on the class or its fields, the default scope is assumed. During the filtering, first, the annotations on root model class and its fields are considered. For all composite fields that have not been filtered out, the annotations on the referenced child class and its fields are considered next, and so on.

## 21.3.1. Server-side Entity Filtering

To pass entity-filtering annotations via Response [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.html>] returned from a resource method you can leverage the

`Response.ResponseBuilder#entity(Object, Annotation[])` [[https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.ResponseBuilder.html#entity\(java.lang.Object, java.lang.annotation.Annotation\[\]\)](https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Response.ResponseBuilder.html#entity(java.lang.Object, java.lang.annotation.Annotation[]))] method. The next example illustrates this approach. You will also see why every custom entity-filtering annotation should contain a factory for creating instances of the annotation.

### Example 21.13. ProjectsResource - Response entity-filtering annotations

```
1 @Path("projects")
2 @Produces("application/json")
3 public class ProjectsResource {
4
5     @GET
6     public Response getProjects(@QueryParam("detailed") final boolean isDetailed) {
7         return Response
8             .ok()
9             .entity(new GenericEntity<List<Project>>(EntityStore.getProjectList(),
10                 isDetailed ? new Annotation[] {ProjectDetailedView.Factory.INSTANCE} : null))
11             .build();
12     }
13 }
```

Annotating a resource method / class is typically easier although it is less flexible and may require more resource methods to be created to cover all the alternative use case scenarios. For example:

### Example 21.14. ProjectsResource - Entity-filtering annotations on methods

```
1 @Path("projects")
2 @Produces("application/json")
3 public class ProjectsResource {
4
5     @GET
6     public List<Project> getProjects() {
7         return getDetailedProjects();
8     }
9
10    @GET
11    @Path("detailed")
12    @ProjectDetailedView
13    public List<Project> getDetailedProjects() {
14        return EntityStore.getProjectList();
15    }
16 }
```

To see how entity-filtering scopes can be applied using a `Configuration` property, see the Example 21.1, “Registering and configuring entity-filtering feature on server.” example.

When a `Project` model from the example above is requested in a scope represented by `@ProjectDetailedView` entity-filtering annotation, the `Project` model data sent over the wire would contain:

- Project - id, name, description, tasks, users
- Task - id, name, description
- User - id, name, email

Or, to illustrate this in JSON format:

```
1 {
2   "description" : "Jersey is the open source (under dual EPL+GPL license) JAX
3   "id" : 1,
4   "name" : "Jersey",
5   "tasks" : [ {
6     "description" : "Entity Data Filtering",
7     "id" : 1,
8     "name" : "ENT_FLT"
9   }, {
10    "description" : "OAuth 1 + 2",
11    "id" : 2,
12    "name" : "OAUTH"
13  } ],
14  "users" : [ {
15    "email" : "very@secret.com",
16    "id" : 1,
17    "name" : "Jersey Robot"
18  } ]
19 }
```

For the *default entity-filtering scope* the filtered model would look like:

- Project - id, name, description

Or in JSON format:

```
1 {
2   "description" : "Jersey is the open source (under dual EPL+GPL license) JAX
3   "id" : 1,
4   "name" : "Jersey"
5 }
```

## 21.3.2. Client-side Entity Filtering

As mentioned above you can define applied entity-filtering scopes using a property set either in the client run-time Configuration (see Example 21.4, “Registering and configuring entity-filtering feature on client.”) or by passing the entity-filtering annotations during a creation of an individual request to be sent to the server.

### Example 21.15. Client - Request entity-filtering annotations

```
1 ClientBuilder.newClient(config)
2   .target(uri)
3   .request()
4   .post(Entity.entity(project, new Annotation[] {ProjectDetailedView.Factory
```

You can use the mentioned method with client injected into a resource as well.

### Example 21.16. Client - Request entity-filtering annotations

```
1 @Path("clients")
2 @Produces("application/json")
3 public class ClientsResource {
```

```
4
5     @Uri("projects")
6     private WebTarget target;
7
8     @GET
9     public List<Project> getProjects() {
10         return target.request()
11             .post(Entity.entity(project, new Annotation[] {ProjectDetailedView
12         })
13     }
```

## 21.4. Role-based Entity Filtering using (jakarta.annotation.security) annotations

Filtering the content sent to the client (or server) based on the authorized security roles is a commonly required use case. By registering `SecurityEntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/SecurityEntityFilteringFeature.html>] you can leverage the Jersey Entity Filtering facility in connection with standard `jakarta.annotation.security` annotations exactly the same way as you would with custom entity-filtering annotations described in previous chapters. Supported security annotations are:

- `@PermitAll` [<https://jakarta.ee/specifications/annotations/2.0/apidocs/jakarta.annotation/jakarta/annotation/security/permitall>],
- `@RolesAllowed` [<https://jakarta.ee/specifications/annotations/2.0/apidocs/jakarta.annotation/jakarta/annotation/security/rolesallowed>], and
- `@DenyAll` [<https://jakarta.ee/specifications/annotations/2.0/apidocs/jakarta.annotation/jakarta/annotation/security/denyall>]

Although the mechanics of the Entity Data Filtering feature used for the security annotation-based filtering is the same as with the entity-filtering annotations, the processing of security annotations differs in a few important aspects:

- Custom `SecurityContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/SecurityContext.html>] should be set by a container request filter in order to use `@RolesAllowed` for role-based filtering of domain model data (server-side)
- There is no need to provide entity-filtering (or security) annotations on resource methods in order to define entity-filtering scopes for `@RolesAllowed` that is applied to the domain model components, as all the available roles for the current user are automatically determined using the information from the provided `SecurityContext` (server-side only).

### Note

Instances of security annotations (to be used for programmatically defined scopes either on client or server) can be created using one of the methods in the `SecurityAnnotations` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/SecurityAnnotations.html>] factory class that is part of the Jersey Entity Filtering API.



## 21.5. Entity Filtering based on dynamic and configurable query parameters

Filtering the content sent to the client (or server) dynamically based on query parameters is another commonly required use case. By registering `SelectableEntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/SelectableEntityFilteringFeature.html>] you can leverage the Jersey Entity Filtering facility in connection with query parameters exactly the same way as you would with custom entity-filtering annotations described in previous chapters.

### Example 21.17. Sever - Query Parameter driven entity-filtering

```
1 @XmlElement
2 public class Address {
3
4     private String streetAddress;
5
6     private String region;
7
8     private PhoneNumber phoneNumber;
9 }
```

Query parameters are supported in comma delimited "dot notation" style similar to `BeanInfo` objects and Spring path expressions. As an example, the following URL: `http://jersey.example.com/addresses/51234?select=region,streetAddress` may render only the address's region and street address properties as in the following example:

### Example 21.18.

```
1 {
2     "region" : "CA",
3     "streetAddress" : "1234 Fake St."
4 }
```

## 21.6. Defining custom handling for entity-filtering annotations

To create a custom entity-filtering annotation with special handling, i.e. an field aggregator annotation used to annotate classes like the one in Example 21.19, “Entity-filtering annotation with custom meaning” it is, in most cases, sufficient to implement and register the following SPI contracts:

- `EntityProcessor` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/spi/EntityProcessor.html>]

Implementations of this SPI are invoked to process entity class and its members. Custom implementations can extend from `AbstractEntityProcessor` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/spi/AbstractEntityProcessor.html>].

- `ScopeResolver` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/spi/ScopeResolver.html>]

Implementations of this SPI are invoked to retrieve entity-filtering scopes from an array of provided annotations.

**Example 21.19. Entity-filtering annotation with custom meaning**

```
1 @Target({ElementType.TYPE})
2 @Retention(RetentionPolicy.RUNTIME)
3 @EntityFiltering
4 public @interface FilteringAggregator {
5
6     /**
7      * Entity-filtering scope to add given fields to.
8      */
9     Annotation filteringScope();
10
11     /**
12      * Fields to be a part of the entity-filtering scope.
13      */
14     String[] fields();
15 }
```

## 21.7. Supporting Entity Data Filtering in custom entity providers or frameworks

To support Entity Data Filtering in custom entity providers (e.g. as in Example 21.20, “Entity Data Filtering support in MOXy JSON binding provider”), it is sufficient in most of the cases to implement and register the following SPI contracts:

- `ObjectProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/spi/ObjectProvider.html>]

To be able to obtain an instance of a filtering object model your provider understands and can act on. The implementations can extend `AbstractObjectProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/spi/AbstractObjectProvider.html>].

- `ObjectGraphTransformer` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/spi/ObjectGraphTransformer.html>]

To transform a read-only generic representation of a domain object model graph to be processed into an entity-filtering object model your provider understands and can act on. The implementations can extend `AbstractObjectProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/spi/AbstractObjectProvider.html>].

**Example 21.20. Entity Data Filtering support in MOXy JSON binding provider**

```
1 @Singleton
2 public class FilteringMoxyJsonProvider extends ConfigurableMoxyJsonProvider {
3
4     @Inject
5     private Provider<ObjectProvider<ObjectGraph>> provider;
6
7     @Override
```

```
8     protected void preWriteTo(final Object object, final Class<?> type, final
9                               final MediaType mediaType, final MultivaluedMap<
10                              final Marshaller marshaller) throws JAXBException {
11         super.preWriteTo(object, type, genericType, annotations, mediaType, headers);
12
13         // Entity Filtering.
14         if (marshaller.getProperty(MarshallerProperties.OBJECT_GRAPH) == null)
15             final Object objectGraph = provider.get().getFilteringObject(genericType);
16
17         if (objectGraph != null) {
18             marshaller.setProperty(MarshallerProperties.OBJECT_GRAPH, objectGraph);
19         }
20     }
21 }
22
23 @Override
24 protected void preReadFrom(final Class<Object> type, final Type genericType,
25                             final MediaType mediaType, final MultivaluedMap<String, Object>
26                             final Unmarshaller unmarshaller) throws JAXBException {
27     super.preReadFrom(type, genericType, annotations, mediaType, headers);
28
29     // Entity Filtering.
30     if (unmarshaller.getProperty(MarshallerProperties.OBJECT_GRAPH) == null)
31         final Object objectGraph = provider.get().getFilteringObject(genericType);
32
33     if (objectGraph != null) {
34         unmarshaller.setProperty(MarshallerProperties.OBJECT_GRAPH, objectGraph);
35     }
36 }
37 }
38 }
```

## 21.8. Modules with support for Entity Data Filtering

List of modules from Jersey workspace that support Entity Filtering:

- MOXy
- Jackson (2.x)

In order to use Entity Filtering in mentioned modules you need to explicitly register either `EntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/EntityFilteringFeature.html>], `SecurityEntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/SecurityEntityFilteringFeature.html>] or `SelectableEntityFilteringFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/filtering/SelectableEntityFilteringFeature.html>] to activate Entity Filtering for particular module.

## 21.9. Examples

To see a complete working examples of entity-filtering feature refer to the:

- Entity Filtering example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/entity-filtering>]
- Entity Filtering example (with security annotations) [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/entity-filtering-security>]
- Entity Filtering example (based on dynamic and configurable query parameters) [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/entity-filtering-selectable>]

---

# Chapter 22. MVC Templates

Jersey provides an extension to support the Model-View-Controller (MVC) design pattern. In the context of Jersey components, the Controller from the MVC pattern corresponds to a resource class or method, the View to a template bound to the resource class or method, and the model to a Java object (or a Java bean) returned from a resource method (Controller).

## Note

Some of the passages/examples from this chapter have been created by Paul Sandoz.

In Jersey 2, the base MVC API consists of two classes (`org.glassfish.jersey.server.mvc` package) that can be used to bind model to view (template), namely `Viewable` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Viewable.html>] and `@Template` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Template.html>]. These classes determine which approach (explicit/implicit) you would be taking when working with Jersey MVC templating support.

## 22.1. Viewable

In this approach a resource method explicitly returns a reference to a view template and the data model to be used. For this purpose the `Viewable` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Viewable.html>] class has been introduced in Jersey 1 and is also present (under a different package) in Jersey 2 and 3. A simple example of usage can be seen in Example 22.1, “Using `Viewable` in a resource class”.

### Example 22.1. Using `Viewable` in a resource class

```
package com.example;

@Path("foo")
public class Foo {

    @GET
    public Viewable get() {
        return new Viewable("index.foo", "FOO");
    }
}
```

In this example, the `Foo` JAX-RS resource class is the controller and the `Viewable` instance encapsulates the provided data model (`FOO` string) and a named reference to the associated view template (`index.foo`).

## Tip

All HTTP methods may return `Viewable` instances. Thus a `POST` method may return a template reference to a template that produces a view as a result of processing an HTML Form [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Form.html>].

## 22.2. @Template

### 22.2.1. Annotating Resource methods

There is no need to use `Viewable` every time you want to bind a model to a template. To make the resource method more readable (and to avoid verbose wrapping of a template reference and model into `Viewable`) you can simply annotate a resource method with `@Template` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Template.html>] annotation. An updated example, using `@Template`, from previous section is shown in Example 22.2, “Using `@Template` on a resource method” example.

#### Example 22.2. Using `@Template` on a resource method

```
package com.example;

@Path("foo")
public class Foo {

    @GET
    @Template(name = "index.foo")
    public String get() {
        return "F00";
    }
}
```

In this example, the `Foo` JAX-RS resource class is still the controller as in previous section but the MVC model is now represented by the return value of annotated resource method.

The processing of such a method is then essentially the same as if the return type of the method was an instance of the `Viewable` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Viewable.html>] class. If a method is annotated with `@Template` and is also returning a `Viewable` instance then the values from the `Viewable` instance take precedence over those defined in the annotation. Producible media types are for both cases, `Viewable` and `@Template`, determined by the method or class level `@Produces` annotation.

### 22.2.2. Annotating Resource classes

A resource class can have templates implicitly associated with it via `@Template` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Template.html>] annotation. For example, take a look at the resource class listing in Example 22.3, “Using `@Template` on a resource class”.

#### Example 22.3. Using `@Template` on a resource class

```
@Path("foo")
@Template
public class Foo {

    public String getFoo() {
        return "F00";
    }
}
```

The example relies on Jersey MVC conventions a lot and requires more explanation as such. First of all, you may have noticed that there is no resource method defined in this JAX-RS resource. Also, there is no template reference defined. In this case, since the `@Template` annotation placed on the resource class does not contain any information, the default relative template reference `index` will be used (for more on this topic see Section 22.3, “Absolute vs. Relative template reference”). As for the missing resource methods, a default `@GET` method will be automatically generated by Jersey for the `Foo` resource (which is the MVC Controller now). The implementation of the generated resource method performs the equivalent of the following explicit resource method:

```
@GET
public Viewable get() {
    return new Viewable("index", this);
}
```

You can see that the resource class serves in this case also as the model. Producible media types are determined based on the `@Produces` annotation declared on the resource class, if any.

### Note

In case of "resource class"-based implicit MVC view templates, the controller is also the model. In such case the template reference `index` is special, it is the template reference associated with the controller instance itself.

In the following example, the MVC controller represented by a JAX-RS `@GET` sub-resource method, is also generated in the resource class annotated with `@Template`:

```
@GET
@Path("/{implicit-view-path-parameter}")
public Viewable get(@PathParam("/{implicit-view-path-parameter}") String template) {
    return new Viewable(template, this);
}
```

This allows Jersey to support also implicit sub-resource templates. For example, a JAX-RS resource at path `foo/bar` will try to use relative template reference `bar` that resolves to an absolute template reference `/com/foo/Foo/bar`.

In other words, a HTTP GET request to a `/foo/bar` would be handled by this auto-generated method in the `Foo` resource and would delegate the request to a registered template processor supports processing of the absolute template reference `/com/foo/Foo/bar`, where the model is still an instance of the same JAX-RS resource class `Foo`.

## 22.3. Absolute vs. Relative template reference

As discussed in the previous section, both `@Template` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Template.html>] and `Viewable` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Viewable.html>] provide means to define a reference to a template. We will now discuss how these values are interpreted and how the concrete template is found.

### 22.3.1. Relative template reference

Relative reference is any path that does not start with a leading `'/'` (slash) character (i.e. `index.foo`). This kind of references is resolved into absolute ones by pre-pending a given value with a fully qualified name of the last matched resource.

Consider the Example 22.3, “Using `@Template` on a resource class” from the previous section, the template name reference `index` is a relative value that Jersey will resolve to its absolute template reference using a fully qualified class name of `Foo` (more on resolving relative template name to the absolute one can be found in the JavaDoc of `Viewable` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Viewable.html>] class), which, in our case, is:

```
"/com/foo/Foo/index"
```

Jersey will then search all the registered template processors (see Section 22.7, “Writing Custom Templating Engines”) to find a template processor that can resolve the absolute template reference further to a “processable” template reference. If a template processor is found then the “processable” template is processed using the supplied data model.

### Note

If none or empty template reference is provided (either in `Viewable` or via `@Template`) then the `index` reference is assumed and all further processing is done for this value.

## 22.3.2. Absolute template reference

Let's change the resource GET method in our `Foo` resource a little:

### Example 22.4. Using absolute path to template in `Viewable`

```
@GET
public Viewable get() {
    return new Viewable("/index", "Foo");
}
```

In this case, since the template reference begins with `"/`, Jersey will consider the reference to be absolute already and will not attempt to absolutize it again. The reference will be used “as is” when resolving it to a “processable” template reference as described earlier.

Absolute template references start with leading `/'` (i.e. `/com/example/index.foo`) character and are not further resolved (with respect to the resolving resource class) which means that the template is looked for at the provided path directly.

Note, however, that template processors for custom templating engines may modify (and the supported ones do) absolute template reference by pre-pending ‘base template path’ (if defined) and appending template suffix (i.e. `.foo`) if the suffix is not provided in the reference.

For example assume that we want to use Mustache templates for our views and we have defined ‘base template path’ as `pages`. For the absolute template reference `/com/example/Foo/index` the template processor will transform the reference into the following path: `/pages/com/example/Foo/index.mustache`.

## 22.4. Handling errors with MVC

In addition to `@Template` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Template.html>] an `@ErrorTemplate` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/ErrorTemplate.html>] annotation has been introduced in Jersey 2.3 (valid for Jersey 3.x as well). The purpose of this annotation is to bind the model to an error view in case an exception has been raised during processing of a request. This is true



for any exception thrown after the resource matching phase (i.e. this not only applies to JAX-RS resources but providers and even Jersey runtime as well). The model in this case is the thrown exception itself.

Example 22.5, “Using `@ErrorTemplate` on a resource method” shows how to use `@ErrorTemplate` on a resource method. If all goes well with the method processing, then the `/short-link` template is used as page sent to the user. Otherwise if an exception is raised then the `/error-form` template is shown to the user.

### Example 22.5. Using `@ErrorTemplate` on a resource method

```
@POST
@Produces({"text/html"})
@Consumes(MediaType.APPLICATION_FORM_URLENCODED)
@Template(name = "/short-link")
@ErrorTemplate(name = "/error-form")
public ShortenedLink createLink(@FormParam("link") final String link) {
    // ...
}
```

Note that `@ErrorTemplate` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/ErrorTemplate.html>] can be used on a resource class or a resource method to merely handle error states. There is no need to use `@Template` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Template.html>] or `Viewable` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/Viewable.html>] with it.

The annotation is handled by custom `ExceptionHandler<E extends Throwable>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ExceptionHandler.html>] which creates an instance of `Viewable` that is further processed by Jersey. This exception mapper is registered automatically with a `MvcFeature`.

## 22.4.1. MVC & Bean Validation

`@ErrorTemplate` can also be used with Bean Validation to display specific error pages in case the validation of input/output values fails for some reason. Everything works as described above except the model is not the thrown exception but rather a list of `ValidationError` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/validation/ValidationError.html>]. This list can be iterated in the template and all the validation errors can be shown to the user in a desirable way.

### Example 22.6. Using `@ErrorTemplate` with Bean Validation

```
@POST
@Produces({"text/html"})
@Consumes(MediaType.APPLICATION_FORM_URLENCODED)
@Template(name = "/short-link") @ErrorTemplate(name = "/error-form")
@Valid
public ShortenedLink createLink(@NotEmpty @FormParam("link") final String link) {
    // ...
}
```

### Example 22.7. Iterating through `ValidationError` in JSP

```
<c:forEach items="${model}" var="error">
```

```
    ${error.message} "<strong>${error.invalidValue}</strong>"<br/>
  </c:forEach>
```

Support for Bean Validation in Jersey MVC Templates is provided by a `jersey-mvc-bean-validation` extension module. The JAX-RS Feature [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html] provided by this module (`MvcBeanValidationFeature`) has to be registered in order to use this functionality (see Section 22.5, “Registration and Configuration”).

Maven users can find this module at coordinates

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-mvc-bean-validation</artifactId>
  <version>4.0.0</version>
</dependency>
```

and for non-Maven users the list of dependencies is available at `jersey-mvc-bean-validation` [https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-bean-validation/dependencies.html].

## 22.5. Registration and Configuration

To use the capabilities of Jersey MVC templating support in your JAX-RS/Jersey application you need to register specific JAX-RS Feature [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Feature.html]s provided by the MVC modules. For `jersey-mvc` module it is `MvcFeature` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/MvcFeature.html] for others it could be, for example, `FreemarkerMvcFeature` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/freemarker/FreemarkerMvcFeature.html] (`jersey-mvc-freemarker`).

### Example 22.8. Registering `MvcFeature`

```
new ResourceConfig()
    .register(org.glassfish.jersey.server.mvc.MvcFeature.class)
    // Further configuration of ResourceConfig.
    .register( ... );
```

### Example 22.9. Registering `FreemarkerMvcFeature`

```
new ResourceConfig()
    .register(org.glassfish.jersey.server.mvc.freemarker.FreemarkerMvcFeature.class)
    // Further configuration of ResourceConfig.
    .register( ... );
```

### Note

Modules that uses capabilities of the base Jersey MVC module register `MvcFeature` automatically, so you don't need to register this feature explicitly in your code.

Almost all of the MVC modules are further configurable and either contain a `*Properties` (e.g. `FreemarkerMvcProperties`) class describing all the available properties which could be set in a

JAX-RS Application / ResourceConfig. Alternatively, the properties are listed directly in the module \*Feature class.

### Example 22.10. Setting MvcFeature.TEMPLATE\_BASE\_PATH value in ResourceConfig

```
new ResourceConfig()
    .property(MvcFeature.TEMPLATE_BASE_PATH, "templates")
    .register(MvcFeature.class)
    // Further configuration of ResourceConfig.
    .register( ... );
```

### Example 22.11. Setting FreemarkerMvcProperties.TEMPLATE\_BASE\_PATH value in web.xml

```
<servlet>
  <servlet-name>org.glassfish.jersey.examples.freemarker.MyApplication</servlet-name>
  <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
  <init-param>
    <param-name>jakarta.ws.rs.Application</param-name>
    <param-value>org.glassfish.jersey.examples.freemarker.MyApplication</param-value>
  </init-param>
  <init-param>
    <param-name>jersey.config.server.mvc.templateBasePath.freemarker</param-name>
    <param-value>freemarker</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

## 22.6. Supported templating engines

Jersey provides extension modules that enable support for several templating engines. This section lists all the supported engines and their modules as well as discusses any module-specific details.

### 22.6.1. Mustache

An integration module for Mustache [<https://github.com/spullara/mustache.java>]-based templating engine.

Mustache template processor resolves absolute template references to processable template references represented as Mustache templates as follows:

#### Procedure 22.1. Resolving Mustache template reference

1. if the absolute template reference does not end in `.mustache` append this suffix to the reference; and
2. if `ServletContext.getResource`, `Class.getResource` or `File.exists` returns a non-null value for the reference then return the reference as the processable template reference otherwise return `null` (to indicate the absolute reference has not been resolved by the Mustache template processor).

Thus the absolute template reference `/com/foo/Foo/index` would be resolved as `/com/foo/Foo/index.mustache`, provided there exists a `/com/foo/Foo/index.mustache` Mustache template in the application.

Available configuration properties:

- `MustacheMvcFeature.TEMPLATE_BASE_PATH` -  
`jersey.config.server.mvc.templateBasePath.mustache`  
  
The base path where Mustache templates are located.
- `MustacheMvcFeature.CACHE_TEMPLATES` -  
`jersey.config.server.mvc.caching.mustache`  
  
Enables caching of Mustache templates to avoid multiple compilation.
- `MustacheMvcFeature.TEMPLATE_OBJECT_FACTORY` -  
`jersey.config.server.mvc.factory.mustache`  
  
Property used to pass user-configured `MustacheFactory`.
- `MustacheMvcFeature.ENCODING` -  
`jersey.config.server.mvc.encoding.mustache`  
  
Property used to configure a default encoding that will be used if none is specified in `@Produces` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html>] annotation. If property is not defined the UTF-8 encoding will be used as a default value.

Maven users can find this module at coordinates

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-mvc-mustache</artifactId>
  <version>4.0.0</version>
</dependency>
```

and for non-Maven users the list of dependencies is available at `jersey-mvc-mustache` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-mustache/dependencies.html>].

## 22.6.2. Freemarker

An integration module for Freemarker [<https://freemarker.apache.org/>]-based templating engine.

Freemarker template processor resolves absolute template references to processable template references represented as Freemarker templates as follows:

### Procedure 22.2. Resolving Freemarker template reference

1. if the absolute template reference does not end in `.ftl` append this suffix to the reference; and
2. if `ServletContext.getResource`, `Class.getResource` or `File.exists` returns a non-null value for the reference then return the reference as the processable template reference otherwise return null (to indicate the absolute reference has not been resolved by the Freemarker template processor).

Thus the absolute template reference `/com/foo/Foo/index` would be resolved to `/com/foo/Foo/index.ftl`, provided there exists a `/com/foo/Foo/index.ftl` Freemarker template in the application.

Jersey will assign the model instance to an attribute named `model`. So it is possible to reference the `foo` key from the provided Map (MVC Model) resource from the Freemarker template as follows:

```
<h1>${model.foo}</h1>
```

Available configuration properties:

- `FreemarkerMvcFeature.TEMPLATE_BASE_PATH` -  
`jersey.config.server.mvc.templateBasePath.freemarker`

The base path where Freemarker templates are located.

- `FreemarkerMvcFeature.CACHE_TEMPLATES` -  
`jersey.config.server.mvc.caching.freemarker`

Enables caching of Freemarker templates to avoid multiple compilation.

- `FreemarkerMvcFeature.TEMPLATE_OBJECT_FACTORY` -  
`jersey.config.server.mvc.factory.freemarker`

Property used to pass user-configured `FreemarkerFactory`.

- `FreemarkerMvcFeature.ENCODING` -  
`jersey.config.server.mvc.encoding.freemarker`

Property used to configure a default encoding that will be used if none is specified in `@Produces` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/Produces.html>] annotation. If property is not defined the UTF-8 encoding will be used as a default value.

Maven users can find this module at coordinates

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-mvc-freemarker</artifactId>
  <version>4.0.0</version>
</dependency>
```

and for non-Maven users the list of dependencies is available at [jersey-mvc-freemarker \[https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-freemarker/dependencies.html\]](https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-freemarker/dependencies.html).

## 22.6.3. JSP

An integration module for JSP-based templating engine.

### Limitations of Jersey JSP MVC Templates

Jersey web applications that want to use JSP templating support should be registered as Servlet filters rather than Servlets in the application's `web.xml`. The `web.xml`-less deployment style introduced in Servlet 3.0 is not supported at the moment for web applications that require use of Jersey MVC templating support.

JSP template processor resolves absolute template references to processable template references represented as JSP pages as follows:

### Procedure 22.3. Resolving JSP template reference

1. if the absolute template reference does not end in `.jsp` append this suffix to the reference; and
2. if `ServletContext.getResource` returns a non-null value for the reference then return the reference as the processable template reference otherwise return `null` (to indicate the absolute reference has not been resolved by the JSP template processor).

Thus the absolute template reference `/com/foo/Foo/index` would be resolved to `/com/foo/Foo/index.jsp`, provided there exists a `/com/foo/Foo/index.jsp` JSP page in the web application.

Jersey will assign the model instance to the attribute named `model` or `it`. So it is possible to reference the `foo` property on the `Foo` resource from the JSP template as follows:

```
<h1>${model.foo}</h1>
```

or

```
<h1>${it.foo}</h1>
```

To include another JSP page in the currently processed one a custom `include` tag can be used. Mandatory parameter `page` represents a relative template name which would be absolutized using the same resolving resource class as the parent JSP page template.

### Example 22.12. Including JSP page into JSP page

```
<%@page contentType="text/html"%>
<%@page pageEncoding="UTF-8"%>

<%@taglib prefix="rbt" uri="urn:org:glassfish:jersey:servlet:mvc" %>

<html>
  <body>

    <rbt:include page="include.jsp"/>

  </body>
</html>
```

Available configuration properties:

- `JspMvcFeature.TEMPLATE_BASE_PATH` -  
`jersey.config.server.mvc.templateBasePath.jsp`

The base path where JSP templates are located.

Maven users can find this module at coordinates

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-mvc-jsp</artifactId>
  <version>4.0.0</version>
</dependency>
```

and for non-Maven users the list of dependencies is available at `jersey-mvc-jsp` [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc-jsp/dependencies.html>].

## 22.7. Writing Custom Templating Engines

To add support for other (custom) templating engines into Jersey MVC Templating facility, you need to implement the `TemplateProcessor` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/spi/TemplateProcessor.html>] and register this class into your application.

### Tip

When writing template processors it is recommend that you use an appropriate unique suffix for the processable template references, in which case it is then possible to easily support mixing of multiple templating engines in a single application without conflicts.

**Example 22.13. Custom TemplateProcessor** [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/spi/TemplateProcessor.html>]

```
@Provider
class MyTemplateProcessor implements TemplateProcessor<String> {

    @Override
    public String resolve(String path, final MediaType mediaType) {
        final String extension = ".testp";

        if (!path.endsWith(extension)) {
            path = path + extension;
        }

        final URL u = this.getClass().getResource(path);
        return u == null ? null : path;
    }

    @Override
    public void writeTo(String templateReference,
                       Viewable viewable,
                       MediaType mediaType,
                       OutputStream out) throws IOException {
        final PrintStream ps = new PrintStream(out);
        ps.print("path=");
        ps.print(templateReference);
        ps.println();
        ps.print("model=");
        ps.print(viewable.getModel().toString());
        ps.println();
    }
}
```

**Example 22.14. Registering custom TemplateProcessor** [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/mvc/spi/TemplateProcessor.html>]

```
new ResourceConfig()
```

```
.register(MyTemplateProcessor.class)
// Further configuration of ResourceConfig.
.register( ... );
```

## Note

In a typical set-up projects using the Jersey MVC templating support would depend on the base module that provides the API and SPI and a single templating engine module for the templating engine of your choice. These modules need to be mentioned explicitly in your `pom.xml` file.

If you want to use just templating API infrastructure provided by Jersey for the MVC templating support in order to implement your custom support for a templating engine other than the ones provided by Jersey, you will need to add the base jersey-mvc [<https://eclipse-ee4j.github.io/jersey.github.io/project-info/4.0.0/jersey/project/jersey-mvc/dependencies.html>] module into the list of your dependencies:

```
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-mvc</artifactId>
  <version>4.0.0</version>
</dependency>
```

## 22.8. Other Examples

To see an example of MVC (JSP) templating support in Jersey refer to the MVC (Bookstore) Example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/bookstore-webapp>].



---

# Chapter 23. Logging

## 23.1. Logging traffic

### 23.1.1. Introduction

Jersey Logging supports the logging request and response via internal client and server filters, which are configured and registered by `LoggingFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html>] 's properties. `LoggingFeature` has been introduced in Jersey 2.23 version and deprecates an older `LoggingFilter`. Jersey of version 3.x fully support all logging features of Jersey 2.x.

`LoggingFeature` might be discovered by auto-discoverable mechanism or initialized by registering on client or server components. Client or server logging filter is initialized depending on which context is `LoggingFeature` registered with.

### 23.1.2. Configuration and registering

#### 23.1.2.1. Configuration options

##### Configurable options

- `Logger name`

Defines a logger used to log request and response messages.

Default value is `LoggingFeature.DEFAULT_LOGGER_NAME` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT\\_LOGGER\\_NAME](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT_LOGGER_NAME)].

- `Logger level`

Defines level that will be used to log messages by logging filters. Messages will be logged only if the effective level of the logger allows it.

Default value is `LoggingFeature.DEFAULT_LOGGER_LEVEL` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT\\_LOGGER\\_LEVEL](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT_LOGGER_LEVEL)].

- `Verbosity`

Verbosity determines how detailed message will be logged. See `LoggingFeature.Verbosity` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.Verbosity.html>] javadoc.

- The lowest verbosity `LoggingFeature.Verbosity.HEADERS_ONLY` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.Verbosity.html#HEADERS\\_ONLY](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.Verbosity.html#HEADERS_ONLY)] will log only request/response headers.
- The medium verbosity (`LoggingFeature.Verbosity.PAYLOAD_TEXT` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.Verbosity.html#PAYLOAD\\_TEXT](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.Verbosity.html#PAYLOAD_TEXT)]) will log request/response headers, as well as

an entity if considered a readable text. The entity is considered a readable text, if `MediaType` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/MediaType.html>] is `text/*` or is one of

- `application/atom+xml`
- `application/json`
- `application/svg+xml`
- `application/x-www-form-urlencoded`
- `application/xhtml+xml`
- `application/xml`
- The highest verbosity `LoggingFeature.Verbosity.PAYLOAD_ANY` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.Verbosity.html#PAYLOAD\\_ANY](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.Verbosity.html#PAYLOAD_ANY)] will log all types of an entity (besides the request/response headers).  
Note that the entity is logged up to the specified maximum number of bytes (see `LoggingFeature.LOGGING_FEATURE_MAX_ENTITY_SIZE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING\\_FEATURE\\_MAX\\_ENTITY\\_SIZE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_MAX_ENTITY_SIZE)]).

Default value is `LoggingFeature.DEFAULT_VERBOSITY` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT\\_VERBOSITY](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT_VERBOSITY)].

- **Maximum entity size**

Maximum number of entity bytes to be logged (and buffered) - if the entity is larger, logging filter will print (and buffer in memory) only the specified number of bytes and print "...more..." string at the end. Negative values are interpreted as zero.

Default value `LoggingFeature.DEFAULT_MAX_ENTITY_SIZE` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT\\_MAX\\_ENTITY\\_SIZE](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT_MAX_ENTITY_SIZE)].

- **Redact HTTP headers**

HTTP headers with sensitive information can be configured to print "[redacted]" in place of their real values. This should be a string with the names of the HTTP headers to be redacted, each entry separated by a semicolon (;). Header names will be compared in a case-insensitive manner and ignoring initial or trailing whitespaces.

Default value `LoggingFeature.DEFAULT_REDACT_HEADERS` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT\\_REDACT\\_HEADERS](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#DEFAULT_REDACT_HEADERS)].

### 23.1.2.2. Configuration properties

The feature is enabled on when auto-discoverable mechanism is not disabled and at least one of the feature's property is set. For enabling client or server logging filter one of the common properties or `_CLIENT` suffixed properties, or `_SERVER` properties respectively.

An example of initializing server-side logging with the highest verbosity.

**Example 23.1. Logging on the client side**

```
1      ClientConfig clientConfig = new ClientConfig();
2      clientConfig.property(LoggingFeature.LOGGING_FEATURE_VERBOSITY_CLIENT, Log
3      Client client = ClientBuilder.newClient(clientConfig);
4
```

The `LoggingFeature` might be registered explicitly on `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>] for server-side logging or on `Client` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/Client.html>] for client-side logging.

**Example 23.2. Register `LoggingFeature` via constructor**

```
1      ResourceConfig config = new ResourceConfig(HelloWorldResource.class);
2      config.register(new LoggingFeature(LOGGER, LoggingFeature.Verbosity.PA
3
```

Following examples demonstrate registering `LoggingFeature` on server-side with default values and values defined by one of the public constructors (see `LoggingFeature` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html>]).

**Example 23.3. Register `LoggingFeature` class**

```
1      ResourceConfig config = new ResourceConfig(HelloWorldResource.class);
2      config.register(LoggingFeature.class);
3
```

An example of server-side logging with entity `Hello World!`

```
1 May 09, 2020 2:55:33 PM org.glassfish.jersey.logging.LoggingInterceptor log
2 INFO: 1 * Server has received a request on thread grizzly-http-server-0
3 1 > GET http://localhost:9998/helloworld
4 1 > accept: text/plain
5 1 > accept-encoding: gzip, deflate
6 1 > connection: Keep-Alive
7 1 > host: localhost:9998
8 1 > user-agent: Jersey/3.0.0 (Apache HttpClient 4.5.9)
9
10 May 09, 2020 2:55:33 PM org.glassfish.jersey.logging.LoggingInterceptor log
11 INFO: 1 * Server responded with a response on thread grizzly-http-server-0
12 1 < 200
13 1 < Content-Type: text/plain
14 Hello World!
```

---

# Chapter 24. Monitoring and Diagnostics

## 24.1. Monitoring Jersey Applications

### 24.1.1. Introduction

#### Important

Jersey monitoring support has been released as a *beta release* in Jersey 2.1 version. As such, the exposed monitoring public APIs and functionality described in this section may change in the future Jersey releases.

Jersey provides functionality for monitoring JAX-RS/Jersey applications. Application monitoring is useful when you need to identify the performance hot-spots in your JAX-RS application, observe execution statistics of particular resources or listen to application or request lifecycle events. Note that this functionality is Jersey-specific extension to JAX-RS API.

Jersey monitoring support is divided into three functional areas:

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Event Listeners            | Event listeners allow users to receive and process a predefined set of events that occur during an application lifecycle (such as application initialization, application destroy) as well as request processing lifecycle events (request started, resource method finished, exception thrown, etc.). This feature is always enabled in Jersey server runtime and is leveraged by the other monitoring features.                                                                                                                                                                                                                                                                                                                                                                                         |
| Monitoring Statistics      | Jersey can be configured to process lifecycle events in order to expose a wide range of runtime monitoring statistics to the end user. The statistics are accessible through an injectable <code>MonitoringStatistics</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatistics.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatistics.html</a> ] interface. The statistics provide general information about the application as well as fine-grained execution statistics on particular resources and sub resources and exposed URIs. For performance reasons, this functionality must be explicitly enabled prior using. |
| JMX MBeans with statistics | In addition to the injectable <code>MonitoringStatistics</code> data, Jersey is able to expose the statistics as JMX MBeans (for example <code>ApplicationMXBean</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ApplicationMXBean.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ApplicationMXBean.html</a> ]). Jersey monitoring MBeans can be accessed programmatically using JMX APIs or browsed via JMX-enabled tool ( <code>JConsole</code> for example). This functionality is also disabled by default for performance reasons and must be enabled if needed.                                                                              |

All monitoring related APIs (beta!) can be found in the `jersey-server` module in `org.glassfish.jersey.server.monitoring` package. Monitoring in Jersey is currently supported on the server side.

## 24.1.2. Event Listeners

Jersey defines two types of event listeners that you can implement and register with your application:

- `ApplicationEventListener` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ApplicationEventListener.html>] for listening to application events, and
- `RequestEventListener` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/RequestEventListener.html>] for listening to events of request processing.

Only the first type, `ApplicationEventListener` can be directly registered as an application-wide provider. The `RequestEventListener` is designed to be specific to every request and can be only returned from the `ApplicationEventListener` as such.

Let's start with an example. The following examples show simple implementations of Jersey event listeners as well as a test JAX-RS resource that will be monitored.

### Example 24.1. Application event listener

```
1 public class MyApplicationEventListener
2     implements ApplicationEventListener {
3     private volatile int requestCnt = 0;
4
5     @Override
6     public void onEvent(ApplicationEvent event) {
7         switch (event.getType()) {
8             case INITIALIZATION_FINISHED:
9                 System.out.println("Application "
10                     + event.getResourceConfig().getApplicationName()
11                     + " was initialized.");
12                 break;
13             case DESTROY_FINISHED:
14                 System.out.println("Application "
15                     + event.getResourceConfig().getApplicationName() + " destroyed");
16                 break;
17         }
18     }
19
20     @Override
21     public RequestEventListener onRequest(RequestEvent requestEvent) {
22         requestCnt++;
23         System.out.println("Request " + requestCnt + " started.");
24         // return the listener instance that will handle this request.
25         return new MyRequestEventListener(requestCnt);
26     }
27 }
```

### Example 24.2. Request event listener

```
1 public class MyRequestEventListener implements RequestEventListener {
2     private final int requestNumber;
3     private final long startTime;
```

```
4
5     public MyRequestEventListener(int requestNumber) {
6         this.requestNumber = requestNumber;
7         startTime = System.currentTimeMillis();
8     }
9
10    @Override
11    public void onEvent(RequestEvent event) {
12        switch (event.getType()) {
13            case RESOURCE_METHOD_START:
14                System.out.println("Resource method "
15                    + event.getUriInfo().getMatchedResourceMethod()
16                    .getHttpMethod()
17                    + " started for request " + requestNumber);
18                break;
19            case FINISHED:
20                System.out.println("Request " + requestNumber
21                    + " finished. Processing time "
22                    + (System.currentTimeMillis() - startTime) + " ms.");
23                break;
24        }
25    }
26 }
```

### Example 24.3. Event listener test resource

```
1 @Path("resource")
2 public class TestResource {
3     @GET
4     public String getSomething() {
5         return "get";
6     }
7
8     @POST
9     public String postSomething(String entity) {
10        return "post";
11    }
12 }
```

Once the listeners and the monitored resource are defined, it's time to initialize our application. The following piece of code shows a `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>] that is used to initialize the application (please note that only `ApplicationEventListener` is registered as provider).

```
1 ResourceConfig resourceConfig =
2     new ResourceConfig(TestResource.class, MyApplicationEventListener.
3         .setApplicationName("my-monitored-application");
```

Our example application now contains a simple resource `TestResource` that defines resource methods for GET and POST and a custom `MyApplicationEventListener` event listener.

The registered `MyApplicationEventListener` implements two methods defined by the `ApplicationEventListener` interface. A method `onEvent()` handles all application lifecycle events. In our case the method handles only 2 application events - initialization and destroy. Other event types are ignored. All application event types are defined

in `ApplicationEvent` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ApplicationEvent.html>]. `Type`. The second method `onRequest` is invoked by Jersey runtime every time a new request is received. The request event type passed to the method is always `START`. If you want to listen to any other request lifecycle events for the new request, you are expected to return an instance of `RequestEventListener` that will handle the request. It is important to understand, that the instance will handle only the request for which it has been returned from an `ApplicationEventListener.onRequest` method and not any other requests. In our case the returned request event listener keeps information about the request number of the current request and a start time of the request which is later used to print out the request processing times statistics. This demonstrates the principle of listening to request events: for one request there is one instance which can be used to hold all the information about the particular request. In other words, `RequestEventListener` is designed to be implicitly request-scoped.

Jersey represents lifecycle events via `RequestEvent` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/RequestEvent.html>] and `ApplicationEvent` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ApplicationEvent.html>] types. Instances of these classes contain information about respective events. The most important information is the event type `Type` retrievable via `getType()` method, which identifies the type of the event. Events contain also additional information that is dependent on a particular event type. This information can be retrieved via event getters. Again, some getters return valid information for all event types, some are specific to a sub-set of event types. For example, in the `RequestEvent`, the `getExceptionCause()` method returns valid information only when event type is `ON_EXCEPTION`. On the other hand, a `getContainerRequest()` can be used to return current request context for any request event type. See javadoc of events and event types to get familiar with event types and information valid for each event type.

Our `MyRequestEventListener` implementation is focused on processing 2 request events. First, it listens for an event that is triggered before a resource method is executed. Also, it hooks to a "request finished" event. As mentioned earlier, the request event `START` is handled only in the `MyApplicationEventListener`. The `START` event type will never be invoked on `RequestEventListener`. Therefore the logic for measuring the `startTime` is in the constructor which is invoked from `MyApplicationEventListener.onRequest()`. An attempt to handling the request `START` event in a `RequestEventListener.onEvent()` method would be a mistake.

Let's deploy the application and use a simple test client code to produce some activity in order to spawn new events:

```
1 target.path("resource").request()  
2     .post(Entity.entity("post", MediaType.TEXT_PLAIN_TYPE));  
3     target.path("resource").request().get();
```

In the code above, the `target` is a `WebTarget` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/client/WebTarget.html>] instance pointing to the application context root path. Using the Chapter 5, *Client API*, we invoke `GET` and `POST` methods on the `MyResource` JAX-RS resource class that we implemented earlier.

When we start the application, run the test client and then stop the application, the console output for the deployed server-side application would contain the following output:

```
Application my-monitored-application was initialized.  
Request 1 started.  
Resource method POST started for request 1  
Request 1 finished. Processing time 330 ms.  
Request 2 started.  
Resource method GET started for request 2
```

Request 2 finished. Processing time 4 ms.  
Application my-monitored-application destroyed.

### 24.1.2.1. Guidelines for implementing Jersey event listeners

- Implement event listeners as thread safe. While individual events will be arriving serially, individual listener invocations may occur from different threads. Thus make sure that your listeners are processing data safely with respect to their Java Memory Model [[http://en.wikipedia.org/wiki/Java\\_Memory\\_Model](http://en.wikipedia.org/wiki/Java_Memory_Model)] visibility (in the example above the fields `requestNumber`, `startTime` of `MyRequestEventListener` are final and therefore the same value is visible for all threads executing the `onEvent()` method).
- Do not block the thread executing the event listeners by performing long-running tasks. Execution of event listeners is a part of the standard application and request processing and as such needs to finish as quickly as possible to avoid negative impact on overall application performance.
- Do not try to modify mutable objects returned from `ApplicationEvent` and `RequestEvent` getters to avoid experiencing undefined behavior. Events listeners should use the information for read only purposes only. Use different techniques like filters, interceptors or other providers to modify the processing of requests and applications. Even though modification might be possible and might work as desired now, your code is in risk of producing intermittent failures or unexpected behaviour (for example after migrating to new Jersey version).
- If you do not want to listen to request events, do not return an empty listener in the `onRequest()` method. Return `null` instead. Returning empty listener might have a negative performance impact. Do not rely on JIT optimizing out the empty listener invocation code.
- If you miss any event type or any detail in the events, let us know via Jersey user mailing list.

### 24.1.2.2. Monitoring Statistics

Event listeners described in the previous section are all-purpose facility. For example, you may decide to use them to measure various execution statistics of your application. While this might be an easy task for simple statistics like "how much time was spent on execution of each Java method?", nevertheless, if you want to measure statistics based on URIs and individual resources, the implementation might get rather complex soon, especially when considering sub-resources and sub-resource locators. To save you the trouble, Jersey provides feature for collecting events and calculating a pre-defined set of monitoring and execution statistics, including application configuration, exception mappers execution, minimum/maximum/average execution times for individual resource methods as well as entire request processing etc.

Calculating the monitoring statistics has obviously a performance impact, therefore this feature is disabled by default. To enable the feature, set the following configuration property to `true`:

```
jersey.config.server.monitoring.statistics.enabled=true
```

The property description can be found in `ServerProperties.MONITORING_STATISTICS_ENABLED` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING\\_STATISTICS\\_ENABLED](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_STATISTICS_ENABLED)] This will calculate the statistics. The easiest way how to get statistics is to let Jersey to inject them. See the following example:

#### Example 24.4. Injecting MonitoringStatistics

```
1 @Path("resource")
2 public static class StatisticsResource {
```



```
3    @Inject
4    Provider<MonitoringStatistics> monitoringStatisticsProvider;
5
6    @GET
7    public String getSomething() {
8        final MonitoringStatistics snapshot
9            = monitoringStatisticsProvider.get().snapshot();
10
11        final TimeWindowStatistics timeWindowStatistics
12            = snapshot.getRequestStatistics()
13                .getTimeWindowStatistics().get(0l);
14
15        return "request count: " + timeWindowStatistics.getRequestCount()
16            + ", average request processing [ms]: "
17            + timeWindowStatistics.getAverageDuration();
18    }
19 }}
```

`MonitoringStatistics` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatistics.html>] are injected into the resource using an `@Inject` [<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/inject>] annotation. Please note the usage of the `Provider` [<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/provider>] for injection (it will be discussed later). Firstly, the snapshot of statistics is retrieved by the `snapshot()` method. The snapshot of statistics is an immutable copy of statistics which does not change over the time. Additionally, data in a snapshot are consistent. It's recommended to create snapshots before working with the statistics data and then process the snapshot data. Working with original non-snapshot data makes sense when data consistency is not important and performance is of highest concern. While it is currently not the case, the injected non-snapshot data may be implemented as mutable for performance reasons in a future release of Jersey.

The injected monitoring statistics represent the root of the collected statistics hierarchy. The hierarchy can be traversed to retrieve any partial statistics data. In the example, we retrieve certain request `TimeWindowStatistics` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/TimeWindowStatistics.html>] data. In our case, those are the request execution statistics for a time window defined by long value 0 which means unlimited time window. This means we are retrieving the global request execution statistics measured since a start of the application. Finally, request count and average duration from the statistics are used to produce the String response. When we invoke few GET requests on the `StatisticsResource`, we get the following console output:

```
request count: 1, average request processing [ms]: 260
request count: 2, average request processing [ms]: 135
request count: 3, average request processing [ms]: 93
request count: 4, average request processing [ms]: 73
```

Let's look closer at `MonitoringStatistics` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatistics.html>] interface. `MonitoringStatistics` interface defines getters by which other nested statistics can be retrieved. All statistics are in the same package and ends with `Statistics` postfix. Statistics interfaces are the following:

```
MonitoringStatistics [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/
main top level statistics
```

jersey/server/monitoring/  
MonitoringStatistics.html]

ResponseStatistics [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ResponseStatistics.html>] response statistics (eg. response status codes and their count)

ResourceStatistics [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ResourceStatistics.html>] statistics of execution of resources (resource classes or resource URIs)

ResourceMethodStatistics [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ResourceMethodStatistics.html>] statistics of execution of resource methods

ExecutionStatistics [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ExecutionStatistics.html>] statistic of execution of a target (resource, request, resource method)

TimeWindowStatistics [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/TimeWindowStatistics.html>] statistics of execution time in specific interval (eg. executions in last 5 minutes)

Each time-monitored target contains `ExecutionStatistics`. So, for example resource method contains execution statistics of its execution. Each `ExecutionStatistics` contains multiple `TimeWindowStatistics`. Currently, each `ExecutionStatistics` contains `TimeWindowStatistics` for these time windows:

- 0: unlimited=> all execution since start of the application
- 1000: 1s => stats measured in last 1 second
- 15000: 15s => stats measured in last 15 seconds
- 60000: 1min => stats measured in last 1 minute
- 900000: 15min => stats measured in last 15 minutes
- 3600000: 1hour => stats measured in last hour minutes

All the time window statistics can be retrieved from a `Map<Long, TimeWindowStatistics>` map returned from `ExecutionStatistics.getTimeWindowStatistics()`. Key of the map is the number of milliseconds of interval (so, for example key 60000 points to statistics for last one minute).

Note, that `snapshot()` method was called in the example only on the top level `MonitoringStatistics`. This produced a snapshot of the entire tree of statistics and therefore we do not need to call `snapshot()` on `TimewindowStatistics` again.

Statistics are injected using the Provider [<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/provider>]. This is preferred way of injecting statistics. The reason is simple. Statistics might change over time and contract of `MonitoringStatistics` does not make any assumptions about mutability of monitoring statistics instances (to allow future optimizations and changes in implementation strategy). In order to get always latest statistics, we recommend injecting a Provider [<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/provider>] rather than a direct reference and use its `get()` method to retrieve the latest statistics. For example, in singleton resources the use of the technique is very important otherwise statistics might correspond to the time when singleton was firstly created and might not update since that time.

#### 24.1.2.2.1. Listening to statistics changes

Statistics are not calculated for each request or each change. Statistics are calculated only from the collected data in regular intervals for performance reasons (for example once per second). If you want to be notified about new statistics, register an implementation of `MonitoringStatisticsListener` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatisticsListener.html>] as one of your custom application providers. Your listener will be called every time the new statistics are calculated and the updated statistics data will be passed to the listener method. This is another way of receiving statistics. See the linked listener API documentation for more information.

#### 24.1.2.3. Monitoring Statistics as MBeans

Jersey provides feature to expose monitoring statistics as JMX MBeans. In order to enable monitoring statistics MBeans exposure, the `ServerProperties.MONITORING_STATISTICS_MBEANS_ENABLED` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING\\_STATISTICS\\_MBEANS\\_ENABLED](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_STATISTICS_MBEANS_ENABLED)] must be set to `true`.

```
jersey.config.server.monitoring.statistics.mbeans.enabled=true
```

Note that enabling exposure of monitoring MBeans causes that also the calculation of `MonitoringStatistics` is automatically enabled as the exposed MBean statistics are extracted from `MonitoringStatistics`.

The easiest way is to browse the MBeans in the JConsole. Open the JConsole (`$JAVA_HOME/bin/jconsole`). Then connect to the process where JAX-RS application is running (server on which the application is running). Switch to a MBean tab and in the MBean tree on the left side find a group `org.glassfish.jersey`. All deployed Jersey applications are located under this group. If you don't see such this group, then MBeans are not exposed (check the configuration property and logs if they not contain any exceptions or errors). The following figure is an example of an output from the JConsole:

The screenshot shows the Java Monitoring & Management Console. The left pane displays a tree view of MBeans under the `org.glassfish.jersey` group. The right pane shows the 'Attributes' sub-group of the 'myApplication' resource, displaying a table of attribute values.

| Name                                  | Value              |
|---------------------------------------|--------------------|
| AverageTime[ms].15m                   | 4                  |
| AverageTime[ms].15s                   | 0                  |
| AverageTime[ms].1h                    | 5                  |
| AverageTime[ms].1m                    | 0                  |
| AverageTime[ms].1s                    | 0                  |
| AverageTime[ms].total                 | 4                  |
| MaxTime[ms].15m                       | 220                |
| MaxTime[ms].15s                       | 1                  |
| MaxTime[ms].1h                        | 220                |
| MaxTime[ms].1m                        | 1                  |
| MaxTime[ms].1s                        | 0                  |
| MaxTime[ms].total                     | 220                |
| MinTime[ms].15m                       | 0                  |
| MinTime[ms].15s                       | 0                  |
| MinTime[ms].1h                        | 0                  |
| MinTime[ms].1m                        | 0                  |
| MinTime[ms].1s                        | 0                  |
| MinTime[ms].total                     | 0                  |
| RequestCount.15m                      | 50                 |
| RequestCount.15s                      | 9                  |
| RequestCount.1h                       | 45                 |
| RequestCount.1m                       | 38                 |
| RequestCount.1s                       | 1                  |
| RequestCount.total                    | 51                 |
| RequestRate[requestsPerSeconds].15m   | 0.6172839506172839 |
| RequestRate[requestsPerSeconds].15s   | 0.6                |
| RequestRate[requestsPerSeconds].1h    | 0.625              |
| RequestRate[requestsPerSeconds].1m    | 0.6333333333333333 |
| RequestRate[requestsPerSeconds].1s    | 1.0                |
| RequestRate[requestsPerSeconds].total | 0.620271946680937  |

Under the root `org.glassfish.jersey` Jersey MBean group you can find your application. If the server contains more Jersey application, all will be present under the root Jersey group. In the screen-shot, the deployed JAX-RS application is named `myApplication` (the name can be defined via `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>] directly or by setting the `ServerProperties.APPLICATION_NAME` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#APPLICATION\\_NAME](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#APPLICATION_NAME)] property). Each application contains `Global`, `Resource` and `Uri` sub-groups. The `Global` group contains all global statistics like overall requests statistics of the entire application (`AllRequestTimes`), configuration of the JAX-RS application (`Configuration`), statistics about `ExceptionHandler` <E extends Throwable> [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ExceptionHandler.html>] execution (`ExceptionHandler`) and statistics about produced responses (`Responses`).

`Resources` and `Uri` groups contains monitoring statistics specific to individual resources. Statistics in `Resources` are bound to the JAX-RS resource Java classes loaded by the application. `Uri` contains statistics of resources based on the matched application `Uri` (one `Uri` entry represents all methods bound to the particular `Uri`, e.g. `/resource/exception`). As Jersey provides programmatic resource builders (described in the chapter "Programmatic API for Building Resources"), one Java resource class can be an endpoint for resource methods on many different `URIs`. And also one `Uri` can be served by method from many different Java classes. Therefore both views are not to be compared 1:1. Instead they provide different logical views on your JAX-RS application. This monitoring feature can also be helpful when designing the JAX-RS APIs as it provides nice view on available root application `URIs`.

Both logical views on the resources exposed by application share few common principles. A single resource entry is always a set of resource methods which are available under the `methods` sub-group.

Statistics can be found in MBeans `MethodTimes` and `RequestTimes`. `MethodTimes` contains statistics measured on resource methods (duration of execution of a code of the a resource method), whereas `RequestTimes` contains statistics of an entire request execution (not only a time of the execution of the resource method but the overall time of the execution of whole request by Jersey runtime). Another useful information is that statistics directly under resource (not under the `methods` sub-group) contains summary of statistics for all resource methods grouped in the resource entry.

Additional useful details about statistics

- `Global->Configuration->Registered(Classes/Instances)`: registered resource classes and instances by the user (i.e., not added by `ModelProcessor` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/model/ModelProcessor.html>] during deployment for example).
- `Global->ExceptionHandler->ExceptionHandlerCount`: map that contains exception mapper classes as keys and number of their execution as values.
- `Global->Responses->ResponseCodesToCountMap`: map that contains response codes as keys and their total occurrence in responses as values.
- Resource groups contain also entries for resources that were added by Jersey at deployment time using `ModelProcessor` (for example all `OPTIONS` methods, `WADL`). `HEAD` methods are not present in the `MXBeans` view (even `HEAD` methods are in resources).
- Execution statistics for different time windows have different update intervals. The shorter the time window, the shorter the update interval. This causes that immediately after the application start, the shorter time windows (such as 15 seconds) may contain higher values than longer ones (e.g. 1 hour time window). The reason is that 1 hour interval will show information that is not up to date and therefore has smaller value. This inconsistency is not so much significant when application is running longer time. Total unlimited time windows contains always up-to-date data. This inconsistency will get fixed in a future Jersey release.

`MXBeans` can be also accessed using `JMX`. To do so, you would need to use the interfaces of `MXBeans`. These interfaces are even useful when working with `MXBeans` only trough `JConsole` as they contain Javadocs for each `MXBean` and attribute. Monitoring MBeans are defined by following interfaces:

- `ApplicationMXBean` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ApplicationMXBean.html>]: contains configuration statistics
- `ExceptionHandlerMXBean` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ExceptionHandlerMXBean.html>]: contains statistics of exception mappers
- `ResourceMethodMXBean` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ResourceMethodMXBean.html>]: contains statistics of resource method
- `ResourceMXBean` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ResourceMXBean.html>]: contains statistics of resource
- `ResponseMXBean` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ResponseMXBean.html>]: contains statistics of responses

The list does not contain `MXBean` for the execution and time window statistics. The reason is that this bean is defined as a `DynamicMBean` [<http://docs.oracle.com/javase/6/docs/api/javax/management/DynamicMBean.html>]. Attributes of this dynamic MBean contains statistics for all time windows available.

MXBeans do not reference each other but can be retrieved by their `ObjectName` [<http://docs.oracle.com/javase/6/docs/api/javax/management/ObjectName.html>]s which are designed in the way, that final MBean tree looks nicely organized in *JConsole*. Each MXBean is uniquely identified by its `ObjectName` and properties of `ObjectName` are structured hierarchically, so that each MXBean can be identified to which parent it belong to (e.g. execution statistics dynamic MXBean belongs to resource method MXBean, which belongs to resource and which belongs to application). Check the `ObjectNames` of exposed MXBeans to investigate the structure (for example through *JConsole*).

To reiterate, exposing Jersey MXBeans and the calculating monitoring statistics may have an performance impact on your application and therefore should be enabled only when needed. Also, please note, that Jersey monitoring is exposing quite a lot of information about the monitored application which might be viewed as problematic in some cases (e.g. in production server deployments).

## 24.2. Tracing Support

Apart from monitoring and collecting application statistics described in Section 24.1, “Monitoring Jersey Applications”, Jersey can also provide tracing or diagnostic information about server-side processing of individual requests. This facility may provide vital information when troubleshooting your misbehaving Jersey or JAX-RS application. When enabled, Jersey tracing facility collects useful information from all parts of JAX-RS server-side request processing pipeline: `PreMatchRequestFilter`, `ResourceMatching`, `RequestFilter`, `ReadIntercept`, `MBR`, `Invoke`, `ResponseFilter`, `WriteIntercept`, `MBW`, as well as `ExceptionHandling`.

The collected tracing information related to a single request is returned to the requesting client in the HTTP headers of a response for the request. The information is also logged on the server side using a dedicated Java Logger instance.

### 24.2.1. Configuration options

Tracing support is disabled by default. You can enable it either "globally" for all application requests or selectively per request. The tracing support activation is controlled by setting the `jersey.config.server.tracing.type` application configuration property. The property value is expected to be one of the following:

- **OFF** - tracing support is disabled (default value).
- **ON\_DEMAND** - tracing support is in a stand-by mode; it is enabled selectively per request, via a special `X-Jersey-Tracing-Accept` HTTP request header.
- **ALL** - tracing support is enabled for all request.

The level of detail of the information provided by Jersey tracing facility - the tracing threshold - can be customized. The tracing threshold can be set at the application level via `jersey.config.server.tracing.threshold` application configuration property, or at a request level, via `X-Jersey-Tracing-Threshold` HTTP request header. The request level configuration overrides any application level setting. There are 3 supported levels of detail for Jersey tracing:

- **SUMMARY** - very basic summary information about the main request processing stages.
- **TRACE** - detailed information about activities in all the main request processing stages (default threshold value).
- **VERBOSE** - most verbose mode that provides extended information similar to **TRACE** level, however with details on entity providers (`MBR/MBW`) that were skipped during the provider selection phase for any

reason (lower priority, pattern matching, etc). Additionally, in this mode all received request headers are echoed as part of the tracing information.

## 24.2.2. Tracing Log

As mentioned earlier, all tracing information is also logged using a dedicated Java Logger. The individual tracing messages are logged immediately as the tracing events occur. The default name of the tracing logger is prefixed `org.glassfish.jersey.tracing.` with a default suffix `general`. This logger name can be customized per request by including a `X-Jersey-Tracing-Logger` HTTP request header as will be shown later.

## 24.2.3. Configuring tracing support via HTTP request headers

Whenever the tracing support is active (`ON_DEMAND` or `ALL`) you can customize the tracing behaviour by including one or more of the following request HTTP headers in your individual requests:

- `X-Jersey-Tracing-Accept` - used to enable the tracing support for the particular request. It is applied only when the application-level tracing support is configured to `ON_DEMAND` mode. The value of the header is not used by the Jersey tracing facility and as such it can be any arbitrary (even empty) string.
- `X-Jersey-Tracing-Threshold` - used to override the tracing level of detail. Allowed values are: `SUMMARY`, `TRACE`, `VERBOSE`.
- `X-Jersey-Tracing-Logger` - used to override the tracing Java logger name suffix.

## 24.2.4. Format of the HTTP response headers

At the end of request processing all tracing messages are appended to the HTTP response as individual headers named `X-Jersey-Tracing-nnn` where *nnn* is index number of message starting at 0.

Each tracing message is in the following format: `CATEGORY [TIME] TEXT`, e.g.

`X-Jersey-Tracing-007: WI [85.95 / 183.69 ms | 46.77 %] WriteTo summary: 4`

The `CATEGORY` is used to categorize tracing events according to the following event types:

- `START` - start of request processing information
- `PRE-MATCH` - pre-matching request filter processing
- `MATCH` - matching request URI to a resource method
- `REQ-FILTER` - request filter processing
- `RI` - entity reader interceptor processing
- `MBR` - message body reader selection and invocation
- `INVOKE` - resource method invocation
- `RESP-FILTER` - response filter processing
- `WI` - write interceptor processing

- MBW - message body writer selection and invocation
- MVC - template engine integration
- EXCEPTION - exception mapping
- FINISHED - processing finish summary

The `TIME`, if present, is a composite value that consists of 3 parts [ `duration` / `time_from_start` | `total_req_ratio` ]:

1. `duration` - the duration of the current trace event [milliseconds]; e.g. duration of filter processing
2. `time_from_start` - the end time of the current event with respect to the request processing start time [milliseconds]
3. `total_req_ratio` - the duration of the current event with respect to the total request processing time [percentage]; this value tells you how significant part of the whole request processing time has been spent in the processing phase described by the current event

There are certain tracing events that do not have any duration. In such case, duration values are not set (---- literal).

The tracing event `TEXT` is a free-form detailed text information about the current diagnostic event.

## Tip

For better identification, instances of JAX-RS components are represented by class name, identity hash code and `@Priority` value if set, e.g. `[org.glassfish.jersey.tests.integration.tracing.ContainerResponseFilter5001@494a8227 #5001]`.

## 24.2.5. Tracing Examples

Example of SUMMARY level messages from `tests/integration/tracing-support` module:

### Example 24.5. Summary level messages

```

1 $ curl -i http://localhost:9998/ALL/root/sub-resource-locator/sub-resource-met
2
3 X-Jersey-Tracing-000: START      [ ---- / ---- ms | ---- %] baseUrl=[http:/
4 X-Jersey-Tracing-001: PRE-MATCH [ 0.01 / 0.68 ms | 0.01 %] PreMatchRequest
5 X-Jersey-Tracing-002: MATCH     [ 8.44 / 9.15 ms | 4.59 %] RequestMatching
6 X-Jersey-Tracing-003: REQ-FILTER [ 0.01 / 9.20 ms | 0.00 %] Request summary
7 X-Jersey-Tracing-004: RI        [86.14 / 95.49 ms | 46.87 %] ReadFrom summar
8 X-Jersey-Tracing-005: INVOKE    [ 0.04 / 95.70 ms | 0.02 %] Resource [org.g
9 X-Jersey-Tracing-006: RESP-FILTER [ 0.01 / 96.55 ms | 0.00 %] Response summar
10 X-Jersey-Tracing-007: WI        [85.95 / 183.69 ms | 46.77 %] WriteTo summar
11 X-Jersey-Tracing-008: FINISHED  [ ---- / 183.79 ms | ---- %] Response statu

```

Example TRACE level messages of `jersey-mvc-jsp` integration, from `examples/bookstore-webapp` module:

### Example 24.6. On demand request, snippet of MVC JSP forwarding

```

1 $ curl -i http://localhost:9998/items/3/tracks/0 -H X-Jersey-Tracing-Accept:wh

```



```
2
3 ...
4 X-Jersey-Tracing-033: WI      [ 0.00 / 23.39 ms | 0.02 %] [org.glassfish.
5 X-Jersey-Tracing-034: WI      [ 0.01 / 23.42 ms | 0.02 %] [org.glassfish.
6 X-Jersey-Tracing-035: MBW     [ ---- / 23.45 ms | ---- %] Find MBW for ty
7 X-Jersey-Tracing-036: MBW     [ ---- / 23.52 ms | ---- %] [org.glassfish.
8 X-Jersey-Tracing-037: MVC     [ ---- / 24.05 ms | ---- %] Forwarding view
9 X-Jersey-Tracing-038: MBW     [ 1.09 / 24.63 ms | 4.39 %] WriteTo by [org
10 X-Jersey-Tracing-039: WI      [ 0.00 / 24.67 ms | 0.01 %] [org.glassfish.
11 X-Jersey-Tracing-040: WI      [ 0.00 / 24.70 ms | 0.01 %] [org.glassfish.
12 ...
```

---

# Chapter 25. Custom Injection and Lifecycle Management

Since version 2.0, Jersey uses Glassfish-HK2 [<https://eclipse-ee4j.github.io/glassfish-hk2/>] library for component life cycle management and dependency injection. Rather than spending a lot of effort in maintaining Jersey specific API (as it used to be before Jersey 2.0 version), Jersey defines several extension points where end-user application can directly manipulate Jersey HK2 bindings using the HK2 public API to customize life cycle management and dependency injection of application components.

Jersey user guide can by no means supply an exhaustive documentation of HK2 API in its entire scope. This chapter only points out the most common scenarios related to dependency injection in Jersey and suggests possible options to implement these scenarios. It is highly recommended to check out the Glassfish-HK2 [<https://eclipse-ee4j.github.io/glassfish-hk2/>] website and read HK2 documentation in order to get better understanding of suggested approaches. HK2 documentation should also help in resolving use cases that are not discussed in this writing.

There are typically three main use cases, where your application may consider dealing with HK2 APIs exposed in Jersey:

- Implementing a custom injection provider that allows an application to define additional types to be injectable into Jersey-managed JAX-RS components.
- Defining a custom injection annotation (other than `@Inject` [<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/inject>] or `@Context` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>]) to mark application injection points.
- Specifying a custom component life cycle management for your application components.

Since Jersey 2.26, the injection has been abstracted, so that Glassfish-HK2 [<https://eclipse-ee4j.github.io/glassfish-hk2/>] can be eventually replaced by the CDI or any other injection framework. In the next chapters, we document possibilities provided directly by Glassfish-HK2 [<https://eclipse-ee4j.github.io/glassfish-hk2/>] and by Jersey abstraction components.

## 25.1. InjectionManager

Since Jersey 2.26, Jersey comes with the main abstraction interface to communicate with the DI container, the `InjectionManager` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectionManager.html>]. What is `ServiceLocator` for Glassfish-HK2 [<https://eclipse-ee4j.github.io/glassfish-hk2/>], or `BeanManager` for CDI, that's `InjectionManager` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectionManager.html>] for Jersey.

`InjectionManager` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectionManager.html>] can be injected into the user provided classes instantiated by Jersey. It can also be obtained programmatically by `InjectionManagerClientProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/InjectionManagerClientProvider.html>] and `InjectionManagerProvider` from Jakarta REST components, such as `FeatureContext` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/FeatureContext.html>], or `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>] and `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>].

Customers used to the ServiceLocator [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/ServiceLocator.html>] can still use it directly; the ServiceLocator [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/ServiceLocator.html>] can be obtained either directly by injection, or programmatically as `InjectionManager.getInstance(ServiceLocator.class)`.

## 25.2. Implementing Custom Injection Provider

Relying on Servlet HTTP session concept is not very RESTful. It turns the originally state-less HTTP communication schema into a state-full manner. However, it could serve as a good example that will help me demonstrate implementation of the use cases described above. The following examples should work on top of Jersey Servlet integration module. The approach that will be demonstrated could be further generalized. Below we will show how to make actual Servlet HttpSession [<https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/httpsession>] injectable into JAX-RS components and how to make this injection work with a custom inject annotation type. Finally, we will demonstrate how you can write HttpSession-scoped JAX-RS resources.

Jersey implementation allows you to directly inject `HttpServletRequest` [<https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/httpServletRequest>] instance into your JAX-RS components. It is quite straight forward to get the appropriate `HttpSession` instance out of the injected request instance. Let say, you want to get `HttpSession` instance directly injected into your JAX-RS types like in the code snippet below.

```
@Path("di-resource")
public class MyDiResource {

    @Inject HttpSession httpSession;

    ...

}
```

### 25.2.1. Using HK2 classes

To make the above injection work, you will need to define an additional HK2 binding in your application ResourceConfig [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>]. Let's start with a custom HK2 Factory [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/Factory.html>] implementation that knows how to extract `HttpSession` out of given `HttpServletRequest`.

```
import org.glassfish.hk2.api.Factory;

...

public class HttpSessionFactory implements Factory<HttpSession> {

    private final HttpServletRequest request;

    @Inject
    public HttpSessionFactory(HttpServletRequest request) {
        this.request = request;
    }

    @Override
    public HttpSession provide() {
        return request.getSession();
    }
}
```

```
    }

    @Override
    public void dispose(HttpSession t) {
    }
}
```

Please note that the factory implementation itself relies on having the actual `HttpServletRequest` instance injected. In your implementation, you can of course depend on other types (and inject them conveniently) as long as these other types are bound to the actual HK2 service locator by Jersey or by your application. The key notion to remember here is that your HK2 **Factory** implementation is responsible for implementing the `provide()` method that is used by HK2 runtime to retrieve the injected instance. Those of you who worked with Guice binding API in the past will most likely find this concept very familiar.

Once implemented, the factory can be used in a custom HK2 **Binder** to define the new injection binding for `HttpSession`. Finally, the implemented binder can be registered in your `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>]:

```
import org.glassfish.hk2.utilities.binding.AbstractBinder;
...

public class MyApplication extends ResourceConfig {

    public MyApplication() {

        ...

        register(new AbstractBinder() {
            @Override
            protected void configure() {
                bindFactory(HttpSessionFactory.class).to(HttpSession.class)
                    .proxy(true).proxyForSameScope(false).in(RequestScoped.class);
            }
        });
    }
}
```

Note that if we did not define any explicit injection scope for the new injection binding, By default, HK2 factories are bound in a HK2 `PerLookup` [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/PerLookup.html>] scope, which is in most cases a good choice, and it is suitable also in our example.

To summarize the approach described above, here is a list of steps to follow when implementing custom injection provider in your Jersey application :

- Implement your own HK2 **Factory** to provide the injectable instances.
- Use the HK2 **Factory** to define an injection binding for the injected instance via custom HK2 **Binder**.
- Register the custom HK2 **Binder** in your application `ResourceConfig`.

While the **Factory**-based approach is quite straight-forward and should help you to quickly prototype or even implement final solutions, you should bear in mind, that your implementation does not need to be based on factories. You can for instance bind your own types directly, while still taking advantage of

HK2 provided dependency injection. Also, in your implementation you may want to pay more attention to defining or managing injection binding scopes for the sake of performance or correctness of your custom injection extension.

## Important

While the individual injection binding implementations vary and depend on your use case, to enable your custom injection extension in Jersey, you must register your custom HK2 Binder [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/utilities/Binder.html>] implementation in your application ResourceConfig [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>].

## 25.2.2. Injection Provider Using Jersey API

To make the `HttpSession` injection work without using HK2 API, we will need to create a custom supplier that knows how to extract `HttpSession` out of given `HttpServletRequest`.

```
import java.util.function.Supplier
...

public class HttpSessionSupplier implements Supplier<HttpSession> {

    private final HttpServletRequest request;

    @Inject
    public HttpSessionSupplier(HttpServletRequest request) {
        this.request = request;
    }

    @Override
    public HttpSession get() {
        return request.getSession();
    }

}
```

Once implemented, the supplier can be used in a custom Jersey `AbstractBinder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/AbstractBinder.html>] to define the new injection binding for `HttpSession`. Finally, the implemented binder can be registered in your `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>]:

```
import org.glassfish.jersey.internal.inject.AbstractBinder;
...
public class MyApplication extends ResourceConfig {

    public MyApplication() {

        ...

        register(new AbstractBinder() {
            @Override
            protected void configure() {
                bindFactory(HttpSessionSupplier.class).to(HttpSession.class)
            }
        });
    }

}
```

```
        .proxy(true).proxyForSameScope(false).in(RequestScoped.class);  
    }  
    });  
}  
}
```

The default scope for Jersey binder is similarly as for the HK2, the `PerLookup` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/PerLookup.html>].

To summarize the approach described above, here is a list of steps to follow when implementing custom injection provider in your Jersey application :

- Implement your own `Supplier` to provide the injectable instances.
- Use the `Supplier` to define an injection binding for the injected instance via custom `AbstractBinder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/AbstractBinder.html>].
- Register the custom `AbstractBinder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/AbstractBinder.html>] in your application `ResourceConfig`.

### Important

Similarly to the HK2, to enable your custom injection extension in Jersey, you must register your custom `AbstractBinder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/AbstractBinder.html>] implementation in your application `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>].

## 25.3. Defining Custom Injection Annotation

Java annotations are a convenient way for attaching metadata to various elements of Java code. Sometimes you may even decide to combine the metadata with additional functionality, such as ability to automatically inject the instances based on the annotation-provided metadata. The described scenario is one of the use cases where having means of defining a custom injection annotation in your Jersey application may prove to be useful. Obviously, this use case applies also to re-used existing, 3rd-party annotation types.

In the following example, we will describe how a custom injection annotation can be supported. Let's start with defining a new custom `SessionInject` injection annotation that we will specifically use to inject instances of `HttpSession` [<https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/httpsession>] (similarly to the previous example):

```
@Retention(RetentionPolicy.RUNTIME)  
@Target(ElementType.FIELD)  
public @interface SessionInject { }
```

The above `@SessionInject` annotation should be then used as follows:

```
@Path("di-resource")  
public class MyDiResource {  
  
    @SessionInject HttpSession httpSession;  
  
    ...  
}
```

```
}
```

Again, the semantics remains the same as in the example described in the previous section. You want to have the actual HTTP Servlet session instance injected into your `MyDiResource` instance. This time however, you expect that the `httpSession` field to be injected must be annotated with a custom `@SessionInject` annotation. Obviously, in this simplistic case the use of a custom injection annotation is an overkill, however, the simplicity of the use case will help us to avoid use case specific distractions and allow us better focus on the important aspects of the job of defining a custom injection annotation.

## 25.3.1. Custom Injection Annotation using HK2

If you remember from the previous section, to make the injection in the code snippet above work, you first need to implement the injection provider (HK2 Factory [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/Factory.html>]) as well as define the injection binding for the `HttpSession` type. That part we have already done in the previous section. We will now focus on what needs to be done to inform the HK2 runtime about our `@SessionInject` annotation type that we want to support as a new injection point marker annotation. To do that, we need to implement our own HK2 `InjectionResolver` [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/InjectionResolver.html>] for the annotation as demonstrated in the following listing:

```
import jakarta.inject.Inject;
import jakarta.inject.Named;

import jakarta.servlet.http.HttpSession;

import org.glassfish.hk2.api.InjectionResolver;
import org.glassfish.hk2.api.ServiceHandle;

...

public class SessionInjectResolver implements InjectionResolver<SessionInject> {

    @Inject
    @Named(InjectionResolver.SYSTEM_RESOLVER_NAME)
    InjectionResolver<Inject> systemInjectionResolver;

    @Override
    public Object resolve(Injectee injectee, ServiceHandle<?> handle) {
        if (HttpSession.class == injectee.getRequiredType()) {
            return systemInjectionResolver.resolve(injectee, handle);
        }

        return null;
    }

    @Override
    public boolean isConstructorParameterIndicator() {
        return false;
    }

    @Override
    public boolean isMethodParameterIndicator() {
        return false;
    }
}
```

```
    }  
}
```

The `SessionInjectResolver` above just delegates to the default HK2 system injection resolver to do the actual work.

You again need to register your injection resolver with your Jersey application, and you can do it the same way as in the previous case. Following listing includes HK2 binder that registers both, the injection provider from the previous step as well as the new HK2 inject resolver with Jersey application `ResourceConfig`. Note that in this case we're explicitly binding the `SessionInjectResolver` to a `@Singleton` [<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/singleton>] scope to avoid the unnecessary proliferation of `SessionInjectResolver` instances in the application:

```
import org.glassfish.hk2.api.TypeLiteral;  
import org.glassfish.hk2.utilities.binding.AbstractBinder;  
  
import jakarta.inject.Singleton;  
  
...  
  
public class MyApplication extends ResourceConfig {  
    public MyApplication() {  
        ...  
        register(new AbstractBinder() {  
            @Override  
            protected void configure() {  
                bindFactory(HttpSessionFactory.class).to(HttpSession.class);  
  
                bind(SessionInjectResolver.class)  
                    .to(new TypeLiteral<InjectionResolver<SessionInject>>(){} )  
                    .in(Singleton.class);  
            }  
        });  
    }  
}
```

## 25.3.2. Custom Injection Annotation using Jersey InjectionResolver

Jersey also comes with its `InjectionResolver` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectionResolver.html>] used to translate into the HK2 `InjectionResolver` [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/InjectionResolver.html>] during runtime. The abstraction is important for allowing to support the custom injection annotation in various DI containers. For instance, the abstraction is used when supporting injection using `@Context` [<https://jakarta.ee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Context.html>] in the CDI container (`jersey-cdi2-se` module).

The `SessionInjectResolver` then looks as follows:

```
import jakarta.inject.Inject;
```



```
import jakarta.servlet.http.HttpSession;

import org.glassfish.jersey.internal.inject.InjectorResolver;

...

public class SessionInjectResolver implements InjectorResolver<SessionInject> {

    private final InjectionManager injectionManager;

    public SessionInjectResolver(InjectionManager) {
        this.injectionManager = injectionManager;
    }

    @Override
    public Object resolve(Injectee injectee) {
        if (HttpSession.class == injectee.getRequiredType()) {
            return injectionManager.getInstance(HttpServletRequest.class).getSession();
        }
        return null;
    }

    @Override
    public boolean isConstructorParameterIndicator() {
        return false;
    }

    @Override
    public boolean isMethodParameterIndicator() {
        return false;
    }

    @Override
    public Class<SessionInject> getAnnotation() {
        return SessionInject.class;
    }
}
```

The `SessionInjectResolver` uses `InjectionManager` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectionManager.html>] described in Section 25.1, “`InjectionManager`”.

Unlike with `Glassfish-HK2` [<https://eclipse-ee4j.github.io/glassfish-hk2/>], `Jersey InjectorResolver` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectorResolver.html>] can only be bound as instance in the `AbstractBinder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/AbstractBinder.html>]. That is why the `InjectionManager` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectionManager.html>] is used in the `InjectorResolver` to resolve the `HttpSession` instance.

The `InjectorResolver` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/InjectorResolver.html>] can be registered in the with `Jersey application ResourceConfig` as follows:

```
import jakarta.ws.rs.core.Feature;
```

```
import org.glassfish.jersey.InjectionManagerProvider;
import org.glassfish.jersey.internal.inject.AbstractBinder;
import org.glassfish.jersey.internal.inject.InjectionManager;

import jakarta.inject.Singleton;

...

public class MyApplication extends ResourceConfig {

    public MyApplication() {

        ...

        register(new Feature() {
            @Override
            public boolean configure(FeatureContext context) {
                final InjectionManager injectionManager = InjectionManagerProvider.get
                context.register(new AbstractBinder() {
                    @Override
                    protected void configure() {
                        bind(new SessionInjectResolver(injectionManager)).to(HttpSessi
                    }
                });
                return true;
            }
        });
    }
}
```

## 25.4. Custom Life Cycle Management

The last use case discussed in this chapter will cover managing custom-scoped components within a Jersey application. If not configured otherwise, then all JAX-RS resources are by default managed on a per-request basis. A new instance of given resource class will be created for each incoming request that should be handled by that resource class. Let say you want to have your resource class managed in a per-session manner. It means a new instance of your resource class should be created only when a new Servlet HttpSession [<https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/httpsession>] is established. (As with previous examples in the chapter, this example assumes the deployment of your application to a Servlet container.)

Following is an example of such a resource class that builds on the support for HttpSession injection from the earlier examples described in this chapter. The `PerSessionResource` class allows you to count the number of requests made within a single client session and provides you a handy sub-resource method to obtain the number via a HTTP GET method call:

```
@Path("session")
public class PerSessionResource {

    @SessionInject HttpSession httpSession;

    AtomicInteger counter = new AtomicInteger();
```

```
@GET
@Path("id")
public String getSession() {
    counter.incrementAndGet();
    return httpSession.getId();
}

@GET
@Path("count")
public int getSessionRequestCount() {
    return counter.incrementAndGet();
}
}
```

Should the above resource be per-request scoped (default option), you would never be able to obtain any other number but 1 from its `getReqs` sub-resource method, because then for each request a new instance of our `PerSessionResource` class would get created with a fresh instance `counter` field set to 0. The value of this field would get incremented to 1 in the the `getSessionRequestCount` method before this value is returned. In order to achieve what we want, we have to find a way how to bind the instances of our `PerSessionResource` class to `HttpSession` instances and then reuse those bound instances whenever new request bound to the same HTTP client session arrives. Let's see how to achieve this.

To get better control over your Jersey component instantiation and life cycle, you need to implement a custom Jersey `ComponentProvider` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/spi/ComponentProvider.html>] SPI, that would manage your custom components. Although it might seem quite complex to implement such a thing, the component provider concept in Jersey is in fact very simple. It allows you to define your own HK2 injection bindings for the types that you are interested in, while informing the Jersey runtime at the same time that it should back out and leave the component management to your provider in such a case. By default, if there is no custom component provider found for any given component type, Jersey runtime assumes the role of the default component provider and automatically defines the default HK2 binding for the component type.

Following example shows a simple `ComponentProvider` implementation, for our use case. Some comments on the code follow.

```
import jakarta.inject.Inject;
import jakarta.inject.Provider;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpSession;
...
import org.glassfish.hk2.api.DynamicConfiguration;
import org.glassfish.hk2.api.DynamicConfigurationService;
import org.glassfish.hk2.api.Factory;
import org.glassfish.hk2.api.PerLookup;
import org.glassfish.hk2.api.ServiceLocator;
import org.glassfish.hk2.utilities.binding.BindingBuilderFactory;
import org.glassfish.jersey.server.spi.ComponentProvider;

@jakarta.ws.rs.ext.Provider
public class PerSessionComponentProvider implements ComponentProvider {

    private ServiceLocator locator;

    static class PerSessionFactory implements Factory<PerSessionResource>{
        static ConcurrentHashMap<String, PerSessionResource> perSessionMap
```

```
        = new ConcurrentHashMap<String, PerSessionResource>();

    private final Provider<HttpServletRequest> requestProvider;
    private final ServiceLocator locator;

    @Inject
    public PerSessionFactory(
        Provider<HttpServletRequest> request,
        ServiceLocator locator) {

        this.requestProvider = request;
        this.locator = locator;
    }

    @Override
    @PerLookup
    public PerSessionResource provide() {
        final HttpSession session = requestProvider.get().getSession();

        if (session.isNew()) {
            PerSessionResource newInstance = createNewPerSessionResource();
            perSessionMap.put(session.getId(), newInstance);

            return newInstance;
        } else {
            return perSessionMap.get(session.getId());
        }
    }

    @Override
    public void dispose(PerSessionResource r) {
    }

    private PerSessionResource createNewPerSessionResource() {
        final PerSessionResource perSessionResource = new PerSessionResource();
        locator.inject(perSessionResource);
        return perSessionResource;
    }
}

@Override
public void initialize(ServiceLocator locator) {
    this.locator = locator;
}

@Override
public boolean bind(Class<?> component, Set<Class<?>> providerContracts) {
    if (component == PerSessionResource.class) {

        final DynamicConfigurationService dynamicConfigService =
            locator.getService(DynamicConfigurationService.class);
        final DynamicConfiguration dynamicConfiguration =
            dynamicConfigService.createDynamicConfiguration();
    }
}
```

```
        BindingBuilderFactory
            .addBinding(BindingBuilderFactory.newFactoryBinder(PerSessionFactory
                .to(PerSessionResource.class), dynamicConfiguration));

        dynamicConfiguration.commit();

        return true;
    }
    return false;
}

@Override
public void done() {
}
}
```

The first and very important aspect of writing your own `ComponentProvider` in Jersey is to store the actual HK2 `ServiceLocator` [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/ServiceLocator.html>] instance that will be passed to you as the only argument of the provider `initialize` method. Your component provider instance will not get injected at all so this is more or less your only chance to get access to the HK2 runtime of your application. Please bear in mind, that at the time when your component provider methods get invoked, the `ServiceLocator` is not fully configured yet. This limitation applies to all component provider methods, as the main goal of any component provider is to take part in configuring the application's `ServiceLocator`.

Now let's examine the `bind` method, which is where your provider tells the HK2 how to bind your component. Jersey will invoke this method multiple times, once for each type that is registered with the actual application. Every time the `bind` method is invoked, your component provider needs to decide if it is taking control over the component or not. In our case we know exactly which Java type we are interested in (`PerSessionResource` class), so the logic in our `bind` method is quite straightforward. If we see our `PerSessionResource` class it is our turn to provide our custom binding for the class, otherwise we just return false to make Jersey poll other providers and, if no provider kicks in, eventually provide the default HK2 binding for the component. Please, refer to the Glassfish-HK2 [<https://eclipse-ee4j.github.io/glassfish-hk2/>] documentation for the details of the concrete HK2 APIs used in the `bind` method implementation above. The main idea behind the code is that we register a new HK2 Factory [<https://javaee.github.io/hk2/apidocs/org/glassfish/hk2/api/Factory.html>] (`PerSessionFactory`), to provide the `PerSessionResource` instances to HK2.

The implementation of the `PerSessionFactory` is also included above. Please note that as opposed to a component provider implementation that should never itself rely on an injection support, the factory bound by our component provider would get injected just fine, since it is only instantiated later, once the Jersey runtime for the application is fully initialized including the fully configured HK2 runtime. Whenever a new session is seen, the factory instantiates and injects a new `PerSessionResource` instance. The instance is then stored in the `perSessionMap` for later use (for future calls).

In a real life scenario, you would want to pay more attention to possible synchronization issues. Also, we do not consider a mechanism that would clean-up any obsolete resources for closed, expired or otherwise invalidated HTTP client sessions. We have omitted those considerations here for the sake of brevity of our example.

---

# Chapter 26. Jersey CDI Container Agnostic Support

## 26.1. Introduction

At the time of this writing, Java SE support is being discussed as one of important additions to CDI 3.0 specification. Existing CDI implementations brought this feature already, only container bootstrapping has not yet been standardized. In Jersey version 2.15 we introduced Weld SE support, so that people could take advantage of CDI features also when running in Java SE environment. As part of this work, the old Jersey CDI module has been refactored so that it supports CDI integration in any CDI-enabled HTTP container.

### Note

This chapter is mainly focused on server-side Jersey Weld SE support. We will mention other containers that are known to be working with Jersey CDI integration modules. We will also describe features demonstrated in Jersey HelloWorld Weld example and provide some hints on how to enable Java SE support for other (non Weld) CDI implementations.

## 26.2. Containers Known to Work With Jersey CDI Support

To stick with JAX-RS specification, Jersey has to support JAX-RS/CDI integration in Java/Jakarta EE environment. The two containers supporting JAX-RS/CDI integration out of the box are Oracle GlassFish and Oracle WebLogic application server.

Apache Tomcat is another Servlet container that is known to work fine with Jersey CDI support. However, things do not work there out of the box. You need to enable CDI support in Tomcat e.g. using Weld. Jersey CDI example [<https://github.com/eclipse-ee4j/jersey/tree/3.x/examples/cdi-webapp>] shows how a WAR application could be packaged (see `tomcat - packaging` profile in the pom file) in order to enable JAX-RS/CDI integration in Tomcat with Jersey using Weld.

If not bundled already with underlying Servlet container, the following Jersey module needs to be packaged with the application or otherwise included in the container class-path:

```
<dependency>
  <groupId>org.glassfish.jersey.ext.cdi</groupId>
  <artifactId>jersey-cdi1x</artifactId>
  <version>4.0.0</version>
</dependency>
```

## 26.3. Request Scope Binding

There is a common pattern for all above mentioned containers. Jersey CDI integration builds upon existing CDI/Servlet integration there. In other words, in all above cases, Jersey application must be deployed as a Servlet, where the underlying Servlet container has CDI integrated already and CDI container bootstrapped properly.

The key feature in CDI/Servlet integration is proper request scope binding. If this feature was missing, you would not be able to use any request scoped CDI beans in your Jersey application. To make Jersey work with CDI in containers that do not have request scope binding resolved, some extra work is required.

To allow smooth integration with Jersey request scope a new SPI, `ExternalRequestScope` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/spi/ExternalRequestScope.html>], was introduced in Jersey version 2.15. An SPI implementation should be registered via the standard `META-INF/services` mechanism and needs to make sure CDI implementation request scope has been properly managed and request scoped data kept in the right context. For performance reasons, at most a single external request scope provider is allowed by Jersey runtime.

## 26.4. Jersey Weld SE Support

The extra work to align HTTP request with CDI request scope was already done by Jersey team for Weld 4.x implementation. In order to utilize Jersey/Weld request scope binding, you need to use the following module:

```
<dependency>
  <groupId>org.glassfish.jersey.ext.cdi</groupId>
  <artifactId>jersey-weld2-se</artifactId>
  <version>4.0.0</version>
</dependency>
```

Then you could use your CDI backed JAX-RS components in a Jersey application running in Grizzly HTTP container bootstrapped as follows:

### Example 26.1. Bootstrapping Jersey application with Weld support on Grizzly

```
1      Weld weld = new Weld();
2      weld.initialize();
3
4      final HttpServer server = GrizzlyHttpServerFactory.createHttpServe
5
6      // ...
7
8      server.shutdownNow();
9      weld.shutdown();
```

The above pattern could be applied also for other Jersey supported HTTP containers as long as you stick with CDI Weld 4.x implementation. You simply add the above mentioned `jersey-weld2-se` module into you class-path and bootstrap the Weld container manually before starting the HTTP container.

---

# Chapter 27. GraalVM native-image generation

This chapter describes Jersey's compatibility with GraalVM native image. This functionality is available since Jersey 2.35 and is under development and configuration. For now Jersey provides native image configuration basics for some modules and example on how to really generate native image for an existing application.

## 27.1. Modules with GraalVM native image support

Currently Jersey provides basic support for native image generation within following modules: `jersey-common`, `jersey-server`, `jersey-client`, `jersey-hk2`. This support means that most of reflection and resource related settings are extracted into `reflect-config` and `resource-config` JSON files to be used while generating a native image. Those files are included in the native image generation process automatically (unless some tricky configuration is applied), so there is no need to include those files manually and/or duplicate their contents in some custom configuration.

## 27.2. HelloWorld native image generation

The example for the GraalVM native image generation is hidden under `examples/helloworld` example. To generate native image there it's required to perform some preliminary steps:

Download GraalVM at least 20.3.2 version  
Set `JAVA_HOME` to point to that `[GraalVM_HOME]`  
Perform `$JAVA_HOME/bin/gu install native-image` because native-image tool is not bundled within GraalVM itself  
Download Jersey examples source codes (preferable some released version like 2.35), and go to `[path_to_jersey_examples]/examples/helloworld`  
Run `mvn -Pnative-image clean package -DskipTests`

If all was correctly performed from previous steps the native image shall be already generated inside the target folder of the `helloworld` example with the name `helloworld-native` and it's possible to run it by

`target/./helloworld-native`

After it's run, console should print our following output:

```
"Hello World" Jersey Example App
May 27, 2021 1:37:49 PM org.glassfish.jersey.server.wadl.WadlFeature
WARNING: JAX-B API not found . WADL feature is disabled.
May 27, 2021 1:37:49 PM org.glassfish.grizzly.http.server.NetworkListener
INFO: Started listener bound to [localhost:8080]
May 27, 2021 1:37:49 PM org.glassfish.grizzly.http.server.HttpServer
INFO: [HttpServer] Started.
Application started.
Try out http://localhost:8080/base/helloworld
Stop the application using CTRL+C
```



If you see this, you can open given link in browser and check how application actually works. In general we are done here and you can use that example to generate native images for your own projects.

## 27.3. What's under the cover

For the example above the following command line was used:

```
-H:EnableURLProtocols=http,https
--initialize-at-build-time=org.glassfish.jersey.client.internal.Ht
-H:+ReportExceptionStackTraces
--verbose
--no-fallback
--report-unsupported-elements-at-runtime
```

This might be useful to generate another native image. It's possible to add another bunch of parameters to the command line (and put those into the `native-image.properties` file inside of your project). Important parameter here is `--initialize-at-build-time` (opposite to `--initialize-at-run-time`) and `--no-fallback` which says to the native image to generate pure native image with everything bundled inside the image and not just fall back wrapper for JDK.

Another important aspect for generating the native image is the proper listing of reflection classes (classes that use reflection in an application). For those needs, there is a native image agent which helps to generate those lists automatically. In order to generate a list of reflection classes (and JNI classes and resources), it is required to run:

```
$JAVA_HOME/bin/java -agentlib:native-image-agent=config-output-dir=[output_location]
```

And afterwards, the `[output_location]` directory will be created with generated lists (in JSON format). Those files can be included as is into native image generation, but it's very preferable to edit them manually to reduce possible ambiguous classes listings.

---

# Chapter 28. Jersey Test Framework

Jersey Test Framework originated as an internal tool used for verifying the correct implementation of server-side components. Testing RESTful applications became a more pressing issue with "modern" approaches like test-driven development and users started to look for a tool that could help with designing and running the tests as fast as possible but with many options related to test execution environment.

Current implementation of Jersey Test Framework supports the following set of features:

- pre-configured client to access deployed application
- support for multiple containers - grizzly, in-memory, jdk, simple, jetty
- able to run against any external container
- automated configurable traffic logging

Jersey Test Framework is primarily based on JUnit but you can run tests using TestNG as well. It works almost out-of-the box and it is easy to integrate it within your Maven-based project. While it is usable on all environments where you can run JUnit, we support primarily the Maven-based setups.

## 28.1. Basics

```
1 public class SimpleTest extends JerseyTest {
2
3     @Path("hello")
4     public static class HelloResource {
5         @GET
6         public String getHello() {
7             return "Hello World!";
8         }
9     }
10
11     @Override
12     protected Application configure() {
13         return new ResourceConfig(HelloResource.class);
14     }
15
16     @Test
17     public void test() {
18         final String hello = target("hello").request().get(String.class);
19         assertEquals("Hello World!", hello);
20     }
21 }
```

If you want to develop a test using Jersey Test Framework, you need to subclass `JerseyTest` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html>] and configure the set of resources and/or providers that will be deployed as part of the test application. This short code snippet shows basic resource class `HelloResource` used in tests defined as part of the `SimpleTest` class. The overridden `configure` method returns a `ResourceConfig` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ResourceConfig.html>] of the test application, that contains only the `HelloResource` resource class. `ResourceConfig` is a sub-class of JAX-RS Application [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Application.html>]. It is a Jersey convenience class for configuring JAX-

RS applications. `ResourceConfig` also implements JAX-RS Configurable [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/core/Configurable.html>] interface to make the application configuration more flexible.

## 28.2. Supported Containers

`JerseyTest` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html>] supports deploying applications on various containers, all (except the external container wrapper) need to have some "glue" code to be supported. Currently Jersey Test Framework provides support for Grizzly, In-Memory, JDK (`com.sun.net.httpserver.HttpServer`), Simple HTTP container (`org.simpleframework.http`) and Jetty HTTP container (`org.eclipse.jetty`) - since Jersey 3.x HTTP Jetty container requires JDK 11+.

A test container is selected based on various inputs. `JerseyTest#getTestContainerFactory()` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#getTestContainerFactory\(\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#getTestContainerFactory())] is always executed, so if you override it and provide your own version of `TestContainerFactory` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/spi/TestContainerFactory.html>], nothing else will be considered. Setting a system variable `TestProperties#CONTAINER_FACTORY` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/TestProperties.html#CONTAINER\\_FACTORY](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/TestProperties.html#CONTAINER_FACTORY)] has similar effect. This way you may defer the decision on which containers you want to run your tests from the compile time to the test execution time. Default implementation of `TestContainerFactory` looks for container factories on classpath. If more than one instance is found and there is a Grizzly test container factory among them, it will be used; if not, a warning will be logged and the first found factory will be instantiated.

Following is a brief description of all container factories supported in Jersey Test Framework.

- Jersey provides 2 different test container factories based on Grizzly. The `GrizzlyTestContainerFactory` creates a container that can run as a light-weight, plain HTTP container. Almost all Jersey tests are using Grizzly HTTP test container factory. Second factory is `GrizzlyWebTestContainerFactory` that is Servlet-based and supports Servlet deployment context for tested applications. This factory can be useful when testing more complex Servlet-based application deployments.

```
<dependency>
  <groupId>org.glassfish.jersey.test-framework.providers</groupId>
  <artifactId>jersey-test-framework-provider-grizzly2</artifactId>
  <version>4.0.0</version>
</dependency>
```

- In-Memory container is not a real container. It starts Jersey application and directly calls internal APIs to handle request created by client provided by test framework. There is no network communication involved. This containers does not support servlet and other container dependent features, but it is a perfect choice for simple unit tests.

```
<dependency>
  <groupId>org.glassfish.jersey.test-framework.providers</groupId>
  <artifactId>jersey-test-framework-provider-inmemory</artifactId>
  <version>4.0.0</version>
</dependency>
```

- `HttpServer` from JDK is another supported test container.

```
<dependency>
```

```
<groupId>org.glassfish.jersey.test-framework.providers</groupId>
<artifactId>jersey-test-framework-provider-jdk-http</artifactId>
<version>4.0.0</version>
</dependency>
```

- Simple container (`org.simpleframework.http`) is another light-weight HTTP container that integrates with Jersey and is supported by Jersey Test Framework.

```
<dependency>
  <groupId>org.glassfish.jersey.test-framework.providers</groupId>
  <artifactId>jersey-test-framework-provider-simple</artifactId>
  <version>4.0.0</version>
</dependency>
```

- Jetty container (`org.eclipse.jetty`) is another high-performance, light-weight HTTP server that integrates with Jersey and is supported by Jersey Test Framework. Shall be used along with JDK 11+.

```
<dependency>
  <groupId>org.glassfish.jersey.test-framework.providers</groupId>
  <artifactId>jersey-test-framework-provider-jetty</artifactId>
  <version>4.0.0</version>
</dependency>
```

## 28.3. Running TestNG Tests

It is possible to run not only JUnit tests but also tests based on TestNG. In order to do this you need to make sure the following 2 steps are fulfilled:

- Extend `JerseyTestNg` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTestNg.html>], or one of its inner classes `JerseyTestNg.ContainerPerClassTest` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTestNg.ContainerPerClassTest.html>] / `JerseyTestNg.ContainerPerMethodTest` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTestNg.ContainerPerMethodTest.html>], instead of `JerseyTest` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html>].
- Add TestNG to your class-patch, i.e.:

```
<dependency>
  <groupId>org.glassfish.jersey.test-framework</groupId>
  <artifactId>jersey-test-framework-core</artifactId>
  <version>4.0.0</version>
</dependency>
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>...</version>
</dependency>
```

To discuss the former requirement in more depth we need to take a look at the differences between JUnit and TestNG. JUnit creates a new instance of a test class for every test present in that class which, from the point of view of Jersey Test Framework, means that new test container and client is created for each test of a test class. However, TestNG creates only one instance of a test class and the initialization of the test container depends more on setup/teardown methods (driven by `@BeforeXXX`

and `@AfterXXX` annotations) than in JUnit. This means that, basically, you can start one instance of test container for all tests present in a test class or separate test container for each and every test. For this reason a separate subclasses of `JerseyTestNg` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTestNg.html>] have been created:

- `JerseyTestNg.ContainerPerClassTest` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTestNg.ContainerPerClassTest.html>] creates one container to run all the tests in. Setup method is annotated with `@BeforeClass`, teardown method with `@AfterClass`.

For example take a look at `ContainerPerClassTest` test. It contains two test methods (`first` and `second`), one singleton resource that returns an increasing sequence of number. Since we spawn only one instance of a test container for the whole class the value expected in the first test is 1 and in the second it's 2.

```
1 public class ContainerPerClassTest extends JerseyTestNg.ContainerPerClassTest {
2
3     @Path("/")
4     @Singleton
5     @Produces("text/plain")
6     public static class Resource {
7
8         private int i = 1;
9
10        @GET
11        public int get() {
12            return i++;
13        }
14    }
15
16    @Override
17    protected Application configure() {
18        return new ResourceConfig(Resource.class);
19    }
20
21    @Test(priority = 1)
22    public void first() throws Exception {
23        test(1);
24    }
25
26    @Test(priority = 2)
27    public void second() throws Exception {
28        test(2);
29    }
30
31    private void test(final Integer expected) {
32        final Response response = target().request().get();
33
34        assertEquals(response.getStatus(), 200);
35        assertEquals(response.readEntity(Integer.class), expected);
36    }
37 }
```

- `JerseyTestNg.ContainerPerMethodTest` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTestNg.ContainerPerMethodTest.html>] creates separate

container for each test. Setup method is annotated with `@BeforeMethod`, teardown method with `@AfterMethod`.

We can create a similar test to the previous one. Take a look at `ContainerPerMethodTest` test. It looks the same except the expected values and extending class: it contains two test methods (`first` and `second`), one singleton resource that returns an increasing sequence of number. In this case we create a separate test container for each test so value expected in the first test is `1` and in the second it's also `1`.

```
1 public class ContainerPerMethodTest extends JerseyTestNg.ContainerPerMethodTest {
2
3     @Path("/")
4     @Singleton
5     @Produces("text/plain")
6     public static class Resource {
7
8         private int i = 1;
9
10        @GET
11        public int get() {
12            return i++;
13        }
14    }
15
16    @Override
17    protected Application configure() {
18        return new ResourceConfig(Resource.class);
19    }
20
21    @Test
22    public void first() throws Exception {
23        test(1);
24    }
25
26    @Test
27    public void second() throws Exception {
28        test(1);
29    }
30
31    private void test(final Integer expected) {
32        final Response response = target().request().get();
33
34        assertEquals(response.getStatus(), 200);
35        assertEquals(response.readEntity(Integer.class), expected);
36    }
37 }
```

If you need more complex setup of your test you can achieve this by directly extending the `JerseyTestNg` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTestNg.html>] class create setup/teardown methods suited to your needs and provide a strategy for storing and handling a test container / client instance (see `JerseyTestNg.configureStrategy(TestNgStrategy)` method).

## 28.4. Advanced features

### 28.4.1. JerseyTest Features

JerseyTest provide enable(...) [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#enable\(java.lang.String\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#enable(java.lang.String))], forceEnable(...) [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#forceEnable\(java.lang.String\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#forceEnable(java.lang.String))] and disable(...) [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#disable\(java.lang.String\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#disable(java.lang.String))] methods, that give you control over configuring values of the properties defined and described in the `TestProperties` class. A typical code that overrides the default property values is listed below:

```
1 public class SimpleTest extends JerseyTest {
2     // ...
3
4     @Override
5     protected Application configure() {
6         enable(TestProperties.LOG_TRAFFIC);
7         enable(TestProperties.DUMP_ENTITY);
8
9         // ...
10
11     }
12 }
```

The code in the example above enables test traffic logging (inbound and outbound headers) as well as dumping the HTTP message entity as part of the traffic logging.

### 28.4.2. External container

Complicated test scenarios may require fully started containers with complex setup configuration, that is not easily doable with current Jersey container support. To address these use cases, Jersey Test Framework providers general fallback mechanism - an External Test Container Factory. Support of this external container "wrapper" is provided as the following module:

```
<dependency>
  <groupId>org.glassfish.jersey.test-framework.providers</groupId>
  <artifactId>jersey-test-framework-provider-external</artifactId>
  <version>4.0.0</version>
</dependency>
```

As indicated, the "container" exposed by this module is just a wrapper or stub, that redirects all request to a configured host and port. Writing tests for this container is similar to any other but you have to provide the information about host and port during the test execution:

```
mvn test -Djersey.test.host=myhost.org -Djersey.config.test.container.port=8080
```

### 28.4.3. Test Client configuration

Tests might require some advanced client configuration. This is possible by overriding `configureClient(ClientConfig clientConfig)` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/>]

`JerseyTest.html#configureClient(org.glassfish.jersey.client.ClientConfig)]` method. Typical use case for this is registering more providers, such as `MessageBodyReader<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyReader.html>]s or `MessageBodyWriter<T>` [<https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html>]s, or enabling additional features.

## 28.4.4. Accessing the logged test records programmatically

Sometimes you might need to check a logged message as part of your test assertions. For this purpose Jersey Test Framework provides convenient access to the logged records via `JerseyTest#getLastLoggedRecord()` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#getLastLoggedRecord\(\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#getLastLoggedRecord())] and `JerseyTest#getLoggedRecords()` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#getLoggedRecords\(\)](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/JerseyTest.html#getLoggedRecords())] methods. Note that this feature is not enabled by default, see `TestProperties#RECORD_LOG_LEVEL` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/TestProperties.html#RECORD\\_LOG\\_LEVEL](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/TestProperties.html#RECORD_LOG_LEVEL)] for more information.

## 28.5. Parallel Testing with Jersey Test Framework

For a purpose of running multiple test containers in parallel you need to set the `TestProperties.CONTAINER_PORT` [[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/TestProperties.CONTAINER\\_PORT.html](https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/test/TestProperties.CONTAINER_PORT.html)] to `0` value. This will tell Jersey Test Framework (and the underlying test container) to use the first available port.

You can set the value as a system property (via command line option) or directly in the test (to not affect ports of other tests):

```
1 @Override
2     protected Application configure() {
3         // Find first available port.
4         forceSet(TestProperties.CONTAINER_PORT, "0");
5
6         return new ResourceConfig(Resource.class);
7     }
```

The easiest way to setup your JUnit or TestNG tests to run in parallel is to configure Maven Surefire plugin. You can do this via configuration options `parallel` and `threadCount`, i.e.:

```
...
<configuration>
  <parallel>methods</parallel>
  <threadCount>5</threadCount>
  ...
</configuration>
...
```

For more information about this topic consult the following Maven Surefire articles:

- Fork Options and Parallel Test Execution [<http://maven.apache.org/surefire/maven-surefire-plugin/examples/fork-options-and-parallel-execution.html>]



- Using TestNG - Running tests in parallel [[http://maven.apache.org/surefire/maven-surefire-plugin/examples/testng.html#Running\\_tests\\_in\\_parallel](http://maven.apache.org/surefire/maven-surefire-plugin/examples/testng.html#Running_tests_in_parallel)]
- Using JUnit - Running tests in parallel [[http://maven.apache.org/surefire/maven-surefire-plugin/examples/junit.html#Running\\_tests\\_in\\_parallel](http://maven.apache.org/surefire/maven-surefire-plugin/examples/junit.html#Running_tests_in_parallel)]

---

# Chapter 29. Building and Testing Jersey

## 29.1. Checking Out the Source

Jersey source code is available on GitHub. You can browse the sources at <https://github.com/eclipse-ee4j/jersey>.

In case you are not familiar with Git, we recommend reading some of the many "Getting Started with Git" articles you can find on the web. For example this DZone RefCard [<http://refcardz.dzone.com/refcardz/getting-started-git>].

To clone the Jersey repository you can execute the following command on the command-line (provided you have a command-line Git client installed on your machine):

```
git clone git://github.com/eclipse-ee4j/jersey.git
```

This creates read-only copy of Jersey workspace. If you want to contribute, please use "pull request": <https://help.github.com/articles/creating-a-pull-request>.

Milestones and releases of Jersey are tagged. You can list the tags by executing the standard Git command in the repository directory:

```
git tag -l
```

or by visiting <https://github.com/eclipse-ee4j/jersey/tags>.

## 29.2. Building the Source

Jersey source code requires Java SE 8 or higher. Support of Jetty HTTP container/client requires JDK 11+. The build is based on Maven. Maven 3.6.3 or higher is highly recommended. Also it is recommended you use the following Maven options when building the workspace (can be set in MAVEN\_OPTS environment variable):

```
-Xmx1048m -XX:PermSize=64M -XX:MaxPermSize=128M
```

It is recommended to build all of Jersey after you cloned the source code repository. To do that execute the following commands in the directory where jersey source repository was cloned (typically the directory named "jersey"):

```
mvn -Dmaven.test.skip=true -DskipTests clean install
```

This command will build Jersey, but skip the test execution. If you don't want to skip the tests, execute the following instead:

```
mvn clean install
```

Building the whole Jersey project including tests could take significant amount of time.

## 29.3. Testing

Jersey contains many tests. Unit tests are in the individual Jersey modules, integration and end-to-end tests are in `jersey/tests/e2e` directory. You can run tests related to a particular area using the following command:

```
mvn -Dtest=<pattern> test -pl :[modulename]
```

where `pattern` may be a comma separated set of names matching tests classes or individual methods (like `LinkTest#testDelimiters`) and `[modulename]` is name of a module from which tests are called. If module does not contain those tests build failure (as no tests run) occurs.

## 29.4. Using NetBeans

NetBeans IDE [<http://netbeans.org>] has excellent maven support. The Jersey maven modules can be loaded, built and tested in NetBeans without any additional NetBeans-specific project files.

---

# Chapter 30. Migration Guide

## 30.1. Migrating from Jersey 3.x to 4.0.x.

### 30.1.1. Breaking Changes

- Jersey 4.x is implementation of Jakarta REST 4.0, part of Jakarta EE 11. For APIs and implementations in Jakarta EE 11, JDK 17 minimum is required. However, some modules may require higher versions of Java SE. See Section 2.1, “Java SE Compatibility”.
- Jakarta REST 4.0 dropped classes bound to JAX-B, since JAX-B has been removed from Jakarta EE 11 Core profile. These classes were `jakarta.ws.rs.core.Link.JaxbLink` and `jakarta.ws.rs.core.Link.JaxbAdapter` along with related methods working with these classes.
- Some modules supporting obsolete libraries were dropped. The Apache 4 HTTP Client supported by `jersey-apache-connector` is replaced by `jersey-apache5-connector`, using the Apache 5 HTTP Client.

`jersey-jetty11-connector` based on Jetty 11 HTTP Client has been dropped, with `jersey-jetty-connector` based on Jetty 12 HTTP Client as a replacement.

The same with `jersey-container-jetty11-http` with `jersey-container-jetty-http` atop Jetty 12 as a replacement.

`jersey-container-simple-http` module was removed without a replacement.

- Given JDK 17 does not allow reflection on JDK internal classes, it is no longer possible to support additional HTTP methods with the default `HttpURLConnection`, such as popular HTTP Patch.
- Jersey 4.x still uses HK2 as a main DI framework. However, in the future releases (5.x +), the main DI container would be the CDI container. In CDI, it is not possible to register beans in the injection container in runtime the way it is possible in HK2 using `AbstractBinder`; the `AbstractBinder` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/internal/inject/AbstractBinder.html>] is moved to the `jersey-hk2` package. Also, a related `org.glassfish.jersey.internal.inject.Bindings` class was moved into the same package. A few internal `Binding` subclasses and related Jersey most innate classes are hidden from public usage. See Section Chapter 25, *Custom Injection and Lifecycle Management*.
- Jersey stopped providing jersey as a bundle, known as `jaxrs-ri`. Jersey jars used in the bundle was only a subset of possibilities Jersey provides with a variety of modules each of which handles an additional customer use-case. Some modules are not designed to be together on a classpath with a module from the `jaxrs-ri` bundle and that has been leading to a confusion.

The `jaxrs-ri` bundle also contained third party libraries with a different licensing than Jersey, which also led to a confusion about whole `jaxrs-ri` bundle and Jersey itself license.

### 30.1.2. Application servers for Jersey

Note that only a few servers support Jakarta EE 11 compatible Servlet API and they are tested with Jersey. Those are: GlassFish 8, Tomcat 11. The other containers will follow.

# Appendix A. Configuration Properties

## A.1. Common (client/server) configuration properties

List of common configuration properties that can be found in `CommonProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html>] class. All of these properties can be overridden by their server/client counterparts.

**Table A.1. List of common configuration properties**

| Constant                                                                                                                                                                                                                                                                                                                                                                                                | Value                                                                                                            | Description                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CommonProperties.ALLOW_SYSTEM_PROPERTIES_PROVIDER</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#ALLOW_SYSTEM_PROPERTIES_PROVIDER">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#ALLOW_SYSTEM_PROPERTIES_PROVIDER</a> ]                | <code>System.getProperties().provider</code>                                                                     | Enable/disable <code>SystemPropertiesProvider</code> (if true) default System properties configuration provider. Default value is true.    |
| <code>CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE</a> ]                      | <code>jersey.config.client.disableAutoDiscovery</code><br><code>jersey.config.server.disableAutoDiscovery</code> | Disable/enforce auto discovery globally on the client/server. Default value is true.                                                       |
| <code>CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE_CLIENT</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE_CLIENT">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE_CLIENT</a> ] |                                                                                                                  |                                                                                                                                            |
| <code>CommonProperties.FEATURE_AUTO_DISCOVERY_DISABLE_SERVER</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE_SERVER">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE_SERVER</a> ] |                                                                                                                  |                                                                                                                                            |
| <code>CommonProperties.JSON_BINDING_FEATURE_DISABLE</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_BINDING_FEATURE_DISABLE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_BINDING_FEATURE_DISABLE</a> ]                            | <code>jersey.config.client.disableJsonBinding</code><br><code>jersey.config.server.disableJsonBinding</code>     | Disable/enforce configuration of Json Binding (JSR-367) feature. Default value is false. Can also be set as a system property. Since 2.45. |
| <code>CommonProperties.JSON_BINDING_FEATURE_DISABLE_CLIENT</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_BINDING_FEATURE_DISABLE_CLIENT">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_BINDING_FEATURE_DISABLE_CLIENT</a> ]       |                                                                                                                  |                                                                                                                                            |

| Constant                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Value                                                                                                         | Description                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey/<br>CommonProperties.html#JSON_BINDING_FEATURE_DISABLE_CLIENT]<br>/<br>CommonProperties.JSON_BINDING_FEATURE_DISABLE_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_BINDING_FEATURE_DISABLE_SERVER]                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                               |                                                                                                                                                                                                                                                                                                                                                              |
| CommonProperties.JSON_BINDING_FEATURE_DISABLE_APPLICATION<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_BINDING_FEATURE_DISABLE_APPLICATION]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | jersey.config.client.disableJsonBinding<br>/<br>jersey.config.server.disableJsonBinding                       | Disables configuration of Json Binding (JSR-367) feature for jakarta.ws.rs.core.Application subclasses whose package names are specified as a value. The APPLICATION-separated string defining prefixes of the application package names. Can also be set as a system property.<br><br>By default, Json Binding is automatically enabled.<br><br>Since 2.45. |
| CommonProperties.JSON_PROCESSING_FEATURE_DISABLE_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_PROCESSING_FEATURE_DISABLE_SERVER]<br>/<br>CommonProperties.JSON_PROCESSING_FEATURE_DISABLE_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_PROCESSING_FEATURE_DISABLE_CLIENT]<br>/<br>CommonProperties.JSON_PROCESSING_FEATURE_DISABLE_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_PROCESSING_FEATURE_DISABLE_SERVER] | jersey.config.client.disableJsonProcessing<br>/<br>jersey.config.server.disableJsonProcessing                 | Disables configuration of Json Processing (JSR-353) feature. Disables Json Processing                                                                                                                                                                                                                                                                        |
| CommonProperties.METAINFO_SERVICES_LOOKUP_DISABLE_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#METAINFO_SERVICES_LOOKUP_DISABLE_SERVER]<br>/<br>CommonProperties.METAINFO_SERVICES_LOOKUP_DISABLE_CLIENT                                                                                                                                                                                                                                                                                                                                                                                                                            | jersey.config.client.disableMetaInfoServicesLookup<br>/<br>jersey.config.server.disableMetaInfoServicesLookup | Disables METAINFO services lookup globally on client/server. Disables MetaInfoServicesLookup                                                                                                                                                                                                                                                                 |

| Constant                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Value                                                                                                          | Description                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#METAINF_SERVICES_LOOKUP_DISABLE_CLIENT]<br>/<br>CommonProperties.METAINF_SERVICES_LOOKUP_DISABLE_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#METAINF_SERVICES_LOOKUP_DISABLE_SERVER]                                                                                                                                                                                                                                                         |                                                                                                                |                                                                                                                                                                                                   |
| CommonProperties.MOXY_JSON_FEATURE_DISABLE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#MOXY_JSON_FEATURE_DISABLE]<br>/<br>CommonProperties.MOXY_JSON_FEATURE_DISABLE_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#MOXY_JSON_FEATURE_DISABLE_CLIENT]<br>/<br>CommonProperties.MOXY_JSON_FEATURE_DISABLE_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#MOXY_JSON_FEATURE_DISABLE_SERVER] | <code>jersey.config.client.disableMoxyJson</code><br><code>jersey.config.server.disableMoxyJson</code>         | Disables configuration of MOXY Json feature. Default value is <code>false</code>                                                                                                                  |
| CommonProperties.OUTBOUND_CONTENT_LENGTH_BUFFER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#OUTBOUND_CONTENT_LENGTH_BUFFER]<br>/<br>CommonProperties.OUTBOUND_CONTENT_LENGTH_BUFFER_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#OUTBOUND_CONTENT_LENGTH_BUFFER_CLIENT]<br>/<br>CommonProperties.OUTBOUND_CONTENT_LENGTH_BUFFER_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/                                              | <code>jersey.config.client.contentLengthBuffer</code><br><code>jersey.config.server.contentLengthBuffer</code> | Authn buffer value that defines the buffer size used to buffer the content length header in order to determine its size and set content length HTTP Content-Length header. Default value is 8192. |

| Constant                                                                                                                                                                                                                  | Value                                                            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey/<br>CommonProperties.html#OUTBOUND_CONTENT_LENGTH_BUFFER_SERVER]                                                                                                                                                   |                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| CommonProperties.PROVIDER_DEFAULT_DISABLE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#PROVIDER_DEFAULT_DISABLE]                | jersey.config.provider.DefaultDisable                            | Enable or disable the default providers from being loaded. The following providers extend application footprint by XML dependencies, which is too heavy for native image, or by AWT which may possibly be not available by JDK 11 desktop:<br><br>java.awt.image.RenderedImage<br>javax.xml.transform.Source<br>javax.xml.transform.dom.DOMSource<br>javax.xml.transform.sax.SAXSource<br>javax.xml.transform.stream.StreamSource<br>The following are the options to disable the provides: {@code DOMSOURCE, RENDEREDIMAGE, SAXSOURCE, SOURCE, STREAMSOURCE}, or to disable all: {@code ALL}. Multiple options can be disabled by adding multiple comma separated values. Default value is NULL @since 2.30 |
| CommonProperties.JSON_JACKSON_ENABLED_MODULES<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_JACKSON_ENABLED_MODULES]        | jersey.config.client.json.jack<br>jersey.config.server.json.jack | Common enabled modules for Jackson modules which enabled for those modules will be used for JSON processing. Default value is NULL @since 2.36                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| CommonProperties.JSON_JACKSON_ENABLED_MODULES_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_JACKSON_ENABLED_MODULES] |                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| CommonProperties.JSON_JACKSON_ENABLED_MODULES_SERVER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/<br>CommonProperties.html#JSON_JACKSON_ENABLED_MODULES] |                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| CommonProperties.JSON_JACKSON_DISABLED_MODULES<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/                                                              | jersey.config.client.json.jack<br>jersey.config.server.json.jack | Common disabled modules for Jackson modules which disabled for those modules will be excluded                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |



| Constant                                                                                                                                                                                                                                     | Value                                             | Description                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CommonProperties.html#JSON_JACKSON_DISABLED_MODULES_CLIENT<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_JACKSON_DISABLED_MODULES]                             | JACKSON_DISABLED_MODULES_CLIENT                   | Excludes JSON processing. Default value is NULL @since 2.36                                                                                                                                                                                                                                                                                                                                                                        |
| CommonProperties.html#JSON_JACKSON_DISABLED_MODULES_SERVER<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#JSON_JACKSON_DISABLED_MODULES]                             | JACKSON_DISABLED_MODULES_SERVER                   | Excludes JSON processing. Default value is NULL @since 2.36                                                                                                                                                                                                                                                                                                                                                                        |
| CommonProperties.PARAM_CONVERTERS_THROW_ILLEGAL_ARGUMENT_EXCEPTION<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#PARAM_CONVERTERS_THROW_ILLEGAL_ARGUMENT_EXCEPTION] | PARAM_CONVERTERS_THROW_ILLEGAL_ARGUMENT_EXCEPTION | Enter the ParamConverter<T> [https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/ParamConverter.html] to throw <code>IllegalArgumentException</code> (as mandated in javadoc. Must be convertible to <code>Boolean</code> value.<br><br>Internally the <code>Exception</code> is caught by Jersey and usually converted to null Therefore, the default value is set to <code>false</code> to speed up the conversion. |
| CommonProperties.THREAD_FACTORY<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#THREAD_FACTORY](Jersey 3.1.7 or later)                                                | Jersey.config.threads.factories                   | Defines the <code>java.util.concurrent.ThreadFactory</code> to be used by internal default <code>ExecutorServices</code> .<br><br>The default is <code>java.util.concurrent.Executors#defaultPlatformThreadFactory()</code> on virtual threads.                                                                                                                                                                                    |
| CommonProperties.USE_VIRTUAL_THREADS<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/CommonProperties.html#USE_VIRTUAL_THREADS](Jersey 3.1.7 or later)                                      | Jersey.config.threads.useVirtualThreads           | Defines whether the virtual threads should be used by Jersey on JDK 21+ when not using an exact number of threads by <code>FixedThreadPool</code> .<br><br>The default is <code>false</code> for this version of Jersey.                                                                                                                                                                                                           |
| LoggingFeature.LOGGING_FILTER_NAME<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html]                                                                             | JerseyConfig.Logging.loggerName                   | logger name of the logging filter. See logging chapter for more information. The default value is <code>org.glassfish.jersey.logging.Logging</code>                                                                                                                                                                                                                                                                                |

| Constant                                                                                                                                                                                                                     | Value                                               | Description                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_LOGGER_NAME]                                                                                                                                                          | <code>jersey.config.logging.LoggerLevel</code>      |                                                                                                                                                                                                                                                                                                                    |
| LoggingFeature.LOGGING_FEATURE_LOGGER_LEVEL<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_LOGGER_LEVEL]       |                                                     | Logger of logging filter's logger at which the messages will be logged. See logging chapter for more information.                                                                                                                                                                                                  |
| LoggingFeature.LOGGING_FEATURE_VERBOSITY<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_VERBOSITY]             | <code>jersey.config.logging.verbosity</code>        | Verbosity of logging filter describes how verbose the logging filter will be. There are 3 possible values <code>LoggingFeature.Verbosity.HEADERS_ONLY</code> , <code>LoggingFeature.Verbosity.PAYLOAD_TEXT</code> or <code>LoggingFeature.Verbosity.PAYLOAD_ANY</code> . See logging chapter for more information. |
| LoggingFeature.LOGGING_FEATURE_MAX_ENTITY_SIZE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_MAX_ENTITY_SIZE] | <code>jersey.config.logging.entity.maxSize</code>   | The maximum number of bytes of the entity which will be logged. See logging chapter for more information.                                                                                                                                                                                                          |
| LoggingFeature.LOGGING_FEATURE_SEPARATOR<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_SEPARATOR]             | <code>jersey.config.logging.entity.separator</code> | Custom logging delimiter for new lines separation. See logging chapter for more information.                                                                                                                                                                                                                       |
| LoggingFeature.LOGGING_FEATURE_REDACT_HEADERS<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_REDACT_HEADERS]   | <code>jersey.config.logging.headers</code>          | The HTTP headers (semicolon separated) to be redacted when logging. See logging chapter for more information.                                                                                                                                                                                                      |

## A.2. Server configuration properties

List of server configuration properties that can be found in `ServerProperties` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html] class.

**Table A.2. List of server configuration properties**

| Constant                                                                                                                             | Value                                             | Description                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ServerProperties.APPLICATION_NAME<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/ | <code>jersey.config.server.applicationName</code> | Definition of application name. The name is an arbitrary user defined name which is used to distinguish between Jersey applications in the case that more applications are |

| Constant                                                                                                                                                                                                                                                          | Value                        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey/server/<br>ServerProperties.html#APPLICATION_NAME]                                                                                                                                                                                                         |                              | deployed on the same runtime (container). The name can be used for example for purposes of monitoring by JMX when name identifies to which application deployed MBeans belong to. The name should be unique in the runtime. The property does not have a default value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| ServerProperties.BV_FEATURE_DISABLE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#BV_FEATURE_DISABLE]                                                             | jersey.config.beanValidation | Disables Bean Validation support. Default value is false.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| ServerProperties.BV_DISABLE_VALIDATE_ON_EXECUTABLE_OVERRIDE_CHECK<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#BV_DISABLE_VALIDATE_ON_EXECUTABLE_OVERRIDE_CHECK] | jersey.config.beanValidation | Disables <code>disable.validateOnExecutableOverrideCheck</code> check. Default value is false.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| ServerProperties.BV_SEND_ERROR_IN_RESPONSE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#BV_SEND_ERROR_IN_RESPONSE]                                               | jersey.config.beanValidation | Enables sending validation <code>ValidationError</code> information to the client. Default value is false.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| ServerProperties.EMPTY_REQUEST_MEDIA_TYPE_MATCHES_ANY<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#EMPTY_REQUEST_MEDIA_TYPE_MATCHES_ANY]                         | jersey.config.server.mvc     | <p>When <code>any</code> request media type is used in <code>@Consumes</code> annotation to accept only HTTP requests with compatible HTTP Content-Type header. However, when <code>any</code> is used in <code>@Consumes</code> annotation, the <code>any</code> wildcard media type is used to match the <code>@Consumes</code> annotation.</p> <p>HTTP/1.1 RFC recommends that missing HTTP Content-Type header MAY default to <code>application/octet-stream</code>. This property makes Jersey consider the missing HTTP Content-Type header to be <code>application/octet-stream</code> rather than a wildcard media type. However, for a resource method without an entity argument, such as for HTTP GET, a wildcard media type is still considered to accept the HTTP</p> |

| Constant                                                                                                                                                                                                        | Value                                      | Description                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                 |                                            | request for the missing HTTP Content-Type header.<br><br>Set this property to <b>false</b> , if the empty request media type should not to match applied <b>@Consumes</b> annotation on a resource method with an entity argument. The default is <b>true</b> .                                                                                                       |
| ServerProperties.FEATURE_AUTO_DISCOVERY_DISABLE<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE]   | jersey.config.server.disableAutoDiscovery  | Disable automatic discovery on server. Default value is <b>false</b> .                                                                                                                                                                                                                                                                                                |
| ServerProperties.HTTP_METHOD_OVERRIDE<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#HTTP_METHOD_OVERRIDE]                       | jersey.config.server.httpMethodOverride    | Defines method overriding of HTTP method overriding. This property is used by <b>HttpMethodOverrideFilter</b> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/filter/HttpMethodOverrideFilter.html] to determine where it should look for method override information (e.g. request header or query parameters). |
| ServerProperties.JSON_BINDING_FEATURE_DISABLE<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#JSON_BINDING_FEATURE_DISABLE]       | jersey.config.server.disableJsonBinding    | Disable configuration of Json Processing (JSR-353) feature. Default value is <b>false</b> .                                                                                                                                                                                                                                                                           |
| ServerProperties.JSON_PROCESSING_FEATURE_DISABLE<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#JSON_PROCESSING_FEATURE_DISABLE] | jersey.config.server.disableJsonProcessing | Disable configuration of Json Processing (JSR-353) feature. Default value is <b>false</b> .                                                                                                                                                                                                                                                                           |
| ServerProperties.LANGUAGE_MAPPINGS<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#LANGUAGE_MAPPINGS]                             | jersey.config.server.languageMappings      | Defines Mappings of URI extensions to languages. The property is used by <b>UriConnegFilter</b> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/filter/UriConnegFilter.html].                                                                                                                                    |
| ServerProperties.MEDIA_TYPE_MAPPINGS<br>[https://eclipse-ee4j.github.io/                                                                                                                                        | jersey.config.server.mediaTypeMappings     | Defines Mappings of URI extensions to media types.                                                                                                                                                                                                                                                                                                                    |

| Constant                                                                                                                                                                                                       | Value                                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MEDIA_TYPE_MAPPINGS]                                                                                                |                                                    | The property is used by UriConnegFilter [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/filter/UriConnegFilter.html].                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| ServerProperties.METAINFO_SERVICES_LOOKUP_DISABLE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#METAINFO_SERVICES_LOOKUP_DISABLE] | jersey.config.server.disableMetaInfoServicesLookup | Disable MetaInfoServicesLookup lookup on server. Default value is false.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| ServerProperties.MOXY_JSON_FEATURE_DISABLE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MOXY_JSON_FEATURE_DISABLE]               | jersey.config.server.disableMoxyJson               | Disable configuration of MOXY Json feature. Default value is false.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| ServerProperties.MONITORING_ENABLED [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_ENABLED] (Jersey 2.12 or later)      | jersey.config.server.monitoring.enabled            | Monitoring, that is application monitoring will be enabled. This will enable the possibility of injecting ApplicationInfo [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/ApplicationInfo.html] into resource and providers. Default value is false.                                                                                                                                                                                                                                                                                                                                                           |
| ServerProperties.MONITORING_STATISTICS_ENABLED [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_STATISTICS_ENABLED]       | jersey.config.server.monitoring.enabled            | Monitoring, that is application monitoring statistics will be enabled. This will enable the possibility of injecting MonitoringStatistics [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatistics.html] into resource and providers and also the registered listeners implementing MonitoringStatisticsListener [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatisticsListener.html] will be called when statistics are available for processing. Monitoring statistics extends basic monitoring feature. Therefore |

| Constant                                                                                                                                                                                                                                                       | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                |       | when enabled, the monitoring gets automatically enabled too (the same result as setting the property <code>ServerProperties.MONITORING_ENABLED</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_ENABLED] to <code>true</code> ). Note that enabling statistics may have a negative performance impact and therefore should be enabled only when needed. Default value is <code>false</code> .                                                                                                                                                                                                                                                                                                                                                                                        |
| <code>ServerProperties.MONITORING_STATISTICS_MBEANS_ENABLED</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_STATISTICS_MBEANS_ENABLED]                            |       | When enabled, Jersey will expose MBeans for the collected monitoring statistics. Exposed JMX MBeans are based on <code>MonitoringStatistics</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/monitoring/MonitoringStatistics.html] data and therefore when enabled, the calculation of monitoring statistics gets automatically enabled too (the same result as setting the property <code>ServerProperties.MONITORING_STATISTICS_ENABLED</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_STATISTICS_ENABLED] to <code>true</code> ). Note that enabling MBeans for monitoring statistics may have a negative performance impact and therefore should be enabled only when needed. Default value is <code>false</code> . |
| <code>ServerProperties.MONITORING_STATISTICS_REFRESH_INTERVAL</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#MONITORING_STATISTICS_REFRESH_INTERVAL] (Jersey 2.10 or later) |       | Monitor (in ms) statistics how often will be monitoring statistics refreshed ( <code>onStatistics</code> method called). Default value is <code>500</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <code>ServerProperties.OUTBOUND_CONTENT_LENGTH_BUFFER_LENGTH</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/                                                                                      |       | An integer value that defines the buffer size used to buffer the outbound message entity in order to determine its size and set the value of <code>HTTP Content-</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

| Constant                                                                                                                                                                                                                                                                                                                                                                                              | Value                                                          | Description                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ServerProperties.html#OUTBOUND_CONTENT_LENGTH_BUFFER_SIZE<br>(Jersey 2.2 or later)                                                                                                                                                                                                                                                                                                                    | <code>config.server.outbound.content.length.buffer.size</code> | Length header. Default value is 8192.                                                                                                                                                                                                       |
| ServerProperties.PROVIDER_CLASSNAMES<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_CLASSNAMES">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_CLASSNAMES</a> ]                                                 | <code>config.server.provider.classnames</code>                 | Defines classnames are class names that implement application-specific resources and providers. If the property is set, the specified classes will be instantiated and registered as either application JAX-RS root resources or providers. |
| ServerProperties.PROVIDER_CLASSPATH<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_CLASSPATH">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_CLASSPATH</a> ]                                                    | <code>config.server.provider.classpath</code>                  | Defines classpath that contains application-specific resources and providers. If the property is set, the specified packages will be scanned for JAX-RS root resources and providers.                                                       |
| ServerProperties.PROVIDER_PACKAGES<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_PACKAGES">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_PACKAGES</a> ]                                                       | <code>config.server.provider.packages</code>                   | Defines packages are packages that contain application-specific resources and providers. If the property is set, the specified packages will be scanned for JAX-RS root resources and providers.                                            |
| ServerProperties.PROVIDER_SCANNING_RECURSIVE<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_SCANNING_RECURSIVE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#PROVIDER_SCANNING_RECURSIVE</a> ]                         | <code>config.server.provider.scanning.recursive</code>         | Provider scanning strategy for package scanning. Default value is <code>true</code> .                                                                                                                                                       |
| ServerProperties.REDUCE_CONTEXT_PATH_SLASHES_ENABLED<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#REDUCE_CONTEXT_PATH_SLASHES_ENABLED">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#REDUCE_CONTEXT_PATH_SLASHES_ENABLED</a> ] | <code>config.server.reduce.context.path.slashes.enabled</code> | Does context path slashes are enabled a port and a context path and will resolve it as URI with only one slash. Default value is <code>false</code> .                                                                                       |
| ServerProperties.RESOURCE_VALIDATION_DISABLE<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#RESOURCE_VALIDATION_DISABLE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#RESOURCE_VALIDATION_DISABLE</a> ]                         | <code>config.server.resource.validation.disable</code>         | Disable resource validation. Default value is <code>false</code> .                                                                                                                                                                          |
| ServerProperties.RESOURCE_VALIDATION_IGNORE_ERRORS<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#RESOURCE_VALIDATION_IGNORE_ERRORS">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#RESOURCE_VALIDATION_IGNORE_ERRORS</a> ]       | <code>config.server.resource.validation.ignore.errors</code>   | Denotes validation ignore of application resource models should fail even in case of a fatal validation errors. Default value is <code>false</code> .                                                                                       |
| ServerProperties.WADL_FEATURE_DISABLE<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#WADL_FEATURE_DISABLE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#WADL_FEATURE_DISABLE</a> ]                                              | <code>config.server.wadl.feature.disable</code>                | Disables WADL generation. Default value is <code>false</code> .                                                                                                                                                                             |

| Constant                                                                                                                                                                                                                    | Value                                                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#WADL_FEATURE_DISABLE]                                                                                                                             |                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| ServerProperties.WADL_GENERATOR_CONFIG<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#WADL_GENERATOR_CONFIG]                 | jersey.config.server.wadl.generator.config           | Defines the WadlGenerator configuration that provides a WadlGenerator [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/wadl/WadlGenerator.html].                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| ServerProperties.RESPONSE_SET_STATUS_OVER_SEND<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#RESPONSE_SET_STATUS_OVER_SEND] | jersey.config.server.response.setStatusOverSendError | When a servlet container sets 4xx or 5xx it is possible to choose between <code>sendError</code> or <code>setStatus</code> on container specific <code>Response</code> implementation. On servlet container Jersey can call <code>HttpServletResponse.setStatus(...)</code> or <code>HttpServletResponse.sendError(...)</code> . Calling <code>sendError(...)</code> method usually resets entity, response headers and provide error page for specified status code (e.g. servlet <code>error-page</code> configuration). However if you want to post-process response (e.g. by servlet filter) the only way to do it is calling <code>setStatus(...)</code> on container <code>Response</code> object. If property value is <code>true</code> the method <code>Response.setStatus(...)</code> is used over default <code>Response.sendError(...)</code> . Type of the property value is <code>boolean</code> . The default value is <code>false</code> . |
| ServerProperties.TRACING<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#TRACING]                                             | jersey.config.server.tracing.type                    | Enables tracing support. Possible values are <code>OFF</code> (default), <code>ON_DEMAND</code> and <code>ALL</code> . See Section 24.2.1, “Configuration options” for more detail.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| ServerProperties.TRACING_THRESHOLD<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/                                                                                       | jersey.config.server.tracing.threshold               | Setting the threshold of detail provided by tracing. Possible values are <code>SUMMARY</code> , <code>TRACE</code> and <code>VERBOSE</code> . See Section 24.2.1,                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |



| Constant                                                                                                                                                                                                                                                                            | Value                                                                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey/server/<br>ServerProperties.html#TRACING_THRESHOLD]                                                                                                                                                                                                                          |                                                                          | “Configuration options” to learn more about the levels.                                                                                                                                                                                                                                                                                                                                                                          |
| ServerProperties.PROCESSING_RESPONSE_ERRORS_ENABLED<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#PROCESSING_RESPONSE_ERRORS_ENABLED]                                               | jersey.config.server.enableProcessingResponseErrors                      | The property value is <code>ResponseErrors</code> the errors raised during response processing are tried to be handled using available response error mappers.                                                                                                                                                                                                                                                                   |
| ServerProperties.SUBRESOURCE_LOCATOR_CACHE_SIZE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#SUBRESOURCE_LOCATOR_CACHE_SIZE]                                                       | jersey.config.server.subresourceLocatorCacheSize                         | A resource cache defines the size of cache for sub-resource locator models. The cache is used to provide better performance for application that uses JAX-RS sub-resource locators.                                                                                                                                                                                                                                              |
| ServerProperties.SUBRESOURCE_LOCATOR_CACHE_AGE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#SUBRESOURCE_LOCATOR_CACHE_AGE]                                                         | jersey.config.server.subresourceLocatorCacheAge                          | A resource cache defines the maximum age (in seconds) for cached for sub-resource locator models. The age of an cache entry is defined as the time since the last access (read) to the entry in the cache. Entry aging is not enabled by default.                                                                                                                                                                                |
| ServerProperties.SUBRESOURCE_LOCATOR_CACHE_JERSEY_RESOURCE_CLASSES_ENABLED<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#SUBRESOURCE_LOCATOR_CACHE_JERSEY_RESOURCE_CLASSES_ENABLED] | jersey.config.server.subresourceLocatorCacheJerseyResourceClassesEnabled | Whether Jersey resources in addition to caching sub-resource locator classes and instances (which are cached by default). To make sure the caching is effective, you need to return same Jersey Resource instances for same input parameters from resource method. This means that generating new Jersey Resource instances for same input parameters would not have any performance effect and it would only fill-up the cache. |
| ServerProperties.LOCATION_HEADER_RELATIVE_URI_RESOLUTION_RFC7231<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#LOCATION_HEADER_RELATIVE_URI_RESOLUTION_RFC7231]                     | jersey.config.server.resolveRelativeURIRFC7231                           | By default, relative URIs in the Location http header against the request URI according to RFC7231 (new HTTP Specification)                                                                                                                                                                                                                                                                                                      |
| ServerProperties.LOCATION_HEADER_RELATIVE_URI_RESOLUTION_DISABLED<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/server/<br>ServerProperties.html#LOCATION_HEADER_RELATIVE_URI_RESOLUTION_DISABLED]                   | jersey.config.server.resolveRelativeURIDisabled                          | By default, relative URIs in the Location http header.                                                                                                                                                                                                                                                                                                                                                                           |

| Constant                                                                                                                                                                                                                                                      | Value                                        | Description                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ServerProperties.UNWRAP_COMPLETION_STAGE_IN_MESSAGE_BODY_WRITER_ENABLED<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#UNWRAP_COMPLETION_STAGE_IN_MESSAGE_BODY_WRITER_ENABLED] | <code>jersey.config.server.properties</code> | Whether the <code>MessageBodyWriter</code> will not use <code>CompletionStage</code> as a generic type. The <code>CompletionStage</code> value will be unwrapped and the <code>MessageBodyWriter</code> will be invoked with the unwrapped type.                                                                        |
| LoggingFeature.LOGGING_FEATURE_LOGGER_NAME_SERVER<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_LOGGER_NAME_SERVER]                                            | <code>jersey.config.server.properties</code> | Logging name of the logging filter. See logging chapter for more information. The default value is <code>org.glassfish.jersey.logging.LoggingFeature</code> .                                                                                                                                                           |
| LoggingFeature.LOGGING_FEATURE_LOGGER_LEVEL_SERVER<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_LOGGER_LEVEL_SERVER]                                          | <code>jersey.config.server.properties</code> | Level of logging filter's logger at which the messages will be logged. See logging chapter for more information.                                                                                                                                                                                                        |
| LoggingFeature.LOGGING_FEATURE_VERBOSITY_SERVER<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_VERBOSITY_SERVER]                                                | <code>jersey.config.server.properties</code> | Verbosity of logging filter describes how verbose the logging filter will be. There are 3 possible values <code>LoggingFeature.Verbosity.HEADERS_ONLY</code> , <code>LoggingFeature.Verbosity.PAYLOAD_TEXT</code> or <code>LoggingFeature.Verbosity.PAYLOAD_ANYTHING</code> . See logging chapter for more information. |
| LoggingFeature.LOGGING_FEATURE_MAX_ENTITY_SIZE_SERVER<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_MAX_ENTITY_SIZE_SERVER]                                    | <code>jersey.config.server.properties</code> | Maximum number of bytes of the entity which will be logged. See logging chapter for more information.                                                                                                                                                                                                                   |
| LoggingFeature.LOGGING_FEATURE_MAX_ENTITY_SIZE_SERVER<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_MAX_ENTITY_SIZE_SERVER]                                    | <code>jersey.config.server.properties</code> | Logging entity separation. See logging chapter for more information.                                                                                                                                                                                                                                                    |
| LoggingFeature.LOGGING_FEATURE_REDACT_HEADERS_SERVER<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_REDACT_HEADERS_SERVER]                                      | <code>jersey.config.server.properties</code> | Whether the headers (semicolon separated) to be redacted when logging. See logging chapter for more information.                                                                                                                                                                                                        |

## A.3. SeBootstrap and WebServer related configuration properties

List of SeBootstrap configuration properties that can be found in ServerProperties [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html] class.

**Table A.3. List of SeBootstrap and WebServer configuration properties**

| Constant                                                                                                                                                                                                       | Value                                                                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ServerProperties.WEBSERVER_ALLOW_PRIVILEGED_PORTS [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#WEBSERVER_ALLOW_PRIVILEGED_PORTS] | <code>jersey.config.server.bootstrap.WebServerAllowPrivilegedPorts</code> | <p>Describes whether a <code>WebServer</code> is allowed to use privileged ports (0-1023) to be used to start the <code>WebServer</code> implementation to be chosen from the unused ports when the <code>SeBootstrap.Configuration</code> <code>PORT</code> is set to <code>-1</code> or unset.</p> <p>The default ports are 80 for HTTP and 443 for HTTPS when <code>WEBSERVER_ALLOW_PRIVILEGED_PORTS</code> is <code>true</code> or 8080 for HTTP and 8443 for HTTPS when <code>WEBSERVER_ALLOW_PRIVILEGED_PORTS</code> is <code>false</code>.</p> <p>If <code>SeBootstrap.Configuration</code> <code>PORT</code> is set to <code>0</code>, the implementation scans for random port (0-65535) when <code>WEBSERVER_ALLOW_PRIVILEGED_PORTS</code> is <code>true</code>, or (1024-65535) when <code>WEBSERVER_ALLOW_PRIVILEGED_PORTS</code> is <code>false</code>.</p> <p>The default this is <code>false</code>. Use <code>true</code> to allow a restricted port number.</p> |
| ServerProperties.WEBSERVER_AUTO_START [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#WEBSERVER_AUTO_START]                         | <code>jersey.config.server.bootstrap.WebServerAutoStart</code>            | <p>Describes whether to automatically start <code>WebServer</code> at bootstrap.</p> <p>By default, servers are immediately listening to connections after bootstrap, so no explicit invocation of <code>WebServer#start()</code> is needed.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| ServerProperties.WEBSERVER_IMPL [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/server/ServerProperties.html#WEBSERVER_IMPL]                                     | <code>jersey.config.server.bootstrap.WebServerImpl</code>                 | Describes the implementation of <code>WebServer</code> to bootstrap.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

| Constant                                                 | Value | Description                                              |
|----------------------------------------------------------|-------|----------------------------------------------------------|
| jersey/server/<br>ServerProperties.html#WEBSERVER_CLASS] |       | By default auto-selects the first server provider found. |

## A.4. Servlet configuration properties

List of servlet configuration properties that can be found in ServletProperties [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/servlet/ServletProperties.html] class.

**Table A.4. List of servlet configuration properties**

| Constant                                                                                                                                                                                                   | Value                                           | Description                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ServletProperties.FILTER_CONTEXT_PATH<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/servlet/ServletProperties.html#FILTER_CONTEXT_PATH]                 | jersey.config.servlet.filter.contextPath        | filter, indicates the PATH pattern of the Jersey servlet filter context path.                                                                                                                                                                                                                                                                        |
| ServletProperties.FILTER_FORWARD_ON_404<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/servlet/ServletProperties.html#FILTER_FORWARD_ON_404]             | jersey.config.servlet.filter.forwardOn404       | filter, forwardOn404 a 404 response with no entity body is returned from either the runtime or the application then the runtime forwards the request to the next filter in the filter chain. This enables another filter or the underlying servlet engine to process the request. Before the request is forwarded the response status is set to 200. |
| ServletProperties.FILTER_STATIC_CONTENT_REGEX<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/servlet/ServletProperties.html#FILTER_STATIC_CONTENT_REGEX] | jersey.config.servlet.filter.staticContentRegex | filter, staticContentRegex to match an incoming servlet path URI to some web page content such as static resources or JSPs to be handled by the underlying servlet engine.                                                                                                                                                                           |
| ServletProperties.JAXRS_APPLICATION_CLASS<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/servlet/ServletProperties.html#JAXRS_APPLICATION_CLASS]         | jakarta.ws.rs.Application                       | Application configuration initialization property whose value is a fully qualified class name of a class that implements JAX-RS Application.                                                                                                                                                                                                         |
| ServletProperties.PROVIDER_WEB_APP<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/servlet/ServletProperties.html#PROVIDER_WEB_APP]                       | jersey.config.servlet.provider.webApp           | provider, webApp should scan the whole web app for application-specific resources and providers.                                                                                                                                                                                                                                                     |
| ServletProperties.QUERY_PARAMETERS_AS_FORM<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/                                                                                                    | jersey.config.servlet.queryParametersAsForm     | filter, queryParameters, disabled will not be treated as form parameters (e.g. injectable using                                                                                                                                                                                                                                                      |

| Constant                                                                                                                                                                                        | Value                                           | Description                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------|
| snapshot/jersey/org/glassfish/<br>jersey/servlet/<br>ServletProperties.html#QUERY_PARAMS_AS_FORM_PARAMS_DISABLED]                                                                               |                                                 | @FormParam) in case a Form request is processed by server. DISABLED]                      |
| ServletProperties.SERVICE_LOCATOR [https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/servlet/<br>ServletProperties.html#SERVICE_LOCATOR] | jersey.config.servlet.component.service_locator | Default service locator to be used as a parent ServiceLocator in the Jersey WebComponent. |

## A.5. Client configuration properties

List of client configuration properties that can be found in ClientProperties [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html] class.

**Table A.5. List of client configuration properties**

| Constant                                                                                                                                                                                                                               | Value                                                     | Description                                                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClientProperties.ASYNC_THREADPOOL_SIZE [https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#ASYNC_THREADPOOL_SIZE]                               | jersey.config.client.async.threadpool.size                | Asynchronous thread pool size. Default value is not set. Supported with GrizzlyConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/grizzly/connector/<br>GrizzlyConnectorProvider.html] only.. |
| ClientProperties.BACKGROUND_SCHEDULER_THREADPOOL_SIZE [https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#BACKGROUND_SCHEDULER_THREADPOOL_SIZE] | jersey.config.client.background.scheduler.threadpool.size | Background scheduler. Default value is not set. Support is undefined.                                                                                                                                                                                      |
| ClientProperties.CHUNKED_ENCODING_SIZE [https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#CHUNKED_ENCODING_SIZE]                               | jersey.config.client.chunked.encoding.size                | Chunked encoding size. Default value is 4096.                                                                                                                                                                                                              |
| ClientProperties.CONNECT_TIMEOUT [https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#CONNECT_TIMEOUT]                                           | jersey.config.client.connect.timeout                      | Read timeout interval, in milliseconds. Default value is 0 (infinity).                                                                                                                                                                                     |
| ClientProperties.CONNECTOR_PROVIDER [https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/                                                                                  | jersey.config.client.connector.provider                   | Select the connector provider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/spi/ConnectorProvider.html]                                                                                             |

| Constant                                                                                                                                                                                                           | Value                                                                 | Description                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClientProperties.html#CONNECTOR_PROVIDER](Jersey 2.40 or later)                                                                                                                                                    | <code>JerseyClientConfig.Client.connectorProvider</code>              | class. Overrides the value from META-INF/services.<br><br>The value MUST be an instance of <code>String</code> .<br><br>The property is recognized by <code>ClientBuilder</code> .                                                                                    |
| ClientProperties.DIGESTAUTH_URI_CACHE_SIZELIMIT [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#DIGESTAUTH_URI_CACHE_SIZELIMIT]         | <code>JerseyClientConfig.Client.digestAuthUriCacheSizeLimit</code>    | The property defines the limit of HTTP proxy the client connector should use.                                                                                                                                                                                         |
| ClientProperties.EXPECT_100_CONTINUE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#EXPECT_100_CONTINUE]                               | <code>JerseyClientConfig.client.requestExpect100Continue</code>       | Allows for Expect:100-Continue processing. Continue being handled by the <code>URLConnection</code> (default Jersey connector). Since 2.32                                                                                                                            |
| ClientProperties.EXPECT_100_CONTINUE_THRESHOLD_SIZE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#EXPECT_100_CONTINUE_THRESHOLD_SIZE] | <code>JerseyClientConfig.Client.expect100ContinueThresholdSize</code> | Property expect100ContinueThresholdSize for content length after which Expect:100-Continue header would be applied before the main request. Default threshold size (64Kb) after which Expect:100-Continue header would be applied before the main request. Since 2.32 |
| ClientProperties.FEATURE_AUTO_DISCOVERY_DISABLE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#FEATURE_AUTO_DISCOVERY_DISABLE]         | <code>JerseyClientConfig.Client.disableAutoDiscovery</code>           | Disable automatic discovery on client. Default value is <code>false</code> .                                                                                                                                                                                          |
| ClientProperties.FOLLOW_REDIRECTS [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#FOLLOW_REDIRECTS]                                     | <code>JerseyClientConfig.client.followRedirects</code>                | Does the client will automatically redirect to the URI declared in 3xx responses. Default value is <code>true</code> .                                                                                                                                                |
| ClientProperties.JSON_BINDING_FEATURE_DISABLE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#JSON_BINDING_FEATURE_DISABLE]             | <code>JerseyClientConfig.Client.disableJsonBinding</code>             | Disable configuration of Json Binding (JSR-367) feature. Default value is <code>false</code> .                                                                                                                                                                        |
| ClientProperties.JSON_PROCESSING_FEATURE_DISABLE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/]                                                                                                        | <code>JerseyClientConfig.Client.disableJsonProcessing</code>          | Disable configuration of Json Processing (JSR-353) feature. Default value is <code>false</code> .                                                                                                                                                                     |

| Constant                                                                                                                                                                                                                                               | Value                                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#JSON_PROCESSING_FEATURE_DISABLE]                                                                                                                                             |                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| ClientProperties.METAINFO_SERVICES_LOOKUP_DISABLE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#METAINFO_SERVICES_LOOKUP_DISABLE]                      | jersey.config.client.MetaInfoServicesLookupDisable | Disables METAINFO services lookup on client. Default value is false.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| ClientProperties.MOXY_JSON_FEATURE_DISABLE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#MOXY_JSON_FEATURE_DISABLE]                                    | jersey.config.client.DisableMoxyJson               | Disables configuration of MOXY Json feature. Default value is false.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| ClientProperties.OUTBOUND_CONTENT_LENGTH_BUFFER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#OUTBOUND_CONTENT_LENGTH_BUFFER<br>(Jersey 2.2 or later)] | jersey.config.client.OutboundContentLengthBuffer   | Attempts to get the buffer defines the buffer size used to buffer the outbound message entity in order to determine its size and set the value of HTTP Content-Length header. Default value is 8192.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| ClientProperties.PROXY_URI<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#PROXY_URI]                                                                    | jersey.config.client.proxy.uri                     | URI of a HTTP proxy the client connector should use. Default value is not set. Currently supported with Apache5ConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/apache5/connector/ Apache5ConnectorProvider.html], GrizzlyConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/grizzly/connector/ GrizzlyConnectorProvider.html], HelidonConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/helidon/connector/ HelidonConnectorProvider.html], NettyConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/ NettyConnectorProvider.html], and JettyConnectorProvider |

| Constant                                                                                                                                                                                               | Value                                          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                        |                                                | [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyConnectorProvider.html] only.                                                                                                                                                                                                                                                                                                                                |
| ClientProperties.PROXY_USERNAME<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#PROXY_USERNAME]<br>(Jersey 2.5 or later) | Jersey.config.client.proxy.username            | Proxy username which will be used for HTTP proxy authentication. Default value is not set. Currently supported with ApacheConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/apache/connector/apache/connector/ConnectorProvider.html] and JettyConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyConnectorProvider.html] only. |
| ClientProperties.PROXY_PASSWORD<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#PROXY_PASSWORD]<br>(Jersey 2.5 or later) | Jersey.config.client.proxy.password            | Proxy password which will be used for HTTP proxy authentication. Default value is not set. Currently supported with ApacheConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/apache/connector/apache/connector/ConnectorProvider.html] and JettyConnectorProvider [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyConnectorProvider.html] only. |
| ClientProperties.READ_TIMEOUT<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#READ_TIMEOUT]<br>(Jersey 2.5 or later)     | Jersey.config.client.read.timeout              | Read timeout interval, in milliseconds. Default value is 0 (infinity).                                                                                                                                                                                                                                                                                                                                                                                                          |
| ClientProperties.REQUEST_ENTITY_PROCESSING<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/                                                                  | Jersey.config.client.request.entity.processing | Defines whether the request should be serialized using internal buffer to evaluate content length or chunk encoding should be used.                                                                                                                                                                                                                                                                                                                                             |



| Constant                                                                                                                                                                                                                                                         | Value                                         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey/client/<br>ClientProperties.html#REQUEST_ENTITY_PROCESSING]<br>(Jersey 2.5 or later)                                                                                                                                                                      |                                               | Possible values are <b>BUFFERED</b> or <b>CHUNKED</b> . Default value is <b>BUFFERED</b> .                                                                                                                                                                                                                                                                                                                                                                                                                                |
| ClientProperties.SNI_HOST_NAME<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#SNI_HOST_NAME]<br>(Jersey 2.43 or later)                                            | <code>Jersey.config.client.sniHostName</code> | <p>The <b>sniHostName</b> host name to be used for calculating the <code>javax.net.ssl.SNIHostName</code> during the HTTPS request. Takes precedence over the HTTP HOST header, if set.</p> <p>By default, the <b>SNIHostName</b> is set when the HOST HTTP header differs from the HTTP request host. When the property value host name matches the HTTPS request host, the <b>SNIHostName</b> is not set, and the HTTP HOST header is not used for setting the <b>SNIHostName</b>. This allows for Domain Fronting.</p> |
| ClientProperties.SUPPRESS_HTTP_COMPLIANCE_VALIDATION<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#SUPPRESS_HTTP_COMPLIANCE_VALIDATION]<br>(Jersey 2.2 or later) | <code>Jersey.config.client.supplier</code>    | <p>Whether the compliance validation HTTP specification compliance for client-side requests will be suppressed. When compliance checks are suppressed, any violations will be merely logged as warnings, rather than causing exceptions being raised in Jersey runtime. Default value is <b>false</b>.</p>                                                                                                                                                                                                                |
| ClientProperties.SSL_CONTEXT_SUPPLIER<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#SSL_CONTEXT_SUPPLIER]                                                        | <code>Jersey.config.client.sniHostName</code> | <p>The <b>context.supplier</b> <code>javax.net.ssl.SSLContext</code> <code>java.util.function.Supplier</code> to be used to set ssl context in the current HTTP request. Has precedence over the <code>Client#getSslContext()</code>.</p> <p>Currently supported by the default <code>HttpURLConnection</code> and by <code>NettyConnector</code> only.</p>                                                                                                                                                               |
| ClientProperties.USE_ENCODING<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#USE_ENCODING]                                                                        | <code>Jersey.config.client.useEncoding</code> | <p>Whether the value of Content-Encoding property the <code>EncodingFilter</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/filter/EncodingFilter.html] should be adding. Default value is not set.</p>                                                                                                                                                                                                                                                        |
| ClientProperties.SNI_HOST_NAME<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/                                                                                                                                                                  | <code>Jersey.config.client.sniHostName</code> | <p>Host name to be used as an <b>SNIHostName</b>. However,</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

| Constant                                                                                                                                                                                                                                                       | Value                                      | Description                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| snapshot/jersey/org/glassfish/<br>jersey/client/<br>ClientProperties.html#SNI_HOST_NAME]                                                                                                                                                                       |                                            | the HOST header is restricted in JDK. <code>URLConnection</code> and <code>JavaNetHttpConnector</code> need an extra System Property set to allow HOST header. As an option to HOST header, this property allows the HOST name to be pre-set on a Client and does not need to be set on each request. Since 3.1.2<br><br>The value MUST be an instance of <code>String</code> |
| LoggingFeature.LOGGING_FEATURE_LOGGER_NAME_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_LOGGER_NAME_CLIENT]                             | <code>JERSEY_LOGGER_NAME_CLIENT</code>     | Logging name of the logging filter. See logging chapter for more information. The default value is <code>org.glassfish.jersey.logging.Logging</code>                                                                                                                                                                                                                          |
| LoggingFeature.LOGGING_FEATURE_LOGGER_LEVEL_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_LOGGER_LEVEL_CLIENT]                           | <code>JERSEY_LOGGER_LEVEL_CLIENT</code>    | Logging level of the logger at which the messages will be logged. See logging chapter for more information.                                                                                                                                                                                                                                                                   |
| LoggingFeature.LOGGING_FEATURE_VERBOSE_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_VERBOSE_CLIENT]                                     | <code>JERSEY_VERBOSE_CLIENT</code>         | Logging verbosity filter describes how verbose the logging filter will be. There are 3 possible values <code>LoggingFeature.Verbosity.HEADERS_ONLY</code> , <code>LoggingFeature.Verbosity.PAYLOAD_TEXT</code> or <code>LoggingFeature.Verbosity.PAYLOAD_ANY</code> . See logging chapter for more information.                                                               |
| LoggingFeature.LOGGING_FEATURE_MAX_ENTITY_SIZE_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_MAX_ENTITY_SIZE_CLIENT]                     | <code>JERSEY_MAX_ENTITY_SIZE_CLIENT</code> | Logging entity maximum size of the entity which will be logged. See logging chapter for more information.                                                                                                                                                                                                                                                                     |
| LoggingFeature.LOGGING_FEATURE_MAX_ENTITY_SIZE_SEPARATOR_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/logging/<br>LoggingFeature.html#LOGGING_FEATURE_MAX_ENTITY_SIZE_SEPARATOR_CLIENT] | <code>JERSEY_SEPARATOR_CLIENT</code>       | Logging entity separator (the side). See logging chapter for more information.                                                                                                                                                                                                                                                                                                |
| LoggingFeature.LOGGING_FEATURE_REDACT_HEADERS_CLIENT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/                                                                                                                                          | <code>JERSEY_REDACT_HEADERS_CLIENT</code>  | Logging headers (each colon separated) to be redacted when                                                                                                                                                                                                                                                                                                                    |

| Constant                                                                                                | Value | Description                                        |
|---------------------------------------------------------------------------------------------------------|-------|----------------------------------------------------|
| snapshot/jersey/org/glassfish/jersey/logging/LoggingFeature.html#LOGGING_FEATURE_REDACT_HEADERS_CLIENT] |       | logging. See logging chapter for more information. |

## A.6. Jersey configuration properties for message & entity processing

List of client configuration properties that can be found in `MessageProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html>] class.

**Table A.6. List of Apache HTTP client configuration properties**

| Constant                                                                                                                                                                                                                                                                                                                                                                                 | Value                                         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MessageProperties.DEFLATE_WITHOUT_ZLIB<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#DEFLATE_WITHOUT_ZLIB">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#DEFLATE_WITHOUT_ZLIB</a> ]                            | <code>Jersey.config.deflate.noZlib</code>     | noZlib set to true, DeflateEncoder deflate encoding interceptor will use non-standard version of the deflate content encoding, skipping the zlib wrapper. Unfortunately, deflate encoding implementations in some products use this non-compliant version, hence the switch.<br><br>The default value is <b>false</b> .                                                                                                                                                                                                                                                                                               |
| MessageProperties.IO_BUFFER_SIZE<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#IO_BUFFER_SIZE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#IO_BUFFER_SIZE</a> ]                                              | <code>Jersey.config.io.bufferSize</code>      | Value of the property indicates the buffer size to be used for I/O operations on byte and character streams. The property value is expected to be a positive integer otherwise it will be ignored.<br><br>The default value is value of <code>MessageProperties.IO_DEFAULT_BUFFER_SIZE</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#IO_DEFAULT_BUFFER_SIZE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#IO_DEFAULT_BUFFER_SIZE</a> ] |
| MessageProperties.JAXB_PROCESS_XML_ROOT_ELEMENT<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#JAXB_PROCESS_XML_ROOT_ELEMENT">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#JAXB_PROCESS_XML_ROOT_ELEMENT</a> ] | <code>Jersey.config.jaxb.noRootElement</code> | noRootElement set to true, root element tag name for collections will be derived from <code>jakarta.xml.bind.annotation.XmlRootElement</code> annotation value and won't be capitalized.<br><br>The default value is <b>false</b> .                                                                                                                                                                                                                                                                                                                                                                                   |

| Constant                                                                                                                                                                                                                                                                                                                                                            | Value                          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MessageProperties.JSON_MAX_STRING_LENGTH<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#JSON_MAX_STRING_LENGTH">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#JSON_MAX_STRING_LENGTH</a> ] | <code>Integer#MAX_VALUE</code> | <p>The default value is not set and the JSON provider default maximum value is used.</p> <p>If supported by Jackson provider, the default value can differ for each Jackson version. For instance, Jackson 2.14 does not support this setting and the default value is <code>Integer#MAX_VALUE</code>, Jackson 2.15.0 has the default value <code>5_000_000</code>, Jackson 2.15.2 has the default value <code>20_000_000</code>.</p>                                                                                                                                                                                                                                                                                                                                                                                                              |
| MessageProperties.XML_SECURITY_DISABLE<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#XML_SECURITY_DISABLE">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#XML_SECURITY_DISABLE</a> ]       | <code>Boolean</code>           | <p>If <code>true</code>, XML security features when parsing XML documents will be disabled.</p> <p>The default value is <code>false</code>.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| MessageProperties.XML_FORMAT_OUTPUT<br>[ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#XML_FORMAT_OUTPUT">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/message/MessageProperties.html#XML_FORMAT_OUTPUT</a> ]                | <code>Boolean</code>           | <p>If <code>true</code> indicates that produced XML output should be formatted if possible.</p> <p>A XML message entity written by a <code>MessageBodyWriter&lt;T&gt;</code> [<a href="https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html">https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html</a>] may be formatted for the purposes of human readability provided the respective <code>MessageBodyWriter&lt;T&gt;</code> [<a href="https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html">https://jakartaee.github.io/rest/apidocs/4.0.0/jakarta/ws/rs/ext/MessageBodyWriter.html</a>] supports XML output formatting. All JAXB-based message body writers support this property.</p> <p>The default value is <code>false</code>.</p> |

## A.7. Apache 5 HTTP client configuration properties

List of client configuration properties that can be found in `Apache5ClientProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/apache5/connector/Apache5ClientProperties.html>] class.

### Table A.7. List of Apache 5 HTTP client configuration properties

| Constant                                                                                                                                                                                                                              | Value                                                             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Apache5ClientProperties.CONNECTION_CLOSING_STRATEGY<br>[https://eclipse-ee4j.github.io/ jersey.github.io/apidocs/ snapshot/ jersey/org/glassfish/ jersey/apache5/connector/ Apache5ClientProperties.html#CONNECTION_CLOSING_STRATEGY] | org.apache.hc.client5.classic.HttpClientConnectionClosingStrategy | String, connection closing strategy. Apache Connection. Accepts an instance of Apache5ConnectionClosingStrategy [https://eclipse-ee4j.github.io/ jersey.github.io/apidocs/snapshot/ jersey/org/glassfish/ jersey/apache5/connector/ Apache5ConnectionClosingStrategy.html].                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Apache5ClientProperties.CONNECTION_MANAGER<br>[https://eclipse-ee4j.github.io/ jersey.github.io/apidocs/ snapshot/ jersey/org/glassfish/ jersey/apache5/connector/ Apache5ClientProperties.html#CONNECTION_MANAGER]                   | org.apache.hc.client5.http.classic.HttpClientConnectionManager    | Connection manager. will be used to create org.apache.hc.client5.http.classic.HttpClient. The value MUST be an instance of org.apache.hc.client5.http.io.HttpClientConnectionManager. If the property is absent a default Connection Manager will be used org.apache.hc.client5.http.impl.io.HttpClientConnectionManager. If you want to use this client in multi-threaded environment, be sure you override default value with org.apache.hc.client5.http.impl.io.HttpClientConnectionManager instance.                                                                                                                                                                                                                    |
| Apache5ClientProperties.CONNECTION_MANAGER_SHARED<br>[https://eclipse-ee4j.github.io/ jersey.github.io/apidocs/ snapshot/ jersey/org/glassfish/ jersey/apache5/connector/ Apache5ClientProperties.html#CONNECTION_MANAGER_SHARED]     | org.apache.hc.client5.http.classic.HttpClientConnectionManager    | Client connection manager. configured connection manager should be shared among multiple Jersey client runtime instances. It means that closing a particular client runtime instance does not shut down the underlying connection manager automatically. In such case, the connection manager life-cycle should be fully managed by the application code. To release all allocated resources, caller code should especially ensure org.apache.hc.client5.http.io.HttpClientConnectionManager gets invoked eventually. This property may only be set prior to constructing Apache connector using Apache5ConnectorProvider [https://eclipse-ee4j.github.io/ jersey.github.io/apidocs/snapshot/ jersey/org/glassfish/ jersey/ |

| Constant                                                                                                                                                                                                                                                             | Value                                                                               | Description                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                      |                                                                                     | <p>apache5/connector/<br/>Apache5ConnectorProvider.html]</p> <p>The value MUST be an instance of <code>java.lang.Boolean</code>.</p> <p>The default value is <code>false</code>.</p>                                                                                                                                                         |
| <p>Apache5ClientProperties.CREDENTIALS_PROVIDER<br/>[https://eclipse-ee4j.github.io/<br/>jersey.github.io/apidocs/<br/>snapshot/jersey/org/glassfish/<br/>jersey/apache5/connector/<br/>Apache5ClientProperties.html#CREDENTIALS_PROVIDER]</p>                       | <p><code>org.apache.hc.client5.http.auth.CredentialsProvider</code></p>             | <p>The credentials provider should be used to retrieve credentials from a user. Credentials needed for proxy authentication are stored here as well.</p> <p>The value MUST be an instance of <code>org.apache.hc.client5.http.auth.CredentialsProvider</code>.</p> <p>If the property is absent a default provider will be used.</p>         |
| <p>Apache5ClientProperties.DISABLE_COOKIES<br/>[https://eclipse-ee4j.github.io/<br/>jersey.github.io/apidocs/<br/>snapshot/jersey/org/glassfish/<br/>jersey/apache5/connector/<br/>Apache5ClientProperties.html#DISABLE_COOKIES]</p>                                 | <p><code>java.lang.Boolean</code></p>                                               | <p>The value indicates the client should handle cookies automatically using <code>HttpClient</code>'s default cookie policy. A value of <code>true</code> will cause the client to ignore all cookies.</p> <p>The value MUST be an instance of <code>java.lang.Boolean</code>.</p> <p>The default value is <code>false</code>.</p>           |
| <p>Apache5ClientProperties.KEEPALIVE_STRATEGY<br/>[https://eclipse-ee4j.github.io/<br/>jersey.github.io/apidocs/<br/>snapshot/jersey/org/glassfish/<br/>jersey/apache5/connector/<br/>Apache5ClientProperties.html#KEEPALIVE_STRATEGY]</p>                           | <p><code>org.apache.hc.client5.http.classic.connection.KeepAliveStrategy</code></p> | <p>The <code>KeepAliveStrategy</code> for the <code>org.apache.hc.client5.http.classic.HttpClient</code>.</p> <p>The value MUST be an instance of <code>org.apache.hc.client5.http.classic.connection.KeepAliveStrategy</code>.</p> <p>If the property is absent the default keepalive strategy of the Apache HTTP library will be used.</p> |
| <p>Apache5ClientProperties.PREEMPTIVE_BASIC_AUTHENTICATION<br/>[https://eclipse-ee4j.github.io/<br/>jersey.github.io/apidocs/<br/>snapshot/jersey/org/glassfish/<br/>jersey/apache5/connector/<br/>Apache5ClientProperties.html#PREEMPTIVE_BASIC_AUTHENTICATION]</p> | <p><code>boolean</code></p>                                                         | <p>The value of <code>true</code> indicates that a client should send an authentication request even before the server gives a 401 response.</p> <p>This property may only be set prior to constructing Apache connector using <code>Apache5ConnectorProvider</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/</p>  |

---

343

| Constant | Value | Description                                                                                                                                                                            |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          |       | <p><code>http.keepAlive</code>,<br/><code>http.maxConnections</code>.</p> <p>The value MUST be an instance of <code>Boolean</code></p> <p>The default value is <code>false</code>.</p> |

## A.8. Helidon HTTP client configuration properties

List of client configuration properties that can be found in `HelidonClientProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/helidon/connector/HelidonClientProperties.html>] class.

**Table A.8. List of Helidon HTTP client configuration properties**

| Constant                                                                                                                                                                                                                                                                                                                                                             | Value                                        | Description                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>HelidonClientProperties.CONFIG</code><br><a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/helidon/connector/HelidonClientProperties.html#CONFIG">[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/helidon/connector/HelidonClientProperties.html#CONFIG]</a> | <code>jersey.connector.helidon.config</code> | <p>A <code>io.helidon.Config</code> instance that is passed to <code>io.helidon.webclient.WebClient.Builder</code> if available.</p> |

## A.9. JDK HTTP client configuration properties

List of client configuration properties that can be found in `JdkConnectorProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProperties.html>] class.

**Table A.9. List of Jdk HTTP client configuration properties**

| Constant                                                                                                                                                                                                                                                                                                                                                                                                  | Value                                                                           | Description                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>JdkConnectorProperties.CONTAINER_IDLE_TIMEOUT</code><br><a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProperties.html#CONTAINER_IDLE_TIMEOUT">[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProperties.html#CONTAINER_IDLE_TIMEOUT]</a> | <code>io.helidon.config.client.JdkConnectorProvider.containerIdleTimeout</code> | <p>Connector provider containerIdleTimeout (milliseconds Integer value).</p> <p>The default value is <code>java.net.CookiePolicy#ACCEPT_ORIGINAL</code></p>                                                                                                                                          |
| <code>JdkConnectorProperties.COOKIE_POLICY</code><br><a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProperties.html#COOKIE_POLICY">[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProperties.html#COOKIE_POLICY]</a>                            | <code>io.helidon.config.client.JdkConnectorProvider.cookiePolicy</code>         | <p>Connector provider cookiePolicy cookie manager.</p> <p>When the timeout elapses, the shared thread pool will be destroyed.</p> <p>The default value is <code>JdkConnectorProperties.DEFAULT_CONNECTION</code> [<a href="https://eclipse-ee4j.github.io/">https://eclipse-ee4j.github.io/</a>]</p> |



| Constant                                                                                                                                                                                                                                           | Value                        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                    |                              | jersey.github.io/apidocs/snapshot/<br>jersey/org/glassfish/jersey/jdk/<br>connector/<br>JdkConnectorProperties.html#DEFAULT_CONNECTION_IDLE_TIMEOUT                                                                                                                                                                                                                                                                                                                                          |
| JdkConnectorProperties.CONNECTION_IDLE_TIMEOUT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/jdk/connector/<br>JdkConnectorProperties.html#CONNECTION_IDLE_TIMEOUT]                 | JerseyClientConfig.Tentative | A Connector Provider of maxIdleTime<br>(Integer value) during which an<br>idle connection will be kept open.<br><br>The default value is<br>JdkConnectorProperties.DEFAULT_CONNECTION_IDLE_TIMEOUT<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/snapshot/<br>jersey/org/glassfish/jersey/jdk/<br>connector/<br>JdkConnectorProperties.html#DEFAULT_CONNECTION_IDLE_TIMEOUT]                                                                                               |
| JdkConnectorProperties.MAX_CONNECTIONS_PER_DESTINATION<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/jdk/connector/<br>JdkConnectorProperties.html#MAX_CONNECTIONS_PER_DESTINATION] | JerseyClientConfig.DEFAULT   | A Connector Provider of maxConnectionsPerDestination<br>connections per each destination.<br>A destination is determined by<br>the following triple: host, port,<br>protocol (HTTP/HTTPS).<br><br>The default value is<br>JdkConnectorProperties.DEFAULT_MAX_CONNECTIONS_PER_DESTINATION<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/snapshot/<br>jersey/org/glassfish/jersey/jdk/<br>connector/<br>JdkConnectorProperties.html#DEFAULT_MAX_CONNECTIONS_PER_DESTINATION] |
| JdkConnectorProperties.MAX_HEADER_SIZE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/jdk/connector/<br>JdkConnectorProperties.html#MAX_HEADER_SIZE]                                 | JerseyClientConfig.DEFAULT   | A Configurable Provider of MAXHeaderSize<br>parser. It defines the maximal<br>acceptable size of HTTP response<br>initial line, each header and chunk<br>header.<br><br>The default value is<br>JdkConnectorProperties.DEFAULT_MAX_HEADER_SIZE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/snapshot/<br>jersey/org/glassfish/jersey/jdk/<br>connector/<br>JdkConnectorProperties.html#DEFAULT_MAX_HEADER_SIZE]                                                           |
| JdkConnectorProperties.MAX_REDIRECTS<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/jdk/connector/<br>JdkConnectorProperties.html#MAX_REDIRECTS]                                     | JerseyClientConfig.DEFAULT   | A Connector Provider of maxRedirects<br>during single request.<br><br>Value is expected to be positive<br>Integer. The default value is<br>JdkConnectorProperties.DEFAULT_MAX_REDIRECTS<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/snapshot/<br>jersey/org/glassfish/jersey/jdk/                                                                                                                                                                                        |

| Constant                                                                                                                                                                                                            | Value                                          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                     |                                                | connector/<br>JdkConnectorProperties.html#DEFAULT_MAX_REDIRECTS<br><br>HTTP redirection must be enabled by property ClientProperties.FOLLOW_REDIRECTS [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#FOLLOW_REDIRECTS], otherwise JdkConnectorProperties.MAX_REDIRECTS [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProperties.html#MAX_REDIRECTS] is not applied. |
| JdkConnectorProperties.WORKER_THREAD_POOL_CONFIG [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jdk/connector/JdkConnectorProperties.html#WORKER_THREAD_POOL_CONFIG] | org.glassfish.jersey.connector.WorkerThreadPoo | Configuration of worker thread pool.<br><br>An instance of <code>org.glassfish.jersey.jdk.connector.</code> is expected.                                                                                                                                                                                                                                                                                                                                                                                                  |

## A.10. Jetty HTTP client configuration properties

List of client configuration properties that can be found in JettyClientProperties [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyClientProperties.html] class.

**Table A.10. List of Jetty HTTP client configuration properties**

| Constant                                                                                                                                                                                        | Value                                                                | Description                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JettyClientProperties.DISABLE_COOKIES [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyClientProperties.html#DISABLE_COOKIES] | org.glassfish.jersey.client.config.jetty.client.disableCookies       | AntidisableCookies indicates the client should handle cookies automatically using HttpClient's default cookie policy. A value of <b>false</b> will cause the client to ignore all cookies.<br><br>The value MUST be an instance of <b>Boolean</b> . If the property is absent the default value is <b>false</b> . |
| JettyClientProperties.ENABLE_SSL_HOSTNAME_VERIFICATION [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/                                                  | org.glassfish.jersey.client.config.jetty.client.disableCookies false | AntidisableCookies false indicates the client disable a hostname verification during SSL Handshake. A client will ignore                                                                                                                                                                                          |

| Constant                                                                                                                                                                                                                        | Value                                                   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey/jetty/connector/<br>JettyClientProperties.html#ENABLE_SSL_HOSTNAME_VERIFICATION]                                                                                                                                         |                                                         | <p>CN value defined in a certificate and ignored in a truststore.</p> <p>The value MUST be an instance of <code>Boolean</code>. If the property is absent the default value is <code>true</code>.</p>                                                                                                                                                                                                                                                                                                                                                                                                                       |
| JettyClientProperties.PREEMPTIVE_BASIC_AUTHENTICATION [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyClientProperties.html#PREEMPTIVE_BASIC_AUTHENTICATION] | <code>org.eclipse.jetty.client.api.Authenticator</code> | <p>The preemptive basic authentication mechanism is used to retrieve credentials from a user.</p> <p>If an <code>org.eclipse.jetty.client.api.Authenticator</code> mechanism is found, it is then used for the given request, returning an <code>org.eclipse.jetty.client.api.Authenticator</code> which is then stored in the <code>org.eclipse.jetty.client.api.Authenticator</code> so that subsequent requests can be preemptively authenticated.</p> <p>The value MUST be an instance of <code>org.eclipse.jetty.client.util.BasicAuthenticator</code>. If the property is absent a default provider will be used.</p> |
| JettyClientProperties.SYNC_LISTENER_RESPONSE_MAX_SIZE [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyClientProperties.html#SYNC_LISTENER_RESPONSE_MAX_SIZE] | <code>Integer</code>                                    | <p>Overrides the default response synchronous listener response max buffer size. In practise, this allows you to read larger responses. Size in bytes.</p> <p>If the property is absent, the value is such as specified by Jetty (currently 2MiB).</p>                                                                                                                                                                                                                                                                                                                                                                      |
| JettyClientProperties.TOTAL_TIMEOUT [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jetty/connector/JettyClientProperties.html#TOTAL_TIMEOUT]                                     | <code>long</code>                                       | <p>The total timeout interval for request/response conversation, in milliseconds. Opposed to <code>ClientProperties.READ_TIMEOUT</code> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/client/ClientProperties.html#READ_TIMEOUT].</p> <p>The value MUST be an instance convertible to <code>Integer</code>. The value of zero <code>0</code> is equivalent to an interval of infinity.</p> <p>The default value is <code>0</code> (infinity).</p>                                                                                                                           |

## A.11. Netty HTTP client configuration properties

List of client configuration properties that can be found in `NettyClientProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html>] class.

**Table A.11. List of Netty HTTP client configuration properties**

| Constant                                                                                                                                                                                                                                                                                                                                                                                                                       | Value                                                                   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>NettyClientProperties.HTTP_REDIRECT_CONTROLLER</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#HTTP_REDIRECT_CONTROLLER">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#HTTP_REDIRECT_CONTROLLER</a> ]                | <code>JerseyConnectorClient.netty.http.redirect.controller</code>       | The <code>http.redirect.controller</code> property determines the <code>NettyHttpRedirectController</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyHttpRedirectController.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyHttpRedirectController.html</a> ] redirect logic. Since 2.47 |
| <code>NettyClientProperties.FILTER_HEADERS_FOR_PROXY</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#FILTER_HEADERS_FOR_PROXY">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#FILTER_HEADERS_FOR_PROXY</a> ]                | <code>JerseyConnectorClient.filter.headers.for.proxy</code>             | Filter headers for requests (CONNECT) towards the proxy except for PROXY-prefixed and HOST headers when <code>true</code> .<br><br>The default value is <code>true</code> and the headers are filtered out.                                                                                                                                                                                                                                         |
| <code>NettyClientProperties.IDLE_CONNECTION_PRUNE_TIMEOUT</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#IDLE_CONNECTION_PRUNE_TIMEOUT">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#IDLE_CONNECTION_PRUNE_TIMEOUT</a> ] | <code>JerseyConnectorClient.idle.connection.prune.timeout</code>        | This property determines the number of seconds the idle connections are kept in the pool before pruned. The default is 60. Specify 0 to disable.                                                                                                                                                                                                                                                                                                    |
| <code>NettyClientProperties.MAX_CONNECTIONS</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_CONNECTIONS">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_CONNECTIONS</a> ]                                           | <code>JerseyConfig.client.maxConnections</code>                         | This property determines the maximum number of idle connections that will be simultaneously kept alive, per destination. The default is 5.<br><br>This property is a Jersey alternative to System property <code>http.maxConnections</code> . The Jersey property takes precedence over the system property.                                                                                                                                        |
| <code>NettyClientProperties.MAX_CONNECTIONS_TOTAL</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_CONNECTIONS_TOTAL">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_CONNECTIONS_TOTAL</a> ]                         | <code>JerseyConnectorClient.maxConnectionsTotal</code>                  | This property defines the maximum number of idle connections that will be simultaneously kept alive in total, rather than per destination. The default is 60. Specify 0 to disable.                                                                                                                                                                                                                                                                 |
| <code>NettyClientProperties.MAX_REDIRECTS</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_REDIRECTS">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_REDIRECTS</a> ]                                                 | <code>Jersey.config.client.netty.connector.provider.maxRedirects</code> | This property provides the maximal number of redirects                                                                                                                                                                                                                                                                                                                                                                                              |

| Constant                                                                                                                                                                                                                    | Value                                                  | Description                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/netty/connector/<br>NettyClientProperties.html#MAX_REDIRECTS]                                                                                         |                                                        | during single request. Value is expected to be positive number. Default value is 5. HTTP redirection must be enabled by property org.glassfish.jersey.client.ClientProperties.FOLLOW_REDIRECTS otherwise NettyClientProperties.MAX_REDIRECTS [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_REDIRECTS] is not applied. |
| NettyClientProperties.PRESERVE_METHOD_ON_REDIRECT.<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#PRESERVE_METHOD_ON_REDIRECT] | jersey.config.client.netty.PRESERVE_METHOD_ON_REDIRECT | Indicates if the HTTP POST method to be preserved on HTTP status 301 (MOVED PERMANENTLY) or status 302 (FOUND) when true or redirected as GET when false.<br><br>The default value is true and the HTTP POST request is not redirected as GET.                                                                                                                                                                       |
| NettyClientProperties.EXPECT_100_CONTINUE_TIMEOUT.<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#EXPECT_100_CONTINUE_TIMEOUT] | jersey.config.client.netty.EXPECT_100_CONTINUE_TIMEOUT | This timeout is used for waiting for 100-Continue response when 100-Continue is sent by the client. Default timeout value is 500 ms after which Expect:100-Continue feature is ignored. Since 2.41                                                                                                                                                                                                                   |
| NettyClientProperties.MAX_HEADER_SIZE.<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_HEADER_SIZE]                         | jersey.config.client.netty.MAX_HEADER_SIZE             | Maximum header size extending of the header size for the Netty connector Default header size is 8192b.                                                                                                                                                                                                                                                                                                               |
| NettyClientProperties.MAX_CHUNK_SIZE.<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_CHUNK_SIZE]                           | jersey.config.client.netty.MAX_CHUNK_SIZE              | Maximum chunk size extending of the chunk size for the Netty connector Default chunk size is 8192b.                                                                                                                                                                                                                                                                                                                  |
| NettyClientProperties.MAX_INITIAL_LINE_LENGTH.<br>[https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/netty/connector/NettyClientProperties.html#MAX_INITIAL_LINE_LENGTH]         | jersey.config.client.netty.MAX_INITIAL_LINE_LENGTH     | Maximum initial line length extending of the initial line length for the Netty connector Default initial line length is 4096b.                                                                                                                                                                                                                                                                                       |

## A.12. Java Net HTTP client configuration properties

List of client configuration properties that can be found in `JavaNetHttpClientProperties` [<https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html>] class.

**Table A.12. List of Java Net HTTP client configuration properties**

| Constant                                                                                                                                                                                                                                                                                                                                                                           | Value                                                       | Description                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>JavaNetHttpClientProperties.COOKIE_HANDLER_CONFIG</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html</a> ]           | <code>KEY_COOKIE_HANDLER_CONFIG.jnh.client</code>           | Configuration of the <code>java.net.CookieHandler</code> that should be used by the <code>java.net.http.HttpClient</code> . If this option is not set, <code>java.net.http.HttpClient</code> will return an empty <code>java.util.Optional</code> and therefore no cookie handler will be used.<br><br>A provided value to this option has to be of type <code>java.net.CookieHandler</code> . |
| <code>JavaNetHttpClientProperties.SSL_PARAMETERS</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html</a> ]                  | <code>KEY_SSL_PARAMETERS.jnh.client</code>                  | Configuration of SSL parameters used by the <code>java.net.http.HttpClient</code> . If this option is not set, then the <code>java.net.http.HttpClient</code> will use <i>&lt;it&gt;implementation specific&lt;/it&gt;</i> default values.<br><br>A provided value to this option has to be of type <code>javax.net.ssl.SSLParameters</code> .                                                 |
| <code>JavaNetHttpClientProperties.PREEMPTIVE_BASIC_AUTHENTICATION</code> [ <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html">https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html</a> ] | <code>KEY_PREEMPTIVE_BASIC_AUTHENTICATION.jnh.client</code> | Configuration of <code>PreemptiveBasicAuthenticator</code> class which represents an object that knows how to obtain authentication for a network connection should be used.<br><br>If an <code>java.net.Authenticator</code> instance is found, it is then used for the given request.<br><br>The value MUST be an instance of <code>java.net.Authenticator</code> .                          |

| Constant                                                                                                                                                                                                                                                                                                                                                                            | Value                                             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                     |                                                   | If the property is absent, no authentication is used.                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html#DISABLE_COOKIES">JavaNetHttpClientProperties.DISABLE_COOKIES</a> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html#DISABLE_COOKIES] | <code>HttpConfig.jnh.client.disableCookies</code> | <p>A value of <code>false</code> indicates the client should handle cookies automatically using <code>HttpClient</code>'s default cookie policy. A value of <code>true</code> will cause the client to ignore all cookies.</p> <p>The value MUST be an instance of <code>java.lang.Boolean</code>.</p> <p>The default value is <code>false</code>.</p>                                                                                                                                                |
| <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html#HTTP_VERSION">JavaNetHttpClientProperties.HTTP_VERSION</a> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/jnh/connector/JavaNetHttpClientProperties.html#HTTP_VERSION]          | <code>HttpConfig.jnh.client.httpVersion</code>    | <p>HTTP version of null or instance of <code>HttpClient.Version.HTTP_1_1</code> the version will be set to <code>HTTP_1_1</code>. If version is <code>HttpClient.Version.HTTP_2</code> the client will attempt to perform each request using <code>HTTP_2</code> protocol but if not supported by server, the protocol will be still <code>HTTP_1_1</code>.</p> <p>The value MUST be an instance of <code>java.net.http.HttpClient.Version</code>.</p> <p>The default value is <code>null</code>.</p> |

## A.13. Multipart configuration properties

List of multipart configuration properties that can be found in `MultiPartProperties` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartProperties.html] class.

**Table A.13. List of multipart configuration properties settable in the `MultiPartProperties.MULTI_PART_CONFIG_RESOURCE` [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartProperties.html#MULTI\_PART\_CONFIG\_RESOURCE] configuration file.**

| Constant                                                                                                                                                                                                                                                                                                                                                           | Value                                             | Description                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartProperties.html#BUFFER_THRESHOLD">MultiPartProperties.BUFFER_THRESHOLD</a> [https://eclipse-ee4j.github.io/jersey.github.io/apidocs/snapshot/jersey/org/glassfish/jersey/media/multipart/MultiPartProperties.html#BUFFER_THRESHOLD] | <code>HttpConfig.multipart.bufferThreshold</code> | <p>Name of the threshold property for the threshold size (in bytes) above which a body part entity will be buffered to disk instead of being held in memory.</p> <p>The default value is <code>MessageProperties.IO_DEFAULT_BUFFER_SIZE</code> [https://eclipse-ee4j.github.io/</p> |

| Constant                                                                                                                                                                                                                             | Value                                             | Description                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                      |                                                   | jersey.github.io/apidocs/snapshot/<br>jersey/org/glassfish/jersey/<br>message/<br>MessageProperties.html#IO_DEFAULT_BUFFER_SIZE                                                                                                                               |
| MultiPartProperties.MAX_PARTS<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/media/multipart/<br>MultiPartProperties.html#MAX_PARTS]                                   | jersey.config.multipart.<br>maxParts              | <b>MaxParts</b><br>The maximum number of parts the multipart entity can have. If the limit is over, the error response status 413 - REQUEST_ENTITY_TOO_LARGE is returned.<br><br>By default, the number is unlimited.                                         |
| MultiPartProperties.MULTI_PART_CONFIG_RESOURCE<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/media/multipart/<br>MultiPartProperties.html#MULTI_PART_CONFIG_RESOURCE] | jersey.config.<br>multipart.<br>config.properties | Name of a properties resource that (if found in the classpath for this application) will be used to configure the settings returned by our getter methods.                                                                                                    |
| MultiPartProperties.TEMP_DIRECTORY<br>[https://eclipse-ee4j.github.io/<br>jersey.github.io/apidocs/<br>snapshot/jersey/org/glassfish/<br>jersey/media/multipart/<br>MultiPartProperties.html#TEMP_DIRECTORY]                         | jersey.config.multipart.<br>tempDir               | <b>TempDir</b><br>The resource property for the directory to store temporary files containing body parts of multipart message that extends allowed memory threshold.<br><br>The default value is not set (will be taken from java.io.tmpdir system property). |